



DesignWare DW_ahb_dmac Databook

DW_ahb_dmac – Product Code: 3889-0

Copyright Notice and Proprietary Information Notice

© 2015 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <http://www.synopsys.com/Company/Pages/Trademarks.aspx>.

All other product or company names may be trademarks of their respective owners.

Synopsys, Inc.
690 E. Middlefield Road
Mountain View, CA 94043
www.synopsys.com

Contents

Revision History	7
Preface	11
Chapter 1	
Product Overview	15
1.1 DesignWare System Overview	15
1.2 General Product Description	17
1.2.1 DW_ahb_dmac Block Diagram	17
1.3 Basic Definitions	19
1.4 Features	24
1.4.1 General	24
1.4.2 Address Generation	24
1.4.3 Channel Buffering	25
1.4.4 Channel Control	25
1.4.5 Transfer Initiation	25
1.4.6 Flow Control	26
1.4.7 Interrupts	26
1.5 Standards Compliance	26
1.6 Verification Environment Overview	26
1.7 Licenses	26
1.8 Where To Go From Here	26
Chapter 2	
Building and Verifying a Component or Subsystem	27
2.1 Setting up Your Environment	27
2.2 Overview of the coreConsultant Configuration and Integration Process	28
2.2.1 coreConsultant Usage	28
2.2.2 Configuring the DW_ahb_dmac within coreConsultant	30
2.2.3 Creating Gate-Level Netlists within coreConsultant	30
2.2.4 Verifying the DW_ahb_dmac within coreConsultant	30
2.2.5 Running Leda on Generated Code with coreConsultant	30
2.2.6 Running SpyGlass® Lint and SpyGlass® CDC	30
2.3 Overview of the coreAssembler Configuration and Integration Process	34
2.3.1 coreAssembler Usage	34
2.3.2 Configuring the DW_ahb_dmac within a Subsystem	38
2.3.3 Creating Gate-Level Netlists within coreAssembler	38
2.3.4 Verifying the DW_ahb_dmac within coreAssembler	38
2.3.5 Running Leda on Generated Code with coreAssembler	38

2.3.6 Running Spyglass on Generated Code with coreAssembler	38
2.4 Database Files	38
2.4.1 Design/HDL Files	38
2.4.2 Synthesis Files	40
2.4.3 Verification Reference Files	40
Chapter 3	
Functional Description	41
3.1 Setup/Operation of DW_ahb_dmac Transfers	41
3.2 Block Flow Controller and Transfer Type	41
3.3 Handshaking Interface	42
3.4 Basic Interface Definitions	43
3.5 Memory Peripherals	44
3.6 Software Handshaking	44
3.7 Handshaking Interface – Peripheral Is Not Flow Controller	45
3.7.1 Single Transaction Region	46
3.7.2 Early-Terminated Burst Transaction	47
3.7.3 Hardware Handshaking – Peripheral Is Not Flow Controller	48
3.7.4 Software Handshaking – Peripheral Is Not Flow Controller	54
3.7.5 Single Transactions – Peripheral Is Not Flow Controller	55
3.8 Handshaking Interface – Peripheral Is Flow Controller	56
3.8.1 Hardware Handshaking – Peripheral Is Flow Controller	57
3.8.2 Software Handshaking – Peripheral Is Flow Controller	59
3.8.3 Single Transactions – Peripheral is Flow Controller	60
3.9 Setting Up Transfers	60
3.9.1 Transfer Operation	61
3.9.2 Peripheral Interrupt Request Interface	94
3.10 Flow Control Configurations	95
3.11 Peripheral Burst Transaction Requests	97
3.11.1 Transmit Watermark Level and Transmit FIFO Underflow	97
3.11.2 Choosing the Transmit Watermark Level	97
3.11.3 Selecting CTLx.DEST_MSIZ and Transmit FIFO Overflow	100
3.11.4 Receive Watermark Level and Receive FIFO Overflow	100
3.11.5 Choosing the Receive Watermark level	101
3.11.6 Selecting CTLx.SRC_MSIZ and Receive FIFO Underflow	101
3.12 Generating Requests for the AHB Master Bus Interface	101
3.12.1 Locked DMA Transfers	103
3.13 Arbitration for AHB Master Interface	105
3.14 Latency	106
3.15 Scatter/Gather	108
3.16 Endianness	111
3.16.1 Big Endian-Little Endian Conversion Logic	111
3.16.2 LLI Fetch, and Status and Control Write-Back	112
3.16.3 Endian Selection	112
3.16.4 Static Endian Configuration	114
3.16.5 Dynamic Endian Configuration	114
3.17 AHB Transfer Error Handling	115
3.18 Last Beat of DMA Burst Indication	116
3.18.1 Example 1	116

Chapter 4	
Parameters	119
4.1 Parameter Descriptions	119
Chapter 5	
Signal Descriptions	135
Chapter 6	
Registers	147
6.1 Register Memory Map	147
6.2 Register and Field Descriptions	160
6.2.1 Configuration and Channel Enable Registers	160
6.2.2 Channel Registers	163
6.2.3 Interrupt Registers	190
6.2.4 Software Handshaking Registers	198
6.2.5 Miscellaneous DW_ahb_dmac Registers	205
Chapter 7	
Internal Parameter Descriptions	243
Chapter 8	
Programming the DW_ahb_dmac	245
8.1 Software Drivers	245
8.2 Register Access	246
8.3 Illegal Register Access	246
8.4 DW_ahb_dmac Transfer Types	246
8.4.1 Multi-Block Transfers	247
8.4.2 Auto-Reloading of Channel Registers	251
8.4.3 Contiguous Address Between Blocks	251
8.4.4 Suspension of Transfers Between Blocks	252
8.4.5 Ending Multi-Block Transfers	252
8.5 Programing Examples	253
8.5.1 Programming Example for Linked List Multi-Block Transfer	255
8.6 Programming a Channel	256
8.6.1 Programming Examples	257
8.7 Disabling a Channel Prior to Transfer Completion	282
8.7.1 Abnormal Transfer Termination	283
8.8 Defined-Length Burst Support on DW_ahb_dmac	283
Chapter 9	
Verification	285
9.1 Overview of Vera Tests	285
9.2 Overview of DW_ahb_dmac Testbench	286
Chapter 10	
Integration Considerations	289
10.1 Performance	289
10.1.1 Area	289
10.1.2 Power Consumption	294
10.2 1KB Boundary Crossing	298
10.3 Read Accesses	298

10.4 Write Accesses	299
10.5 Consecutive Write-Read	300
10.6 Accessing Top-level Constraints	301
10.7 Coherency	301
10.7.1 Writing Coherently	302
10.7.2 Reading Coherently	308
Appendix A	
Error and Warning Messages	313
A.1 Warnings During Simulation	313
A.2 Warnings During Synthesis	313
Appendix B	
Channel Locking and Deadlock	315
B.1 Hardware Detection of Deadlock	316
B.1.1 Case 1	316
B.1.2 Case 2	317
B.1.3 Case 3	317
B.1.4 Deadlock Prevention by Hardware	318
B.2 Programming Restrictions to Avoid Deadlock	319
Appendix C	
DW_ahb_dmac Application Notes	321
C.1 Interoperability Between DW_ahb_dmac and PrimeCell Hardware Handshaking Interface	321
C.2 Mapping of PrimeCell Software Handshaking Registers to DW_ahb_dmac	323
C.2.1 PrimeCell Software Handshaking Registers	323
C.2.2 DW_ahb_dmac Software Handshaking Registers	324
C.2.3 Register Interface Mapping	324
Appendix D	
Configuring DW_ahb_dmac to Match ARM PrimeCell PL080/PL081	329
D.1 ARM PL080 Equivalent	329
D.2 ARM PL081 Equivalent	329
Appendix E	
Glossary	333
Index	337

Revision History

This table shows the revision history for the databook from release to release. This is being tracked from version 2.10b onward.

Version	Date	Description
2.20a	June 2015	Added: ■ “Running SpyGlass® Lint and SpyGlass® CDC” on page 30 ■ “Running Spyglass on Generated Code with coreAssembler” on page 38 ■ Chapter 7, “Internal Parameter Descriptions” Updated: ■ Chapter 5, “Signal Descriptions” auto-extracted from the RTL
2.19a	June 2014	Updated: ■ Version for 2014.06a release ■ Big endian BE32 format for data transfer on AHB master interface ■ Performance section in Integration consideration ■ Default Input/Output Delay in the following Signal groups: - Slave Interface - Master <i>N</i> Interface
2.18b	May 2013	Added: ■ An output signal dma_wlast on the AHB interface to indicate the last write data during burst transfers to destination peripherals ■ Configuration parameter DMAH_WLAST_EN to enable the dma_wlast signal ■ Added section “Last Beat of DMA Burst Indication” to explain last write data during bursts transfers feature Removed: ■ Note stating that disabling the channel via software prior to completing a transfer is not supported when DW_ahb_dmac is configured to use defined length bursts; feature now supported Updated: ■ Corrected the sequence of bits in the DMA_COMP_PARAMS_1 register ■ Document template
2.17d	Sep 2012	Added the product code on the cover

(Continued)

Version	Date	Description
2.17d	Mar 2012	Updated: ■ Instructions for setting bit 0 of DmaCfgReg register ■ Descriptions of SSTATARx and DSTATARx registers ■ Information in Early-Terminated Burst Transaction section
2.17c	Nov 2011	Version change for 2011.11a release
2.17b	Oct 2011	Updated: ■ Corrected dma_req and dma_single as being registered ■ SSTATARx and DSTATARx register descriptions
2.17a	Jun 2011	Updated: ■ System diagram in Figure 1-1 ■ “Related Documentation” section in Preface
2.17a	May 2011	Edited “Burst transaction” definition
2.17a	Apr 2011	Added: ■ New “Latency” section ■ New DMAH_REVERSE_WB_OVERRIDE parameter ■ Information to “Memory Peripherals” section about impact of CTLx.SRC_MSIZE, CTLx.DEST_MSIZE values on burst transfers to/from memory peripherals
2.16a	Dec 2010	Version change for 2010.12a release
2.15a	Sep 2010	Updated: ■ Descriptions for dma_req signal ■ Corrected names of include files and vcs command used for simulation ■ Corrected defaults for DMAH_CHx_SMS and DMAH_CHx_DMS
2.14a	May 2010	Updated the DMAH_INTR_POL parameter description
2.14a	Dec 2009	Updated databook to new template for consistency with other IIP/VIP/PHY databooks
2.14a	Jul 2009	Corrected equations for avoiding underflow when programming a source burst transaction
2.14a	May 2009	Removed references to QuickStarts, as they are no longer supported
2.14a	Apr 2009	Added: Note that multi-block transfers not supported Updated: Corrected DMAH_CHx_FIFO_DEPTH equation
2.14a	Nov 2008	Corrected CFG* reset values in memory map

(Continued)

Version	Date	Description
2.14a	Oct 2008	Version change for 2008.10a release
2.12a	Jul 2008	Corrected Setting Up Transfers example #4 calculation
2.12a	Jun 2008	Updated: ■ Descriptions for software handshaking registers to say that channel must be enabled to allow writing to a bit ■ Descriptions for INT_EN register
2.11a	Mar 2008	Updated: ■ CFGx.FIFO_EMPTY default to 0x1 ■ Text for value of 1 for CFGx.FIFO_MODE
2.10b	Jan 2008	Updated: ■ Table 23 (DMAH_CHx_CTL_WB_EN, DMAH_CHx_MULTI_BLK_TYPE) ■ Table 25 (SAR4, SGR4) ■ “Hardware Realignment of SAR/DAR Registers” section ■ Figure 71, LLPI register definition
2.10b	July 2007	Corrected available files in workspace/sim/test_name directory
2.10b	June 2007	Version change for 2007.06a release

Preface

This databook provides information that you need to interface the DesignWare AHB Central Direct Memory Access (DMA) Controller, referred to as DW_ahb_dmac throughout the remainder of this databook. This component conforms to the [AMBA Specification, Revision 2.0](#) from ARM.

The information in this databook includes a functional description, signal and parameter descriptions, and a memory map. Also provided are an overview of the component testbench, a description of the tests that are run to verify the coreKit, and synthesis information for the component.

Organization

The chapters of this databook are organized as follows:

- Chapter 1, “[Product Overview](#)” provides a system overview, a component block diagram, basic features, and an overview of the verification environment.
- Chapter 2, “[Building and Verifying a Component or Subsystem](#)” introduces you to using the DW_ahb_dmac within the coreAssembler and coreConsultant tools.
- Chapter 3, “[Functional Description](#)” describes the functional operation of the DW_ahb_dmac.
- Chapter 4, “[Parameters](#)” identifies the configurable parameters supported by the DW_ahb_dmac.
- Chapter 5, “[Signal Descriptions](#)” provides a list and description of the DW_ahb_dmac signals.
- Chapter 6, “[Registers](#)” describes the programmable registers of the DW_ahb_dmac.
- Chapter 7, “[Internal Parameter Descriptions](#)” describes the programmable registers of the DW_ahb_dmac.
- Chapter 8, “[Programming the DW_ahb_dmac](#)” provides information needed to program the configured DW_ahb_dmac.
- Chapter 9, “[Verification](#)” provides information on verifying the configured DW_ahb_dmac.
- Chapter 10, “[Integration Considerations](#)” includes information you need to integrate the configured DW_ahb_dmac into your design.
- Appendix A, “[Error and Warning Messages](#)” describes errors and warnings that you may encounter when configuring, verifying, or synthesizing the DesignWare component using the coreTools GUIs.
- Appendix B, “[Channel Locking and Deadlock](#)” explains how channel locking can cause deadlock.

- Appendix C, “[DW_ahb_dmac Application Notes](#)” describes the interoperability between DW_ahb_dmac and ARM PrimeCell handshaking interface.
- Appendix D, “[Configuring DW_ahb_dmac to Match ARM PrimeCell PL080/PL081](#)” provides configuration parameter settings for DW_ahb_dmac so that it matches or is equivalent to the ARM PrimeCell PL080 and PL081 devices.
- Appendix E, “[Glossary](#)” provides a glossary of general terms.

Related Documentation

- [DW_ahb_dmac Driver Kit User Guide](#) – Contains information on the Driver Kit for the DW_ahb_dmac; requires source code license (DWC-APB-Periph-Source)
- [Using DesignWare Library IP in coreAssembler](#) – Contains information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools
- [coreAssembler User Guide](#) – Contains information on using coreAssembler
- [coreConsultant User Guide](#) – Contains information on using coreConsultant

To see a complete listing of documentation within the DesignWare Synthesizable Components for AMBA 2, refer to the [Guide to Documentation for DesignWare Synthesizable Components for AMBA 2 and AMBA 3 AXI](#).

Web Resources

- DesignWare IP product information: <http://www.designware.com>
- Your custom DesignWare IP page: <http://www.mydesignware.com>
- Documentation through SolvNet: <http://solvnet.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL): <http://www.synopsys.com/keys>

Customer Support

To obtain support for your product:

- First, prepare the following debug information, if applicable:
 - For environment setup problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, use the following menu entry:
File > Build Debug Tar-file

Check all the boxes in the dialog box that apply to your issue. This menu entry gathers all the Synopsys product data needed to begin debugging an issue and writes it to the file `<core tool startup directory>/debug.tar.gz`.
 - For simulation issues outside of coreConsultant or coreAssembler:
 - Create a waveforms file (such as VPD or VCD)
 - Identify the hierarchy path to the DesignWare instance
 - Identify the timestamp of any signals or locations in the waveforms that are not understood
- Then, contact Support Center, with a description of your question and supplying the above information, using one of the following methods:

- *For fastest response*, use the SolvNet website. If you fill in your information as explained below, your issue is automatically routed to a support engineer who is experienced with your product. The **Sub Product** entry is critical for correct routing.

Go to <http://solvnet.synopsys.com/EnterACall> and click on the link to enter a call.

Provide the requested information, including:

- **Product:** DesignWare Library IP
- **Sub Product:** AMBA
- **Tool Version:** *product version number*
- **Problem Type:**
- **Priority:**
- **Title:** DW_ahb_dmac
- **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood

After creating the case, attach any debug files you created in the previous step.

- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
 - Include the Product name, Sub Product name, and Tool Version number in your e-mail (as identified above) so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood
 - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<http://www.synopsys.com/Support/GlobalSupportCenters>

1

Product Overview

This chapter provides a basic overview of the DW_ahb_dmac, which is an AHB-Central DMA Controller core that transfers data from a source peripheral to a destination peripheral over one or more AHB bus.

1.1 DesignWare System Overview

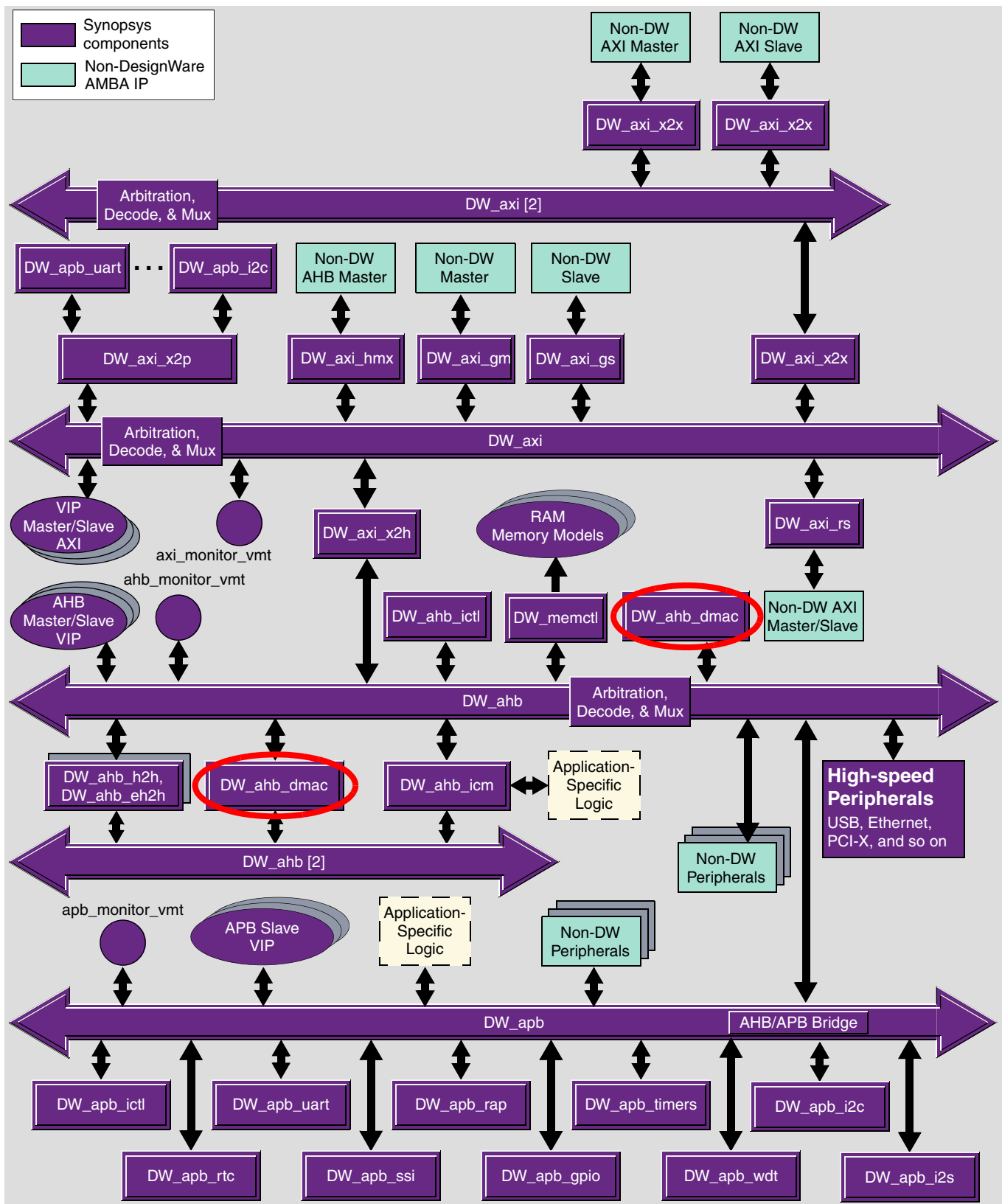
The Synopsys DesignWare Synthesizable Components environment is a parameterizable bus system containing AMBA version 2.0-compliant AHB (Advanced High-performance Bus) and APB (Advanced Peripheral Bus) components, and AMBA version 3.0-compliant AXI (Advanced eXtensible Interface) components.

[Figure 1-1](#) illustrates one example of this environment, including the AXI bus, the AHB bus, and the APB bus. Included in this subsystem are synthesizable IP for AXI/AHB/APB peripherals, bus bridges, and an AXI interconnect and AHB bus fabric. Also included are verification IP for AXI/AHB/APB master/slave models and bus monitors. In order to display the databook for a DW_* component, click on the corresponding component object in the illustration.

**Attention**

Links resolve only if you are viewing this databook from your \$DESIGNWARE_HOME tree, and to only those components that are installed in the tree.

Figure 1-1 Example of DW_ahb_dmac in a Complete System



You can connect, configure, synthesize, and verify the DW_ahb_dmac within a DesignWare subsystem using coreAssembler, documentation for which is available on the web in the [coreAssembler User Guide](#).

If you want to configure, synthesize, and verify a single component such as the DW_ahb_dmac component, you might prefer to use coreConsultant, documentation for which is available in the [coreConsultant User Guide](#).

1.2 General Product Description

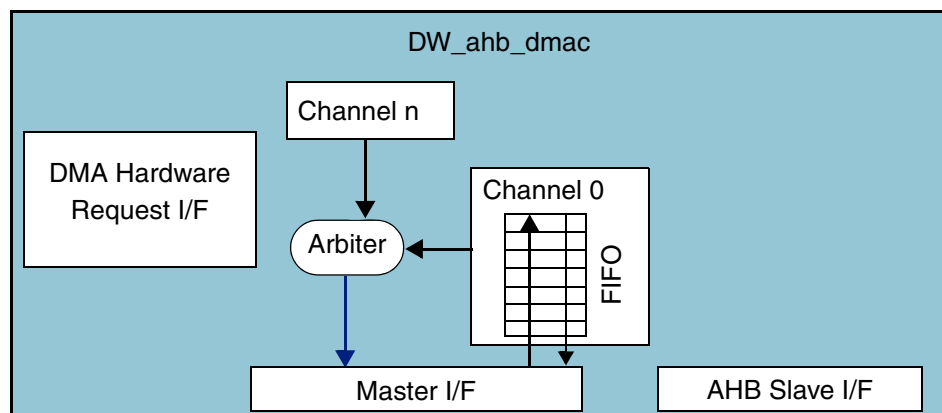
The Synopsys DW_ahb_dmac conforms to the [AMBA Specification, Revision 2.0](#) from ARM.

1.2.1 DW_ahb_dmac Block Diagram

Figure 1-2 shows the following functional groupings of the main interfaces to the DW_ahb_dmac block:

- DMA hardware request interface
- Up to eight channels
- FIFO per channel for source and destination
- Arbiter
- AHB master interface
- AHB slave interface

Figure 1-2 DW_ahb_dmac Block Diagram

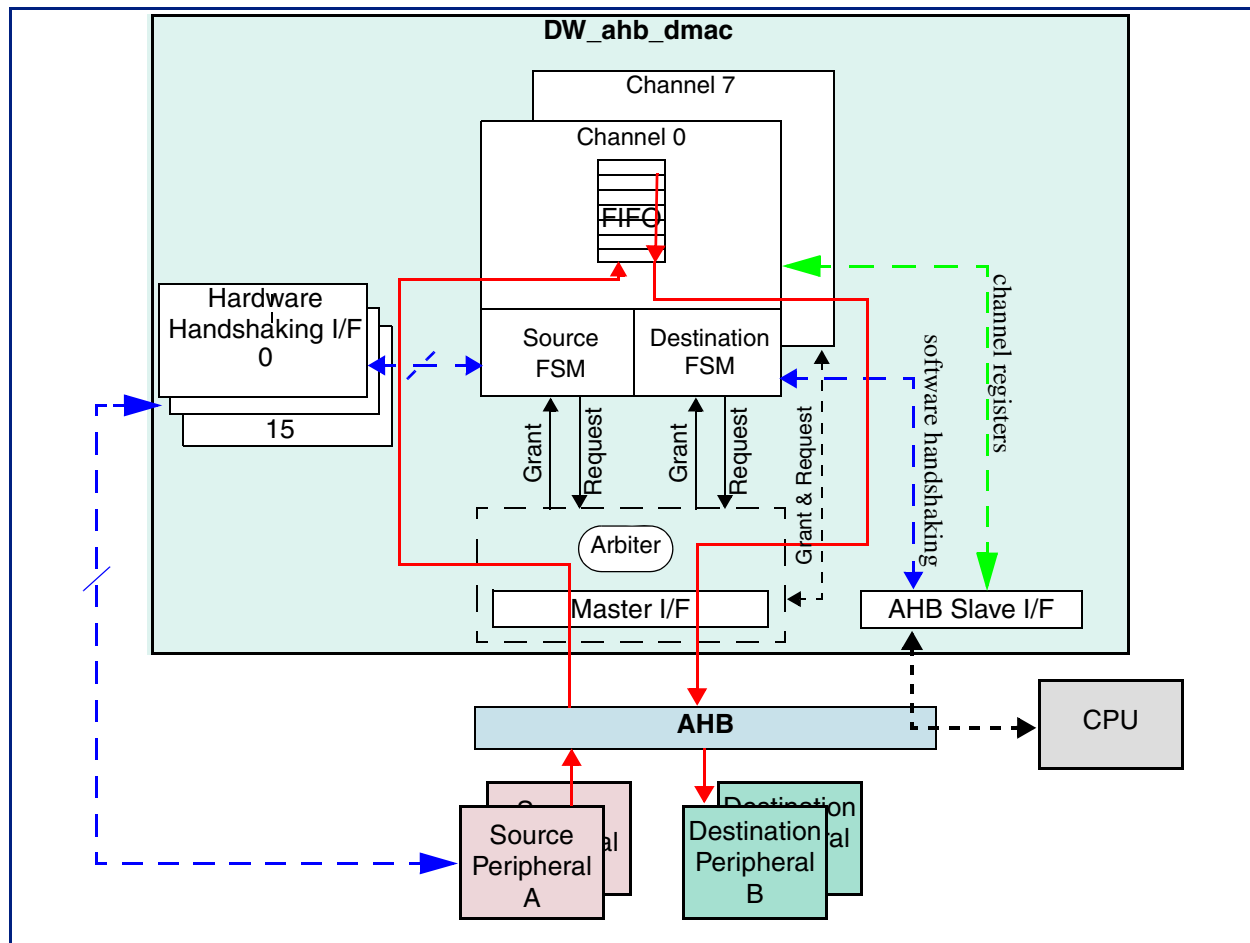


One channel of the DW_ahb_dmac is required for each source/destination pair. In the most basic configurations, as illustrated in [Figure 1-3](#) on page 18, the DW_ahb_dmac has one master interface and one channel. The master interface reads the data from a source peripheral (A) and writes it to a destination peripheral (B). Two AHB transfers are required for each DMA data transfer; this is also known as a dual-access transfer.

[Figure 1-3](#) illustrates a peripheral-to-peripheral DMA transfer, where peripheral A (source) uses a hardware handshaking interface, and peripheral B (destination) uses a software handshaking interface. For example, the request to send data to peripheral B is originated by the CPU, while writing to peripheral B is handled by the DW_ahb_dmac. The channel source and destination arbitrate independently for the AHB master

interface, along with other channels. For more information about the arbitration scheme, refer to [“Arbitration for AHB Master Interface”](#) on page 105.

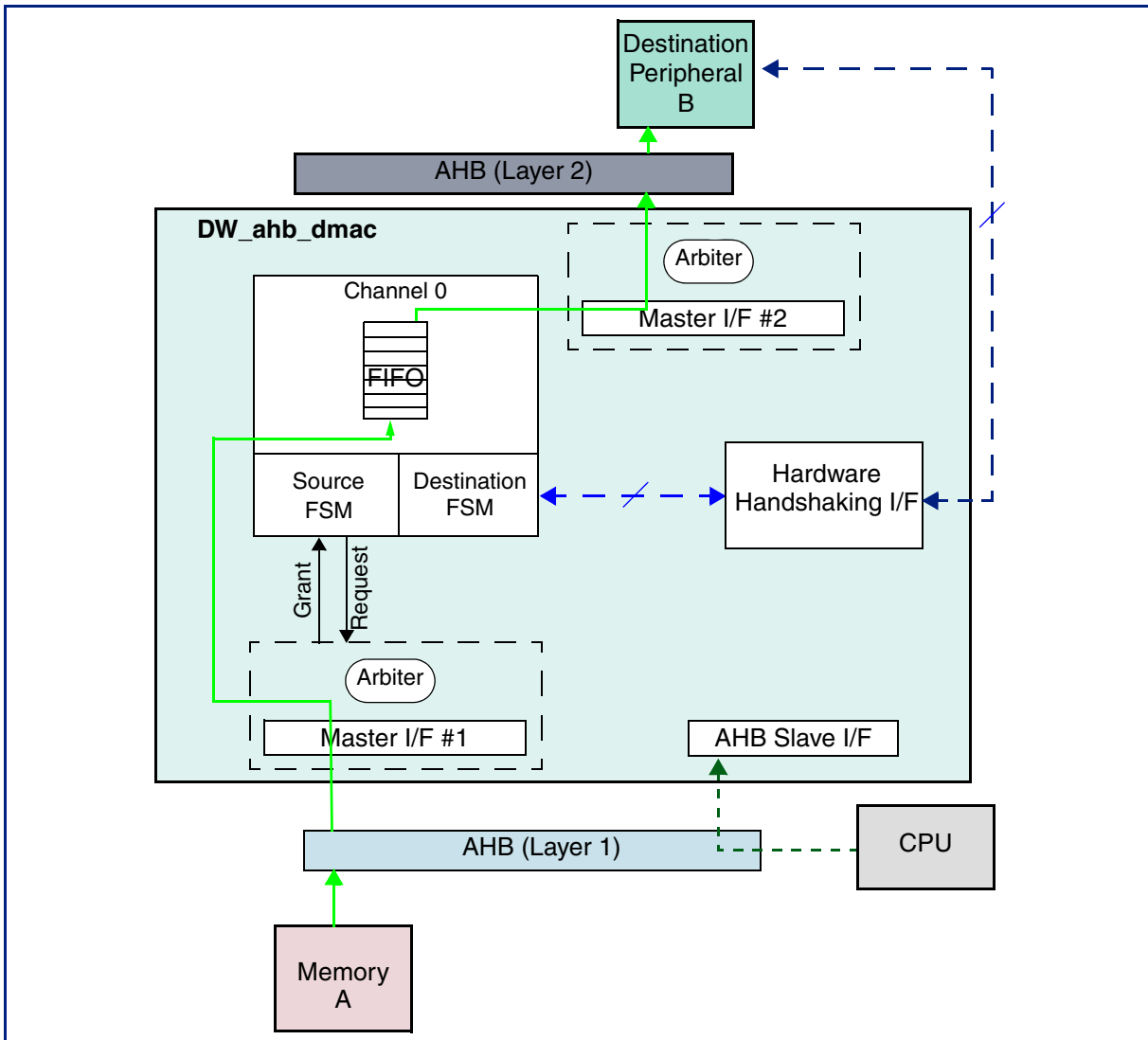
Figure 1-3 Peripheral-to-Peripheral DMA Transfer on Same AHB Layer



The DW_ahb_dmac also supports multi-layer DMA transfers when the source and destination peripherals are on different AHB layers. In this case, you must configure the DW_ahb_dmac to have more than one master interface—one per layer. [Figure 1-4](#) on page 19 illustrates a DW_ahb_dmac with two master interfaces and a DMA transfer between a source and destination on different AHB layers. Peripheral B uses

a hardware handshaking interface. The memory does not use any handshaking interface to the DW_ahb_dmac in order to initiate DMA transfers.

Figure 1-4 Peripheral-to-Memory DMA Transfer on Separate AHB Layers



1.3 Basic Definitions

The following terms are concise definitions of the DMA concepts used throughout this databook:

- **Source peripheral** – Device on a AHB layer from which the DW_ahb_dmac reads data; the DW_ahb_dmac then stores the data in the channel FIFO. The source peripheral teams up with a destination peripheral to form a channel. The source peripheral is either an AHB or APB slave. If the source is an APB slave, it is accessed through the AHB-APB bridge.
- **Destination peripheral** – Device to which the DW_ahb_dmac writes the stored data from the FIFO (previously read from the source peripheral). The destination peripheral is either an AHB or APB slave. If the destination is an APB slave, it is accessed through the AHB-APB bridge.

- **Memory** – Source or destination that is always “ready” for a DMA transfer and does not require a handshaking interface to interact with the DW_ahb_dmac. A peripheral should be assigned as memory only if it does not insert more than 16 wait states. If more than 16 wait states are required, then the peripheral should use a handshaking interface – the default if the peripheral is not programmed to be memory – in order to signal when the peripheral is ready to accept or supply data. A memory peripheral can also generate SPLIT/RETRY responses.
- **Channel** – Read/write data path between a source peripheral on one configured AHB layer and a destination peripheral on the same or different AHB layer that occurs through the channel FIFO. If the source peripheral is not memory, then a source handshaking interface is assigned to the channel. If the destination peripheral is not memory, then a destination handshaking interface is assigned to the channel. Source and destination handshaking interfaces can be assigned dynamically by programming the channel registers.
- **Master interface** – DW_ahb_dmac is a master on the AHB bus, reading data from the source and writing it to the destination over the AHB bus. It is possible to have up to four master interfaces, which means that up to four independent source and destination channels can operate simultaneously. Each channel has to arbitrate for the master interface. You need to have more than one master interface if the source and destination peripherals reside on different AHB layers.
- **Slave interface** – The AHB interface over which the DW_ahb_dmac is programmed. The slave interface in practice can be on the same layer as any of the master interfaces, or it can be on a separate layer.
- **Handshaking interface** – A set of signals or software registers that conform to a protocol and handshake between the DW_ahb_dmac and source or destination peripheral in order to control transferring a single or burst transaction between them. This interface is used to request, acknowledge, and control a DW_ahb_dmac transaction. A channel can receive a request through one of three types of handshaking interface: hardware, software, or peripheral interrupt.
 - **Hardware handshaking interface** – Uses hardware signals to control transferring a single or burst transaction between the DW_ahb_dmac and the source or destination peripheral. For more information about this interface, refer to [“Hardware Handshaking – Peripheral Is Not Flow Controller”](#) on page 48 and [“Hardware Handshaking – Peripheral Is Flow Controller”](#) on page 57.
 - **Software handshaking interface** – Uses software registers to control transferring a single or burst transaction between the DW_ahb_dmac and the source or destination peripheral. No special DW_ahb_dmac handshaking signals are needed on the I/O of the peripheral. This mode is useful for interfacing an existing peripheral to the DW_ahb_dmac without modifying it. For more information about this interface, refer to [“Flow Control Configurations”](#) on page 95.
 - **Peripheral interrupt handshaking interface** – Simple use of the hardware handshaking interface. In this mode, the interrupt line from the peripheral is tied to the dma_req input of the hardware handshaking interface; other interface signals are ignored. For more information about this interface, refer to [“Peripheral Interrupt Request Interface”](#) on page 94.
- **Flow controller** – Device (either the DW_ahb_dmac, or source/destination peripheral) that determines the length of a DMA block transfer and terminates it.
 - If you know the length of a block before enabling the channel, then you should program the DW_ahb_dmac as the flow controller.

- If the length of a block is not known prior to enabling the channel, the source or destination peripheral needs to terminate a block transfer. In this mode, the peripheral is the flow controller.

For more information, refer to “[Setup/Operation of DW_ahb_dmac Transfers](#)” on page 41.

- **Flow control mode** (CFGx.FCMODE) – Special mode that only applies when the destination peripheral is the flow controller. It controls the data pre-fetching from the source peripheral.
- **Transfer hierarchy** – [Figure 1-5](#) illustrates the hierarchy between DW_ahb_dmac transfers, block transfers, transactions (single or burst), and AHB transfers (single or burst) for non-memory peripherals.



Note

Note that for memory peripherals, there is no DMA Transaction Level.

Figure 1-5 DW_ahb_dmac Transfer Hierarchy for Non-Memory Peripherals

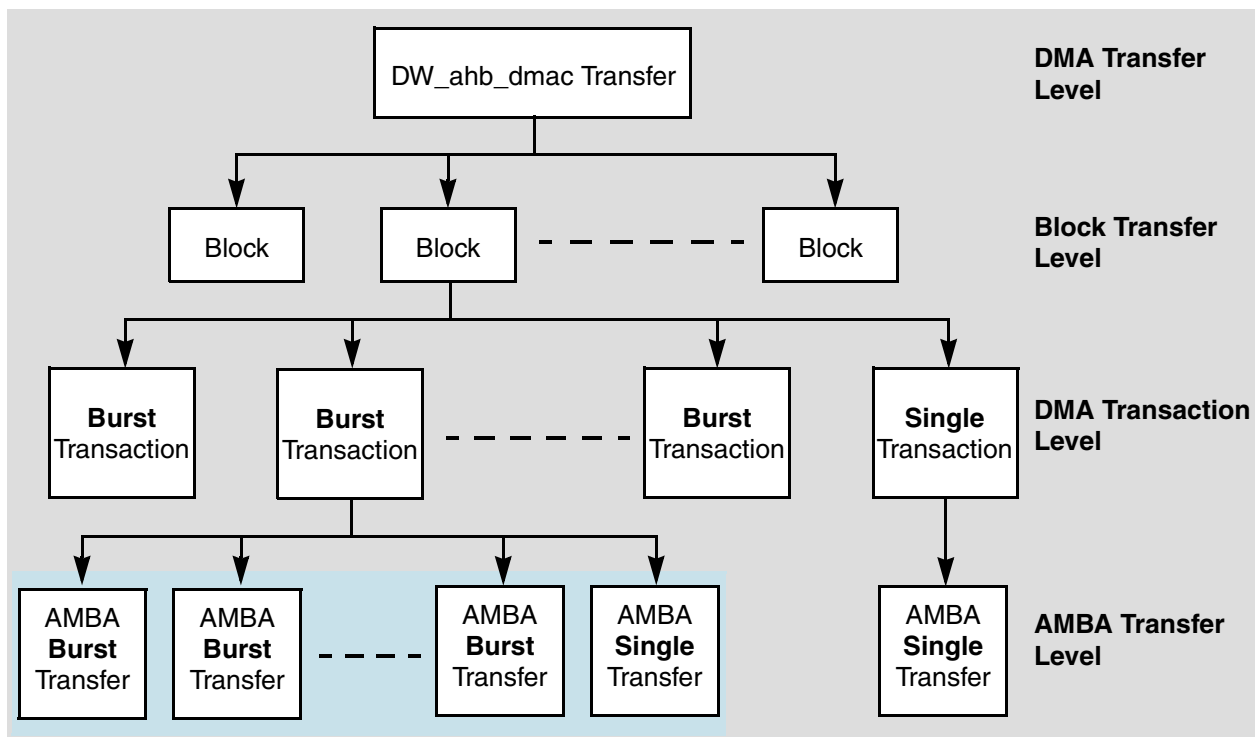
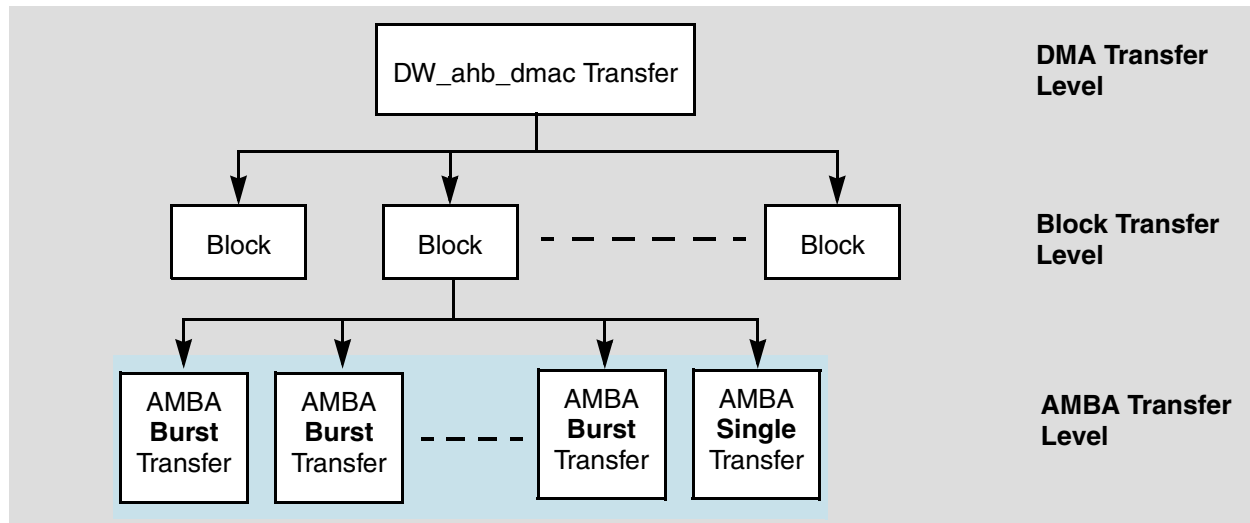


Figure 1-6 shows the transfer hierarchy for memory.

Figure 1-6 DW_ahb_dmac Transfer Hierarchy for Memory



- **Block** – Block of DW_ahb_dmac data, the amount of which is the block length and is determined by the flow controller. For transfers between the DW_ahb_dmac and memory, a block is broken directly into a sequence of bursts and single transfers. For transfers between the DW_ahb_dmac and a non-memory peripheral, a block is broken into a sequence of DW_ahb_dmac transactions (single and bursts). These are in turn broken into a sequence of AHB transfers.
- **Transaction** – Basic unit of a DW_ahb_dmac transfer, as determined by either the hardware or software handshaking interface. A transaction is relevant only for transfers between the DW_ahb_dmac and a source or destination peripheral if the peripheral is a non-memory device. There are two types of transactions:
 - **Single transaction** – Length of a single transaction is always 1 and is converted to a single AHB transfer.
 - **Burst transaction** – Length of a burst transaction is programmed into the DW_ahb_dmac. The burst transaction is converted into a sequence of bursts and AHB single transfers. The burst transaction length is under program control and normally bears some relationship to the FIFO sizes in the DW_ahb_dmac and in the source and destination peripherals.
- **DMA transfer** – Software controls the number of blocks in a DW_ahb_dmac transfer. Once the DMA transfer has completed, the hardware within the DW_ahb_dmac disables the channel and can generate an interrupt to signal the DMA transfer completion. You can then reprogram the channel for a new DMA transfer.
 - **Single-block DMA transfer** – Consists of a single block.
 - **Multi-block DMA transfer** – DMA transfer may consist of multiple DW_ahb_dmac blocks. Multi-block DMA transfers are supported through block chaining (linked list pointers), auto-reloading channel registers, and contiguous blocks. The source and destination can independently select which method to use.
 - **Linked lists (block chaining)** – Linked list pointer (LLP) points to the location in system memory where the next linked list item (LLI) exists. The LLI is a set of registers that describes the next block (block descriptor) and an LLP register. The DW_ahb_dmac fetches the LLI at

the beginning of every block when block chaining is enabled. For more information about linked lists and block chaining, refer to [“Block Chaining Using Linked Lists”](#) on page 247.

LLI accesses are always 32-bit accesses (Hsize = 2) aligned to 32-bit boundaries and cannot be changed or programmed to anything other than 32-bit, even if the AHB master interface of the LLI supports more than a 32-bit data width.

- **Auto-reloading** – DW_ahb_dmac automatically reloads the channel registers at the end of each block to the value when the channel was first enabled. For more information about this function, refer to [“Auto-Reloading of Channel Registers”](#) on page 251.
- **Contiguous blocks** – Address between successive blocks is selected to be a continuation from the end of the previous block. For more information, refer to [“Contiguous Address Between Blocks”](#) on page 251.
- **Scatter** – Relevant to destination transfers within a block. The destination address is incremented or decremented by a programmed amount when a scatter boundary is reached. The number of AHB transfers between successive scatter boundaries is under software control.
- **Gather** – Relevant to source transfers within a block. The source address is incremented or decremented by a programmed amount when a gather boundary is reached. The number of AHB transfers between successive gather boundaries is under software control.

For more information about scatter and gather, refer to [“Scatter/Gather”](#) on page 108.

- **Channel locking** – Software can program a channel to keep the AHB master interface by locking arbitration of the master bus interface for the duration of a DMA transfer, block, or transaction (single or burst). For more information on channel locking, refer to [“Channel Locking”](#) on page 103.
- **Bus locking** – Software can program a channel to maintain control of the AHB bus by asserting hlock for the duration of a DMA transfer, block, or transaction (single or burst). At minimum, channel locking is asserted during bus locking. For more information, refer to [“Bus Locking”](#) on page 103.
- **FIFO mode** – Special mode to improve bandwidth. When enabled, the channel waits until the FIFO is less than half full to fetch the data from the source peripheral, and waits until the FIFO is greater than or equal to half full in order to send data to the destination peripheral. Because of this, the channel can transfer the data using bursts, which eliminates the need to arbitrate for the AHB master interface in each single AHB transfer. When this mode is not enabled, the channel waits only until the FIFO can transmit or accept a single AHB transfer before it requests the master bus interface.
- **Pseudo fly-by operation** – Typically, it takes two AHB bus cycles to complete a transfer – one for reading the source and one for writing to the destination. However, when the source and destination peripherals of a DMA transfer are on different AHB layers, it is possible for the DW_ahb_dmac to fetch data from the source and store it in the channel FIFO at the same time that the DW_ahb_dmac extracts data from the channel FIFO and writes it to the destination peripheral. This activity is known as *pseudo fly-by operation*. In order for this to occur in appropriate sequential order, the source and destination logic in the DW_ahb_dmac should first win their respective master interfaces, and then the master interface for both source and destination layers must win arbitration of their AHB layer.

1.4 Features

The DW_ahb_dmac component includes the following features.

1.4.1 General

- AMBA 2.0-compliant
- AHB slave interface – used to program the DW_ahb_dmac
- Channels
 - Up to eight channels, one per source and destination pair
 - Unidirectional channels – data transfers in one direction only
 - Programmable channel priority
- AHB master interface(s)
 - Up to four independent AHB master interfaces that allows:
 - Up to four simultaneous DMA transfers
 - Masters that can be on different AHB layers (multi-layer support)
 - Source and destination that can be on different AHB layers (pseudo fly-by performance)
 - Configurable data bus width (up to 256 bits) for each AHB master interface
 - Configurable endianness for master interfaces
- Transfers
 - Support for memory-to-memory, memory-to-peripheral, peripheral-to-memory, and peripheral-to-peripheral DMA transfers
 - DW_ahb_dmac to or from APB peripherals through the APB bridge
- Configurable identification register
- Component ID parameters for configurable software driver support
- Configuration of DesignWare AHB Lite system

Source code for this component is available on a per-project basis as a DesignWare Core. Please contact your local sales office for the details.

1.4.2 Address Generation

- Programmable source and destination addresses (on AHB bus)
- Address increment, decrement, or no change
- Multi-block transfers achieved through:
 - Linked Lists (block chaining)
 - Auto-reloading of channel registers
 - Contiguous address between blocks

- Independent source and destination selection of multi-block transfer type
- Scatter/Gather

1.4.3 Channel Buffering

- Single FIFO per channel for source and destination
- Configurable FIFO depth
- D flip-flop-based FIFO
- Automatic data packing or unpacking to fit FIFO width

1.4.4 Channel Control

- Programmable source and destination for each channel
- Programmable transfer type for each channel (memory-to-memory, memory-to-peripheral, peripheral-to-memory, and peripheral-to-peripheral)
- Programmable burst transaction size for each channel
- Programmable enable and disable of DMA channel
- Support for disabling channel without data loss
- Support for suspension of DMA operation
- Support for RETRY, SPLIT, and ERROR responses
- Programmable maximum burst transfer size per channel
- Configurable maximum transaction size to allow gate optimization
- Configurable maximum block size to allow gate optimization
- Bus locking – can be programmed to be over the transaction, block, or DMA transfer level
- Channel locking – can be programmed to be over the transaction, block, or DMA transfer level
- Option to hardcode type of multi-block transfer
- Option to disable the writeback of the Channel Control register at the end of every block transfer

1.4.5 Transfer Initiation

- Handshaking interfaces for source and destination peripherals (up to 16)
 - Hardware handshaking interface
 - Software handshaking interface
 - Peripheral interrupt handshaking interface
- Handshaking interface supports single or burst DMA transactions
- Polarity control for hardware handshaking interface
- Enabling and disabling of individual DMA handshaking interfaces

1.4.6 Flow Control

- Programmable flow control at block transfer level (source, destination, or DW_ahb_dmac)
- Software control of source data pre-fetch when destination is flow controller

1.4.7 Interrupts

- Combined and separate interrupt requests
- Interrupt generation on:
 - DMA transfer (multi-block) completion
 - Block transfer completion
 - Single and burst transaction completion
 - Error condition
- Support of interrupt enabling and masking

1.5 Standards Compliance

The DW_ahb_dmac component conforms to the [AMBA Specification, Revision 2.0](#) from ARM. Readers are assumed to be familiar with this specification.

1.6 Verification Environment Overview

The DW_ahb_dmac includes an extensive verification environment, which sets up and invokes your selected simulation tool to execute tests that verify the functionality of the configured component. You can then analyze the results of the simulation.

[“Verification”](#) on page 285 discusses the specific procedures for verifying the DW_ahb_dmac.

1.7 Licenses

Before you begin using the DW_ahb_dmac, you must have a valid license. For more information, refer to [“Licenses”](#) in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

1.8 Where To Go From Here

At this point, you may want to get started working with the DW_ahb_dmac component within a subsystem or by itself. Synopsys provides several tools within its coreTools suite of products for the purposes of configuration, synthesis, and verification of single or multiple synthesizable IP components—coreConsultant and coreAssembler. For information on the different coreTools, refer to [Guide to coreTools Documentation](#).

For more information about configuring, synthesizing, and verifying just your DW_ahb_dmac component, refer to [“Overview of the coreConsultant Configuration and Integration Process”](#) on page 28.

For more information about implementing your DW_ahb_dmac component within a DesignWare subsystem using coreAssembler, refer to [“Overview of the coreAssembler Configuration and Integration Process”](#) on page 34.

Building and Verifying a Component or Subsystem

DesignWare Synthesizable IP (SIP) components for AMBA 2 and AMBA 3 AXI are packaged using Synopsys coreTools, which enable the user to configure, synthesize, and run simulations on a single SIP title, or to build a configured AMBA subsystem. You do this by generating a workspace view using one of the following coreTools applications:

- coreConsultant – Used for configuration, RTL generation, synthesis, and execution of packaged verification for a single SIP title. The [coreConsultant User Guide](#) provides complete information on using coreConsultant.
- coreAssembler – Used for building and configuration of a subsystem that connects multiple SIP titles, RTL generation, synthesis, and creation of a template subsystem testbench. The [coreAssembler User Guide](#) provides complete information on using coreAssembler.

A workspace is your working version of a DesignWare SIP component or subsystem. In fact, you can create several workspaces to experiment with different design alternatives.



Hint

If you are unfamiliar with coreTools—which is comprised of the coreAssembler, coreConsultant, and coreBuilder tools—you can go to [Using DesignWare Library IP in coreAssembler](#) to “get started” learning how to work with DesignWare SIP components.

2.1 Setting up Your Environment

The DW_ahb_dmac is included in a release of DesignWare SIP components. It is assumed that you have already downloaded and installed the release. If you have not, you can download and install the latest versions of required tools using the [DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide](#).

You also need to set up your environment correctly using specific environment variables, such as DESIGNWARE_HOME, VERA_HOME, PATH, and SYNOPSIS. If you are not familiar with these requirements and the necessary licenses, refer to “[Setting up Your Environment](#)” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

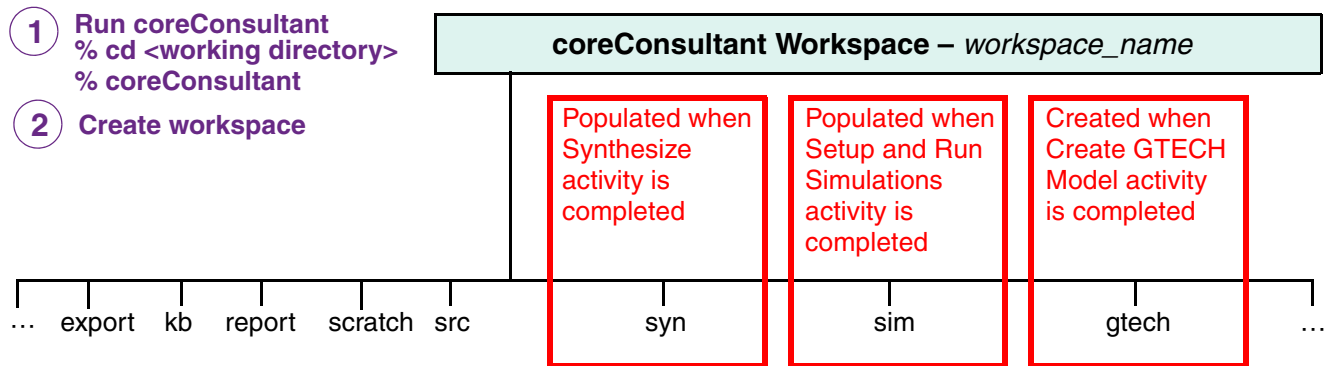
2.2 Overview of the coreConsultant Configuration and Integration Process

Once you have correctly downloaded and installed a release of DesignWare SIP components and then set up your environment, you can begin work on the DW_ahb_dmac using coreConsultant.

2.2.1 coreConsultant Usage

Figure 2-1 illustrates some general directories and files in a coreConsultant workspace.

Figure 2-1 coreConsultant Usage Flow



3 Use coreConsultant to create, synthesize, and verify your component

Table 2-1 provides a description of the implementation workspace directory and subdirectories.

Table 2-1 coreConsultant Implementation Workspace Directory Contents

Directory/Subdirectory	Description
auxiliary	Scripts and text files used by coreConsultant. Generated upon first creating workspace.
custom	Contains RTL preprocessor scripts. Generated during Specify Configuration activity.
doc	Contains local copies of component-specific databooks. Generated upon first creating workspace.
export	Contains files used to integrate results from the completed source configuration and synthesis activities into your design (outside coreConsultant). Generated upon first creating workspace; populated during Specify Configuration activity.
gtech	Contains synthesis scripts and output netlists from gtech generation; also used for RTL simulation of encrypted source code. Generated during Generate GTECH Model activity.

Table 2-1 coreConsultant Implementation Workspace Directory Contents (Continued)

Directory/Subdirectory	Description
kb	Contains knowledge base information used by coreConsultant. These are binary files containing the state of the design. Generated upon first creating workspace; populated and updated throughout activities.
leda	Contains Leda configuration files for the component. Generated upon first creating workspace; updated during Run Leda Coding Checker activity.
report	Contains all of the reports created by coreConsultant during build, configuration, test and synthesis phases. An index.html file in this directory links to many of these generated reports. Generated upon first creating workspace; populated and updated throughout activities.
scratch	Contains temp files used during the coreConsultant processes. Generated upon first creating workspace; populated and updated throughout activities.
sim	Contains test stimulus and output files. Generated upon first creating workspace; updated during Setup and Run Simulations activity.
spyglass	Contains SpyGlass Lint and CDC configuration files for the component. Generated upon first SpyGlass run; updated during Run Spyglass RTL Checker activity.
src	Includes the top-level RTL file, <i>design_name.v</i> . If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated upon first creating workspace; populated during Specify Configuration activity.
syn	Contains synthesis files for the component. Generated upon first creating workspace; updated during Synthesis activity and Formal Verification activity.
tcl	Contains synthesis intent scripts. Generated upon first creating workspace.

For details on some key files created during coreConsultant activities, refer to [“Database Files”](#) on page 38.

For information on using coreConsultant, refer to the [coreConsultant User Guide](#).

2.2.2 Configuring the DW_ahb_dmac within coreConsultant

The “[Parameters](#)” chapter on [page 119](#) describes the DW_ahb_dmac hardware configuration parameters that you configure using the coreConsultant GUI.

The “Creating the RTL View of a Core” chapter in the [coreConsultant User Guide](#) discusses how to specify a configuration for an individual component like the DW_ahb_dmac.

2.2.3 Creating Gate-Level Netlists within coreConsultant

The “Creating the Gate-Level Netlist for a Core” chapter in the [coreConsultant User Guide](#) discusses how to create a translation of the RTL view into a technology-specific netlist for an individual component like the DW_ahb_dmac.

2.2.4 Verifying the DW_ahb_dmac within coreConsultant

The “[Verification](#)” chapter on [page 285](#) provides an overview of the testbench available for DW_ahb_dmac verification using the coreConsultant GUI.

The “Verifying Your Implementation” chapter in the [coreConsultant User Guide](#) discusses how to simulate an individual component like the DW_ahb_dmac.

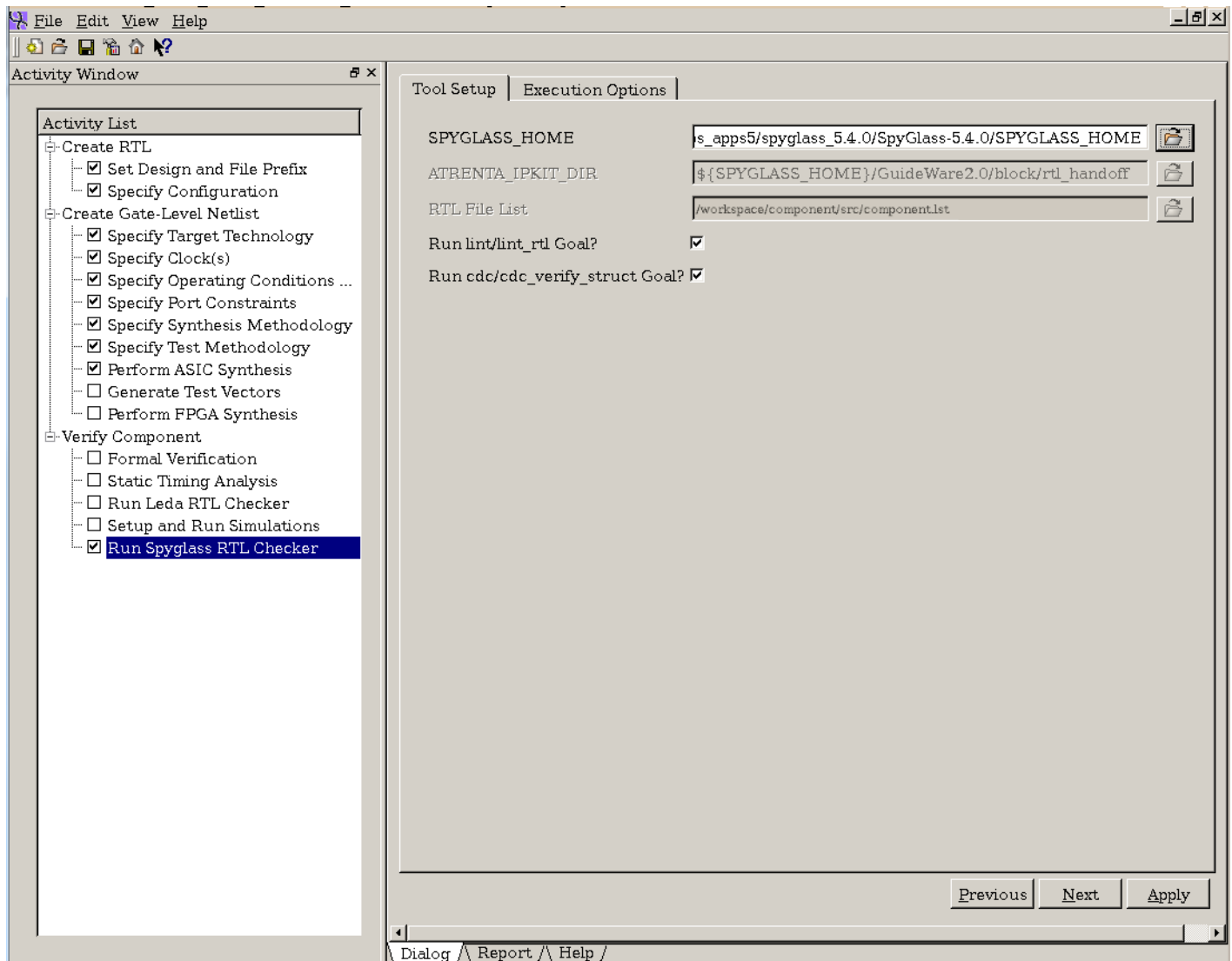
2.2.5 Running Leda on Generated Code with coreConsultant

When you select **Verify Component > Run Leda Coding Checker** from the Activity List, the corresponding Activity View appears. In this Activity View you select rules configuration file and define Leda command line switches.

2.2.6 Running SpyGlass® Lint and SpyGlass® CDC

This section discusses the procedure to run SpyGlass Lint and SpyGlass CDC.

[Figure 2-2](#) shows the coreConsultant GUI in which you run Lint and CDC goals.

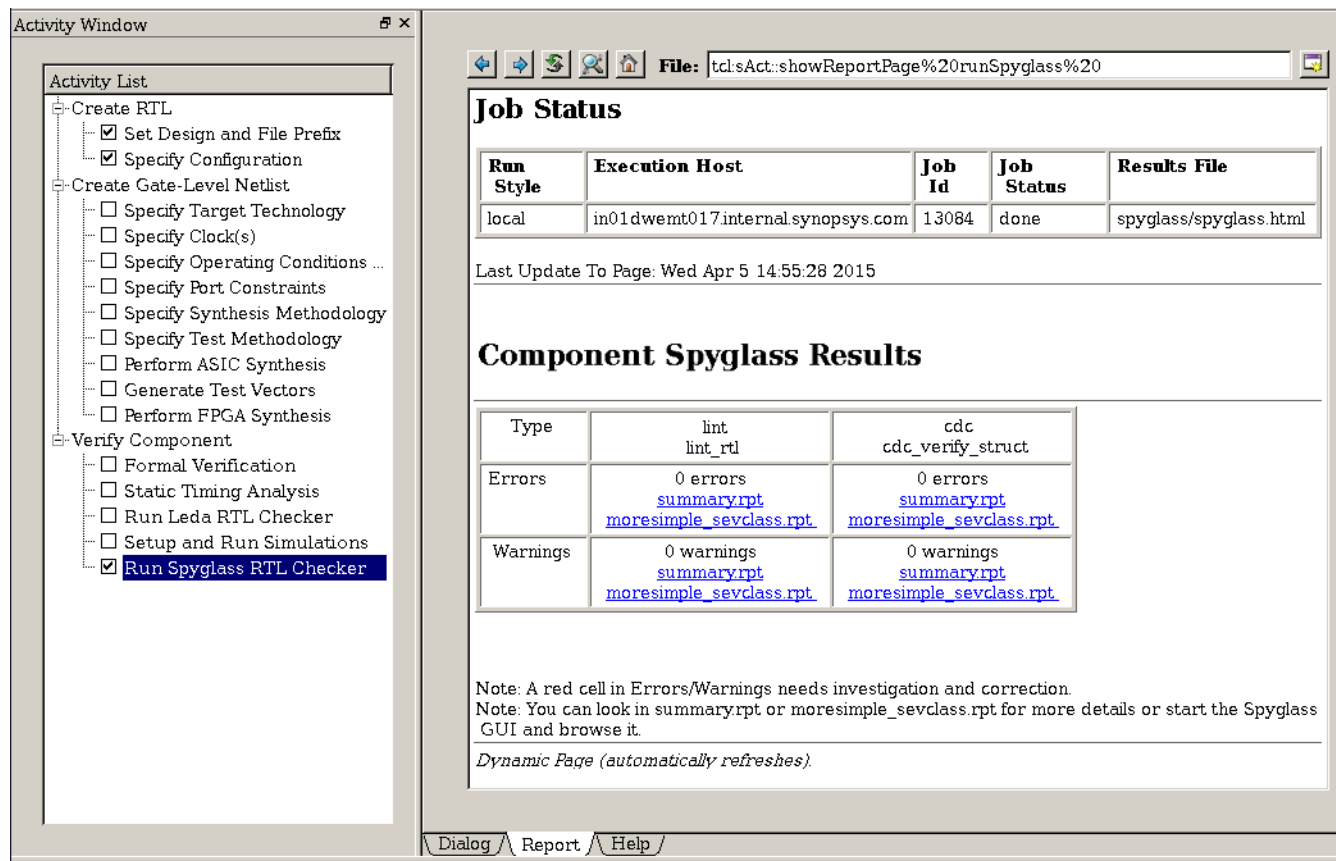
Figure 2-2 SpyGlass Options in coreConsultant

The SpyGlass flow in coreConsultant runs Guidewave 2.0 rules for block/rtl_handoff. Within the block/rtl_handoff, only lint/lint_rtl and cdc/cdc_verify_struct goals are run.

In [Figure 2-2](#), select the type of run goals. You can select either Lint run goal or CDC run goal, or both Lint and CDC run goals. By default, both Lint and CDC are selected.

When the Lint and/or CDC is run, the results are available in the Report tab. Errors (if any) are displayed with a red colored cell and warnings (if any) are displayed in yellow colored cell, as shown in [Figure 2-3](#).

Figure 2-3 coreConsultant SpyGlass Report Summary



- 2.2.6.1 Fixed Settings
- The settings are fixed (hardcoded) when you run SpyGlass in coreConsultant.
- 2.2.6.2 SpyGlass Lint

Table 2-2 lists the SpyGlass Link waiver files that are used by the coreConsultant tool.

Table 2-2 Waiver Files for Sypglass Lint

File Name	Description
<configured_workspace>/spyglass/spyglass_design_specific_waivers.swl	These are DW_ahb_dmac design-specific rule waivers. This file contains Lint waivers for DW_ahb_dmac (if applicable). The reason for each of the waivers are included as comments in the file.
<configured_workspace>/spyglass/spyglass_engineering_council_waivers.swl	This file contains rules that Synopsys waives for its IPs.

2.2.6.3 SpyGlass CDC

To define the SpyGlass CDC constraints, it is important to understand the reset and clock logic used in DW_ahb_dmac. For information on reset and clock logic, refer “[Functional Description](#)” on page 41 and “[Signal Descriptions](#)” on page 135.

2.2.6.3.1 CDC Files

[Table 2-3](#) summarizes files for SpyGlass CDC used by coreConsultant.

Table 2-3 Waiver Files for Sypglass CDC

File Name	Description
<code><configured_workspace>/spyglass/manual.sgdc</code>	These are the constraints pertaining to a given mode.
<code><configured_workspace>/spyglass/ports.sgdc</code>	These are the list of I/O signals and their respective clocks.
<code><configured_workspace>/spyglass/spyglass_design_specific_waivers.swl</code>	These are DW_ahb_dmac design-specific rule waivers. This file contains CDC waivers for DW_ahb_dmac (if applicable). The reason for each of the waivers are included as comments in the file.
<code><configured_workspace>/spyglass/spyglass_engineering_council_waivers.swl</code>	These are rules that Synopsys waives for its IPs.

2.2.6.3.2 CDC Path Debug Using the SpyGlass GUI

For debugging the CDC path, it is necessary to run SpyGlass in interactive mode in the configured workspace. To invoke the SpyGlass GUI and to run CDC, complete the following steps:

1. Go to the `<configured_workspace>/spyglass` directory.
2. Issue `./sh.spyglass` to start the spyGlass GUI or issue `./sh.spyglass -batch` to start the SpyGlass in batch mode.
3. In the SpyGlass GUI, the Goal Setup window opens by default.
4. Uncheck the `lint_rtl` option and click the **Selected Goal (s)** button.
5. After the CDC run is complete, the Analyze Results window displays the results.

Navigate to and select the relevant errors to open a schematic for analysis.

2.3 Overview of the coreAssembler Configuration and Integration Process

Once you have correctly downloaded and installed a release of DesignWare SIP components and then set up your environment, you can begin work on your DesignWare subsystem with coreAssembler.

2.3.1 coreAssembler Usage

Figure 2-4 illustrates some general directories and files in a coreAssembler workspace.

Figure 2-4 coreAssembler Usage Flow

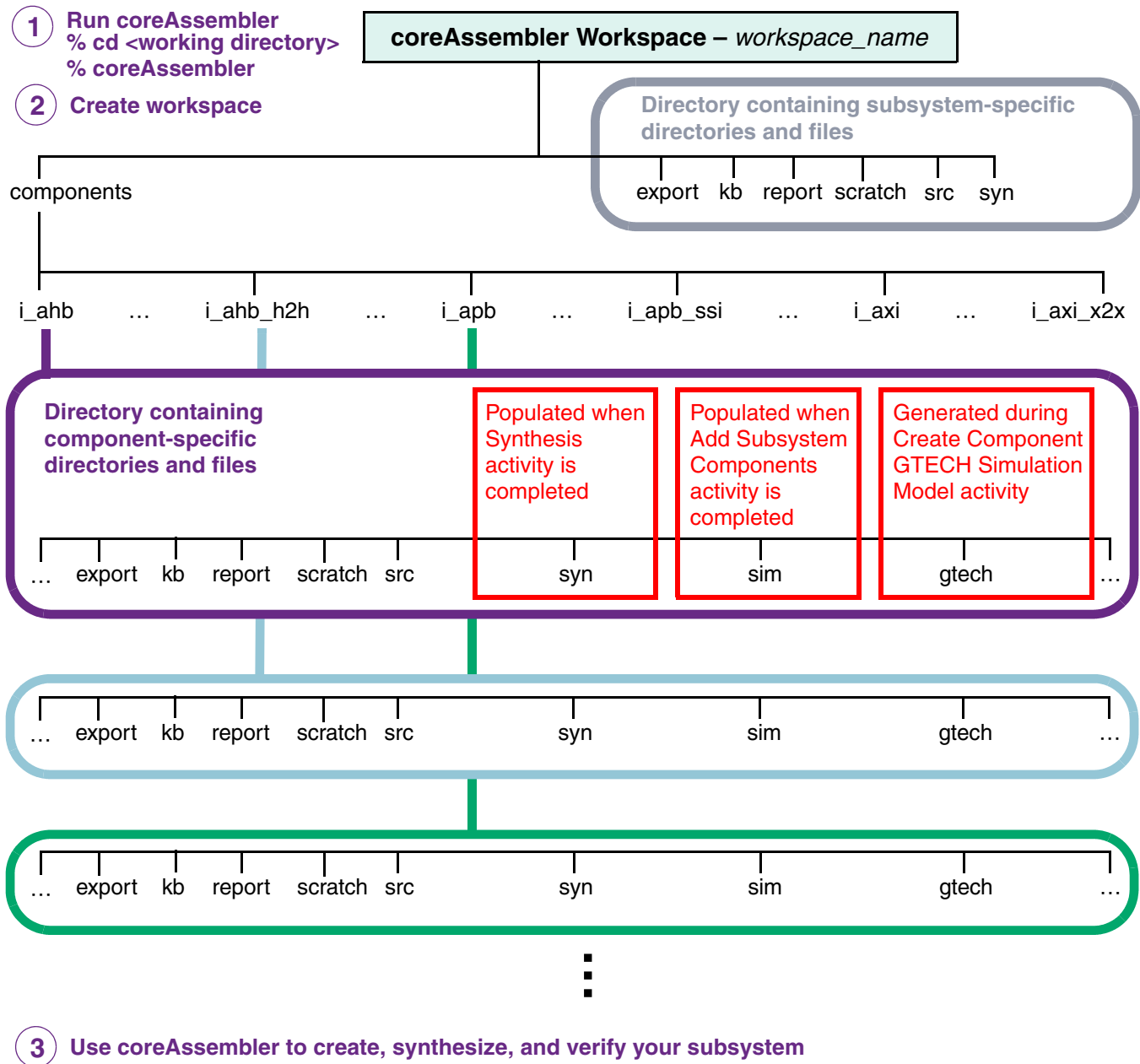


Table 2-4 provides a description of the implementation workspace directory and subdirectories.

Table 2-4 coreAssembler Implementation Workspace Directory Contents

Directory/Subdirectory	Description
components	Contains a directory for each IP component instance connected in the subsystem. Generated and populated with separate component directories upon first adding components; populated and updated throughout activities.
<i>i_component/auxiliary</i>	Scripts and text files used by coreAssembler. Generated during Add Subsystem Components activity.
<i>i_component/custom</i>	Contains RTL preprocessor scripts. Generated during Configure Components activity.
<i>i_component/doc</i>	Contains local copies of component-specific databooks. Generated during Add Subsystem Components activity.
<i>i_component/export</i>	Contains files used to integrate results from the completed source configuration and synthesis activities into your design (outside coreAssembler). Generated during Add Subsystem Components activity; populated during Configure Components activity.
<i>i_component/gtech</i>	Contains synthesis scripts and output netlists from gtech generation; also used for RTL simulation of encrypted source code. Generated during Create Component GTECH Simulation Model activity.
<i>i_component/kb</i>	Contains knowledge base information used by coreAssembler. These are binary files containing the state of the design. Generated during Add Subsystem Components activity; populated and updated throughout activities.
<i>i_component/leda</i>	Contains Leda configuration files for the component. Generated during Add Subsystem Components activity; populated during Run Leda Coding Checker (for <i>/i_component</i>) activity.
<i>i_component/report</i>	Contains all of the reports created by coreAssembler during build, configuration, test and synthesis phases. An index.html file in this directory links to many of these generated reports. Generated during Add Subsystem Components activity; populated and updated throughout activities.
<i>i_component/scratch</i>	Contains temp files used during the coreAssembler processes. Generated during Add Subsystem Components activity; populated and updated throughout activities.
<i>i_component/sim</i>	Contains test stimulus and output files. Generated during Add Subsystem Components activity; updated during Setup and Run Simulations (for <i>/i_component</i>) activity.

Table 2-4 coreAssembler Implementation Workspace Directory Contents (Continued)

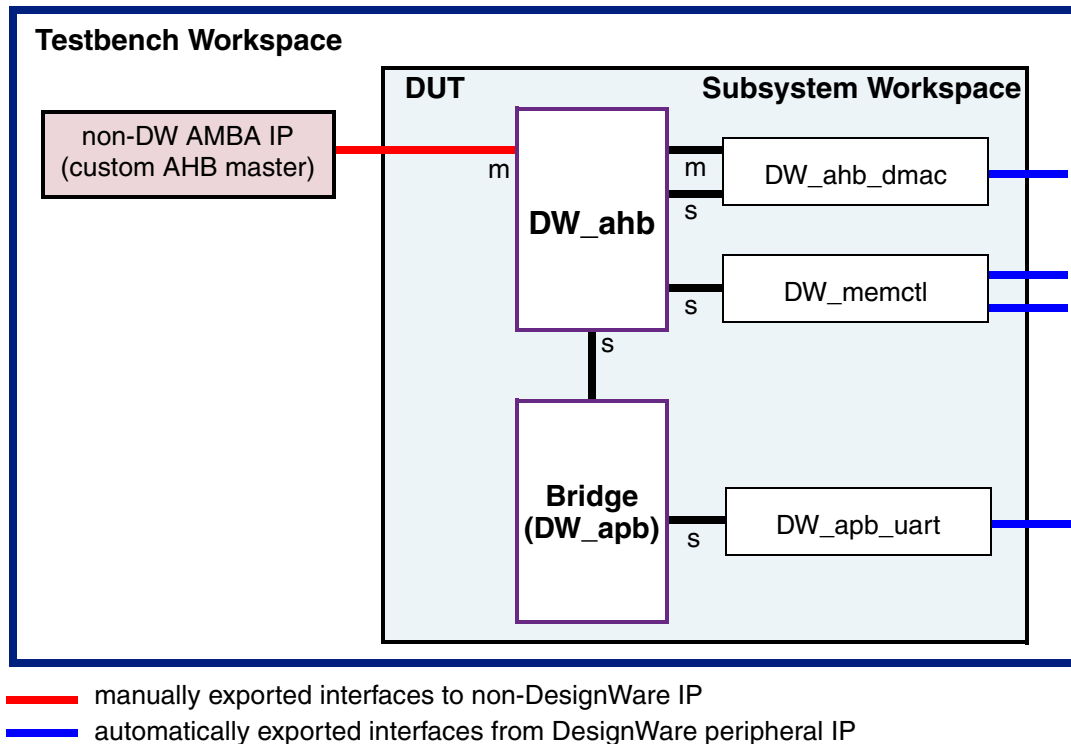
Directory/Subdirectory	Description
<code>i_component/spyglass</code>	Contains SpyGlass Lint and CDC configuration files for the component. Generated upon first SpyGlass run; updated during Run Spyglass RTL Checker activity.
<code>i_component/src</code>	Includes the top-level RTL file, <i>design_name.v</i> . If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated during Add Subsystem Components activity; populated during Specify Configuration activity.
<code>i_component/syn</code>	Contains synthesis files for the component. Generated during Add Subsystem Components activity; updated during Synthesis activity.
<code>i_component/tcl</code>	Contains synthesis intent scripts. Generated during Add Subsystem Components activity.
<code>export</code>	Contains subsystem files used to integrate the results from the completed source configuration and synthesis activities into your design (outside coreAssembler). Generated upon first creating workspace; populated starting with Memory Map Specification activity.
<code>kb</code>	Contains subsystem knowledge base information used by coreAssembler. These are binary files containing the state of the design. Generated upon first creating workspace; populated and updated throughout activities.
<code>report</code>	Contains subsystem reports created by coreAssembler during build, configuration, test and synthesis phases. An index.html file in this directory links to many of these generated reports. Generated upon first creating workspace; populated and updated throughout activities.
<code>scratch</code>	Contains subsystem temp files used during the coreAssembler processes. Generated upon first creating workspace; populated and updated throughout activities.
<code>src</code>	Includes the RTL related to the subsystem. If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated upon first creating workspace; populated starting with Generate Subsystem RTL activity.
<code>syn</code>	Contains synthesis files for the subsystem. Generated upon first creating workspace; updated during Synthesize activity and Formal Verification activity.

For details on some key files created during coreAssembler activities, refer to [“Database Files”](#) on page 38.

For information on using coreAssembler, refer to the [coreAssembler User Guide](#). For information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools, refer to [Using DesignWare Library IP in coreAssembler](#).

Figure 2-5 illustrates the DW_ahb_dmac in a simple subsystem.

Figure 2-5 DW_ahb_dmac in Simple Subsystem



The subsystem in Figure 2-5 contains the following components that you may want to use as you learn to use coreAssembler:

- DW_ahb_dmac
- DW_ahb
- DW_memctl (source)
- DW_apb
- DW_apb_uart
- AHB Master

The AHB Master is meant to be exported out of the design and then replaced by a real AHB Master – such as a CPU – later in the design process; at least one exported AHB master is required in a subsystem if you intend to do a basic simulation that tests connections.

2.3.2 Configuring the DW_ahb_dmac within a Subsystem

The “Parameters” chapter on [page 119](#) describes the DW_ahb_dmac hardware configuration parameters that you configure using the coreAssembler GUI. Corresponding databooks for the other components in a subsystem contain “Parameters” chapters that describe their respective configuration parameters.

The “Creating the RTL View of a Subsystem” chapter in the [coreAssembler User Guide](#) discusses how to configure subsystem components and automatically connect them using the coreAssembler GUI.

2.3.3 Creating Gate-Level Netlists within coreAssembler

The “Creating the Gate-Level Netlist for a Subsystem” chapter in the [coreAssembler User Guide](#) discusses how to create a translation of the RTL view into a technology-specific netlist for a subsystem.

2.3.4 Verifying the DW_ahb_dmac within coreAssembler

The “Verification” chapter on [page 285](#) provides an overview of the testbench available for DW_ahb_dmac verification using the coreAssembler GUI.

The “Verifying Subsystems and Components” chapter in the [coreAssembler User Guide](#) discusses how to simulate a subsystem.

2.3.5 Running Leda on Generated Code with coreAssembler

When you select **Verify Component > Run Leda Coding Checker for /i_component)** from the Activity List, the corresponding Activity View appears. In this Activity View you select rules configuration file and define Leda command line switches.

2.3.6 Running Spyglass on Generated Code with coreAssembler

When you select **Verify Component > Run Spyglass RTL Checker for /i_component** from the Activity List, the corresponding Activity View appears. In this Activity View, you can select to run Spyglass Lint and Spyglass CDC.

2.4 Database Files

The following subsections describe some key files created in coreConsultant and coreAssembler activities.

2.4.1 Design/HDL Files

The following sections describe the design and HDL files that are produced by coreConsultant and coreAssembler when configuring and verifying a DesignWare Synthesizable Component. The following files are created in different directories by coreConsultant and coreAssembler:

- coreConsultant – *workspace/* directory
- coreAssembler – *workspace/components/i_component/* directory

2.4.1.1 RTL-Level Files

The following table describes the RTL files that are generated by the Create RTL activity. They are encrypted except where otherwise noted. Any Synopsys synthesis tool or simulator can read encrypted RTL files.

Table 2-5 RTL-Level Files

Files	Encrypted?	Purpose
<i>./src/component_cc_constants.v</i>	No	Includes definitions and values of all configuration parameters that you have specified for the component.
<i>./src/component.v</i>	No	Top-level HDL file. Include the DesignWare libraries by using the following options in your simulator invocation: +libext+.v+.V -y \${SYNOPSYS}/packages/gtech/src_ver -y \${SYNOPSYS}/dw/sim_ver
<i>./src/component_submodule.v</i>	Yes	Sub-modules of component
<i>./src/component_constants.v</i>	No	Includes the constants used internally in the design.
<i>./src/component_undef.v</i>		Includes an undef for each of the definitions found in the <i>component_cc_constants.v</i> file; compiled in after the last file listed in <i>./src/components.lst</i> when compiling multiple instances of the same IP.
<i>./src/component.lst</i>	No	Lists the order in which the RTL files should be read into tools, such as simulators or dc_shell. For example, use the following option to read the design into VCS: vcs +v2k -f component.lst

2.4.1.2 Simulation Model Files

The following table includes files generated for the component during the Generate GTECH Simulation activity. These files are needed when you are using a non-Synopsys simulator (when you can not use the encrypted RTL).

Table 2-6 Simulation Model Files

Files	Encrypted?	Purpose
<i>./gtech/final/db/component.v</i>	No	Simulation model of the component for use with non-Synopsys simulators. A technology-independent, gate-level netlist; VHDL and Verilog versions are generated. Include the DesignWare libraries by using the following options in your simulator invocation: +libext+.v+.V -y \${SYNOPSYS}/packages/gtech/src_ver -y \${SYNOPSYS}/dw/sim_ver

2.4.2 Synthesis Files

The following table includes files generated after the Create Gate-Level Netlist activity is performed on a component.

Table 2-7 Synthesis Files

Files	Encrypted?	Purpose
./syn/auxScripts	No	Auxiliary files for synthesis.
./syn/final/db/ <i>component</i> .db	Binary format	Synopsys .db files (gate level) that can be read into dc_shell for further synthesis, if desired.
./syn/final/db/ <i>component</i> .v	No	Gate-level netlist that is mapped to technology libraries that you specify.
./syn/constrain/script/*.*	No	Constraint files for the components.
./syn/final/report/*.*	No	Synthesis result files.

2.4.3 Verification Reference Files

Files described in the following table include information pertaining to the component's operation so that you can verify installation and configuration of the component has been successful. These files are not for re-use during system-level verification.

Table 2-8 Verification Reference Files

Files	Encrypted?	Purpose
./sim/runtest	No	Perl script that runs the Setup and Run Simulations activity from the command line.
./sim/runtest.log	No	The overall result of simulation, including pass/fail results.
./sim/test_ <i>testname</i> /test.result	No	Pass/fail of individual test.
./sim/test_ <i>testname</i> /test.log	No	Log file for individual test.

3

Functional Description

This chapter describes the functional details of the DW_ahb_dmac component.

There is an option to configure AHB Lite, which is the DesignWare implementation of AMBA 2.0 AHB-Lite. The DesignWare AHB Lite configuration does not include the following:

- Requesting/granting protocols to the arbiter and split/retry responses from the slaves; all slaves are made non-split capable
- No arbiter as the signals associated with the component are not used: hbusreq and hgrant
- No write data, address, or control multiplexers
- Pause mode not enabled
- Default master number changed to 1
- Number of masters is changed to 1

You use the DMAH_M1_AHB_LITE parameter to configure the DW_ahb_dmac for an AHB Lite configuration.

For more information about AHB Lite, refer to the [DesignWare DW_ahb Databook](#).

3.1 Setup/Operation of DW_ahb_dmac Transfers

“[Programming a Channel](#)” on page 256 describes how to program the DW_ahb_dmac in order to perform DMA transfers. This section discusses how a single block transfer, made up of transactions, is actually performed. The relevant settings of the DW_ahb_dmac are also discussed here.

3.2 Block Flow Controller and Transfer Type

The device that controls the length of a block is known as the flow controller. Either the DW_ahb_dmac, the source peripheral, or the destination peripheral must be assigned as the flow controller.

- If the block size is known prior to when the channel is enabled, then the DW_ahb_dmac should be programmed as the flow controller. The block size should be programmed into the CTLx.BLOCK_TS field.
- If the block size is unknown when the DW_ahb_dmac channel is enabled, either the source or destination peripheral must be the flow controller.

The CTLx.TT_FC field indicates the transfer type and flow controller for that channel. [Table 3-1](#) lists valid transfer types and flow controller combinations.

Table 3-1 Transfer Types and Flow Control Combinations

Transfer Type	Flow Controller
Memory to Memory	DW_ahb_dmac
Memory to Peripheral	DW_ahb_dmac
Memory to Peripheral	Peripheral
Peripheral to Memory	DW_ahb_dmac
Peripheral to Memory	Peripheral
Peripheral to Peripheral	DW_ahb_dmac
Peripheral to Peripheral	Source Peripheral
Peripheral to Peripheral	Destination Peripheral

As an example, the DW_ahb_dmac can be programmed as the flow controller when a DMA block must be transferred from a receive DW_apb_ssi peripheral to memory. In a block transfer, software programs the DW_apb_ssi register – CTRLR1.NDF – with the number of source data items minus 1. Software then programs the CTLx.BLOCK_TS register with the same value and programs the DW_ahb_dmac as the flow controller.

The DW_apb_ssi has no built-in intelligence to signal block completion to the DW_ahb_dmac; this is not required in this case because software knows the block size prior to enabling the channel.

As another example, a peripheral can be a block flow controller when a DMA block must be transferred from an Ethernet controller to memory. In this case, the size of an ethernet packet may not be known prior to enabling the DW_ahb_dmac channel. Therefore, the ethernet controller needs built-in intelligence to indicate to the DW_ahb_dmac when a block transfer has completed.

3.3 Handshaking Interface

Handshaking interfaces are used at the transaction level to control the flow of single or burst transactions. The operation of the handshaking interface is different and depends on whether the peripheral or the DW_ahb_dmac is the flow controller.

The peripheral uses the handshaking interface to indicate to the DW_ahb_dmac that it is ready to transfer or accept data over the AHB bus.

A non-memory peripheral can request a DMA transfer through the DW_ahb_dmac using one of two types of handshaking interfaces:

- Hardware
- Software

Software selects between the hardware or software handshaking interface on a per-channel basis. Software handshaking is accomplished through memory-mapped registers, while hardware handshaking is accomplished using a dedicated handshaking interface.

**Note**

Throughout the remainder of this document, references to both source and destination hardware handshaking interfaces assume an active-high interface (refer to CFGx.SRC(DST)_HS_POL bits in the Channel Configuration register, “CFGx” on page 180). When active-low handshaking interfaces are used, then the active level and edge are reversed from that of an active-high interface.

The type of handshaking interface depends on whether the peripheral is a flow controller or not.

**Note**

Source and destination peripherals can independently select the handshaking interface type; that is, hardware or software handshaking. For more information, refer to the CFGx.HS_SEL_SRC and CFGx.HS_SEL_DST parameters in the CFGx register on page 180.

3.4 Basic Interface Definitions

**Note**

In this chapter and the following equations, references to CTLx.SRC_MSIZ, CTLx.DEST_MSIZ, CTLx.SRC_TR_WIDTH, and CTLx.DST_TR_WIDTH refer to the decoded values of the parameters; for example, CTLx.SRC_MSIZ = 3'b001 decodes to 4, and CTLx.SRC_TR_WIDTH = 3'b010 decodes to 32 bits.

The following definitions are used in this chapter:

- Source single transaction size in bytes

$$src_single_size_bytes = CTLx.SRC_TR_WIDTH/8 \text{ (1)}$$

- Source burst transaction size in bytes

$$src_burst_size_bytes = CTLx.SRC_MSIZ * src_single_size_bytes \text{ (2)}$$

- Destination single transaction size in bytes

$$dst_single_size_bytes = CTLx.DST_TR_WIDTH/8 \text{ (3)}$$

- Destination burst transaction size in bytes

$$dst_burst_size_bytes = CTLx.DEST_MSIZ * dst_single_size_bytes \text{ (4)}$$

- Block size in bytes:

- DW_ahb_dmac is flow controller – With the DW_ahb_dmac as the flow controller, the processor programs the DW_ahb_dmac with the number of data items (block size) of source transfer width (CTLx.SRC_TR_WIDTH) to be transferred by the DW_ahb_dmac in a block transfer; this is programmed into the CTLx.BLOCK_TS field. Therefore, the total number of bytes to be transferred in a block is:

$$blk_size_bytes_dma = CTLx.BLOCK_TS * src_single_size_bytes \text{ (5)}$$

- Source peripheral is block flow controller

$$\text{blk_size_bytes_src} = (\text{Number of source burst transactions in block} * \text{src_burst_size_bytes}) + (\text{Number of source single transactions in block} * \text{src_single_size_bytes})(6)$$

- Destination peripheral is block flow controller

$$\text{blk_size_bytes_dst} = (\text{Number of destination burst transactions in block} * \text{dst_burst_size_bytes}) + (\text{Number of destination single transactions in block} * \text{dst_single_size_bytes})(7)$$

3.5 Memory Peripherals

Figure 1-6 on page 22 shows the DMA transfer hierarchy of the DW_ahb_dmac for a memory peripheral. There is no handshaking interface with the DW_ahb_dmac, and therefore the memory peripheral can never be a flow controller. Once the channel is enabled, the transfer proceeds immediately without waiting for a transaction request.

The alternative to not having a transaction-level handshaking interface is to allow the DW_ahb_dmac to attempt AHB transfers to the peripheral once the channel is enabled. If the peripheral slave cannot accept these AHB transfers, it inserts wait states onto the bus (by de-asserting hready) until it is ready; it is not recommended that more than 16 wait states be inserted onto the bus. By using the handshaking interface, the peripheral can signal to the DW_ahb_dmac that it is ready to transmit or receive data, and then the DW_ahb_dmac can access the peripheral without the peripheral inserting wait states onto the bus.



Note

If a channel is used exclusively for memory-to-memory DMA transfers – that is, no transaction-level handshaking on the source or destination side – then set DMAH_MAX_MULT_SIZE to 4 in order to achieve logic optimization.

The CTLx.SRC_MSIZ and CTLx.DEST_MSIZ are properties valid only for peripherals with a handshaking interface; they cannot be used for defining the burst length for memory peripherals.

When the peripherals are memory, the DW_ahb_dmac is always the flow controller and uses DMA transfers to move blocks; thus the CTLx.SRC_MSIZ and CTLx.DEST_MSIZ values are not used for memory peripherals. The SRC_MSIZ/DEST_MSIZ limitations are used to accommodate devices that have limited resources, such as a FIFO. Memory does not normally have limitations similar to the FIFOs.

Therefore:

- Length of burst transfers *to* memory is always equal to the number of data items available in a channel FIFO or data items required to complete the block transfer, whichever is smaller.
- Length of burst transfers *from* memory is always equal to the space available in a channel FIFO or number of data items required to complete the block transfer, whichever is smaller.



Note

The length of burst transfers to or from memory can be limited by software by programming the CFGx.MAX_ABRST register field.

3.6 Software Handshaking

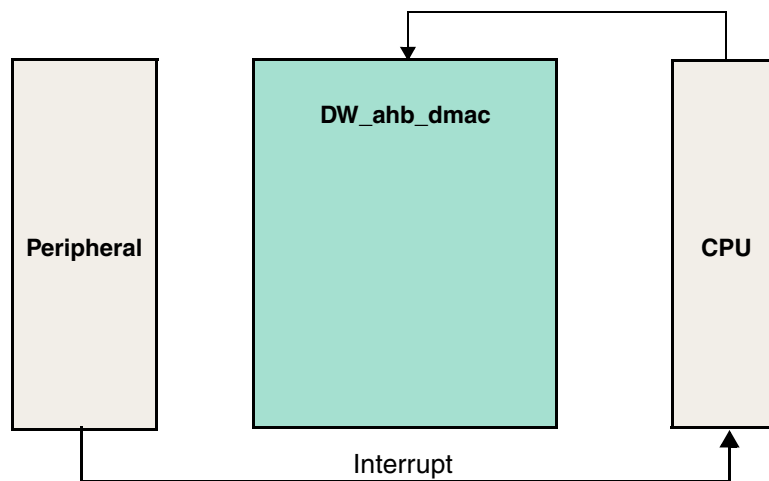
When the slave peripheral requires the DW_ahb_dmac to perform a DMA transaction, it communicates this request by sending an interrupt to the CPU or interrupt controller. The interrupt service routine then uses

the software registers detailed in “[Software Handshaking Registers](#)” on page 198 to initiate and control a DMA transaction. This group of software registers is used to implement the software handshaking interface.

The [HS_SEL_SRC/HS_SEL_DST](#) bit in the [CFGx](#) channel configuration register must be set to enable software handshaking. For examples on how to use the software handshaking interface, refer to “[Example 9](#)” on page 85, “[Example 10](#)” on page 87, and “[Example 11](#)” on page 90.

Figure 3-1 Software Controlled DMA Transfers

- Program and enable channel through “[Channel Registers](#)” on page 163
- After interrupt, initiate and control DMA transaction between peripheral(s) DW_ahb_dmac through “[Software Handshaking Registers](#)” on page 198



The software handshaking registers are:

- [ReqSrcReg](#) – source software transaction request
- [ReqDstReg](#) – destination software transaction request
- [SglReqSrcReg](#) – single source transaction request
- [SglReqDstReg](#) – single destination transaction request
- [LstSrcReg](#) – last source transaction request
- [LstDstReg](#) – last destination transaction request

For details on how the software handshaking flow works, refer to “[Software Handshaking – Peripheral Is Not Flow Controller](#)” on page 54 and “[Software Handshaking – Peripheral Is Flow Controller](#)” on page 59.

3.7 Handshaking Interface – Peripheral Is Not Flow Controller

When the peripheral is not the flow controller, the DW_ahb_dmac tries to efficiently transfer the data using as little of the bus bandwidth as possible. Generally, the DW_ahb_dmac tries to transfer the data using burst transactions and, where possible, fill or empty the channel FIFO in single bursts – provided that the software has not limited the burst length; refer to “[Example 3](#)” on page 65. The DW_ahb_dmac can also lock the arbitration for the master bus interface so that a channel is permanently granted the master bus

interface. Additionally, the DW_ahb_dmac can assert the AMBA hlock signal to lock the DW_ahb system arbiter. For more information, refer to [“Locked DMA Transfers”](#) on page 103.

Before describing the handshaking interface operation when the peripheral is not the flow controller, the following sections define the terms “Single Transaction Region” and “Early-Terminated Burst Transaction.”

3.7.1 Single Transaction Region

There are cases where a DMA block transfer cannot complete using only burst transactions. Typically this occurs when the block size is not a multiple of the burst transaction length; for more information, refer to [Example 4](#) and [Example 5](#). In these cases, the block transfer uses burst transactions up to the point where the amount of data left to complete the block is less than the amount of data in a burst transaction. At this point, the DW_ahb_dmac samples the “single” status flag and completes the block transfer using single transactions; again refer to [Example 4](#) and [Example 5](#).

The peripheral asserts a single status flag to indicate to the DW_ahb_dmac that there is enough data or space to complete a single transaction from or to the source/destination peripheral.



Note

For hardware handshaking, the single status flag is a signal on the hardware handshaking interface; refer to [“Hardware Handshaking – Peripheral Is Not Flow Controller”](#) on page 48. For software handshaking, the single status flag is one of the software handshaking interface registers; refer to [“Software Handshaking – Peripheral Is Not Flow Controller”](#) on page 54.

The Single Transaction Region is the time interval where the DW_ahb_dmac uses single transactions to complete the block transfer; burst transactions are exclusively used outside this region.



Note

Burst transactions can also be used in this region; for more information, refer to [“Early-Terminated Burst Transaction”](#) on page 47.

The Single Transaction Region applies to only a peripheral that is *not* the flow controller. The precise definition of when this region is entered is dependent on what acts as the flow controller:

- The DW_ahb_dmac is the flow controller – The source peripheral enters the Single Transaction Region when the number of bytes left to complete in the source block transfer is less than *src_burst_size_bytes*. If:

$$\text{blk_size_bytes} / \text{src_burst_size_bytes} = \text{integer}(8)$$

then the source never enters this region, and the source block uses only burst transactions.

The destination peripheral enters the Single Transaction Region when the number of bytes left to complete in the destination block transfer is less than *dst_burst_size_bytes*. If:

$$\text{blk_size_bytes} / \text{dst_burst_size_bytes} = \text{integer}(9)$$

then the destination never enters this region, and the destination block uses only burst transactions.

**Note**

The above conditions cause a peripheral to enter the Single Transaction Region. When the peripheral is outside the Single Transaction Region, then the DW_ahb_dmac responds to only burst transaction requests. Whether the peripheral knows that it is in the Single Transaction Region or not, it must always generate burst requests outside the Single Transaction Region, or the DMA block transfer stalls. Once in the Single Transaction Region, the DW_ahb_dmac can complete the block transfer using single transactions.

- Either the source or destination peripheral is the flow controller – The destination or source peripheral enters the Single Transaction Region when the flow control peripheral – that is, the source or destination – signals the last transaction in the block *and* when the amount of data left to be transferred in the destination/source block is less than that which is specified by *dst_burst_size_bytes/src_burst_size_bytes*.

3.7.2 Early-Terminated Burst Transaction

When a source or destination peripheral is in the Single Transaction Region, a burst transaction can still be requested. However, *src_burst_size_bytes/ dst_burst_size_bytes* is greater than the number of bytes left to complete in the source/destination block transfer at the time that the burst transaction is triggered. In this case, the burst transaction is started and “early-terminated” at block completion without transferring the programmed amount of data – that is, *src_burst_size_bytes* or *dst_burst_size_bytes* – but only the amount required to complete the block transfer. An Early-Terminated Burst Transaction occurs between the DW_ahb_dmac and the peripheral only when the peripheral is not the flow controller.

The DW_ahb_dmac never terminates defined-length bursts early.

- If the DW_ahb_dmac is configured with DMAH_INCR_BURSTS = 0 and it detects a burst request when in the Single Transaction Region, the DW_ahb_dmac only issues SINGLE type transfers to complete the burst.
- If DMAH_INCR_BURSTS = 1 and the DW_ahb_dmac detects a burst when in the Single Transaction Region, the DW_ahb_dmac issues an undefined length burst (INCR) of length only large enough to complete the block transfer. It does not transfer all programmed *src_burst_size_bytes* or *dst_burst_size_bytes*.

3.7.3 Hardware Handshaking – Peripheral Is Not Flow Controller

Figure 3-2 illustrates the hardware handshaking interface between a peripheral – whether a destination or source – and the DW_ahb_dmac when the peripheral is not the flow controller.

Figure 3-2 Hardware Handshaking Interface

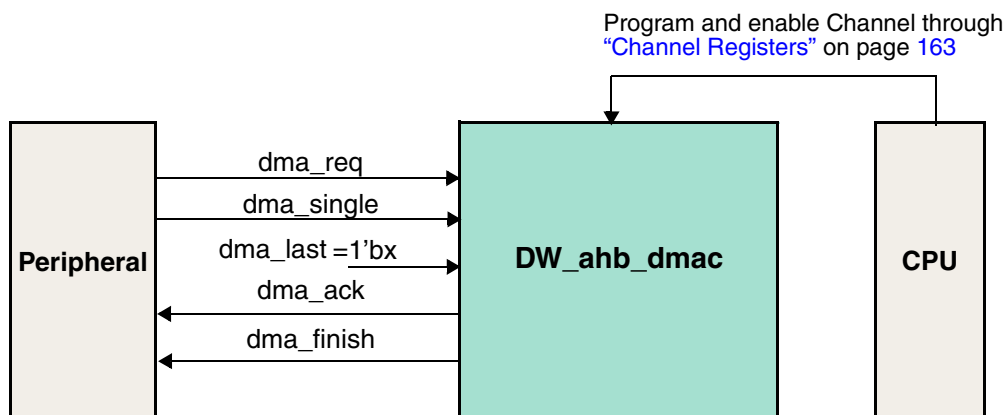


Table 3-2 describes the hardware handshaking signals in the case where the peripheral is not the flow controller; that is, where either the DW_ahb_dmac or the other peripheral is the flow controller. Signal polarity can be programmed using the [CFGx.SRC_HS_POL](#) and [CFGx.DST_HS_POL](#) fields.

Table 3-2 Hardware Handshaking Interface

Signal	Direction	Description
dma_ack	Output	DW_ahb_dmac acknowledge signal to peripheral. The dma_ack signal is asserted after the data phase of the last AHB transfer in the current transaction – single or burst – to the peripheral that has completed. For a single transaction, dma_ack remains asserted until the peripheral de-asserts dma_single; dma_ack is de-asserted one hclk cycle later. For a burst transaction, dma_ack remains asserted until the peripheral de-asserts dma_req; dma_ack is de-asserted one hclk cycle later.
dma_finish	Output	DW_ahb_dmac asserts dma_finish to signal block completion. This has the same timing as dma_ack and forms a handshaking loop with dma_req if the last transaction in the block was a burst transaction, or with dma_single if the last transaction in the block was a single transaction. There is an exception to the above timing definition when dma_finish interfaces with a source peripheral when the destination peripheral is the flow controller; for details, refer to "Example 7" on page 76 Case 1b.
dma_last	Input	Since the peripheral is not the flow controller, dma_last is not sampled by the DW_ahb_dmac and this signal is ignored.

Table 3-2 Hardware Handshaking Interface (Continued)

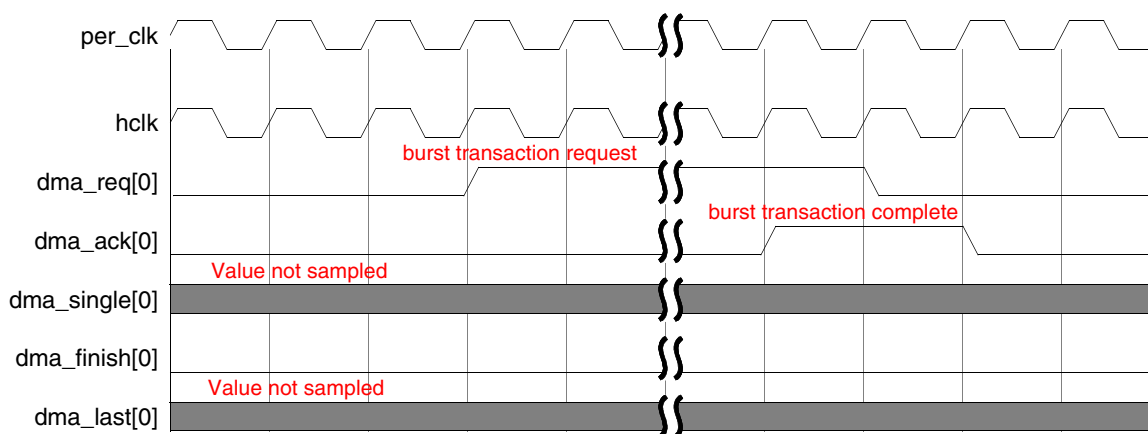
Signal	Direction	Description
dma_req	Input	Burst transaction request from peripheral. The DW_ahb_dmac always interprets the dma_req signal as a burst transaction request, regardless of the level of dma_single. This is a level-sensitive signal; once asserted by the peripheral, dma_req must remain asserted until the DW_ahb_dmac asserts dma_ack. Upon receiving the dma_ack signal from the DW_ahb_dmac to indicate the burst transaction is complete, the peripheral should de-assert the burst request signal, dma_req. Once dma_req is de-asserted by the peripheral, the DW_ahb_dmac de-asserts dma_ack. If an active level on dma_req is detected in the Single Transaction Region, then the block is completed using an Early-Terminated Burst Transaction.
dma_single	Input	Single transfer status. The dma_single signal is a status signal that is asserted by a destination peripheral when it can accept at least one destination data item; otherwise it is cleared. For a source peripheral, the dma_single signal is again a status signal and is asserted by a source peripheral when it can transmit at least one source data item; otherwise it is cleared. Once asserted, dma_single must remain asserted until dma_ack is asserted, at which time the peripheral should de-assert dma_single. This signal is sampled by the DW_ahb_dmac only in the Single Transaction Region of the block transfer. Outside of this region, dma_single is ignored and all transactions are burst transactions.

Figure 3-3 shows the timing diagram of a burst transaction where the peripheral clock, `per_clk`, equals `hclk`. In this example, the peripheral is outside the Single Transaction Region, and therefore the DW_ahb_dmac does not sample `dma_single[0]`.

The handshaking loop is as follows:

- `dma_req` asserted by peripheral
- > `dma_ack` asserted by DW_ahb_dmac
- > `dma_req` de-asserted by peripheral
- > `dma_ack` de-asserted by DW_ahb_dmac.

Figure 3-3 Burst Transaction – `pclk = hclk`



The `per_clk` signal is equal to `hclk` if the peripheral is an AHB peripheral; it is equal to `pclk` if the peripheral is an APB peripheral. The burst transaction request signal, `dma_req`, and the single status signal, `dma_single`, are generated in the peripheral of `per_clk` and sampled by `hclk` in the DW_ahb_dmac. The acknowledge signal, `dma_ack`, is generated in the DW_ahb_dmac of `hclk` and sampled in the peripheral by `per_clk`. The handshaking mechanism between the DW_ahb_dmac and the peripheral supports quasi-synchronous clocks; that is, `hclk` and `per_clk` must be phase-aligned, and the `hclk` frequency must be a multiple of the `per_clk` frequency.

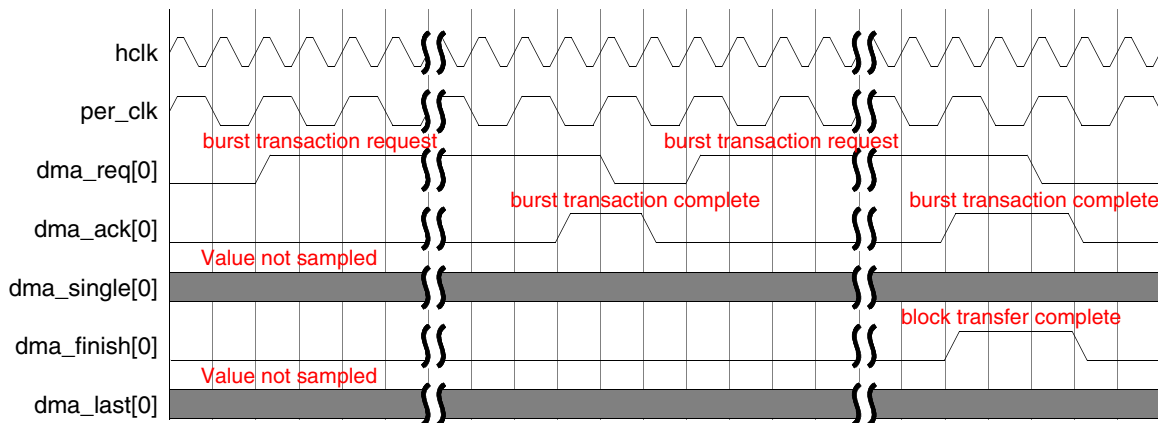
In the case where the destination peripheral is an APB peripheral and for the case of buffered writes through an APB bridge, then caution must be exercised so as not to overflow the destination peripheral FIFO. This can happen when the write is buffered in the APB bridge; that is, the write completes on the AHB before completing on the APB bus. The following scenario can cause an overflow: the DW_ahb_dmac asserts `dma_ack` as soon as the write transaction completes on the AHB. The APB peripheral, on sampling that the acknowledge signal is asserted, de-asserts its request signal and asserts the request signal one APB clock cycle later, as it senses that there is space in its FIFO. The issue here is that there could be space for only a single entry that the first buffered write will consume. The initiation of the second transaction may overflow the FIFO. If the write were not buffered, then the initiation of the second transaction would not occur, as the destination peripheral would sense that its FIFO was full.

To avoid this, do one of the following:

1. Do not use buffered writes.
2. If using buffered writes, then ensure that `dma_ack` signal from the DW_ahb_dmac is delayed until the write completes to the APB peripheral. One way may be to route the `dma_ack` signal through the APB bridge.

Figure 3-4 shows two back-to-back burst transactions at the end of a block transfer where the `hclk` frequency is twice the `pclk` frequency; the peripheral is an APB peripheral. The second burst transaction terminates the block, and `dma_finish[0]` is asserted to indicate block completion.

Figure 3-4 Back-to-Back Burst Transactions – $hclk = 2 * per_clk$



There are two things to note when designing the hardware handshaking interface:

- Once asserted, the `dma_req` burst request signal must remain asserted until the corresponding `dma_ack` signal is received, even if the condition that generates `dma_req` in the peripheral is False.
- The `dma_req` signal should be de-asserted when `dma_ack` is asserted, even if the condition that generates `dma_req` in the peripheral is True.

Figure 3-5 shows a single transaction that occurs in the [Single Transaction Region](#). The handshaking loop is as follows:

- dma_single asserted by peripheral
- > dma_ack asserted by DW_ahb_dmac
- > dma_single de-asserted by peripheral
- > dma_ack de-asserted by DW_ahb_dmac

Figure 3-5 Single Transaction

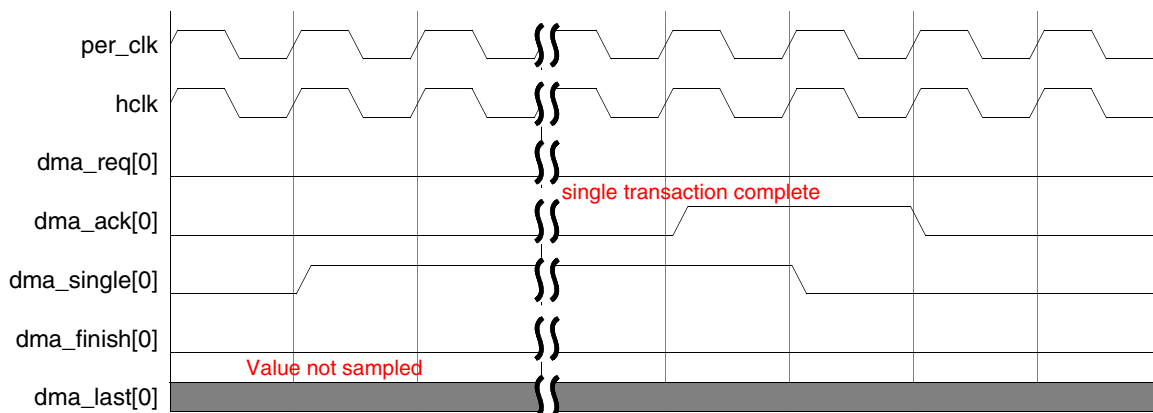
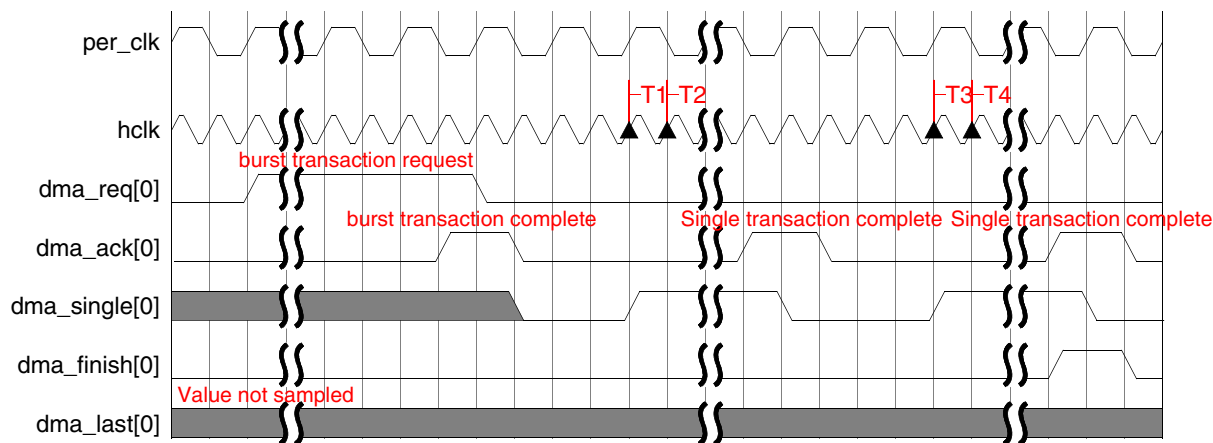


Figure 3-6 shows a burst transaction, followed by two back-to-back single transactions, where the hclk frequency is twice the per_clk frequency; handshaking interface 0 is used.

Figure 3-6 Burst Followed by Back-to-Back Single Transactions



After the first burst transaction, the peripheral enters the [Single Transaction Region](#) and the DW_ahb_dmac starts sampling dma_single[0]. On hclk edges T2 and T4, the DW_ahb_dmac samples that dma_single[0] is asserted and performs single transactions. The second single transaction terminates the block transfer; dma_finish[0] is asserted to indicate block completion.

In the [Single Transaction Region](#), if an active level on dma_req and dma_single occur on the same cycle – or if the active level on dma_single occurs only one cycle before an active level on dma_req – then the burst transaction takes precedence over the single transaction, and the block would be completed using an

Early-Terminated Burst Transaction. If the DW_ahb_dmac samples that `dma_req[0]` was asserted on `hclk` cycles T1-T2 or T3-T4 in Figure 3-6, then the burst request takes precedence.

In Figure 3-7, an active level on `dma_req[0]` after time T4 takes precedence over the active level on `dma_single[0]` after time T3.

Figure 3-7 Early-Terminated Burst Transaction

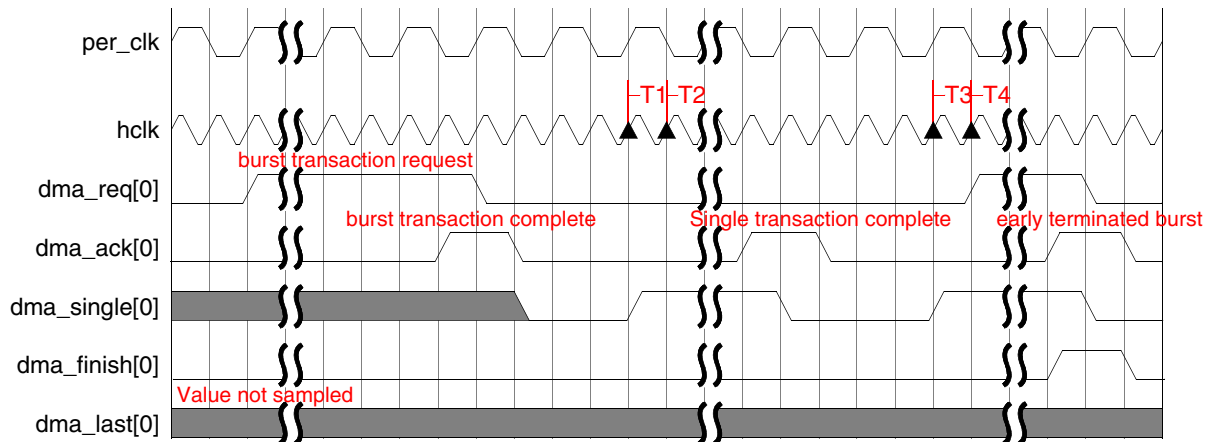
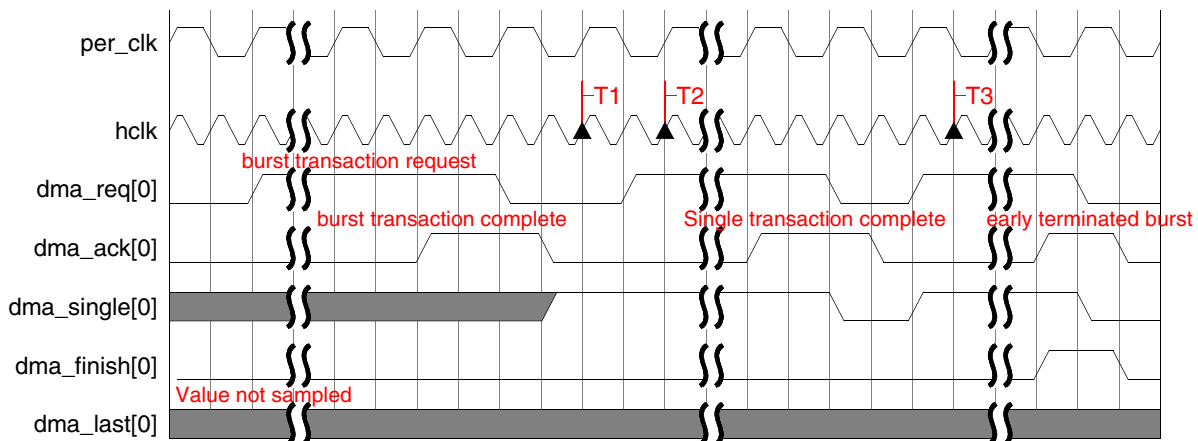


Figure 3-8 shows a burst transaction followed by a single transaction, followed by a burst transaction at the end of a block.

Figure 3-8 Burst Transaction ignored During Active Single Transaction



After the first burst transaction completes, the peripheral is in the **Single Transaction Region** and DW_ahb_dmac samples that `dma_single[0]` is asserted at T1. The `dma_req[0]` signal is triggered in the middle of this single transaction at time T2. This burst transaction request is ignored and is not serviced. An active edge on `dma_req[0]` is re-generated and sampled by DW_ahb_dmac at time T3. This burst transaction completes the block transfer using an **Early-Terminated Burst Transaction**.

3.7.3.1 Generating `dma_req` and `dma_single` Hardware Handshaking Signals

Figure 3-9 illustrates a suggested method of generating `dma_req` and `dma_single` for a source peripheral when the peripheral is not the flow controller. The `single_flag` signal in Figure 3-9 is asserted when the source FIFO has at least one source data item in the FIFO. The `burst_flag` signal in Figure 3-9 is asserted

when the source FIFO contains data items greater than or equal to some watermark-level number of data items in it.

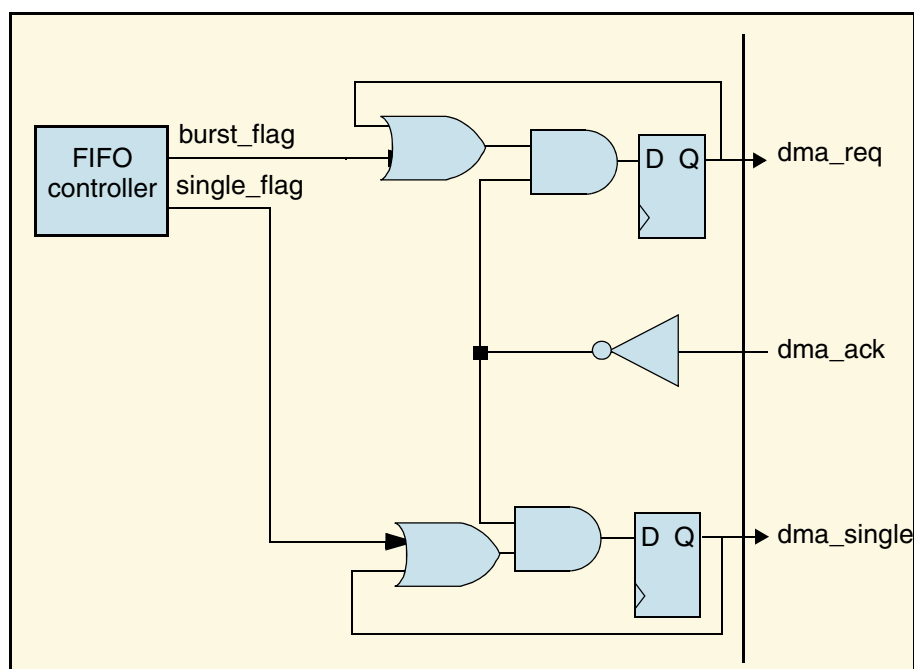


Note

Figure 3-9 shows `dma_req` and `dma_single` being de-asserted when `dma_ack` is asserted. It also shows how, once asserted, `dma_req` and `dma_single` remain asserted until `dma_ack` is asserted. The example assumes active-high handshaking.

The destination peripheral `dma_req` and `dma_single` signals can be generated in a similar fashion, but in this case the `single_flag` signal in Figure 3-9 is asserted when the destination FIFO has at least one free location. The `burst_flag` signal in Figure 3-9 is asserted when the destination FIFO contains free locations greater than or equal to some watermark-level number.

Figure 3-9 Generation of `dma_req` and `dma_single` by Source



3.7.4 Software Handshaking – Peripheral Is Not Flow Controller

When the peripheral is not the flow controller, then the last transaction registers – `LstSrcReg` and `LstDstReg` – are not used, and the values in these registers are ignored.

3.7.4.1 Operation – Peripheral Not In Single Transaction Region

Writing a 1 to the `ReqSrcReg[x]/ReqDstReg[x]` register is always interpreted as a burst transaction request, where x is the channel number. However, in order for a burst transaction request to start, software must write a 1 to the `SglReqSrcReg[x]/SglReqDstReg[x]` register.

You can write a 1 to the `SglReqSrcReg[x]/SglReqDstReg[x]` and `ReqSrcReg[x]/ReqDstReg[x]` registers in any order, but both registers must be asserted in order to initiate a burst transaction. Upon completion of the burst transaction, the hardware clears the `SglReqSrcReg[x]/SglReqDstReg[x]` and `ReqSrcReg[x]/ReqDstReg[x]` registers.

3.7.4.2 Operation – Peripheral In Single Transaction Region

Writing a 1 to the [SglReqSrcReg/SglReqDstReg](#) initiates a single transaction. Upon completion of the single transaction, both the [SglReqSrcReg/SglReqDstReg](#) and [ReqSrcReg/ReqDstReg](#) bits are cleared by hardware. Therefore, writing a 1 to the [ReqSrcReg/ReqDstReg](#) is ignored while a single transaction has been initiated, and the requested burst transaction is not serviced.

Again, writing a 1 to the [ReqSrcReg/ReqDstReg](#) register is always a burst transaction request. However, in order for a burst transaction request to start, the corresponding channel bit in the [SglReqSrcReg/SglReqDstReg](#) must be asserted. Therefore, to ensure that a burst transaction is serviced in this region, you must write a 1 to the [ReqSrcReg/ReqDstReg](#) before writing a 1 to the [SglReqSrcReg/SglReqDstReg](#) register. If the programming order is reversed, a single transaction is started instead of a burst transaction. The hardware clears both the [ReqSrcReg/ReqDstReg](#) and the [SglReqSrcReg/SglReqDstReg](#) registers after the burst transaction request completes. When a burst transaction is initiated in the [Single Transaction Region](#), then the block completes using an [Early-Terminated Burst Transaction](#).

Software can poll the relevant channel bit in the [SglReqSrcReg/SglReqDstReg](#) and [ReqSrcReg/ReqDstReg](#) registers. When both are 0, then either the requested burst or single transaction has completed. Alternatively, the [IntSrcTran](#) or [IntDstTran](#) interrupts can be enabled and unmasked in order to generate an interrupt when the requested source or destination transaction has completed.



Note

The transaction-complete interrupts are triggered when both single and burst transactions are complete. The same transaction-complete interrupt is used for both single and burst transactions.

3.7.5 Single Transactions – Peripheral Is Not Flow Controller

When the source peripheral is not the flow controller, the source peripheral can hardcode `dma_single` to an inactive level (hardware handshaking), or software will never need to initiate single transactions from the source (software handshaking). This can happen if either of the following is true:

- Block size is a multiple of the burst transaction length.
 - If `DW_ahb_dmac` is the flow controller

$$\text{blk_size_bytes_dma}/\text{src_burst_size_bytes} = \text{integer}$$
 - If the destination peripheral is the flow controller

$$\text{blk_size_bytes_dst}/\text{src_burst_size_bytes} = \text{integer}$$
- Block size is not a multiple of the burst transaction length, but the peripheral can dynamically adjust the watermark level that triggers a burst request in order to enable block completion.

When the destination peripheral is not a flow controller, then the destination peripheral may hardcode `dma_single` to an inactive level (hardware handshaking), or software will never need to initiate single transactions to the destination (software handshaking). This can happen when any of the following are true:

- Block size is a multiple of the burst transaction length.
 - If `DW_ahb_dmac` is the flow controller

$$\text{block_size_bytes_dma}/\text{dst_burst_size_bytes} = \text{integer}$$

- If the source peripheral is flow controller

$$\text{block_size_bytes_src} / \text{dst_burst_size_bytes} = \text{integer}$$

- The destination peripheral can dynamically adjust the watermark level upwards so that a burst request is triggered in order to enable a destination block completion.
- It is guaranteed that data at some point will be extracted from the destination FIFO in the “Single transaction region” in order to trigger a burst transaction.

**Note**

The destination peripheral requires data to be extracted in order to empty the destination FIFO below a watermark level for triggering a destination burst request. If it is guaranteed that data at some point will be extracted from the destination FIFO in the [Single Transaction Region](#) in order to trigger a dma_req, then in this case, the dma_single signal from the destination can be tied to an inactive level, and the destination block completes with an [Early-Terminated Burst Transaction](#).

If none of the above are true, then a series of burst transactions followed by single transactions is needed to complete the source/destination block transfer; for more information, refer to “[Example 4](#)” on page 67 and “[Example 5](#)” on page 70.

3.8 Handshaking Interface – Peripheral Is Flow Controller

When the peripheral is the flow controller, it controls the length of the block and must communicate to the DW_ahb_dmac when the block transfer is complete. The peripheral does this by telling the DW_ahb_dmac that the current transaction – burst or single – is the last transaction in the block. When the peripheral is the flow controller and the block size is not a multiple of the CTLx.SRC_MSIZE/CTLx.DEST_MSIZE, then the peripheral must use single transactions to complete a block transfer.

**Note**

Since the peripheral can terminate the block on a single transaction, there is no notion of a [Single Transaction Region](#) such as there is when the peripheral is not the flow controller.

When the peripheral is the flow controller, it indicates directly to DW_ahb_dmac which type of transaction – single or burst – to perform. Where possible, the DW_ahb_dmac uses the maximum possible burst length. It can also lock the arbitration for the master bus so that a channel is permanently granted the master bus interface. The DW_ahb_dmac can also assert the hlock signal to lock the DW_ahb system arbiter. For more information, refer to “[Locked DMA Transfers](#)” on page 103.

3.8.1 Hardware Handshaking – Peripheral Is Flow Controller

Figure 3-10 shows the hardware handshaking interface between a destination or source peripheral and the DW_ahb_dmac when the peripheral is the flow controller.

Figure 3-10 Hardware Handshaking Interface

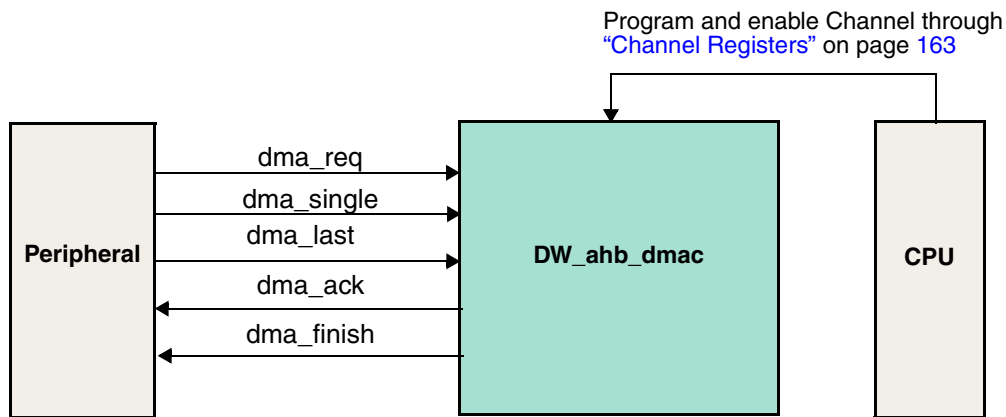


Table 3-3 describes the operation of the hardware handshaking interface signals when the peripheral is the flow controller; timing diagrams are illustrated in Figure 3-11 on page 58 and Figure 3-12 on page 59.

Table 3-3 Hardware Handshaking Interface

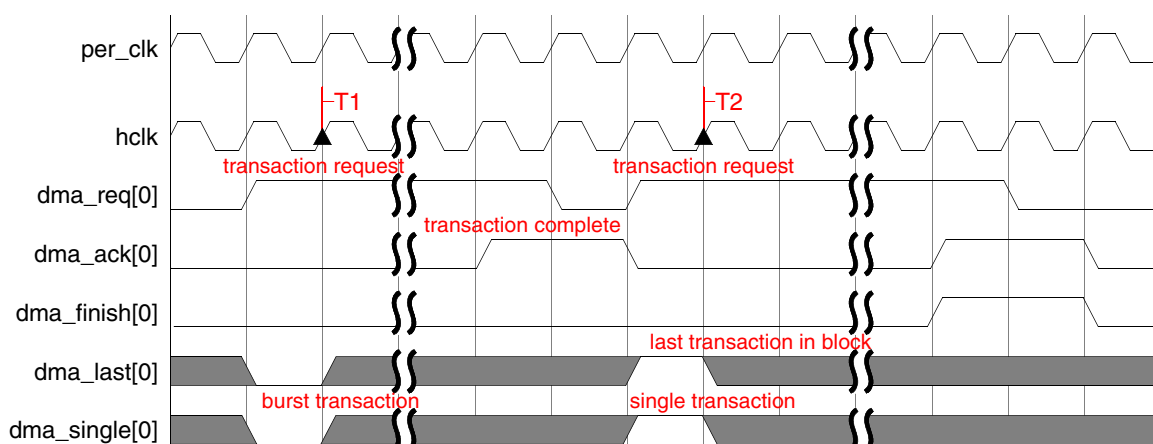
Signal	Direction	Description
dma_ack	Output	DW_ahb_dmac acknowledge signal to the peripheral. This is asserted after the data phase of the last AHB transfer in the current transaction (single or burst) to the peripheral has completed. It forms a handshaking loop with dma_req and remains asserted until the peripheral de-asserts dma_req (de-asserted one hclk cycle later).
dma_finish	Output	DW_ahb_dmac block transfer complete signal. The DW_ahb_dmac asserts dma_finish in order to signal block completion. This uses the same timing as dma_ack and forms a handshaking loop with dma_req.
dma_last	Input	Last transaction in block. When the peripheral is the flow controller, it asserts dma_last on the same cycle as dma_req is asserted in order to signal that this transaction request is the last in the block; the block transfer is complete after this transaction is complete. If dma_single is high in the same cycle, then the last transaction is a single transaction. If dma_single is low in the same cycle, then the last transaction is a burst transaction.
dma_req	Input	Transaction request from peripheral. An active level on dma_req initiates a transaction request. The type of transaction – single or burst – is qualified by dma_single. Once dma_req is asserted, it must remain asserted until dma_ack is asserted. When the peripheral that is driving dma_req determines that dma_ack is asserted, it must de-assert dma_req.

Table 3-3 Hardware Handshaking Interface (Continued)

Signal	Direction	Description
dma_single	Input	Single or burst transaction request. If dma_single is de-asserted in the same clock cycle as a rising edge on dma_req, a burst transaction is requested by the peripheral. If asserted, the peripheral requests a single transaction.

The following timing diagrams assume that handshaking interface 0 is active-high.

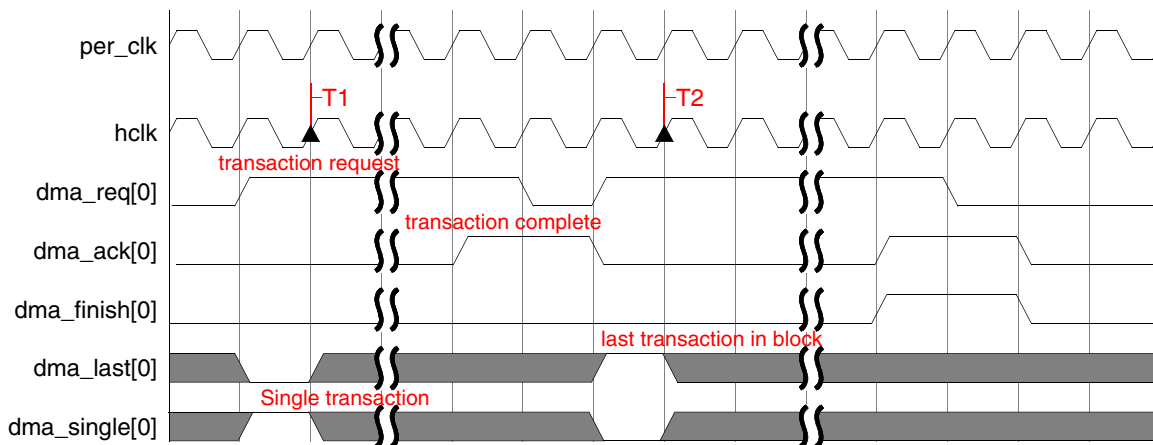
Figure 3-11 shows a burst transaction followed by a single transaction, where the single transaction is the last in the block. On clock edge T1, DW_ahb_dmac samples that dma_req[0] is asserted, dma_single[0] is de-asserted, and dma_last[0] is de-asserted. This is a request for a burst transaction, which is not the last transaction in the block.

Figure 3-11 Burst Transaction Followed by Single Transaction that Terminates Block

On clock edge T2, the DW_ahb_dmac samples that dma_req[0], dma_single[0], and dma_last[0] are all asserted. This is a request for a single transaction, which is the last transaction in the block. The dma_last[0] and dma_single[0] signals need only be valid on the same clock cycle that dma_req is generated.

Similarly, [Figure 3-12](#) shows a single transaction followed by a burst transaction, where the burst transaction is the last transaction in the block.

Figure 3-12 Single Transaction Followed by Burst Transaction that Terminates Block



3.8.2 Software Handshaking – Peripheral Is Flow Controller

Writing a 1 to the Source/Destination Software Transaction Request initiates a transaction; refer to “[ReqSrcReg](#)” on page 199 and “[ReqDstReg](#)” on page 200, respectively. The type of transaction – single or burst – depends on the state of the corresponding channel bit in the Single Source/Destination Transaction Request register; refer to “[SglReqSrcReg](#)” on page 201 or “[SglReqDstReg](#)” on page 202, respectively.

If $SglReqSrcReg[n]/SglReqDstReg[n] = 1$ when a 1 is written to the $ReqSrcReg[n]/ReqDstReg[n]$ register, this means that software is requesting a single transaction on channel n , or a burst transaction otherwise.

The request is the last in the block if the corresponding channel bit in the Last Source/Destination Request register is asserted; refer to “[LstSrcReg](#)” on page 203 and “[LstDstReg](#)” on page 204, respectively.

If $LstSrcReg[n]/LstDstReg[n] = 1$ when a 1 is written to the $ReqSrcReg[n]/ReqDstReg[n]$ register, this means that software is requesting that this transaction is the last transaction in the block. The [SglReqSrcReg/SglReqDstReg](#) and [LstSrcReg/LstDstReg](#) registers must be written to before the [ReqSrcReg/ReqDstReg](#) registers.

On completion of the transaction – single or burst – the relevant channel bit in the [ReqSrcReg/ReqDstReg](#) register is cleared by hardware. Software can therefore poll this bit in order to determine when the requested transaction has completed. Alternatively, the [IntSrcTran](#) or [IntDstTran](#) interrupts can be enabled and unmasked in order to generate an interrupt when the requested transaction – single or burst – has completed.

When the peripheral is the flow controller and the block size is not a multiple of the $CTLx.SRC_MSIZE/CTLx.DEST_MSIZE$, then software must use single transactions to complete the block transfer.

3.8.3 Single Transactions – Peripheral is Flow Controller

When the source peripheral is the flow controller, then it can hardcode `dma_single` to an inactive level (hardware handshaking), or software will never need to initiate single transactions from the source (software handshaking). This occurs when:

$$\text{block_size_bytes_src} / \text{src_burst_size_bytes} = \text{integer} \quad (10)$$

When the destination peripheral is the flow controller, then the destination peripheral can hardcode `dma_single` to an inactive level (hardware handshaking), or software will never need to initiate single transactions to the destination (software handshaking) when:

$$\text{block_size_bytes_dst} / \text{dst_burst_size_bytes} = \text{integer} \quad (11)$$

3.9 Setting Up Transfers

Transfers are set up by programming fields of the `CTLx` and `CFGx` registers for that channel. As shown in [Figure 1-5](#) on page 21, a single block is made up of numerous transactions – single and burst – which are in turn composed of AHB transfers. A peripheral requests a transaction through the handshaking interface to the DW_ahb_dmac; for more information, refer to [“Handshaking Interface”](#) on page 42. The operation of the handshaking interface is different and depends on what is acting as the flow controller.

[Table 3-4](#) on page 60 lists the parameters that are investigated in the following examples. The effects of these parameters on the flow of the block transfer are highlighted. In addition to the software parameters, it includes the channel FIFO depth, `DMAH_CHx_FIFO_DEPTH`, which is configurable only in coreConsultant.

For definitions of the register parameters, refer to [“Register and Field Descriptions”](#) on page 160; for definitions of DW_ahb_dmac parameters in coreConsultant, refer to [“Parameters”](#) on page 119.

Table 3-4 Parameters Used in Transfer Examples

Parameter	Description
<code>DMAH_CHx_FIFO_DEPTH</code>	Channel x FIFO depth in bytes
<code>CTLx.TT_FC</code>	Transfer type and flow control
<code>CTLx.BLOCK_TS</code>	Block transfer size
<code>CTLx.SRC_TR_WIDTH</code>	Source transfer width
<code>CTLx.DST_TR_WIDTH</code>	Destination transfer width
<code>CTLx.SRC_MSIZ</code>	Source burst transaction length
<code>CTLx.DEST_MSIZ</code>	Destination burst transaction length
<code>CFGx.MAX_ABRST</code>	Maximum AMBA burst length
<code>CFGx.FIFO_MODE</code>	FIFO mode select
<code>CFGx.FCMODE</code>	Flow-control mode

3.9.1 Transfer Operation

The following examples show the effect of different settings of each parameter from [Table 3-4](#) on a DMA block transfer. In all examples, it is assumed that no bursts are early-terminated by the DW_ahb system arbiter, unless otherwise stated. [Example 1](#) through [Example 8](#) use hardware handshaking on both the source and destination side. [Example 9](#) through [Example 11](#) use software handshaking on both the source and destination side.

The following is a brief description of each of the examples:

- “[Example 1](#)” on page [62](#) – Block transfer when the DW_ahb_dmac is the flow controller.
- “[Example 2](#)” on page [63](#) – Effect of DMAH_CHx_FIFO_DEPTH on a block transfer.
- “[Example 3](#)” on page [65](#) – Effect of maximum AMBA burst length, CFGx.MAX_ABRST, on a block transfer.
- “[Example 4](#)” on page [67](#) – Block transfer when the DW_ahb_dmac is the flow controller and the source peripheral enters the [Single Transaction Region](#).
- “[Example 5](#)” on page [70](#) – Block transfer when the DW_ahb_dmac is the flow controller and the destination peripheral enters the [Single Transaction Region](#). Also demonstrates channel FIFO flushing to the destination peripheral at the end of a block transfer.
- “[Example 6](#)” on page [72](#) – Effect of CFGx.FIFO_MODE on a block transfer.
- “[Example 7](#)” on page [76](#) – Block transfer when the destination peripheral is the flow controller and data pre-fetching from the source is enabled; CFGx.FCMODE = 0.
- “[Example 8](#)” on page [83](#) – Block transfer when the destination peripheral is the flow controller and data pre-fetching from the source is disabled; CFGx.FCMODE = 1.
- “[Example 9](#)” on page [85](#) – Block transfer when the DW_ahb_dmac is the flow controller and software handshaking is used on both the source and destination side.
- “[Example 10](#)” on page [87](#) – Block transfer when the DW_ahb_dmac is the flow controller and software handshaking is used on both the source and destination side; the source enters the [Single Transaction Region](#).
- “[Example 11](#)” on page [90](#) – Block transfer when the source peripheral is the flow controller and software handshaking is used on both the source and destination side; the destination peripheral enters the [Single Transaction Region](#).

The DW_ahb_dmac is programmed with the number of data items that are to be transferred for each burst transaction request, CTLx.SRC_MSIZ / CTLx.DEST_MSIZ. Similarly, the width of each data item in the transaction is set by the CTLx.SRC_TR_WIDTH and CTLx.DST_TR_WIDTH fields.

3.9.1.1 Example 1

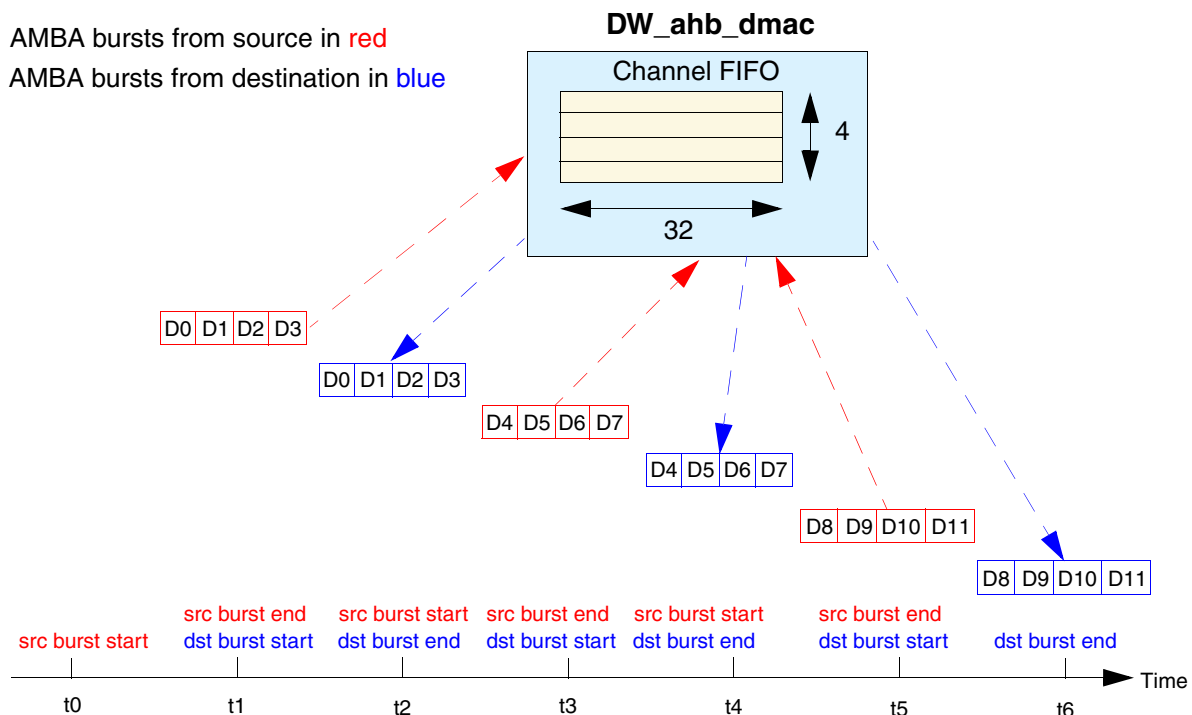
Scenario: Example block transfer when the DW_ahb_dmac is the flow controller. This example is the same for both software and hardware handshaking interfaces. [Table 3-5](#) lists the DMA parameters for this example.

Table 3-5 Parameters in Transfer Operation – Example 1

Parameter	Description
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral transfer with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 12	–
CTLx.SRC_TR_WIDTH = 3'b010	32 bit
CTLx.DST_TR_WIDTH = 3'b010	32 bit
CTLx.SRC_MSIZ = 3'b 001	Source burst transaction length = 4
CTLx.DEST_MSIZ = 3'b 001	Destination burst transaction length = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 16 bytes	–

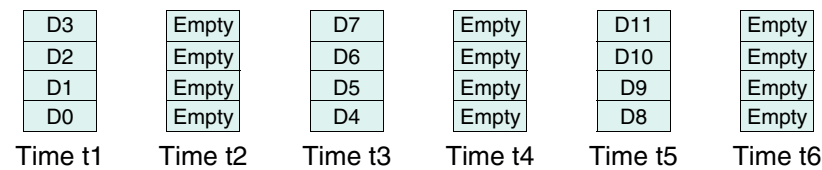
Using equation (5), a total of 48 bytes are transferred in the block; that is, $blk_size_bytes_dma = 48$. As shown in [Figure 3-13](#), this block transfer consists of three bursts of length 4 from the source, interleaved with three bursts, again of length 4, to the destination.

Figure 3-13 Breakdown of Block Transfer



The channel FIFO is alternatively filled by a burst from the source and emptied by a burst to the destination until the block transfer has completed, as shown in [Figure 3-14](#).

Figure 3-14 Channel FIFO Contents at Times Indicated in [Figure 3-13](#)



Burst transactions are completed in one burst. Additionally, because (8) and (9) are both true, neither the source or destination peripherals enter their [Single Transaction Region](#) at any stage throughout the DMA transfer, and the block transfer from the source and to the destination consists of burst transactions only.

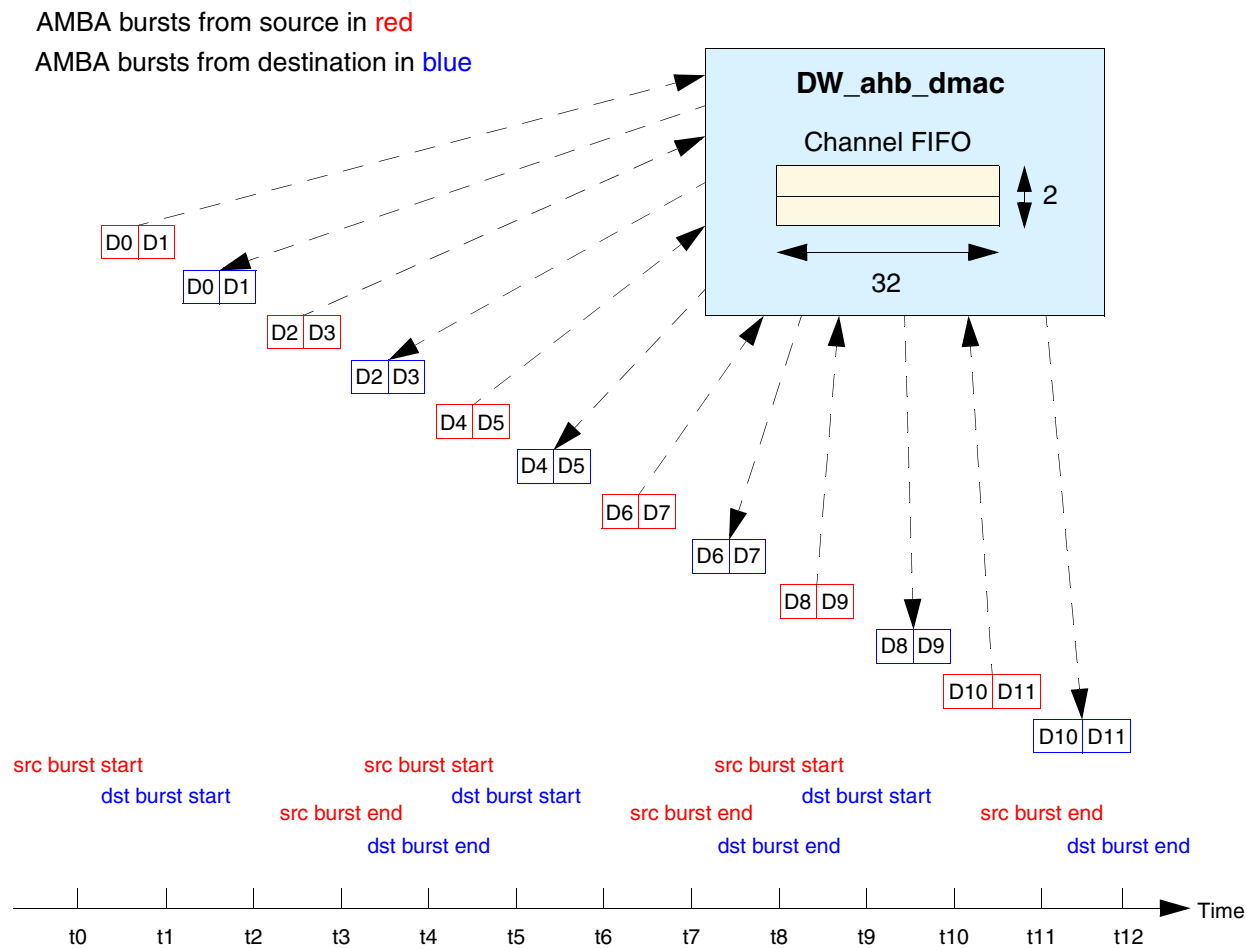
3.9.1.2 Example 2

Scenario: Effect of DMAH_CHx_FIFO_DEPTH on block transfers. This example is the same for both software and hardware handshaking interfaces.

In this example, the coreConsultant DMAH_CHx_FIFO_DEPTH parameter is changed to 8 bytes, and all other parameters are left unchanged from [Example 1](#), [Table 3-5](#) on page 62.

[Example 1](#) shows the source and destination burst transactions completing in a single burst. In general, a burst transaction may take multiple bursts to complete. With the DMAH_CHx_FIFO_DEPTH parameter set to 8 bytes instead of 16 bytes, the block transfer would look like that shown in [Figure 3-15](#).

Figure 3-15 Breakdown of Block Transfer for DMAH_CHx_FIFO_DEPTH=8



The block transfer consists of six bursts of length 2 from the source, interleaved with six bursts – again of length 2 – to the destination, as shown in [Figure 3-13](#). The channel FIFO is alternatively filled by a burst from the source and emptied by a burst to the destination, until the block transfer has completed. In this example, a transfer of each source or destination burst transaction is made up of two bursts, each of length 2.

Therefore, [Example 2](#) has twice the number of bursts per block than [Example 1](#).

Recommendation: To allow a burst transaction to complete in a single burst, the DW_ahb_dmac channel FIFO depth should be large enough to accept an amount of data equal to an entire burst transaction. Therefore, in order to allow both source and destination burst transactions to complete in one burst:

$$\text{DMAH_CHx_FIFO_DEPTH} \geq \max(2 * \text{src_burst_size_bytes}, 2 * \text{dst_burst_size_bytes}) \quad (12)$$

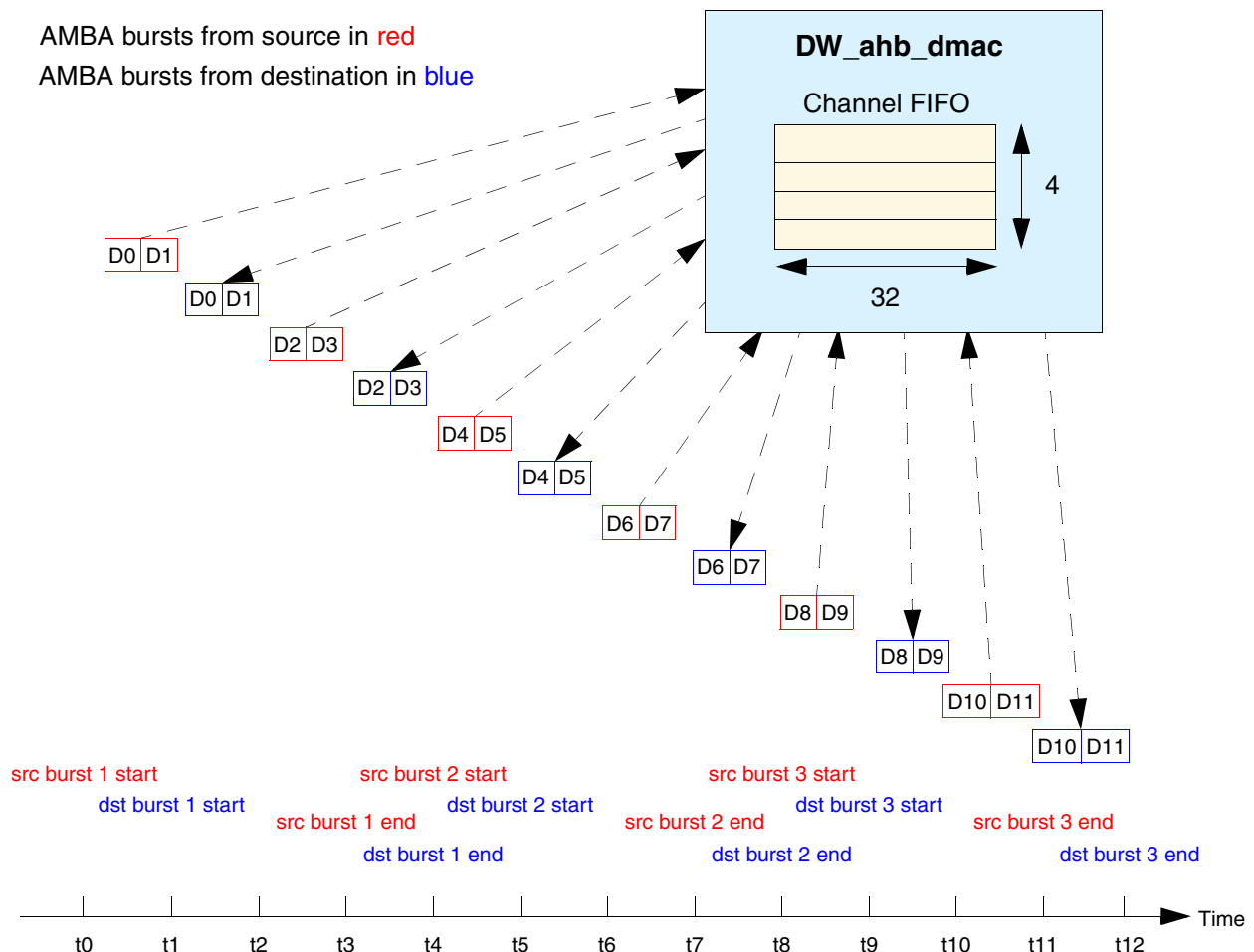
Adhering to the above recommendation results in a reduced number of bursts per block, which in turn results in improved bus utilization and lower latency for block transfers.

3.9.1.3 Example 3

Scenario: Effect of the maximum AMBA burst length, CFGx. MAX_ABRST. This example is the same for both software and hardware handshaking interfaces.

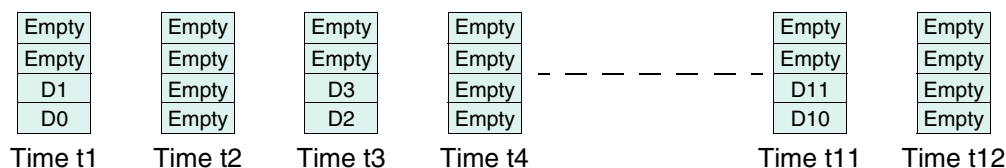
If the CFGx. MAX_ABRST = 2 parameter and all other parameters are left unchanged from [Example 1](#), [Table 3-5](#) on page 62, then the block transfer would look like that shown in [Figure 3-16](#).

Figure 3-16 Breakdown of Block Transfer where max_abrst = 2, Case 1



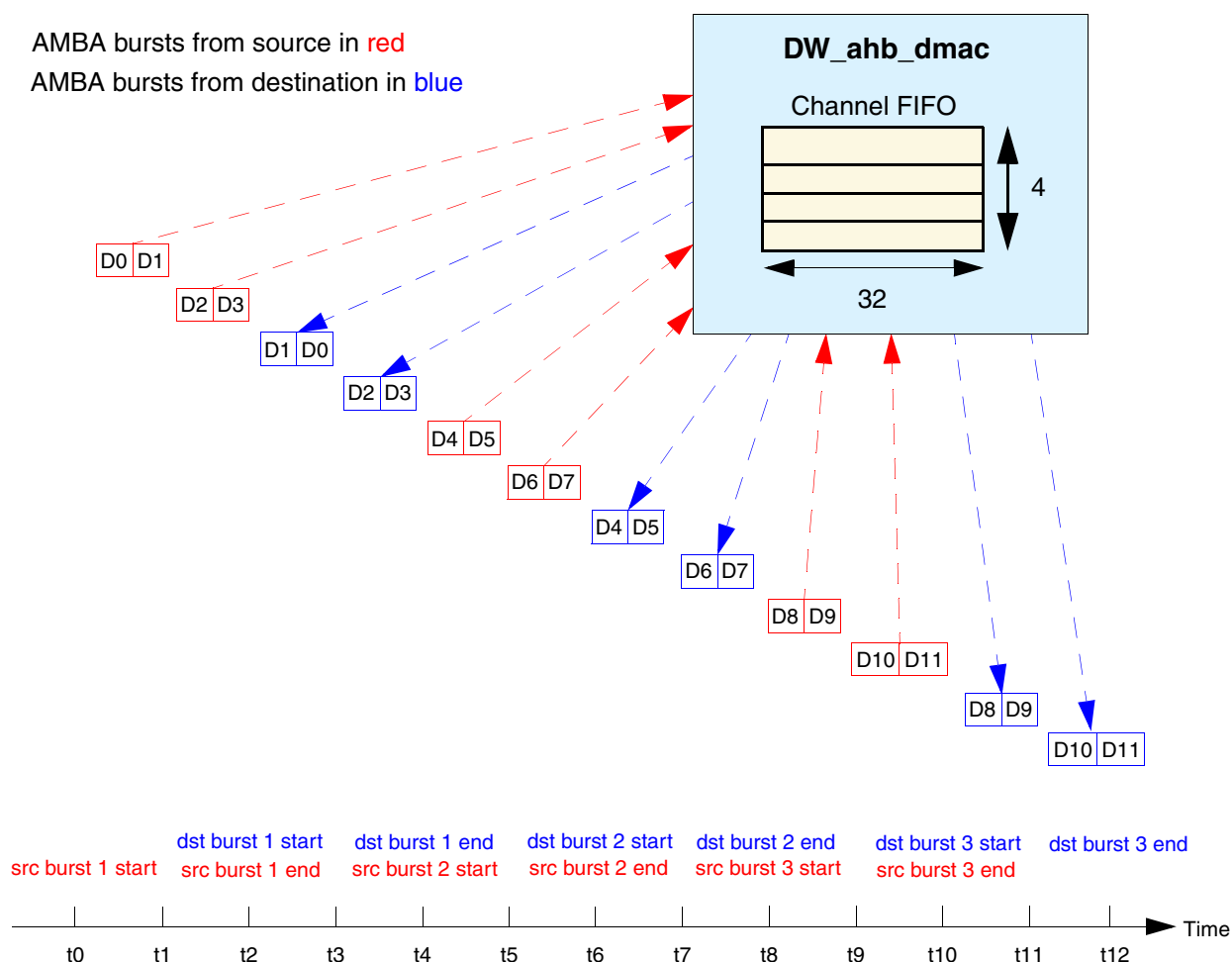
The channel FIFO is alternatively half filled by a burst from the source, and then emptied by a burst to the destination until the block transfer has completed; this is illustrated in [Figure 3-17](#).

Figure 3-17 Channel FIFO Contents at Times Indicated in [Figure 3-16](#)



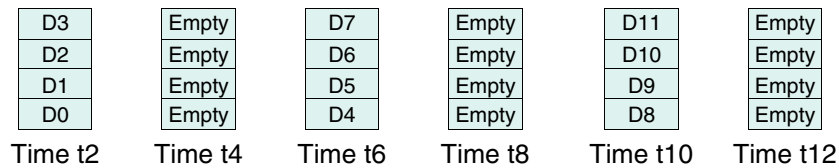
In this example block transfer, each source or destination burst transaction is made up of two bursts, each of length 2. As [Figure 3-17](#) illustrates, the top two channel FIFO locations are redundant for this block transfer. However, this is not the general case. The block transfer could proceed as indicated in [Figure 3-18](#).

Figure 3-18 Breakdown of Block Transfer where max_abrst = 2, Case 2



This depends on the timing of the source and destination transaction requests, relative to each other. [Figure 3-19](#) illustrates the channel FIFO status for [Figure 3-18](#).

Figure 3-19 Channel FIFO Contents at Times Indicated in [Figure 3-18](#)



Recommendation: To allow a burst transaction to complete in a single burst, the following should be true:

$$\text{CFGx.MAX_ABRST} \geq \max(\text{src_burst_size_bytes}, \text{dst_burst_size_bytes})$$

Adhering to the above recommendation results in a reduced number of bursts per block, which in turn results in improved bus utilization and lower latency for block transfers. Limiting a burst to a maximum length prevents the DW_ahb_dmac from saturating the AHB bus when the DW_ahb system arbiter is configured to only allow changing of the grant signals to bus masters at the end of an undefined length burst. It also prevents a channel from saturating a DW_ahb_dmac master bus interface. For more information, refer to “[Arbitration for AHB Master Interface](#)” on page 105.

3.9.1.4 Example 4

Scenario: Source peripheral enters [Single Transaction Region](#); the DW_ahb_dmac is the flow controller.

This example is the same for both hardware and software handshaking and demonstrates how a block from the source can be completed using a series of single transactions. It also demonstrates how the watermark level that triggers a burst request in the source peripheral can be dynamically adjusted so that the block transfer from the source completes with an [Early-Terminated Burst Transaction](#). [Table 3-6](#) lists the parameters used in this example.

Table 3-6 Parameters in Transfer Operation – Example 4

Parameter	Comment
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral transfer with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 12	–
CTLx.SRC_TR_WIDTH = 3'b010	32 bit
CTLx.DST_TR_WIDTH = 3'b010	32 bit
CTLx.SRC_MSIZ = 3'b010	Source burst transaction length = 8
CTLx.DEST_MSIZ = 3'b001	Destination burst transaction length = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 16 bytes	–

In this case, BLOCK_TS is not a multiple of the source burst transaction length, $CTLx.SRC_MSIZE$, so near the end of a block transfer from the source, the amount of data left to be transferred is less than $src_burst_size_bytes$.

In this example, the block size is a multiple of the destination burst transaction length:

$$blk_size_bytes_dma/dst_burst_size_bytes = 48/16 = integer$$

The destination block is made up of three burst transactions to the destination and does not enter the [Single Transaction Region](#).

The block size is not a multiple of the source burst transaction length:

$$blk_size_bytes_dma/src_burst_size_bytes = 48/32 \neq integer$$

Consider the case where the watermark level that triggers a source burst request in the source peripheral is equal to $CTLx.SRC_MSIZE = 8$; that is, eight entries or more need to be in the source peripheral FIFO in order to trigger a burst request.

[Figure 3-20](#) shows how this block transfer is broken into burst and single transactions, and bursts and single transfers.

Figure 3-20 Breakdown of Block Transfer

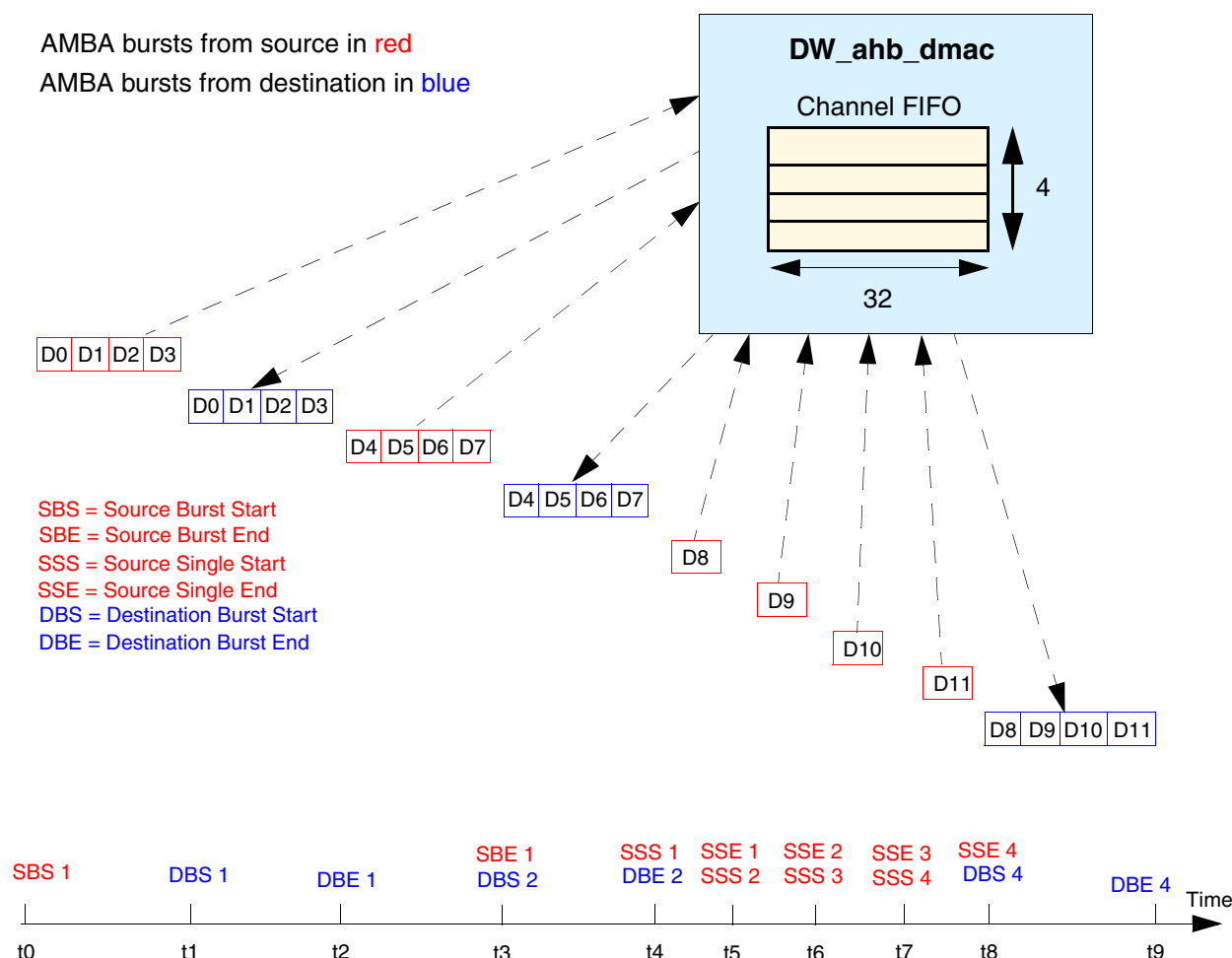
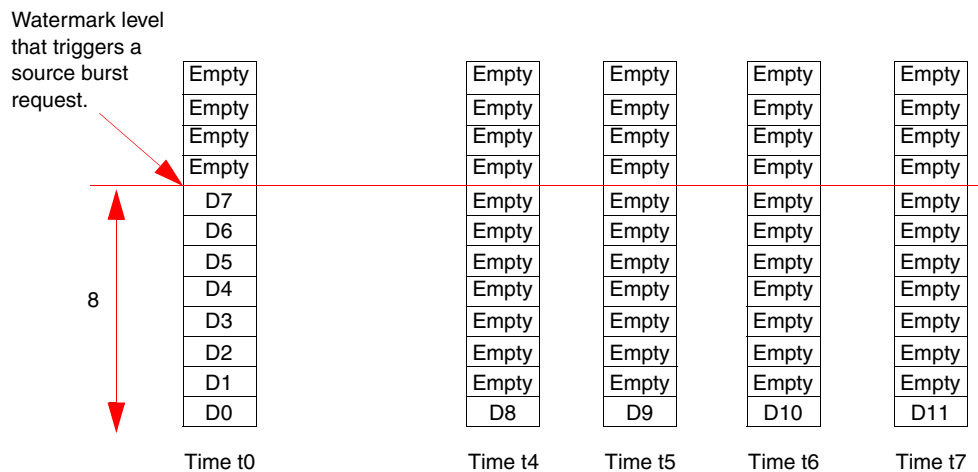


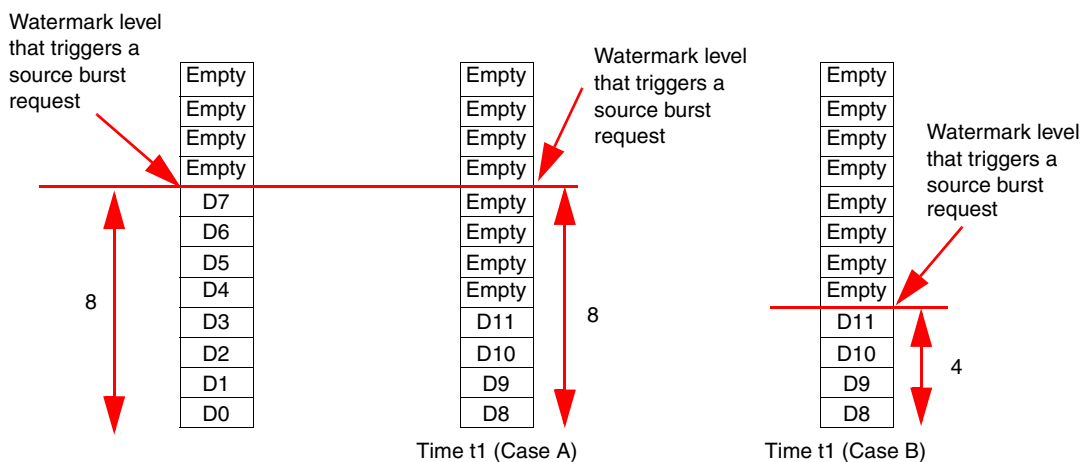
Figure 3-21 shows the status of the source FIFO at various times throughout the source block transfer.

Figure 3-21 Source FIFO Contents at Time Indicated in Figure 3-20



As shown in Figure 3-22, if the DW_ahb_dmac does not perform single transactions, the source FIFO contains four entries at time t1. However, the source has no more data to send. Therefore, if the watermark level remains at 8 (at time t1, Case A in Figure 3-22), the watermark level is never reached and a new burst request is never triggered.

Figure 3-22 Source FIFO Contents where Watermark Level is Dynamically Adjusted



The source peripheral, not knowing the length of a block and only able to request burst transactions, sits and waits for the FIFO level to reach a watermark level before requesting a new burst transaction request. This region, where the amount of data left to transfer in the source block is less than *src_burst_size_bytes*, is known as the [Single Transaction Region](#).

In the Single Transaction Region, the DW_ahb_dmac performs single transactions from the source peripheral until the source block transfer has completed. In this example, the DW_ahb_dmac completes the source block transfer using four single transactions from the source.

Now consider Case B in Figure 3-22, where the source peripheral can dynamically adjust the watermark level that triggers a burst transaction request near the end of a block. After the first source burst transaction

completes, the source peripheral recognizes that it has only four data items left to complete in the block and adjusts the FIFO watermark level that triggers a burst transaction to 4. This triggers a burst request, and the block completes using a burst transaction. However, CTLx.SRC_MSIZE = 8, and there are only four data items left to transfer in the source block. The DW_ahb_dmac terminates the last source burst transaction early and fetches only four of the eight data items in the last source burst transaction. This is called an [Early-Terminated Burst Transaction](#).

Observation: Under certain conditions, it is possible to hardcode dma_single from the source peripheral to an inactive level (hardware handshaking). Under the same conditions, it is possible for software to complete a source block transfer without initiating single transactions from the source. For more information, refer to [“Single Transactions – Peripheral Is Not Flow Controller”](#) on page 55.

3.9.1.5 Example 5

Scenario: The destination peripheral enters the [Single Transaction Region](#) while the DW_ahb_dmac is the flow controller. This example also demonstrates how the DW_ahb_dmac channel FIFO is flushed at the end of a block transfer to the destination; this example is the same for both hardware and software handshaking.

Consider the case with the parameters set to values listed in [Table 3-7](#).

Table 3-7 Parameters in Transfer Operation – Example 5

Parameter	Comment
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral transfer with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 44	–
CTLx.SRC_TR_WIDTH = 3'b000	8 bit
CTLx.DST_TR_WIDTH = 3'b 011	64bit
CTLx.SRC_MSIZE = 3'b001	Source burst transaction length = 4
CTLx.DEST_MSIZE = 3'b001	Destination burst transaction length = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 32 bytes	–

In this example, the block size is a multiple of the source burst transaction length:

$$blk_size_bytes_dma/src_burst_size_bytes = (44 * 1)/4 = 11 = integer$$

The source block transfer is completed using only burst transactions, and the source does not enter the [Single Transaction Region](#).

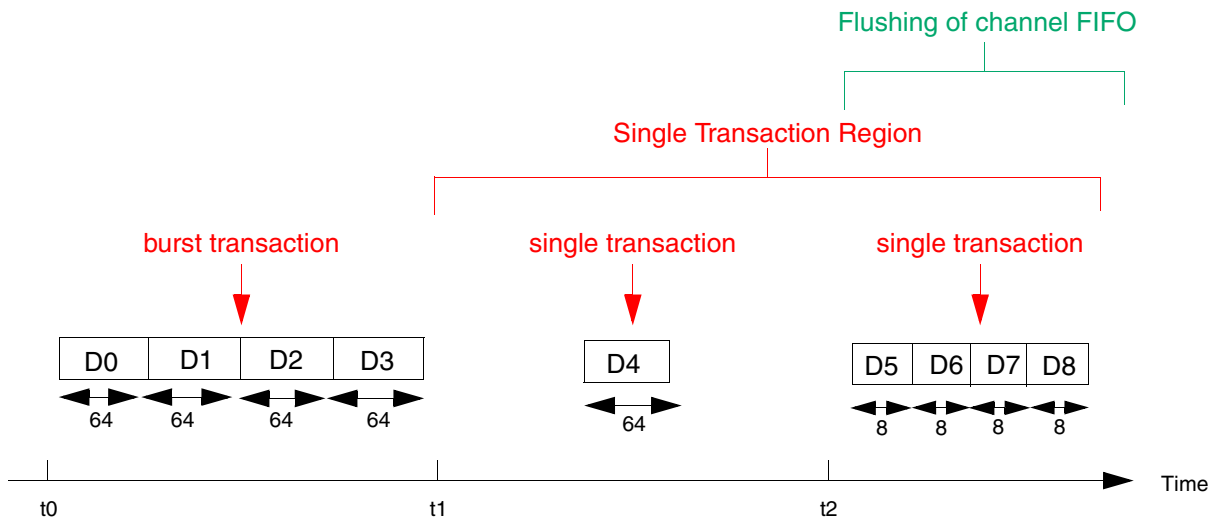
The block size is not a multiple of the destination burst transaction length:

$$blk_size_bytes_dma/dst_burst_size_bytes 44/32 \neq integer$$

So near the end of the block transfer to the destination, the amount of data left to be transferred is less than *dst_burst_size_bytes* and the destination enters the [Single Transaction Region](#).

Figure 3-23 shows one way in which the block transfer to the destination can occur.

Figure 3-23 Block Transfer to Destination



After the first 32 bytes ($dst_burst_size_bytes = 32$) of the destination burst transaction have been transferred to the destination, there are 12 bytes ($blk_size_bytes_dma - dst_burst_size_bytes = 44 - 32$) left to transfer. This is less than the amount of data that is transferred in a destination burst ($dst_burst_size_bytes = 32$). Therefore, the destination peripheral enters the [Single Transaction Region](#) where the DW_ahb_dmac can complete a block transfer to the destination using single transactions.



Note

- In the Single Transaction Region, asserting `dma_single` initiates a single transaction for hardware handshaking. Writing a 1 to the relevant channel bit of the [SglReqDstReg](#) register initiates a single transaction for software handshaking.
- The destination peripheral, not knowing the length of a block and only able to request burst transactions, sits and waits for the FIFO to fall below a watermark level before requesting a new burst transaction request.

At time t_2 in [Figure 3-23](#), a single transaction to the destination has been completed. There are now only four bytes ($12 - dst_single_size_bytes = 12 - 8$) left to transfer in the destination block. However, `CTLx.DST_TR_WIDTH` implies 64-bit AHB transfers to the destination ($dst_single_size_bytes = 8$ byte); therefore, the DW_ahb_dmac cannot form a single word of the specified `CTLx.DST_TR_WIDTH`.

The DW_ahb_dmac channel FIFO has four bytes in it that must be flushed to the destination. The DW_ahb_dmac switches into a “FIFO flush mode,” where the block transfer to the destination is completed by changing the AHB transfer width to the destination to be equal to that of the `CTLx.SRC_TR_WIDTH`; that is, byte AHB transfers in this example. Thus the last single transaction in the destination block is made up of a burst of length 4 and `CTLx.SRC_TR_WIDTH` width.

When the DW_ahb_dmac is in FIFO flush mode, the address on `haddr` is incremented by the value of `hsize` on the bus; that is, `CTLx.SRC_TR_WIDTH`, and not `CTLx.DST_TR_WIDTH`. In cases where the [DARx](#) is selected to be contiguous between blocks (refer to [Table 8-1](#) on page 249 and “[Programming Examples](#)” on page 257), the [DARx](#) will need re-alignment at the start of the next block, since it is aligned to

`CTLx.SRC_TR_WIDTH` and not `CTLx.DST_TR_WIDTH` at the end of the previous block. This is handled by hardware. For more information, refer to “[Hardware Realignment of SAR/DAR Registers](#)” on page 166.

In general, channel FIFO flushing to the destination occurs if all three of the following are true:

- DW_ahb_dmac or the Source peripheral are flow control peripherals
- `CTLx.DST_TR_WIDTH > CTLx.SRC_TR_WIDTH`
- Flow control device:
 - If DW_ahb_dmac is flow controller:

$$\text{blk_size_bytes_dma} / \text{dst_single_size_bytes} \neq \text{integer}$$
 - If source is flow controller:

$$\text{blk_size_bytes_src} / \text{dst_single_size_bytes} \neq \text{integer}$$



Note

When not in FIFO flush mode, a single transaction is mapped to a single AHB transfer. However, in FIFO flush mode, a single transaction is mapped to multiple AHB transfers of `CTLx.SRC_TR_WIDTH` width. The cumulative total of data transferred to the destination in FIFO flush mode is less than `dst_single_size_bytes`.

In the above example, a burst request is not generated in the Single Transaction Region. If a burst request were generated at time t1 in [Figure 3-23](#), then the burst transaction would proceed until there was not enough data left in the destination block to form a single data item of `CTLx.DST_TR_WIDTH` width. The burst transaction would then be early-terminated. In this example, only one data item of the four requested (decoded value of `DEST_MIZE = 4`) would be transferred to the destination in the burst transaction. This is referred to as an [Early-Terminated Burst Transaction](#). If a burst request were generated at time t2 in [Figure 3-23](#), then the destination block would be completed (four byte transfers to the destination to flush the DW_ahb_dmac channel FIFO) and this burst request would again be early-terminated at the end of the destination block.

Observation: If the source transfer width – `CTLx.SRC_TR_WIDTH` in the channel control register (`CTLx`) – is less than the destination transfer width (`CTLx.DST_TR_WIDTH`), then the FIFO may need to be flushed at the end of the block transfer. This is done by setting the AHB transfer width of the last few AHB transfers of the block to the destination so that it is equal to `CTLx.SRC_TR_WIDTH` and not the programmed `CTLx.DST_TR_WIDTH`.

3.9.1.6 Example 6

Scenario: In all examples presented so far, none of the bursts have been early-terminated by the DW_ahb system arbiter. Referring to [Example 1](#), the AHB transfers on the source and destination side look somewhat symmetric. In the examples presented so far, where the bursts are not early-terminated by the DW_ahb system arbiter, the traffic profile on the AHB bus would be the same, regardless of the value of `CFGx.FIFO_MODE`. This example, however, considers the effect of `CFGx.FIFO_MODE`; it is the same for both hardware and software handshaking.

CFGx.FIFO_MODE: Determines how much space or data needs to be available in the FIFO before a burst transaction request is serviced.

0 = Space/data available for single AHB transfer of the specified transfer width.

1 = Data available is greater than or equal to half the FIFO depth for destination transfers and space available is greater than half the fifo depth for source transfers. The exceptions are at the end of a burst transaction request or at the end of a block transfer.

Table 3-8 lists the parameters used in this example.

Table 3-8 Parameters in Transfer Operation – Example 6

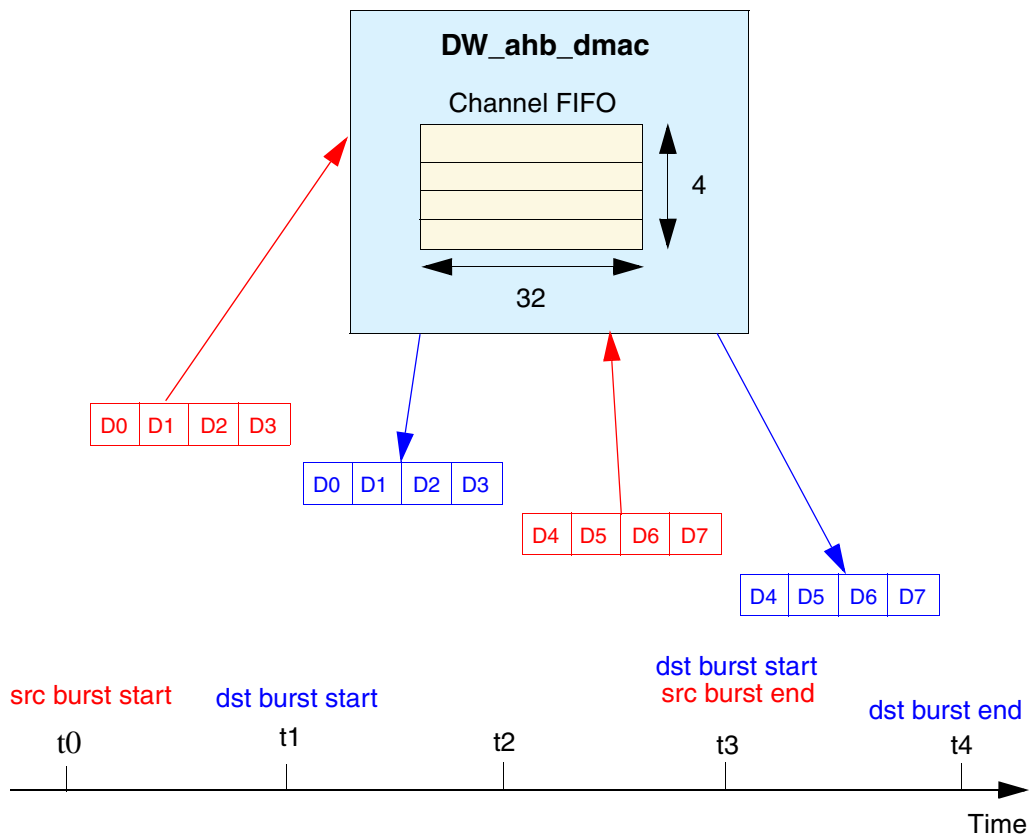
Parameter	Comment
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral transfer with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 32	–
CTLx.SRC_TR_WIDTH = 3'b010	32 bit
CTLx.DST_TR_WIDTH = 3'b010	32 bit
CTLx.SRC_MSIZ = 3'b010	Decoded value = 8
CTLx.DEST_MSIZ = 3'b001	Decoded value = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 16 bytes	–

The block transfer may proceed by alternately filling and emptying the DW_ahb_dmac channel FIFO. Up to time t4, the transfer might proceed like that shown in Figure 3-24.

Figure 3-24 Block Transfer Up to Time “t4”

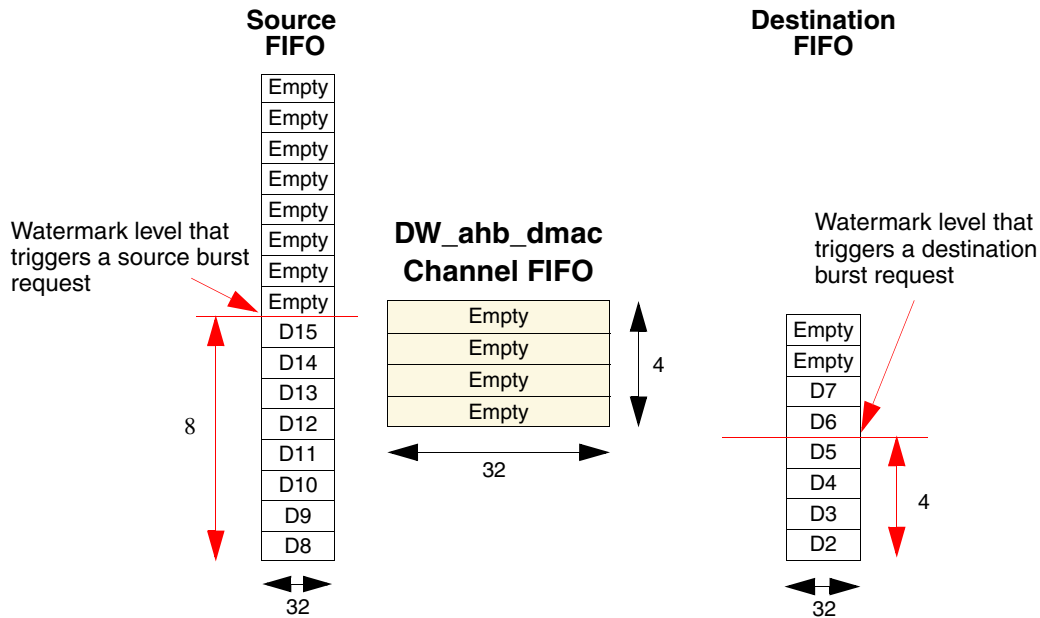
AMBA bursts from source in **red**

AMBA bursts from destination in **blue**



At time t4, the src, channel, and destination FIFOs might look like that shown in [Figure 3-25](#)

Figure 3-25 Source, DW_ahb_dmac Channel and Destination FIFOs at Time 't4' in [Figure 3-22](#)



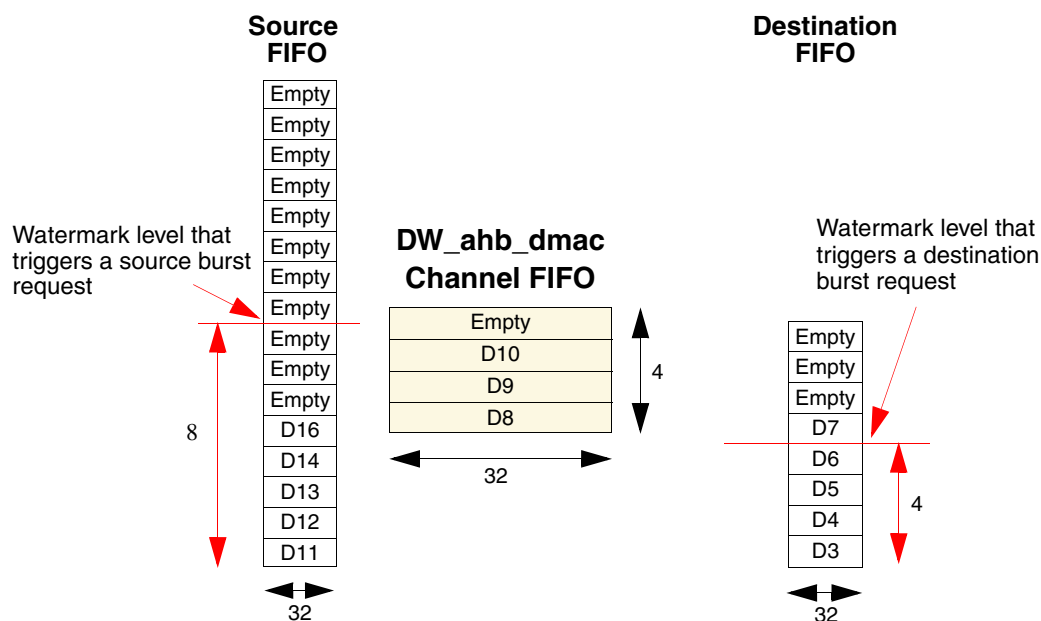
At time t4, a source burst transaction is requested, and the DW_ahb_dmac attempts a burst of length 4. Suppose that this burst is early-burst terminated after three AHB transfers. The FIFO status after this burst might look like that shown in [Figure 3-26](#).

Referring to [Figure 3-26](#), notice that a burst request from the destination is not triggered, since the destination FIFO contents are above the watermark level. The DW_ahb_dmac has space for one data item in the channel FIFO. So, does the DW_ahb_dmac initiate a single AHB transfer from the source peripheral to fill the channel FIFO?

The answer depends on the value of `CFGx.FIFO_MODE`. If `CFGx.FIFO_MODE = 0`, then the DW_ahb_dmac attempts to perform a single AHB transfer in order to fill the channel FIFO. If `CFGx.FIFO_MODE = 1`, then

the DW_ahb_dmac waits until the channel FIFO is less than half-full before initiating a burst from the source, as illustrated in Figure 3-26.

Figure 3-26 FIFO Status After Early-Terminated Burst



Observation: When $\text{CFGx.FIFO_MODE} = 1$, the number of bursts per block is less than when $\text{CFGx.FIFO_MODE} = 0$ and, hence, the bus utilization will improve. This setting favors longer bursts. However, the latency of DMA transfers may increase when $\text{CFGx.FIFO_MODE} = 1$, since the DW_ahb_dmac waits for the channel FIFO contents to be less than half the FIFO depth for source transfers, or greater than or equal to half the FIFO depth for destination transfers. Therefore, system bus occupancy and usage can be improved by delaying the servicing of multiple requests until there is sufficient data/space available in the FIFO to generate a burst (rather than multiple single AHB transfers); this comes at the expense of transfer latency. For reduced block transfer latency, set $\text{CFGx.FIFO_MODE} = 0$. For improved bus utilization, set $\text{CFGx.FIFO_MODE} = 1$.

3.9.1.7 Example 7

Scenario: Example block transfer when the destination is the flow controller; the effect of data pre-fetching ($\text{CFGx.FCMODE} = 0$) and possible data loss. This example uses a hardware handshaking interface, but the same scenario can be explained using a software handshaking interface.



Note

Data pre-fetching is when data is fetched from the source before the destination requests it.

Flow Control Mode. Determines when source transaction requests are serviced when the Destination Peripheral is the flow controller.

0 = Source transaction requests are serviced when they occur. Data pre-fetching is enabled.

1 = Source transaction requests are not serviced until a destination transaction request occurs. In this mode, the amount of data transferred from the source is limited such that it is guaranteed to be

transferred to the destination prior to block termination by the destination. Data pre-fetching is disabled.

Table 3-9 lists the parameters used in this example.

Table 3-9 Parameters in Transfer Operation – Example 7

Parameter	Description
CTLx.TT_FC=3'b111	Peripheral-to-peripheral transfer with destination as flow controller
CTLx.BLOCK_TS = x	–
CTLx.SRC_TR_WIDTH = 3'b010	32 bit
CTLx.DEST_TR_WIDTH=3'b010	32 bit
CTLx.SRC_MSIZ = 3'b010	Decoded value = 8
CTLx.DEST_MSIZ = 3'b001	Decoded value = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 32	–
CFGx.FCMODE = 0	Data pre-fetching enabled
CFGx.SRC_PER = 0	Source assigned handshaking interface 0
CFGx.DEST_PER = 1	Destination assigned handshaking interface 1
CFGx.MAX_ABRST = 7	–

Consider a case where the destination block is made up of one burst transaction, followed by one single transaction.

$$blk_size_bytes_dst = dst_burst_size_bytes = 16 + 4 = 20 \text{ bytes}$$

There are a number of different cases that can arise when CFGx.FCMODE = 0:

1. When the destination peripheral signals a last transaction, there is enough data in the DW_ahb_dmac channel FIFO to complete the last transaction to the destination. Therefore, the DW_ahb_dmac stops transferring data from the source, and the block transfer from the source completes; any surplus data that has been fetched from the source is effectively lost. Two cases arise when the destination peripheral signals the last transaction in a block:
 - a. Active burst transaction request on source side
 - b. No active burst request on source side
2. When the destination peripheral signals a last transaction, there is not enough data in the channel FIFO to complete the last transaction to the destination. The DW_ahb_dmac fetches just enough data to complete the block transfer. Two cases arise.
 - a. Source enters [Single Transaction Region](#) when destination peripheral signals last transaction.
 - b. Source does not enter [Single Transaction Region](#) when destination peripheral signals last transaction.

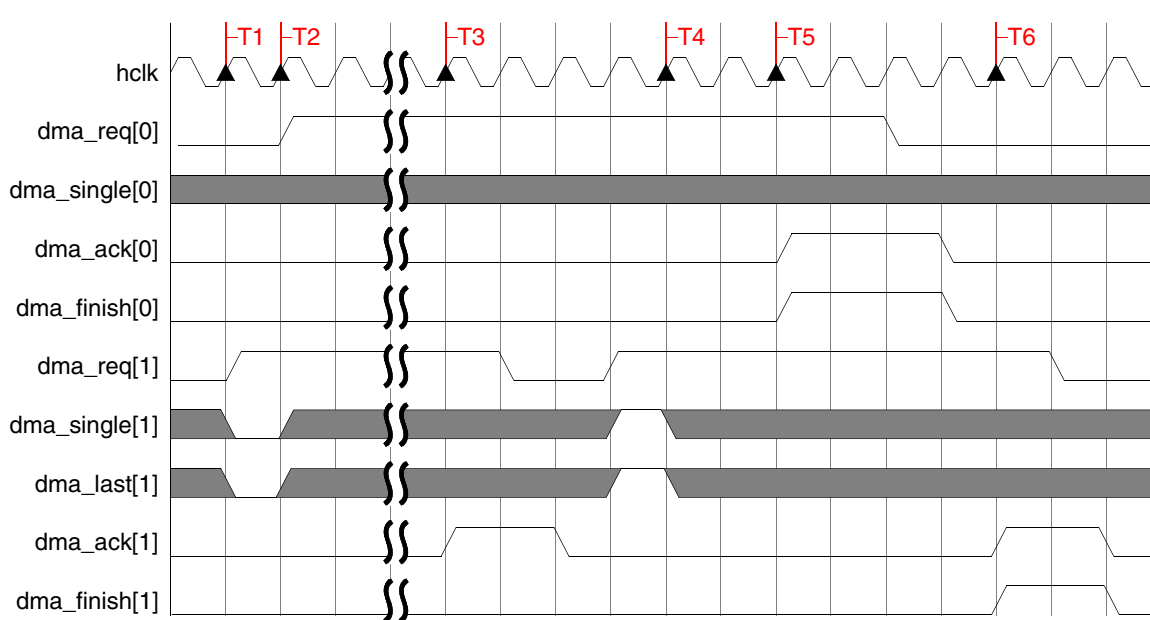
Setting CFGx.FCMODE is pertinent only when the destination peripheral is the flow controller. When CFGx.FCMODE = 0, scenarios arise where not all the data that has been pre-fetched from the source is required to complete the block transfer to the destination. This excess data is not transferred to the destination peripheral and is effectively lost for a read-sensitive source peripheral. In this example, we assume a read-sensitive source peripheral.

Case [a](#) and Case [b](#) highlight instances where data pre-fetching is enabled and data is lost. Case [a](#) and Case [b](#) highlight instances where data pre-fetching is enabled, but no data loss occurs.

In this example, handshaking interface 0 is assigned to the source peripheral, and handshaking interface 1 is assigned to the destination peripheral.

Consider the block transfer shown in [Figure 3-27](#), where the destination is the flow controller and data pre-fetching is enabled (CFGx.FCMODE = 0).

Figure 3-27 Data Loss when Pre-Fetching is Enabled



The source requests a burst transaction at time T2. The destination requests a burst transaction at time T1 and completes this burst request at time T3. At time T4, the destination requests a single transaction, which is to be the last in the block transfer. Suppose that at time T4 the DW_ahb_dmac has fetched seven words from the source and written four words to the destination. Therefore, the DW_ahb_dmac has three words in the channel FIFO, but the destination requires only one word to complete the block transfer.

The DW_ahb_dmac, recognizing that it has enough data in the channel FIFO to complete the block transfer to the destination, fetches no more data from the source and early-terminates the source burst transaction (only seven of the eight data items in the source burst transaction have been fetched from the source) – [Early-Terminated Burst Transaction](#). The DW_ahb_dmac asserts dma_finish[0] to the source at time T5, and this has the same timing as dma_ack[0], as shown in [Figure 3-27](#).

At time T6, the last single transaction to the destination has completed, which has removed one of the remaining three data words in the channel FIFO. At this time, both the source and destination block transfers have completed, and there remain two data words in the channel FIFO that have been fetched

from the source. These two data words are lost because they do not form the start of the next block transfer for multi-block transfers, since the channel FIFO is cleared between blocks for multi-block transfers.

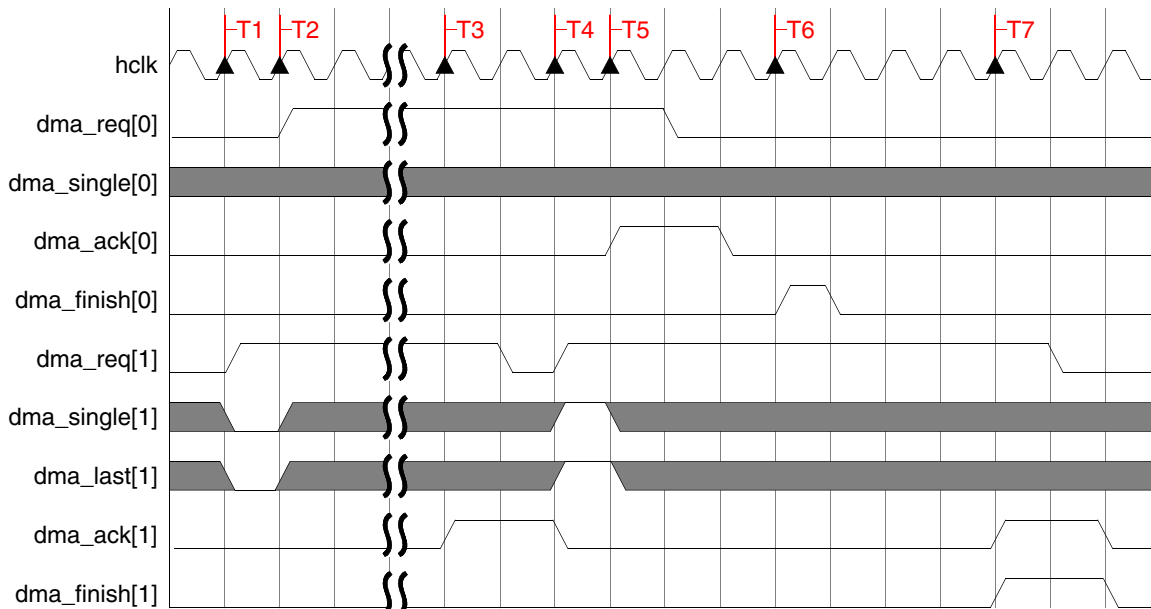
**Note**

There is an exception to Case a. If the last AHB transfer to the source received a SPLIT/RETRY response over the AHB bus, then dma_finish is not asserted until the AHB transfer that received the SPLIT/RETRY response is retried and an OKAY response is received over the hresp AHB bus; to do otherwise would be a violation of the AMBA protocol. This additional word that is fetched is effectively lost, since it is not transferred to the destination.

3.9.1.7.1 Case 1b – Timing exception on dma_finish to the source when data pre-fetching is enabled

Consider the block transfer shown in Figure 3-28, where the destination is the flow controller and data pre-fetching is enabled (CFGx.FCMODE = 0).

Figure 3-28 Timing Exception on dma_finish to Source Peripheral



The source requests a burst transaction at time T2 and completes the burst transaction at time T5. The destination requests a burst transaction at time T1 and completes this burst request at time T3. At time T4, the destination requests a single transaction, which is to be the last in the block transfer. At time T5, the DW_ahb_dmac has completed the burst transaction from the source.

At time T5, the DW_ahb_dmac has fetched eight words from the source and written four words to the destination, which means that the DW_ahb_dmac has four words in the channel FIFO. However, the destination requires only one word to complete the block transfer. The DW_ahb_dmac, recognizing that it has enough data in the channel FIFO to complete the block transfer to the destination, fetches no more data from the source and signals a source block completion by asserting dma_finish[0] for a single cycle at time T6. Since there is no active transaction on the source side – that is, the previous source burst transaction has completed and there has been no new burst request from the source – the dma_finish[0] cannot form a

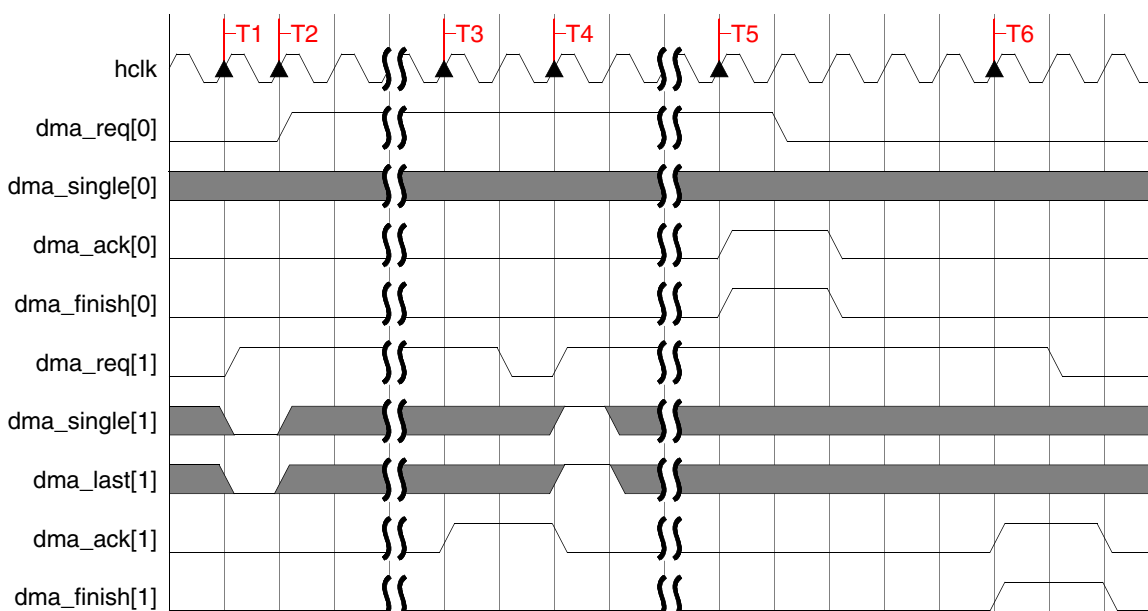
handshaking loop with `dma_req[0]` (there is no active burst request) and therefore is asserted for only a single cycle.

Similar to Case a, when both the source and destination block transfers have completed at time T7, there are three data items left in the channel FIFO that are effectively lost.

3.9.1.7.2 Case 2a – Data pre-fetching enabled but no data loss. Source enters **Single Transaction Region** when destination signals last transaction.

Consider the block transfer as shown in Figure 3-29, where the destination is a flow controller and data pre-fetching is enabled (`CFGx.FCMODE = 0`). The transfer parameters are the same as case 1a, Table 3-9.

Figure 3-29 Case of No Data Loss When Pre-Fetching is Enabled



In this scenario, when `dma_last[1]` is asserted by the destination peripheral at time T4, there is not enough data in the channel FIFO to complete the last single transaction. Assume that the DW_ahb_dmac has fetched four data items from the source peripheral at time T4. In this case, the DW_ahb_dmac fetches one more data item from the source peripheral and then early terminates the source burst using an **Early-Terminated Burst Transaction**. The DW_ahb_dmac signals block completion to the source by asserting `dma_finish[0]` at T5, which forms a handshaking loop with `dma_req[0]`. In this case, there is no data loss, and all data that was fetched from the source has been transferred to the destination.

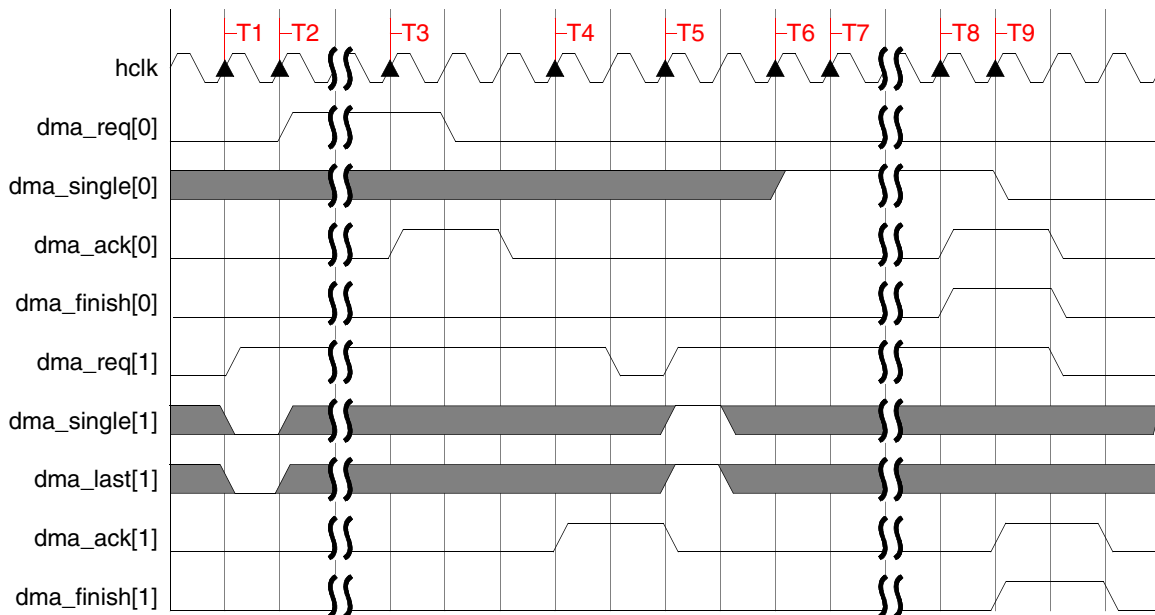
Consider the case where the transfer parameters are as given in Table 3-10.

Table 3-10 Transfer Parameters

Parameter	Description
CTLx.TT_FC=3'b111	Peripheral-to-peripheral transfer with destination as flow controller
CTLx.BLOCK_TS = x	
CTLx.SRC_TR_WIDTH = 3'b010	32 bit

Table 3-10 Transfer Parameters

Parameter	Description
CTLx.DEST_TR_WIDTH=3'b010	32 bit
CTLx.SRC_MSIZ = 3'b001	Decoded value = 4
CTLx.DEST_MSIZ = 3'b001	Decoded value = 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 32	–
CFGx.FCMODE = 0	Data pre-fetching enabled
CFGx.SRC_PER = 0	Source assigned handshaking interface 0
CFGx.DEST_PER = 1	Destination assigned handshaking interface 1
CFGx.MAX_ABRST = 7	–

Figure 3-30 Source Enters Single Transaction Region when Destination Asserts dma_last[1]

As illustrated in [Figure 3-30](#), the source requests a burst transaction at time T2 and completes the burst transaction at time T3. The destination requests a burst transaction at time T1 and completes this burst request at time T4. The destination requests a last single transaction at time T5; the channel FIFO is empty at this time. The amount of data left to complete a source block transfer – 4 bytes – is less than the following:

$$\text{src_burst_size_bytes} = 4 * 4 = 16 \text{ bytes}$$

Therefore, the source enters the [Single Transaction Region](#) at time T6. At time T7, the DW_ahb_dmac samples that `dma_single[0]` from the source peripheral is asserted and initiates a single transaction.

**Note**

If an active level on `dma_req[0]` was triggered at time T6 or T7, a source burst transaction would have taken precedence over the source single transaction. Upon completion of the source block, this burst transaction would have been early-terminated using an Early-Terminated Burst Transaction.

When this transaction completes at time T8, the DW_ahb_dmac recognizes that enough data has been fetched from the source peripheral to complete the block transfer to the destination. The DW_ahb_dmac asserts `dma_finish[0]` to the source peripheral at time T8; this has the same timing as `dma_ack[0]`. The destination block transfer completes as previously described. No data loss occurs.

3.9.1.7.3 Case 2b – Data pre-fetching enabled but no data loss.

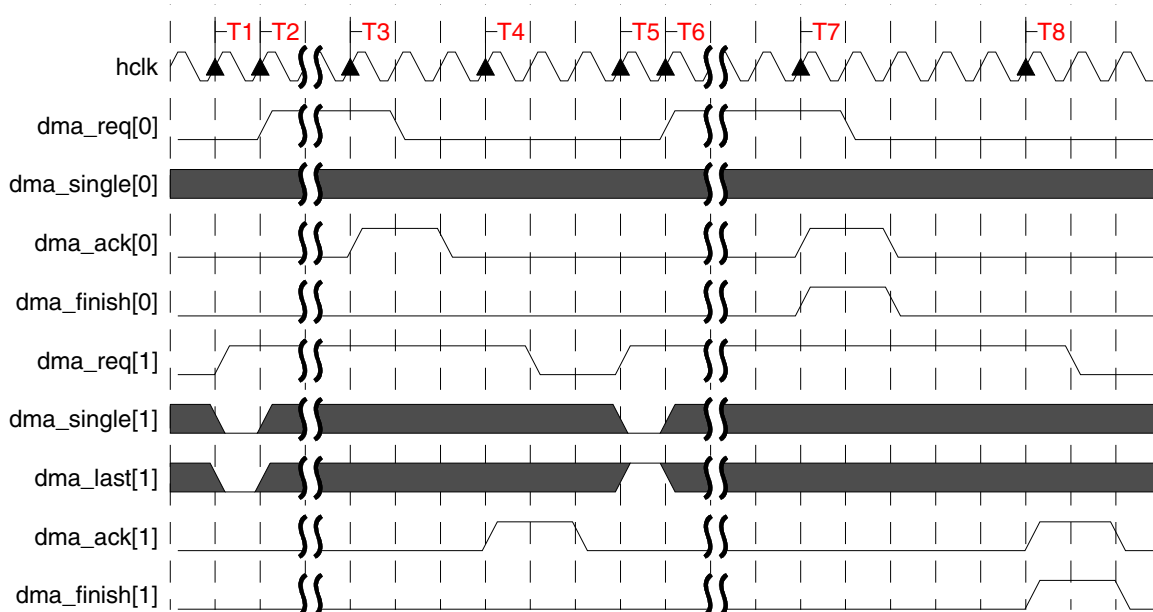
In this case, the source does not enter the [Single Transaction Region](#) when the destination signals the last transaction. This case uses the parameters listed in [Table 3-10](#)

The destination block is made up of two burst transactions.

$$blk_size_bytes_dst = 2 * (4 * 4) = 32 \text{ bytes}$$

As illustrated in [Figure 3-31](#), the source requests a burst transaction at time T2 and completes the burst transaction at time T3. The destination requests a burst transaction at time T1 and completes this burst request at time T4.

Figure 3-31 Case where Source Does Not Enter *Single transaction region* when Destination Asserts `dma_last[1]`



At time T5 the destination peripheral requests the last burst transaction in the block transfer to the destination. At this point, the channel FIFO is empty. The number of bytes that must be fetched from the source peripheral to complete the block transfer to the destination is equal to $4 * 4 = 16$ bytes. Since 16 bytes is not less than `src_msize_bytes` (16 bytes), the source does not enter the [Single Transaction Region](#). The DW_ahb_dmac waits for a burst request from the source peripheral, which occurs at time T6. Upon completion of this burst request at time T7, the DW_ahb_dmac signals a source block transfer completion

and asserts `dma_finish[0]`. Upon completion of the last destination burst transaction at time T8, the DW_ahb_dmac signals a destination block transfer completion and asserts `dma_finish[1]` to the destination.

Note that when data pre-fetching is enabled, `CFGx.FCMODE = 0`, the maximum amount of data that can be lost depends on whether the last transaction in the block transfer to the destination is a single transaction or burst transaction. In the worst case scenario, the DW_ahb_dmac has pre-fetched enough data from the source to fill the channel FIFO when the last transaction is signalled by the destination peripheral.

The maximum amount of data that can be lost is:

- Last transaction in block transfer is a single transaction:

$$\text{DMAH_CHx_FIFO_DEPTH} - \text{dst_single_size_bytes} \quad [\text{refer to equation (1)}]$$

- Last transaction in block transfer is a burst transaction:

$$\text{DMAH_CHx_FIFO_DEPTH} - \text{dst_burst_size_bytes} \quad [\text{refer to equation (2)}]$$

If this equation is ≤ 0 , then no data is lost.

Thus, if the last transaction in the block is a burst transaction and equation (2) is less than zero, then no data can be lost when `CFGx.FCMODE = 0`. There is one exception to this, as outlined in [Example 8](#).

Enabling data pre-fetching may reduce the latency of the DMA transfer when the destination is the flow controller.

Observation: For a source peripheral that is not read-sensitive (such as memory), data pre-fetching should be enabled – that is, `CFGx.FCMODE = 0` – in order to reduce the transfer latency when the destination is the flow controller. If the source peripheral is a read-sensitive device (such as a source FIFO), then data pre-fetching should be disabled – that is, `CFGx.FCMODE = 1` – when the destination peripheral is the flow controller.

3.9.1.8 Example 8

Scenario: Data loss when destination is flow controller and data pre-fetching is disabled; `CFGx.FCMODE = 1`.

This example uses a hardware handshaking interface, but the same scenario can be explained using a software handshaking interface. Two scenarios arise when `CFGx.FCMODE = 1`:

- `CTLx.SRC_TR_WIDTH ≤ CTLx.DST_TR_WIDTH`
- `CTLx.SRC_TR_WIDTH > CTLx.DST_TR_WIDTH`

3.9.1.8.1 Case 1 – `CTLx.SRC_TR_WIDTH ≤ CTLx.DST_TR_WIDTH`

In this case, the DW_ahb_dmac controls the transfer of data from the source, such that at any time there is at most enough data to complete the current transaction – single or burst – to the destination. If there is currently no active transaction to the destination, then the channel FIFO is empty and no data is pre-fetched from the source, even if the source has an active transaction request. If both the source and destination are requesting, then the DW_ahb_dmac fetches only enough data from the source to complete the current destination transaction, and no more. Therefore, there can never be any data loss.

3.9.1.8.2 Case 2 – CTLx.SRC_TR_WIDTH > CTLx.DST_TR_WIDTH

In this example, assume the parameters in [Table 3-11](#).

Table 3-11 Parameters in Transfer Operation – Example 7, Case 2b

Parameter	Description
CFGx.FCMODE = 1	Data pre-fetching disabled
CTLx.BLOCK_TS = x	–
CTLx.SRC_MSIZE = 3'b001	Decoded value = 4
CTLx.DEST_MSIZE = 3'b010	Decoded value = 8
CTLx.SRC_TR_WIDTH = 3'b011	64-bit
CTLx.DST_TR_WIDTH=3'b010	32-bit
CTLx.TT_FC=3'b111	Peripheral to Peripheral transfer with destination as flow controller
DMAH_CHx_FIFO_DEPTH = 32	–
CFGx.SRC_PER = 0	Source assigned handshaking interface 0
CFGx.DEST_PER = 1	Destination assigned handshaking interface 1
CFGx.MAX_ABRST = 8	–

Consider the case where the destination block is made up of a burst transaction, followed by one single transaction:

$$blk_size_bytes_dst = (8 * 4) + 4 = 36 \text{ bytes}$$

$$src_single_size_bytes = 8$$

$$dst_single_size_bytes = 4$$

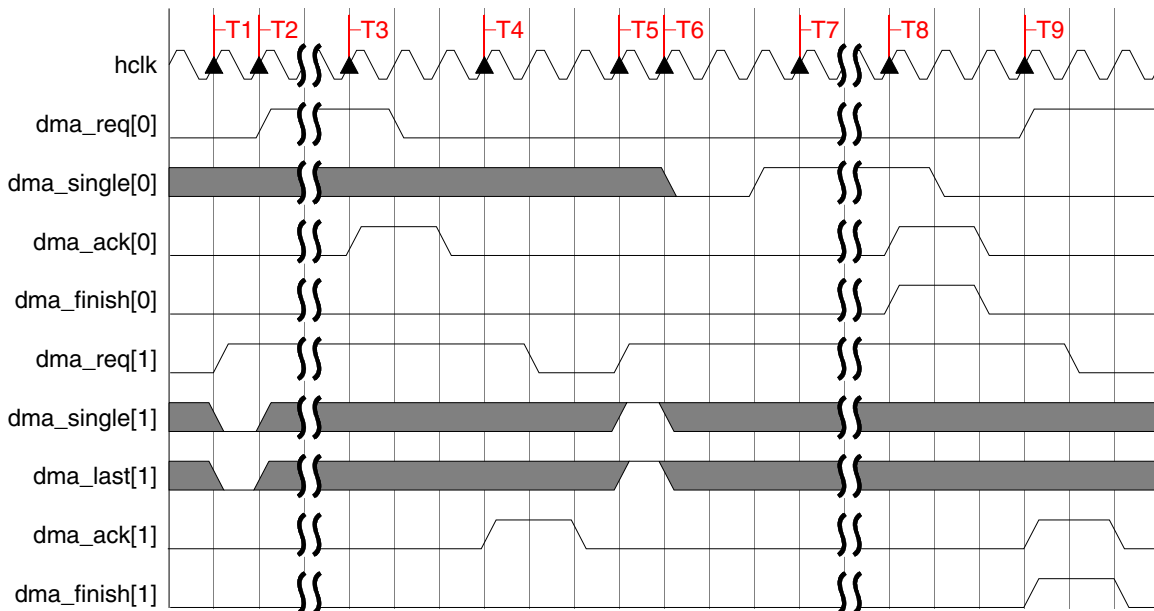
$$src_burst_size_bytes = 4 * 8 = 32$$

$$dst_burst_size_bytes = 8 * 4 = 32$$

As illustrated in [Figure 3-32](#), the source requests a burst transaction at time T2, and completes the burst transaction at time T3. The destination requests a burst transaction at time T1 and completes this burst

request at time T4. The destination requests a last single transaction at time T5. The channel FIFO is empty at this time.

Figure 3-32 Data Loss when Data Pre-Fetching is Disabled



The amount of data left to complete a source block transfer, 4 bytes, is less than *src_burst_size_bytes* (32 bytes). Therefore, the source enters the [Single Transaction Region](#). At time T7, the DW_ahb_dmac samples that *dma_single[0]* is asserted and initiates a single transaction from the source.

The DW_ahb_dmac fetches a single source data item from the source peripheral and stores the eight bytes in the channel FIFO. This single transaction completes at time T8; the source block also completes at time T8. However, the destination requires only four of these eight bytes to complete the block transfer to the destination (the block transfer to destination completes at time T9). The remaining four bytes are lost. Thus, when the destination is the flow controller, data loss occurs when *CFGx.FCMODE* = 1 if both of the following are true:

- $SRC_TR_WITDT > CTLx.DST_TR_WIDTH$
- $blk_size_bytes_dst/src_single_size_bytes \neq \text{integer}$.

The amount of data lost is:

$$src_single_size_bytes - dst_single_size_bytes \quad [\text{refer to equations (2) and (3)}]$$

Observation: Data loss can occur when the destination is the flow controller, even if data pre-fetching is disabled, *CFGx.FCMODE* = 1.

3.9.1.9 Example 9

Scenario: This scenario demonstrates how to use software handshaking on both the source and destination sides when the DW_ahb_dmac is the flow controller. The DW_apb_ssi is used as the source and destination

peripherals; for more information on the DW_apb_ssi, refer to the [DesignWare DW_apb_ssi Databook](#). This example uses the parameters listed in [Table 3-12](#).

Table 3-12 Parameters in Transfer Operation – Example 9

Parameter	Description
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 8	
CTLx.SRC_TR_WIDTH = 3'b001	16 bit
CTLx.DST_TR_WIDTH = 3'b001	16 bit
CTLx.SRC_MSIZ = 3'b 001,	Source burst transaction of length 4
CTLx.DEST_MSIZ = 3'b 010	Destination burst transaction of length 8
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 16 bytes	–
CFGx.HS_SEL_SRC = 1	Source software handshaking
CFGx.HS_SEL_DST = 1	Destination software handshaking

All SSI parameters are prefixed with “SSI”.

The following are some definitions of DW_apb_ssi parameters that are used in this example:

- SSI.TXFTLR: Transmit FIFO Threshold – Controls the level of entries (or below) at which the DW_apb_ssi transmit FIFO controller triggers an ssi_txe_intr interrupt.
- SSI.TXFLR: Transmit FIFO Level – Contains the number of valid data entries in the DW_apb_ssi transmit FIFO.
- SSI.IMR.TXEIM– Writing a 0 to this field masks the ssi_txe_intr interrupt.
- SSI.RXFTLR: Receive FIFO Threshold – Controls the level of entries (or below) at which the DW_apb_ssi transmit FIFO controller triggers an ssi_rxf_intr interrupt.
- SSI.RXFLR: Receive FIFO Level – Contains the number of valid data entries in the DW_apb_ssi transmit FIFO.
- SSI.IMR.RXFIM – Writing a 0 to this field masks the ssi_rxf_intr interrupt.

In this example:

SSI.TXFTLR = CTLx.DEST_MSIZ
SSI.RXFTLR + 1 = CTLx.SRC_MSIZ

The block transfer takes place on channel 0.

It is assumed that the transaction complete interrupts, `IntDstTran` and `IntSrcTran`, are enabled and unmasked at the beginning of the block transfer. For example:

```
MaskSrcTran[0] = 1
MaskDstTran[0] = 1
CTLx.INT_EN = 1
```

The block size is a multiple of the source burst transaction length:

$$\text{blk_size_bytes_dma/src_burst_size_bytes} = 16/8 = 2 = \text{integer}$$

It is also a multiple of the destination burst transaction length:

$$\text{blk_size_bytes_dma/dst_burst_size_bytes} = 16/16 = 1 = \text{integer}$$

This block consists of two source burst transactions and one destination burst transaction. Neither the source nor destination enter the [Single Transaction Region](#). The block transfer proceeds as follows:

1. `SSI.TXFLR` is initially equal to 0 so that an `ssi_txe_intr` interrupt is generated by the `DW_apb_ssi`. The Interrupt Service Routine (ISR) for this interrupt writes hex 0101 to both the [ReqDstReg](#) and [SglReqDstReg](#) registers; the order is not important. This generates a destination burst transaction request. Before exiting, this ISR software should write a 0 to the `SSI.IMR.TXEIM` register in order to mask any further `ssi_txe_intr` interrupts, since these are level-sensitive interrupts.
2. When the `DW_apb_ssi` receive FIFO contains greater than or equal to `SSI.RXFTLR + 1` half-words, an `ssi_rxf_intr` interrupt is generated by the `DW_apb_ssi`. The Interrupt Service Routine (ISR) for this interrupt writes hex 0101 to both the [ReqSrcReg](#) and [SglReqSrcReg](#) registers; the order is not important. This generates a source burst transaction request. Before exiting this ISR, software should write a 0 to the `SSI.IMR.RXFIM` register in order to mask any further `ssi_rxf_intr` interrupts, since this is a level-sensitive interrupt.
3. Upon completion of the source burst transaction, the `DW_ahb_dmac` clears [ReqSrcReg\[0\]](#) and [SglReqSrcReg\[0\]](#), and generates an `IntSrcTran` interrupt. The ISR for this interrupt should write a 1 to the `SSI.IMR.RXFIM` register in order to unmask the generation of `ssi_rxf_intr` interrupts.
4. Same as step 2.
5. Same as step 3.
6. Upon completion of the destination burst transaction, the `DW_ahb_dmac` clears [ReqDstReg\[0\]](#) and [SglReqDstReg\[0\]](#), and generates an `IntDstTran` interrupt. The ISR for this interrupt should write a 1 to the `SSI.IMR.TXEIM` register to unmask the generation of `ssi_txe_intr` interrupts.



Note

An alternative to using interrupts is for software to poll the `DW_apb_ssi` FIFO levels, `SSI.TXFLR/SSI.RXFLR`, until they equal `CTLx.DEST_MSIZE/CTLx.SRC_MIZE` in place of `ssi_txe_intr/ssi_rxf_intr` interrupts. Also, in place of `IntSrcTran/IntDstTran` interrupts, software could poll the [ReqSrcReg\[0\]/ReqDstReg\[0\]](#) registers until cleared by hardware.

3.9.1.10 Example 10

Scenario: This scenario demonstrates how to use software handshaking on both the source and destination side when `DW_ahb_dmac` is the flow controller and the source peripheral enters the [Single Transaction](#)

Region. The DW_apb_ssi is used as the source and destination peripherals; for more information on the DW_apb_ssi, refer to the [DesignWare DW_apb_ssi Databook](#).

This example uses the parameters listed in [Table 3-13](#).

Table 3-13 Parameters in Transfer Operation – Example 10

Parameter	Description
CTLx.TT_FC = 3'b011	Peripheral-to-peripheral DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 12	–
CTLx.SRC_TR_WIDTH = 3'b00	16 bit
CTLx.DST_TR_WIDTH = 3'b001	16 bit
CTLx.SRC_MSIZ = 3'b 010	Source burst transaction of length 8
CTLx.DEST_MSIZ = 3'b 001	Destination burst transaction of length 4
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 8 bytes	–
CFGx.HS_SEL_SRC = 1	Source software handshaking
CFGx.HS_SEL_DST = 1	Destination software handshaking

The block size is not a multiple of the source burst transaction length:

$$blk_size_bytes_dma/src_burst_size_bytes = 24/16 \neq integer$$

Therefore, the block transfer from the source enters the [Single Transaction Region](#) near the end of a block.

The block size is a multiple of the destination burst transaction length:

$$blk_size_bytes_dma/dst_burst_size_bytes = 24/8 = 3 = integer$$

Therefore, the block transfer to the destination consists of three burst transactions, and the destination does not enter the [Single Transaction Region](#).

In this example:

$$SSI.TXFTLR = CTLx.DEST_MSIZ$$

3.9.1.10.1 ssi_rxf_intr Interrupt Service Routine (ISR)

1. Read the SSI.RXFTLR register.

If SSI.RXFTLR = 0:

- a. Write hex 0101 to the [SglReqSrcReg](#) register.
- b. Write 7 (CTLx.SRC_MSIZ - 1) into the SSI.RXFTLR register, which triggers a new ssi_rxf_intr interrupt when the data items in the source FIFO are greater than or equal to 8.

else (SSI.RXFTLR = 7)

- c. Write 0 to the SSI.IMR.RXFIM register in order to mask any further ssi_rxf_intr interrupts in the ISR for the ssi_rxf_intr interrupt.
- d. Write hex 0101 to the [ReqSrcReg](#) register.

3.9.1.10.2 IntSrcTran Interrupt Service Routine (ISR)

1. Write 0 into the SSI.RXFTLR register, which triggers an ssi_rxf_intr interrupt when a single data item is in the DW_apb_ssi receive FIFO.
2. Write a 1 to the SSI.IMR.RXFIM register in order to unmask the generation of ssi_rxf_intr interrupts.

The DMA block transfer might proceed as follows:

1. Software writes a value of 0 into the SSI.RXFTLR register, which triggers an ssi_rxf_intr interrupt when a single data item is in the DW_apb_ssi receive FIFO.
2. SSI.TXFLR is initially equal to 0, so an ssi_txe_intr interrupt is generated by DW_apb_ssi. The Interrupt Service Routine (ISR) for this interrupt writes hex 0101 to both the [ReqDstReg](#) and [SglReqDstReg](#); the order is not important. This generates a destination burst transaction request. Before exiting this ISR, software should write a 0 to the SSI.IMR.TXEIM register in order to mask any further ssi_txe_intr interrupts, since this is a level-sensitive interrupt.
3. When the DW_apb_ssi receive FIFO contains one source data item, the DW_apb_ssi generates an ssi_rxf_intr interrupt. The ssi_rxf_intr ISR is called with SSI.RXFTLR = 0.
4. When the DW_apb_ssi receive FIFO contains half-words greater than or equal to SSI.RXFTLR + 1 = 8, an ssi_rxf_intr interrupt is generated by the DW_apb_ssi. The ssi_rxf_intr ISR is called with SSI.RXFTLR = 7, which generates a source burst transaction request, as the [SglReqSrcReg](#) has already been written to in step 3.
5. On completion of the destination burst transaction, the DW_ahb_dmac clears [ReqDstReg](#)[0] and [SglReqDstReg](#)[0], and generates an IntDstTran interrupt. The ISR for this interrupt should write a 1 to the SSI.IMR.TXEIM register in order to unmask the generation of ssi_txe_intr interrupts.
6. Same as step 2, except the ssi_txe_intr interrupt is generated when the DW_apb_ssi transmit FIFO drops to or below the watermark level.
7. Same as step 5.
8. Upon completion of the source burst transaction, the DW_ahb_dmac clears [ReqSrcReg](#)[0] and [SglReqSrcReg](#)[0], and generates an IntSrcTran interrupt.



Note

The source peripheral has entered the [Single Transaction Region](#), since there are only eight bytes left to complete the source block; however, *src_burst_size_bytes* = 16 bytes.

9. Same as 6.
10. Same as 3.
11. The DW_ahb_dmac performs a single transaction from the source. Upon completion of the source single transaction, the DW_ahb_dmac clears [ReqSrcReg](#)[0] and [SglReqSrcReg](#)[0], and generates an IntSrcTran interrupt.

12. Steps 10 and 11, performed three more times. The block transfer from the source is now complete.
13. Same as step 7. The block transfer to the destination is now complete.

**Note**

The block transfer could proceed by polling the SSI.TXFLR/SSI.RXFLR registers in place of ssi_txe_intr/ssi_rxf_intr interrupts. Also, in place of IntSrcTran/IntDstTran interrupts, software could poll the ReqSrcReg[0]/ReqDstReg[0] registers.

When the ssi_rxf_intr ISR has no knowledge of when the source peripheral is inside or outside the [Single Transaction Region](#), as for the above example, then the ssi_rxf_intr ISR has to be invariant to this. If the ssi_rxf_intr ISR knows when the [Single Transaction Region](#) has been entered, then it can dynamically adjust the threshold level that triggers a source burst transaction, as explained in “[Example 4](#)” on page 67. The source block then completes on an [Early-Terminated Burst Transaction](#). The ssi_rxf_intr and IntSrcTran Interrupt Service Routines in this case would be:

3.9.1.10.3 ssi_rxf_intr Interrupt Service Routine (ISR)

1. Write hex 0101 to the ReqSrcReg register, followed by a write of hex 0101 to the SglReqSrcReg register. This generates a source burst transaction request.
2. Before exiting this ISR, software should write a 0 to the SSI.IMR.RXFIM register in order to mask any further ssi_rxf_intr interrupts, since this is a level-sensitive interrupt.

3.9.1.10.4 IntSrcTran Interrupt Service Routine (ISR)

1. If the source has entered the [Single Transaction Region](#) after this burst transaction, then the ISR writes a value of 3 into the SSI.RXFTLR register. This triggers an ssi_rxf_intr interrupt when four data items are in the DW_apb_ssi receive FIFO.
2. Write a 1 to the SSI.IMR.TXEIM register in order to unmask the generation of ssi_txe_intr interrupts.

**Note**

Knowing CTLx.BLOCK_TS and the number of source burst transactions completed, or by reading CTLx.BLOCK_TS – which reads the total number of data items read from the source peripheral up to this time – software can calculate when the amount of data left to fetch in the source block transfer is less than CTLx.SRC_MSIZ. Therefore, software can calculate when the source peripheral has entered the [Single Transaction Region](#).

3.9.1.11 Example 11

Scenario: This scenario demonstrates how to use software handshaking on both the source and destination side when the source peripheral is the flow controller and the destination peripheral enters the [Single Transaction Region](#). The DW_apb_ssi is used as the destination peripheral; for more information on the

DW_apb_ssi, refer to the [DesignWare DW_apb_ssi Databook](#). This example uses the parameters listed in [Table 3-14](#).

Table 3-14 Parameters in Transfer Operation – Example 11

Parameter	Description
CTLx.TT_FC = 3'b101	Peripheral-to-peripheral with source as flow controller
CTLx.BLOCK_TS = x	–
CTLx.SRC_TR_WIDTH = 3'b001	16 bit
CTLx.DST_TR_WIDTH = 3'b001	16 bit
CTLx.SRC_MSIZE = 3'b 001,	Source burst transaction length = 4
CTLx.DEST_MSIZE = 3'b 010	Destination burst transaction length = 8
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 16 bytes	–
CFGx.HS_SEL_SRC = 1	Source software handshaking
CFGx.HS_SEL_DST = 1	Destination software handshaking

In this example, it is assumed that the source peripheral generates an interrupt when the FIFO level is greater than or equal to some watermark level. For the purposes of this example, assume that the watermark level that triggers an interrupt named *src_burst_intr* is equal to CTLx.SRC_MSIZE. Also assume that this interrupt is level-sensitive and can be masked by writing to a software register in the source peripheral.

Consider the case where the source block is made up of a three-burst transaction:

$$blk_size_bytes_src = 3 * src_burst_size_bytes = 3 * 8 = 24 \text{ bytes}$$

The block size is not a multiple of the destination burst transaction length:

$$blk_size_bytes_src/dst_burst_size_bytes \ 24/16 \neq integer$$

Therefore, the block transfer to the destination enters the [Single Transaction Region](#) near the end of the block.



Attention

Block transfer when *one* of the following is true:

- It is guaranteed that data at some point will be extracted from the destination FIFO in the “Single transaction region” in order to trigger a burst transaction.
- When the watermark level that triggers a burst transaction to the destination can be dynamically adjusted near the end of block.

The DMA block transfer might proceed as follows:

1. Program SSI.TXFTLR = CTL_x.DEST_MSIZEx
2. SSI.TXFLR is initially 0, so an ssi_txe_intr interrupt is generated by the DW_apb_ssi. The Interrupt Service Routine (ISR) for this interrupt writes hex 0101 to both the ReqDstReg and SglReqDstReg; the order is not important. This generates a destination burst transaction request. Before exiting this ISR, software should write a 0 to the SSI.IMR.TXEIM register in order to mask any further ssi_txe_intr interrupts, since these are level-sensitive interrupts.
3. The source peripheral generates an interrupt when the watermark level is reached or exceeded. The Interrupt Service Routine (ISR) for this interrupt writes hex 0100 to the SglReqSrcReg and LstSrcReg registers, followed by hex 0101 to the ReqSrcReg. This generates a source burst transaction request, which is not the last in the block. Software should now mask the src_burst_intr interrupt.
4. Upon completion of the source burst transaction, the DW_ahb_dmac clears ReqSrcReg[0] and generates an IntSrcTran interrupt. The ISR for this interrupt should unmask the generation of the src_burst_intr interrupt in the source peripheral.
5. Repeat step 3.
6. Repeat step 4.
7. Upon completion of the destination burst transaction, the DW_ahb_dmac clears ReqDstReg[0] and SglReqDstReg[0], and generates an IntDstTran interrupt. The ISR for this interrupt should write a 1 to the SSI.IMR.TXEIM register in order to unmask generation of ssi_rxf_intr interrupts.
8. The source peripheral generates an interrupt when the watermark level is reached or exceeded. The Interrupt Service Routine (ISR) for this interrupt writes hex 0100 to the SglReqSrcReg register, followed by hex 0101 to the LstSrcReg register, followed by hex 0101 to the ReqSrcReg register. This generates a source burst transaction request, which is the last in the block. Software should now mask the src_burst_intr interrupt.
9. Repeat 4. The block transfer from the source is now complete.
10. The block transfer to the destination may proceed as follows:
 - a. If it is guaranteed that data at some point will be extracted from the destination FIFO near the end of a block in order to trigger a burst transaction, an ssi_txe_intr interrupt is generated by the destination DW_apb_ssi peripheral. The Interrupt Service Routine (ISR) for this interrupt writes hex 0101 to the ReqDstReg register, followed by a write hex 0101 to the SglReqDstReg; the order is important. This generates a destination burst transaction request, which is an **Early-Terminated Burst Transaction**. Before exiting this ISR, software should write a 0 to the SSI.IMR.TXEIM register in order to mask any further ssi_txe_intr interrupts, since these are level-sensitive interrupts. The block transfer to the destination is now complete.
 - b. If it is *not* guaranteed that data at some point will be extracted from the destination FIFO near the end of a block in order to trigger a burst transaction, and if software can determine that the destination has entered the **Single Transaction Region**, then it can write a value of 4 into the SSI.TXFTLR register. This triggers an ssi_txe_intr interrupt when four free locations are present in the DW_apb_ssi transmit FIFO. On receipt of this interrupt, software can then write hex 0101 to the ReqDstReg register, followed by a write of hex 0101 to the SglReqDstReg register; the order is important. This generates a destination burst transaction request, which is an **Early-Terminated Burst Transaction**. Hardware clears ReqDstReg[0] and SglReqDstReg[0] upon block completion, which occurs after four data items have been transferred to the destination in this burst

transaction, and an IntDstTran interrupt is generated. The block transfer to the destination is now complete.

**Note**

Software can determine when it has entered the [Single Transaction Region](#), since it knows that a last-burst transaction has been requested by the source, and can calculate the number of bytes left to complete in the destination block. It can then compare this to `dst_burst_size_bytes`. For more information, refer to [“Single Transaction Region”](#) on page 46.

**Attention**

Block transfer when *both* of the following are true:

- It is *not* guaranteed that data at some point will be extracted from the destination FIFO in the “Single transaction region” in order to trigger a burst transaction.
- When the watermark level that triggers a burst transaction to the destination *cannot* be dynamically adjusted near the end of block.

The `ssi_txe_intr` and `IntDstTran` Interrupt Service Routines in this case would be:

- `ssi_txe_intr` Interrupt Service Routine (ISR)
 - a. Read the `SSI.TXFTLR` register.
If `SSI.TXFTLR = 1`
 - i. Write hex 0101 to the [SglReqDstReg](#) register.
 - ii. Write 8 (`CTLx.DEST_MSIZ`) to the `SSI.TXFTLR` register. This will trigger a new `ssi_txe_intr` interrupt when the number of data items in the destination FIFO is less than or equal to 8.
 - else (`SSI.TXFTLR = 8`)
 - i. Write 0 to the `SSI.IMR.TXFIM` register in order to mask any further `ssi_txe_intr` interrupts in the ISR for the `ssi_txe_intr` interrupt.
 - ii. Write hex 0101 to the [ReqDstReg](#) register.
- `IntDstTran` Interrupt Service Routine (ISR)
 - a. Write 1 to the `SSI.TXFTLR` register, which triggers an `ssi_txe_intr` interrupt when a single free location is available in the `DW_apb_ssi` transmit FIFO.
 - b. Write a 1 to the `SSI.IMR.TXFIM` register in order to unmask the generation of `ssi_txe_intr` interrupts.

The DMA block transfer might proceed as follows:

1. Software writes a value of 1 into the `SSI.TXFTLR` register, which triggers an `ssi_txe_intr` interrupt when a single free location is in the `DW_apb_ssi` transmit FIFO.
2. `SSI.TXFLR` is initially 0, so an `ssi_txe_intr` interrupt is generated by the `DW_apb_ssi`. The `ssi_txe_intr` ISR is called when `SSI.TXFTLR = 1`.
3. `SSI.TXFLR` is initially 0, so an `ssi_txe_intr` interrupt is generated by the `DW_apb_ssi`. The `ssi_txe_intr` ISR is called with `SSI.TXFTLR = 8`. A destination burst transaction request is generated, as the [SglReqDstReg](#) register has been written to in step 2.

4. The source peripheral generates an interrupt when the watermark level is reached or exceeded. The Interrupt Service Routine (ISR) for this interrupt writes hex 0100 to the [SglReqSrcReg](#) and [LstSrcReg](#) registers, followed by a write of hex 0101 to the [ReqSrcReg](#). This generates a source burst transaction request, which is not the last in the block. Software should now mask the *src_burst_intr* interrupt.
5. Upon completion of the source burst transaction, the DW_ahb_dmac clears [ReqSrcReg\[0\]](#) and generates an *IntSrcTran* interrupt. The ISR for this interrupt should unmask generation of the *src_burst_intr* interrupt in the source peripheral.
6. Repeat step 4.
7. Repeat step 5.
8. Upon completion of the destination burst transaction, the DW_ahb_dmac clears [ReqDstReg\[0\]](#) and [SglReqDstReg\[0\]](#), and generates an *IntDstTran* interrupt.
9. The source peripheral generates an interrupt when the watermark level is reached or exceeded. The Interrupt Service Routine (ISR) for this interrupt writes hex 0100 to the [SglReqSrcReg](#) register, followed by a write of hex 0101 to the [LstSrcReg](#) register, followed by a write of hex 0101 to the [ReqSrcReg](#) register. This generates a source burst transaction request, which is the last in the block. Software should now mask the *src_burst_intr* interrupt.
10. Repeat step 5. The block transfer from the source is now complete.
11. When a free location exists in the destination FIFO, an *ssi_txe_intr* interrupt is generated by the DW_apb_ssi. The *ssi_txe_intr* ISR is called with *SSI.TXFTLR* = 1.
12. The DW_ahb_dmac performs a single transaction to the destination. Upon completion of the destination single transaction, the DW_ahb_dmac clears [ReqDstReg\[0\]](#) and [SglReqDstReg\[0\]](#), and generates an *IntDstTran* interrupt.
13. Repeat steps 11 and 12 four more times. The block transfer to the destination is now complete.

**Note**

“Example 11” on page 90 outlines a DW_ahb_dmac block transfer using software handshaking when interrupts are used to control the block transfer. The same could be achieved using register polling.

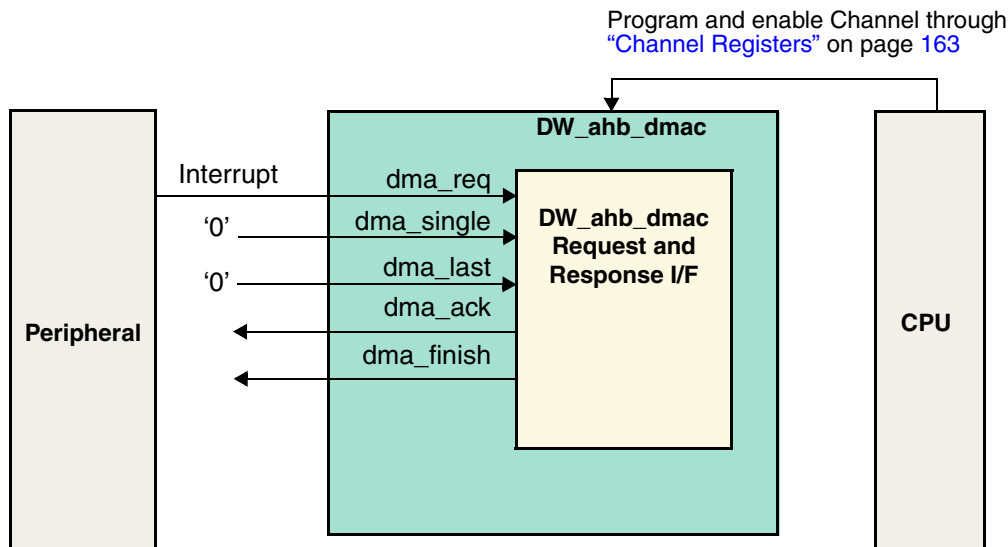
3.9.2 Peripheral Interrupt Request Interface

The interface illustrated in [Figure 3-33](#) is a simplified version of the hardware handshaking interface. In this mode:

- The interrupt line from the peripheral is tied to the *dma_req* input.
- The *dma_single* input is tied low.
- All other interface signals are ignored.

This interface can be used where the slave peripheral does not have hardware handshaking signals. To the DW_ahb_dmac, this is the same “[Hardware Handshaking – Peripheral Is Not Flow Controller](#)” on page 48.

Figure 3-33 Transaction Request Through Peripheral Interrupt



The peripheral can never be the flow controller, since it cannot connect to the dma_last signal. The interrupt line from the peripheral is tied to the dma_req line, as shown in [Figure 3-33](#). The timing of the interrupt line from the peripheral must be the same as the dma_req line, as discussed in “[Hardware Handshaking – Peripheral Is Not Flow Controller](#)” on page 48.

Since the dma_ack line is not sampled by the peripheral, the handshaking loop is as follows:

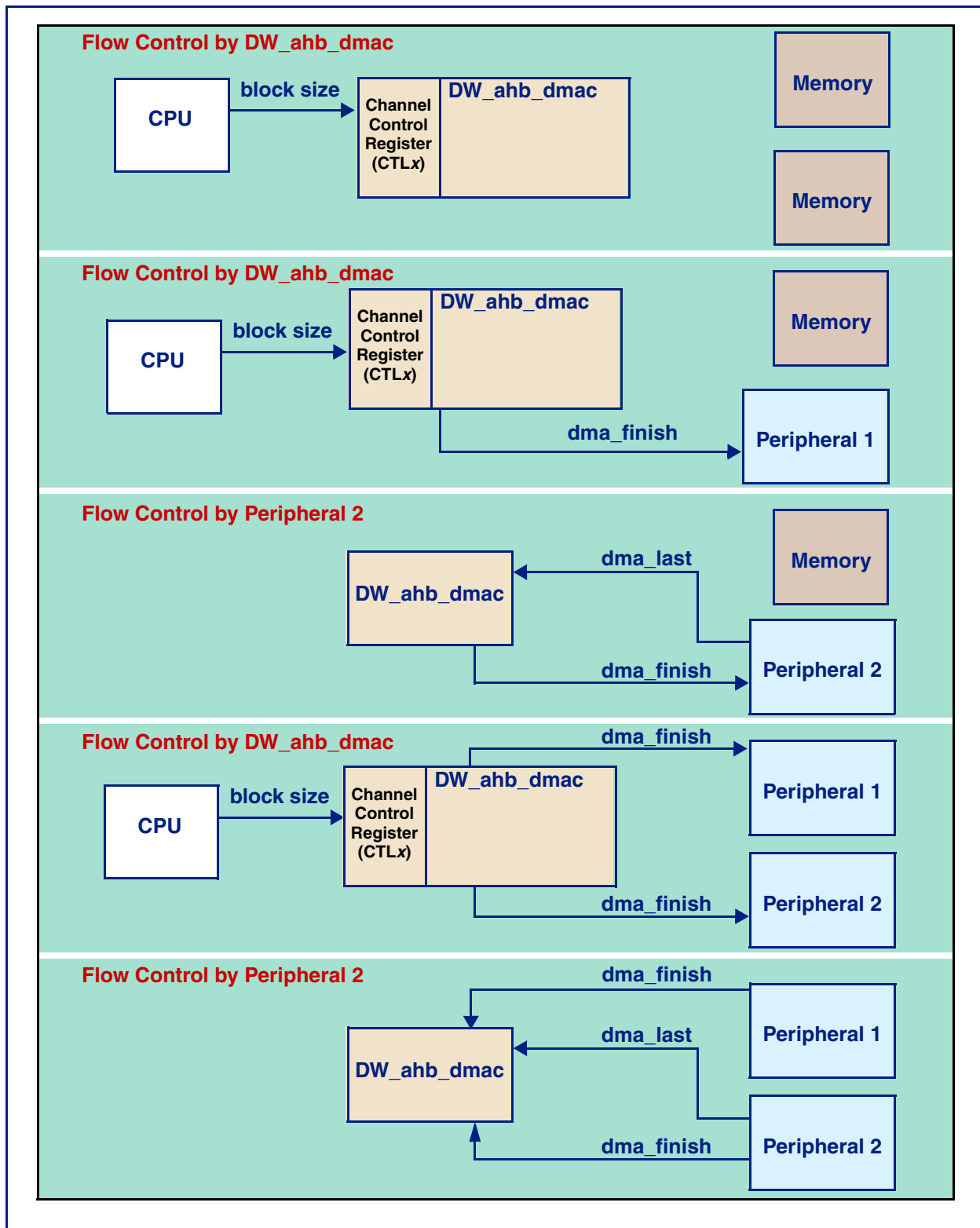
1. Peripheral generates an interrupt that asserts dma_req.
2. DW_ahb_dmac completes the burst transaction and generates an end-of-burst transaction interrupt, IntSrcTran/IntDstTran. Interrupts must be enabled and the transaction complete interrupt unmasked.
3. The interrupt service routine clears the interrupt in the peripheral so that the dma_req is de-asserted.

Notice that dma_single is hardcoded to an inactive level. For conditions where the source/destination peripheral can tie dma_single to an inactive level, refer to “[Single Transactions – Peripheral Is Not Flow Controller](#)” on page 55.

3.10 Flow Control Configurations

[Figure 3-34](#) on page 96 indicates five different flow control configurations using hardware handshaking interfaces – a simplified version of the interface is shown. These scenarios can also be used for software handshaking, which uses software registers instead of signals.

Figure 3-34 Flow Control Configurations



3.11 Peripheral Burst Transaction Requests

For a source FIFO, an active edge is triggered on `dma_req` when the source FIFO exceeds some watermark level. For a destination FIFO, an active edge is triggered on `dma_req` when the destination FIFO drops below some watermark level.

This section investigates the optimal settings of these watermark levels on the source and destination peripherals and their relationship to, respectively:

- Source transaction length, `CTLx.SRC_MSIZ`
- Destination transaction length, `CTLx.DEST_MSIZ`

For demonstration purposes, a receive `DW_apb_ssi` is used as a source peripheral, and a transmit `DW_apb_ssi` is used as a destination peripheral. For more information on the `DW_apb_ssi`, refer to the [DesignWare DW_apb_ssi Databook](#).



Note

Throughout this section, `DW_apb_ssi`-related parameters are prefixed with “SSI.” `DW_ahb_dmac`-related parameters are prefixed with “DMA.”

3.11.1 Transmit Watermark Level and Transmit FIFO Underflow

During `DW_apb_ssi` serial transfers, `DW_apb_ssi` transmit FIFO requests are made to the `DW_ahb_dmac` whenever the number of entries in the `DW_apb_ssi` transmit FIFO is less than or equal to the `DW_apb_ssi` Transmit Data Level Register (`SSI.DMATDLR`) value. This is known as the watermark level. The `DW_ahb_dmac` responds by writing a burst of data to the `DW_apb_ssi` transmit FIFO buffer, of length `DMA.CTLx.DEST_MSIZ`.

Data should be fetched from the `DW_ahb_dmac` often enough for the `DW_apb_ssi` transmit FIFO to continuously perform serial transfers; that is, when the `DW_apb_ssi` transmit FIFO begins to empty, another burst transaction request should be triggered. Otherwise the `DW_apb_ssi` transmit FIFO runs out of data (underflow). To prevent this condition, the user must set the watermark level correctly.

3.11.2 Choosing the Transmit Watermark Level

Consider an example with the following assumption:

$$\text{DMA.CTLx.DEST_MSIZ} = \text{SSI_TX_FIFO_DEPTH} - \text{SSI.DMATDLR}$$



Note

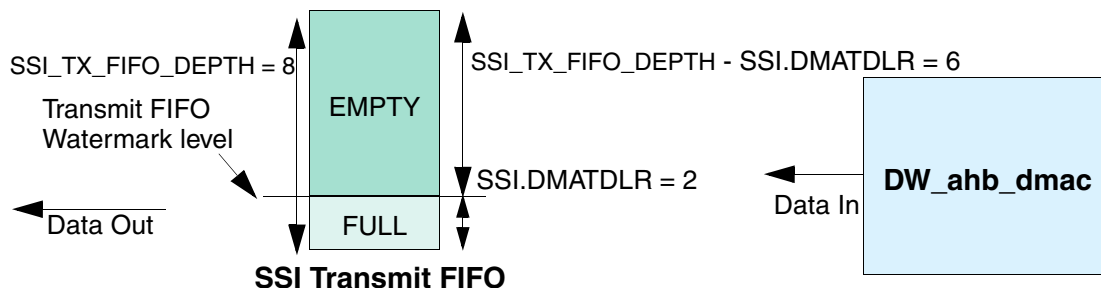
`SSI_TX_FIFO_DEPTH` is the `DW_apb_ssi` transmit FIFO depth. `SSI.DMATDLR` controls the level at which a `DW_ahb_dmac` destination burst request is made by the `DW_apb_ssi` transmit logic. It is equal to the watermark level; that is, a destination burst request is generated (active-edge of `dma_req` triggered) when the number of valid data entries in the `DW_apb_ssi` transmit FIFO is equal to or below this field value.

In this situation, the number of data items to be transferred in a `DW_ahb_dmac` burst is equal to the empty space in the `DW_apb_ssi` transmit FIFO. Consider two different watermark level settings.

3.11.2.1 Case 1: SSI.DMATDLR = 2

Figure 3-35 illustrates the watermark levels in Case 1 where SSI.DMATDLR = 2.

Figure 3-35 Case 1 Watermark Levels where SSI.DMATDLR = 2



Case 1 uses the parameters listed in Table 3-15.

Table 3-15 Transmit Watermark Level – Case 1

Parameter	Comment
SSI.DMATDLR = 2	DW_apb_ssi transmit FIFO watermark level
DMA.CTLx.DEST_MSIZE = SSI_TX_FIFO_DEPTH - SSI.DMATDLR = 6	DMA.CTLx.DEST_MSIZE is equal to the empty space in the transmit FIFO at the time the burst request is made.
SSI_TX_FIFO_DEPTH = 8	DW_apb_ssi transmit FIFO depth
DMA.CTLx.BLOCK_TS = 30	Block size

The number of burst transactions that are needed equals the block size divided by the number of data items per burst:

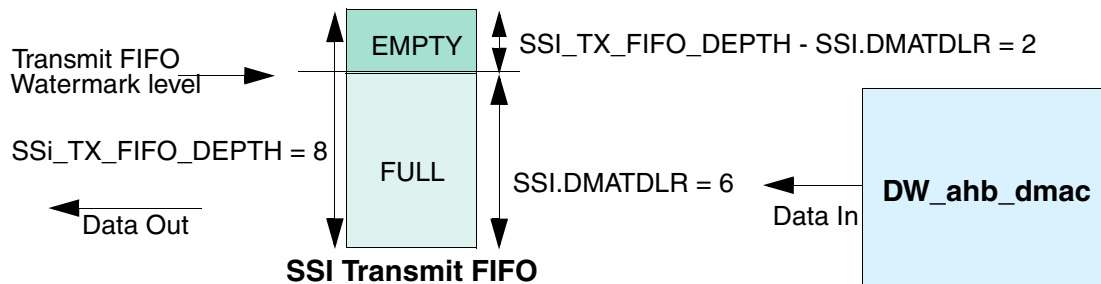
$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 30 / 6 = 5$$

The number of burst transactions in the DW_ahb_dmac block transfer is 5, but the watermark level, SSI.DMATDLR, is quite low. Therefore, the probability of an SSI underflow is high where the SSI serial transmit line needs to transmit data, but where there is no data left in the transmit FIFO. This occurs because the DW_ahb_dmac has not had time to service the DW_ahb_dmac request before the DW_apb_ssi transmit FIFO becomes empty.

3.11.2.2 Case 2: SSI.DMATDLR = 6

Figure 3-36 illustrates the watermark levels in Case 2 where SSI.DMATDLR = 6.

Figure 3-36 Case 2 Watermark Levels where SSI.DMATDLR = 6



Case 2 uses the parameters listed in Table 3-16.

Table 3-16 Transmit Watermark Level – Case 2

Parameter	Description
SSI.DMATDLR = 6	DW_apb_ssi transmit FIFO watermark level
DMA.CTLx.DEST_MSIZE = SSI_TX_FIFO_DEPTH - SSI.DMATDLR = 2	DMA.CTLx.DEST_MSIZE is equal to the empty space in the transmit FIFO at the time the burst request is made.
SSI_TX_FIFO_DEPTH = 8	DW_apb_ssi transmit FIFO depth
DMA.CTLx.BLOCK_TS = 30	Block size

The number of burst transactions in the block are:

$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 30 / 2 = 15$$

In this block transfer, there are fifteen destination burst transactions in a DMA block transfer, but the watermark level, SSI.DMATDLR, is high. Therefore, the probability of an SSI underflow is low because the DW_ahb_dmac has plenty of time to service the destination burst transaction request before the SSI transmit FIFO becomes empty.

Thus, the second case has a lower probability of underflow at the expense of more burst transactions per block. This potentially provides a greater amount of bursts per block and a worse bus utilization than the former case.

Therefore, the goal in choosing a watermark level is to minimize the number of transactions per block, while at the same time keeping the probability of an underflow condition to an acceptable level. In practice, this is a function of the following ratio:

$$\frac{\text{Rate of SSI data transmission}}{\text{Rate of DW_ahb_dmac response to destination burst requests}}$$

For example, promoting the channel to the highest-priority channel in the DW_ahb_dmac, and promoting the DW_ahb_dmac master interface to the highest-priority master in the AHB layer, increases the rate at which the DW_ahb_dmac can respond to burst transaction requests. This in turn allows the user to decrease

the watermark level, which improves bus utilization without compromising the probability of an underflow occurring.

3.11.3 Selecting CTLx.DEST_MSIZ and Transmit FIFO Overflow

As can be seen from [Figure 3-36](#) on page 99, programming DMA.CTLx.DEST_MSIZ to a value greater than the watermark level that triggers the DW_ahb_dmac request may cause overflow when there is not enough space in the DW_apb_ssi transmit FIFO to service the destination burst request. Therefore, the following equation must be adhered to in order to avoid overflow:

$$\text{DMA.CTLx.DEST_MSIZ} \leq \text{SSI_TX_FIFO_DEPTH} - \text{SSI.DMATDLR} \text{ (13)}$$

In “[Case 2: SSI.DMATDLR = 6](#)” on page 99, the amount of space in the transmit FIFO at the time the burst request is made is equal to the destination burst length, DMA.CTLx.DEST_MSIZ. Thus, the transmit FIFO may be full, but not overflowed, upon completion of the burst transaction.

Therefore, for optimal operation, DMA.CTLx.DEST_MSIZ should be set at the FIFO level that triggers a transmit DW_ahb_dmac request; that is:

$$\text{DMA.CTLx.DEST_MSIZ} = \text{SSI_TX_FIFO_DEPTH} - \text{SSI.DMATDLR} \text{ (14)}$$

This is the setting used in [Figure 3-24](#) on page 74.

Adhering to equation (14) reduces the number of DW_ahb_dmac bursts needed for a block transfer, and this in turn improves AHB bus utilization.



Note

The DW_apb_ssi transmit FIFO is not full at the end of a DW_ahb_dmac burst transaction if the SSI has successfully transmitted one data item or more on the SSI serial transmit line before the end of the burst transaction.

3.11.4 Receive Watermark Level and Receive FIFO Overflow

During DW_apb_ssi serial transfers, DW_apb_ssi receive FIFO requests are made to the DW_ahb_dmac whenever the number of entries in the DW_apb_ssi receive FIFO is at or above the DW_ahb_dmac Receive Data Level Register; that is, SSI.DMARDLR+1. This is known as the watermark level. The DW_ahb_dmac responds by reading a burst of data from the receive FIFO buffer of length DMA.CTLx.SRC_MSIZ.



Note

SSI.DMARDLR controls the level at which a source burst request is made by the receive logic. When the number of valid data entries in the DW_apb_ssi receive FIFO is equal to or greater than the watermark level (DMARDLR+1), a source burst request is generated (active-edge of dma_req triggered).

Data should be fetched by the DW_ahb_dmac often enough for the DW_apb_ssi receive FIFO to accept serial transfers continuously; that is, when the DW_apb_ssi receive FIFO begins to fill, another burst transaction request should be triggered. Otherwise, the DW_apb_ssi receive FIFO fills with data (overflow). To prevent this condition, the user must correctly set the watermark level.

3.11.5 Choosing the Receive Watermark level

Similar to choosing the transmit watermark level described earlier, the receive watermark level, `SSI.DMARDLR+1`, should be set to minimize the probability of overflow, as shown in [Figure 3-37](#) on page 101. It is a trade-off between the number of burst transactions required per block versus the probability of an overflow occurring.

3.11.6 Selecting CTLx.SRC_MSIZ and Receive FIFO Underflow

As can be seen in [Figure 3-37](#), programming a source burst transaction length greater than the watermark level may cause underflow when there is not enough data to service the source burst request. Therefore, equation (15) below must be adhered to in order to avoid underflow.

If the number of data items in the `DW_apb_ssi` receive FIFO is equal to the source burst length at the time the source burst request is made – `DMA.CTLx.SRC_MSIZ` – the `DW_apb_ssi` receive FIFO may be emptied, but not underflowed, at the completion of the source burst transaction. For optimal operation, `DMA.CTLx.SRC_MSIZ` should be set at the watermark level; that is:

$$\text{DMA.CTLx.SRC_MSIZ} = \text{SSI.DMARDLR} + 1 \quad (15)$$

Adhering to equation (15) reduces the number of burst transactions in a block transfer, and this in turn can improve AHB bus utilization.



Note

The `DW_apb_ssi` receive FIFO is not empty at the end of the source burst transaction if the SSI has successfully received one data item or more on the SSI serial receive line before the end of the burst, as illustrated in [Figure 3-37](#).

Figure 3-37 SSI Receive FIFO



3.12 Generating Requests for the AHB Master Bus Interface

Each channel has a source state machine and destination state machine running in parallel. These state machines generate the request inputs to the arbiter, which arbitrates for the master bus interface (one arbiter per master bus interface).

When the source/destination state machine is granted control of the master bus interface, and when the master bus interface is granted control of the external AHB bus, then AHB transfers between the peripheral and the `DW_ahb_dmac` (on behalf of the granted state machine) can take place.

AHB transfers from the source peripheral or to the destination peripheral cannot proceed until the channel FIFO is ready. For burst transaction requests and for transfers involving memory peripherals, the criterion for “FIFO readiness” is controlled by the `FIFO_MODE` field of the `CFGx` register.

The definition of FIFO readiness is the same for:

- Single transactions
- Burst transactions, where `CFGx.FIFO_MODE = 0`
- Transfers involving memory peripherals, where `CFGx.FIFO_MODE = 0`

The channel FIFO is deemed ready when the space/ data available is sufficient to complete a single AHB transfer of the specified transfer width. FIFO readiness for source transfers occurs when the channel FIFO contains enough room to accept at least a single transfer of `CTLx.SRC_TR_WIDTH` width. FIFO readiness for destination transfers occurs when the channel FIFO contains data to form at least a single transfer of `CTLx.DST_TR_WIDTH` width.



Note

An exception to FIFO readiness for destination transfers occurs in “FIFO flush mode.” In this mode, FIFO readiness for destination transfers occurs when the channel FIFO contains data to form at least a single transfer of `CTLx.SRC_TR_WIDTH` width (and not `CTLx.DST_TR_WIDTH` width, as is the normal case). For an explanation of FIFO flush mode, refer to “[Example 5](#)” on page 70.

When `CFGx.FIFO_MODE = 1`, then the criteria for FIFO readiness for burst transaction requests and transfers involving memory peripherals are as follows:

- A FIFO is ready for a source burst transfer when the FIFO is less than half empty.
- A FIFO is ready for a destination burst transfer when the FIFO is greater than or equal to half full.

Exceptions to this “readiness” occur. During these exceptions, a value of `CTLx.FIFO_MODE = 0` is assumed. The following are the exceptions:

- Near the end of a burst transaction or block transfer – The channel source state machine does not wait for the channel FIFO to be less than half empty if the number of source data items left to complete the source burst transaction or source block transfer is less than `DMAH_CHx_FIFO_DEPTH/2`. Similarly, the channel destination state machine does not wait for the channel FIFO to be greater than or equal to half full, if the number of destination data items left to complete the destination burst transaction or destination block transfer is less than `DMAH_CHx_FIFO_DEPTH/2`.
- In FIFO flush mode – For an explanation of FIFO flush mode, refer to “[Example 5](#)” on page 70.
- When a channel is suspended – The destination state machine does not wait for the FIFO to become half empty to flush the FIFO, regardless of the value of the `FIFO_MODE` field.
- After receipt of a split/retry response from a source or destination – The AMBA protocol requires that after an AHB master receives a split/retry response, it must re-issue the transfer that received the split/retry before attempting any other transfer. Therefore, a transfer is re-issued to the same address that returned the split/retry, regardless of `FIFO_MODE`, when the DW_ahb_dmac is next granted the AHB bus. This is repeated until an OKAY response is received on the AHB hresp bus.

When the source/destination peripheral is not memory, the source/destination state machine waits for a single/burst transaction request. Upon receipt of a transaction request and only if the channel FIFO is “ready” for source/destination AHB transfers, a request for the master bus interface is made by the source/destination state machine.



There is one exception to this, which occurs when the destination peripheral is the flow controller and `CFGx.FCMODE = 1` (data pre-fetching is disabled). Then the source state machine does not generate a request for the master bus interface (even if the FIFO is “ready” for source transfers and has received a source transaction request) until the destination requests new data. Refer to “[Example 8](#)” on page 83.

When the source/destination peripheral is memory, the source/destination state machine must wait until the channel FIFO is “ready.” A request is then made for the master bus interface. There is no handshaking mechanism employed between a memory peripheral and the DW_ahb_dmac.

3.12.1 Locked DMA Transfers

It is possible to program the DW_ahb_dmac for:

- Bus locking – Asserts the AHB hlock signal.
- Channel locking – Locks the arbitration for the AHB master interface, which grants ownership of the master bus interface to one of the requesting channel state machines (source or destination).

Bus and channel locking can proceed for the duration of a DMA transfer, a block transfer, or a single or burst transaction.

3.12.1.1 Bus Locking

If the `LOCK_B` bit in the channel configuration register (`CFGx`) is set, then the AHB hlock signal is asserted for the duration specified in the `LOCK_B_L` field.

3.12.1.2 Channel Locking

If the `LOCK_CH` field is set, then the arbitration for the master bus interface is exclusively reserved for the source and destination peripherals of that channel for the duration specified in the `LOCK_CH_L` field.

If bus locking is activated for a certain duration, then it follows that the channel is also automatically locked for that duration. Three cases arise:

- `CFGx.LOCK_B = 0` – Programmed values of `CFGx.LOCK_CH` and `CFGx.LOCK_CH_L` are used.
- `CFGx.LOCK_B = 1` and `CFGx.LOCK_CH = 0` – DMA transfer proceeds as if `CFGx.LOCK_CH = 1` and `CFGx.LOCK_CH_L = CFGx.LOCK_B_L`. The programmed values of `CFGx.LOCK_CH` and `CFGx.LOCK_CH_L` are ignored.
- `CFGx.LOCK_B = 1` and `CFGx.LOCK_CH = 1` – Two cases arise:
 - `CFGx.LOCK_B_L ≤ CFGx.LOCK_CH_L` – In this case, the DMA transfer proceeds as if `CFGx.LOCK_CH_L = CFGx.LOCK_B_L` and the programmed value of `CFGx.LOCK_CH_L` is ignored. Thus, if bus locking is enabled over the DMA transfer level, then channel locking is enabled over the DMA transfer level, regardless of the programmed value of `CFGx.LOCK_CH_L`.
 - `CFGx.LOCK_B_L > CFGx.LOCK_CH_L` – The programmed value of `CFGx.LOCK_CH_L` is used. Thus, if bus locking is enabled over the DMA block transfer level and channel locking is enabled over the DMA transfer level, then channel locking is performed over the DMA transfer level.

3.12.1.3 Locking Levels

If locking is enabled for a channel, then locking of the AHB master bus interface at a programmed locking transfer level is activated when the channel is first granted the AHB master bus interface at the start of that locking transfer level. It continues until the locking transfer level has completed; that is, if channel 0 has enabled channel level locking at the block transfer level, then this channel locks the master bus interface when it is first granted the master bus interface at the start of the block transfer, and continues to lock the master bus interface until the block transfer has completed.

Source and destination block transfers occur successively in time, and a new source block cannot commence until the previous destination block has completed. When both source and destination are on the same AHB layer, then block level locking is terminated on completion of the block to the destination. If they are on separate layers, then block-level locking is terminated on completion of the block on that layer – when the source block on the source AHB layer completes, and when the destination block on the destination AHB layer completes. The same is true for DMA transfer-level locking.

Transaction-level locking is different due to the fact that source and destination transactions occur independently in time, and the number of source and destination transactions in a DMA block or DMA transfer do not have to match. When the source and destination are on the same AHB layer, then transaction-level locking is cleared at the end of a source or destination transaction only if the opposing peripheral is not currently in the middle of a transaction.

For example, if locking is enabled at the transaction level and an end-of-source transaction is signaled, then this disables locking only if one of the following is true:

- The destination is on a different AHB layer
- The destination is on the same AHB layer, but the channel is not currently in the middle of a transaction to the destination peripheral.

The same rules apply when an end-of-destination transaction is signalled.

If channel-level or bus-level locking is enabled for a channel at the transaction level, and either the source or destination of the channel is a memory device, then the locking is ignored and the channel proceeds as if locking (bus or channel) is disabled.

**Note**

Since there is no notion of a transaction level for a memory peripheral, then transaction-level locking is not allowed when either source or destination is memory.

3.12.1.4 Channel Locking and Deadlock

Certain combinations of channel-level and bus-level locking may lead to deadlock, where multiple channels are concurrently enabled and no channel can proceed with the DMA transfer. This occurs only for configurations where `DMAH_NUM_MASTER_INT > 1` and `DMAH_NUM_CHANNELS > 1`. The methods used to avoid deadlock are described in more detail in [“Channel Locking and Deadlock”](#) on page 315.

3.13 Arbitration for AHB Master Interface

Each DW_ahb_dmac channel has two request lines that request ownership of a particular master bus interface: channel source and channel destination request lines.

Source and destination arbitrate separately for the bus. Once a source/destination state machine gains ownership of the master bus interface and the master bus interface has ownership of the AHB bus, then AHB transfers can proceed between the peripheral and DW_ahb_dmac. [Figure 3-38](#) on page 106 illustrates the arbitration flow of the master bus interface.

An arbitration scheme decides which of the request lines ($2 * DMAH_NUM_CHANNELS$) is granted the particular master bus interface. Each channel has a programmable priority. A request for the master bus interface can be made at any time, but is granted only after the current AHB transfer (burst or single) has completed. Therefore, if the master interface is transferring data for a lower priority channel and a higher priority channel requests service, then the master interface will complete the current burst for the lower priority channel before switching to transfer data for the higher priority channel.

To prevent a channel from saturating the master bus interface, it can be given a maximum AMBA burst length ([MAX_ABRST](#) field in [CFGx](#) register) at channel setup time. This also prevents the master bus interface from saturating the AHB bus where the system arbiter cannot change the grant lines until the end of an undefined length burst.

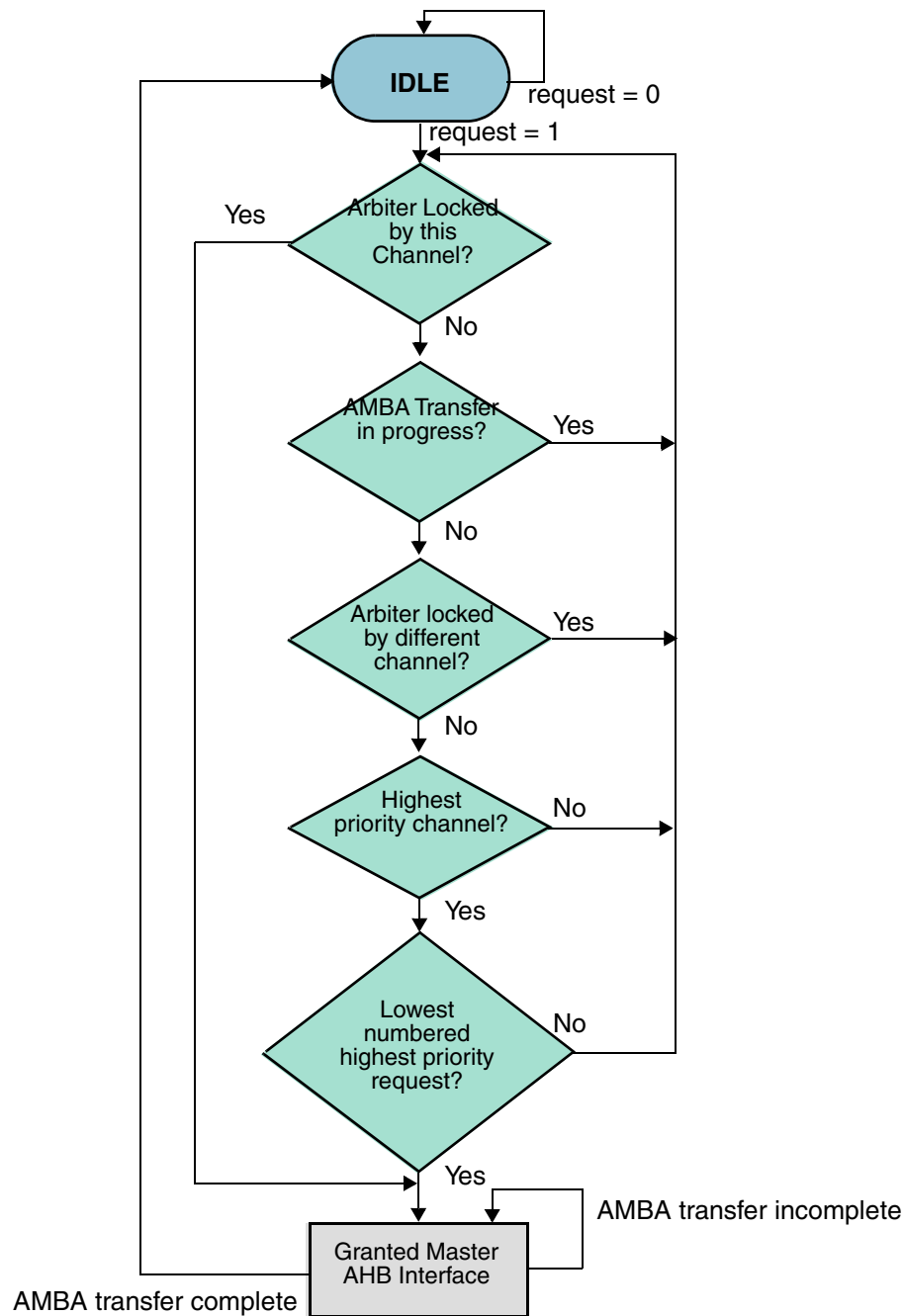
The following is the interface arbitration scheme employed when no channel has locked ([Channel Locking](#)) the arbitration for the master bus interface:

- If only one request line is active at the highest priority level, then the request with the highest priority wins ownership of the AHB master bus interface; it is not necessary for the priority levels to be unique.
If more than one request is active at the highest requesting priority, then these competing requests proceed to a second tier of arbitration.
- If equal priority requests occur, then the lower-numbered channel is granted.

In other words, if a peripheral request attached to Channel 7 and a peripheral request attached to Channel 8 have the same priority, then the peripheral attached to Channel 7 is granted first.

**Note**

A channel source is granted before the destination if both have their request lines asserted when a grant decision is made. A channel source and channel destination inherit their channel priority and therefore always have the same priority.

Figure 3-38 Arbitration Flow for Master Bus Interface

3.14 Latency

By default, the DW_ahb_dmac has:

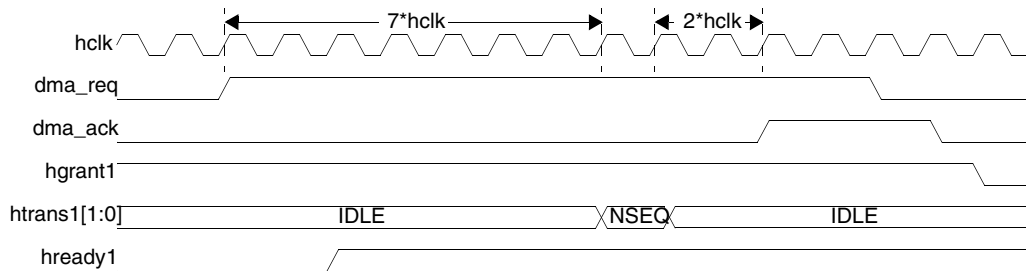
- Seven cycles of latency from hardware handshaking request to NSEQ being issued from appropriate AHB master interface
- Two cycles of latency from completion of AHB burst to assertion of dma_ack



The following latency illustrations are best-case scenarios where the DW_ahb_dmac master interface already owns the AHB bus before issuing the transfer. AHB bus arbitration latency increases the observed latency.

Figure 3-39 shows the default latency from a hardware handshaking request to the AHB transfer, and the latency from the completion of that burst to the assertion of dma_ack.

Figure 3-39 Default DW_ahb_dmac Latency

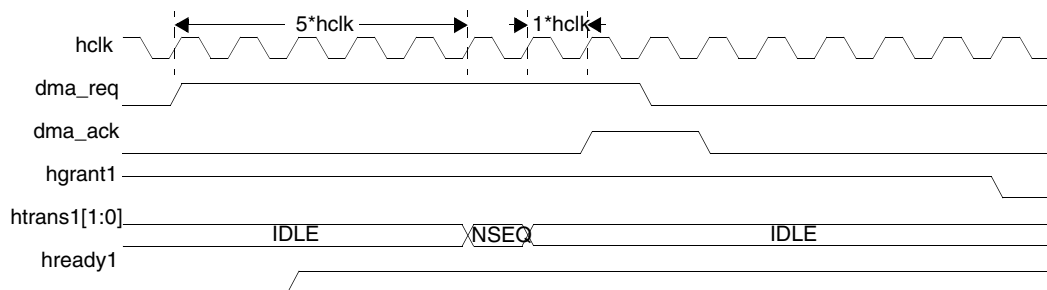


This default latency can be reduced by setting the parameter DMAH_REMOVE_PIPELINING to 1, which reduces the latency to:

- Five cycles from hardware handshaking request to NSEQ
- One cycle from AHB burst completion to dma_ack.

Figure 3-40 shows the latency from a hardware handshaking request to the AHB transfer, and the latency from the completion of that burst to the assertion of dma_ack; in this case, DMAH_REMOVE_PIPELINING is set to 1.

Figure 3-40 DW_ahb_dmac Latency with DMAH_REMOVE_PIPELINING Set to 1



Setting DMAH_REMOVE_PIPELINING to 1 also does the following:

- Results in longer logic paths in the DW_ahb_dmac, which can make it harder to reach the desired operating frequency.
- Reduces latency by one cycle when the hardware handshaking interfaces are not being used. One of the pipeline stages removed is in the master interface arbiter, which is used for all types of transfer.

3.15 Scatter/Gather

Scatter is relevant to a destination transfer. The destination address is incremented or decremented by a programmed amount – the scatter increment – when a scatter boundary is reached. [Figure 3-41](#) on page 109 shows an example destination scatter transfer. The destination address is incremented or decremented by the value stored in the destination scatter increment (DSRx.DSI) field (refer to “[DSRx](#)” on page 188), multiplied by the number of bytes in a single AHB transfer to the destination $\S$ (decoded value of [CTLx.DST_TR_WIDTH](#))/8 – when a scatter boundary is reached. The number of destination transfers between successive scatter boundaries is programmed into the Destination Scatter Count (DSC) field of the DSRx register.

Scatter is enabled by writing a 1 to the [CTLx.DST_SCATTER_EN](#) field. The [CTLx.DINC](#) field determines if the address is incremented, decremented, or remains fixed when a scatter boundary is reached. If the [CTLx.DINC](#) field indicates a fixed-address control throughout a DMA transfer, then the [CTLx.DST_SCATTER_EN](#) field is ignored, and the scatter feature is automatically disabled.

Gather is relevant to a source transfer. The source address is incremented or decremented by a programmed amount when a gather boundary is reached. The number of source transfers between successive gather boundaries is programmed into the Source Gather Count (SGRx.SGC) field. The source address is incremented or decremented by the value stored in the source gather increment (SGRx.SGI) field (refer to “[SGRx](#)” on page 186), multiplied by the number of bytes in a single AHB transfer from the source – (decoded value of [CTLx.SRC_TR_WIDTH](#))/8 – when a gather boundary is reached.

Gather is enabled by writing a 1 to the [CTLx.SRC_GATHER_EN](#) field. The [CTLx.SINC](#) field determines if the address is incremented, decremented, or remains fixed when a gather boundary is reached. If the [CTLx.SINC](#) field indicates a fixed-address control throughout a DMA transfer, then the [CTLx.SRC_GATHER_EN](#) field is ignored, and the gather feature is automatically disabled.

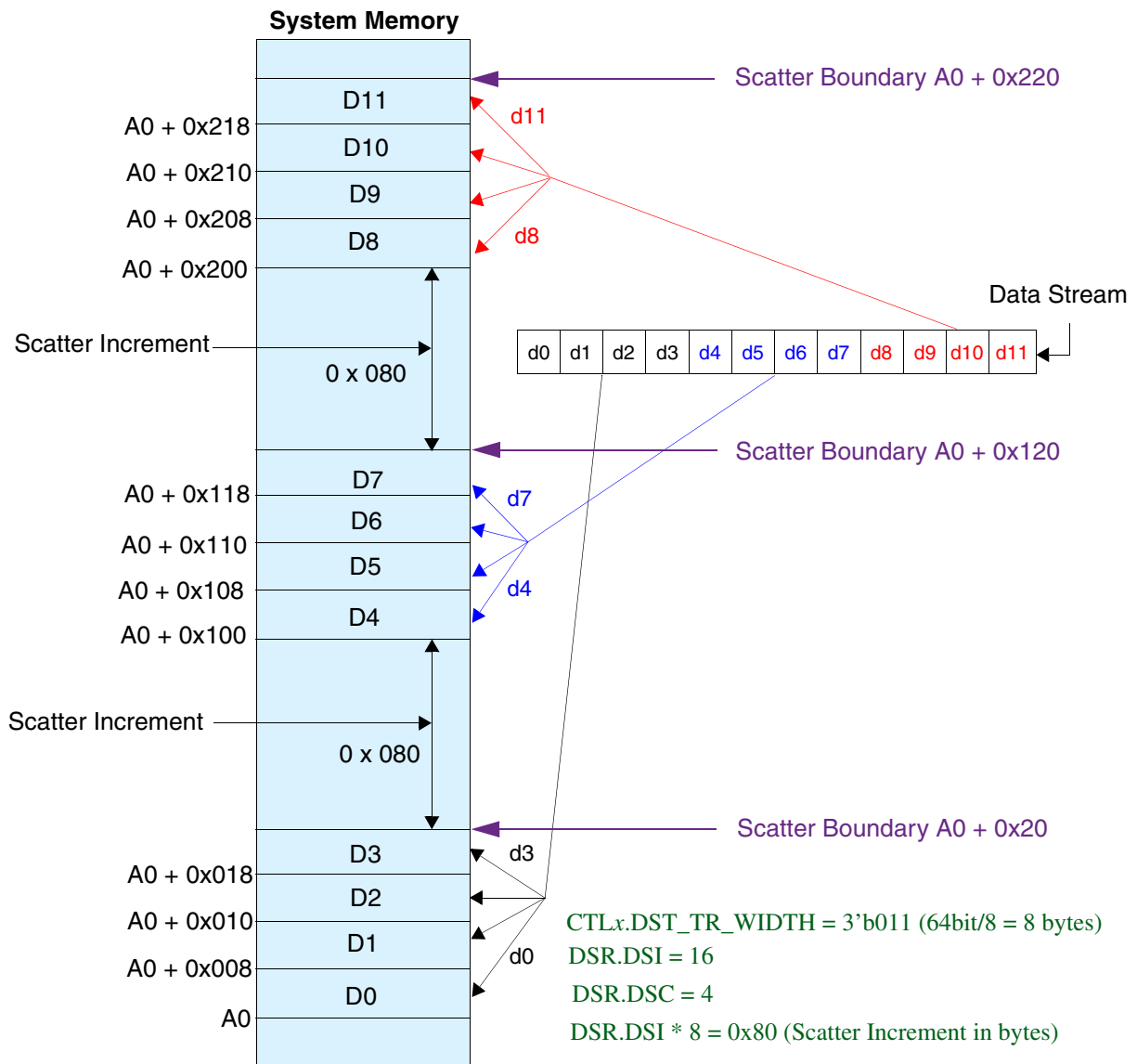


Note

For multi-block transfers, the counters that keep track of the number of transfers left to reach a gather/scatter boundary are re-initialized to the source gather count (SGRx.SGC) and destination scatter count (DSC), respectively, at the start of each block transfer.

Figure 3-41 shows an example of a destination scatter transfer:

Figure 3-41 Example of Destination Scatter Transfer



As an example of gather increment, consider the following:

$SRC_TR_WIDTH = 3'b\ 010$ (32 bit)

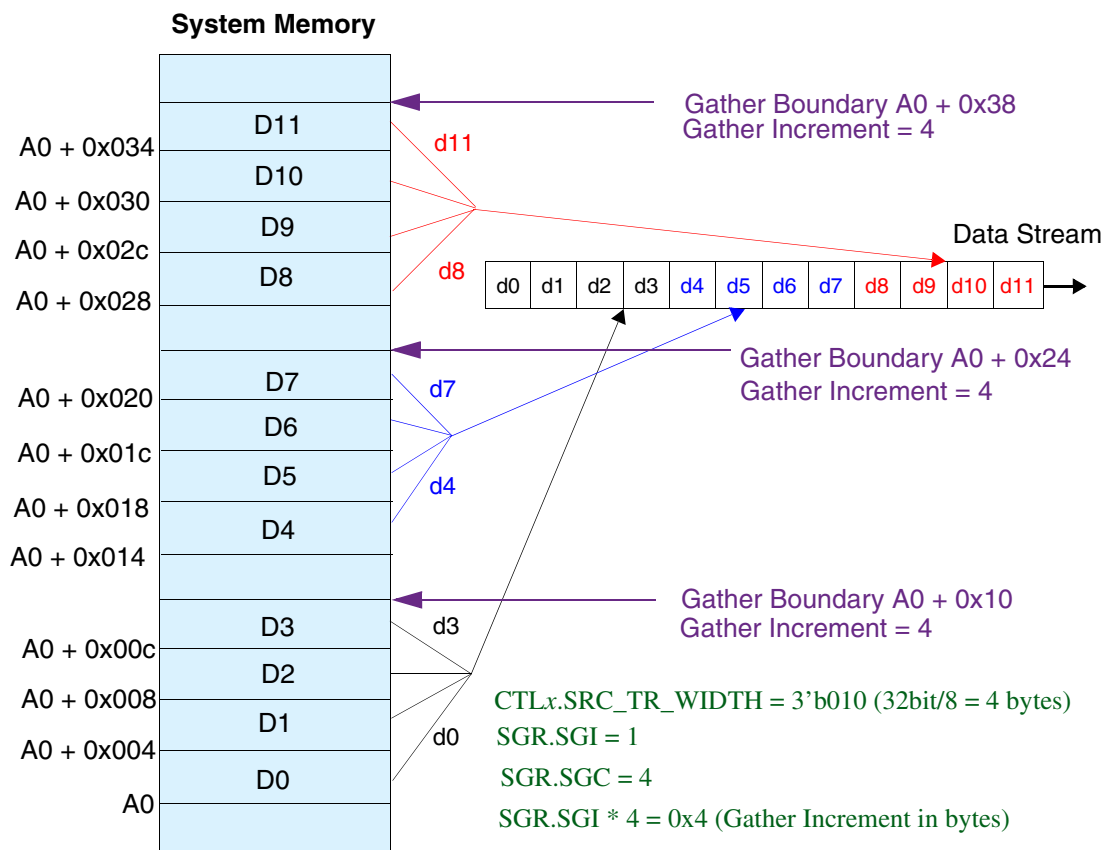
$SGR.SGC = 0x04$ (source gather count)

$CTLx.SRC_GATHER_EN = 1$ (source gather enabled)

$SARx = A0$ (starting source address)

Figure 3-42 shows a source gather when $\text{SGR.SGI} = 0x01$.

Figure 3-42 Source Gather when $\text{SGR.SGI} = 0x1$



In general, if the starting address is $A0$ and $\text{CTLx.SINC} = 2'b00$ (increment source address control), then the transfer will be:

$A0, A0 + \text{TWB}, A0 + 2 * \text{TWB}, \dots, (A0 + (\text{SGR.SGC} - 1) * \text{TWB})$
 <-scatter_increment-> $(A0 + (\text{SGR.SGC} * \text{TWB}) + (\text{SGR.SGI} * \text{TWB}))$

where TWB is the transfer width in bytes, decoded value of $\text{CTLx.SRC_TR_WIDTH} / 8 = \text{src_single_size_bytes}$.

3.16 Endianness

DW_ahb_dmac supports Little Endian, Address Invariant (AI) Big Endian scheme and BE-32 (Word Invariant) scheme of data access on AHB slave interface and each AHB master interface. Endian format can be either hard coded using coreConsultant configuration parameter or selected using a I/O pins (signal).

3.16.1 Big Endian-Little Endian Conversion Logic

The behavior of DW_ahb_dmac for Big Endian-Little Endian conversion using BE-32 (Word Invariant) method is as shown in the following figures.

Figure 3-43 BE-LE Conversion Using BE-32 (Word Invariant) Method for 32-Bit Data Bus

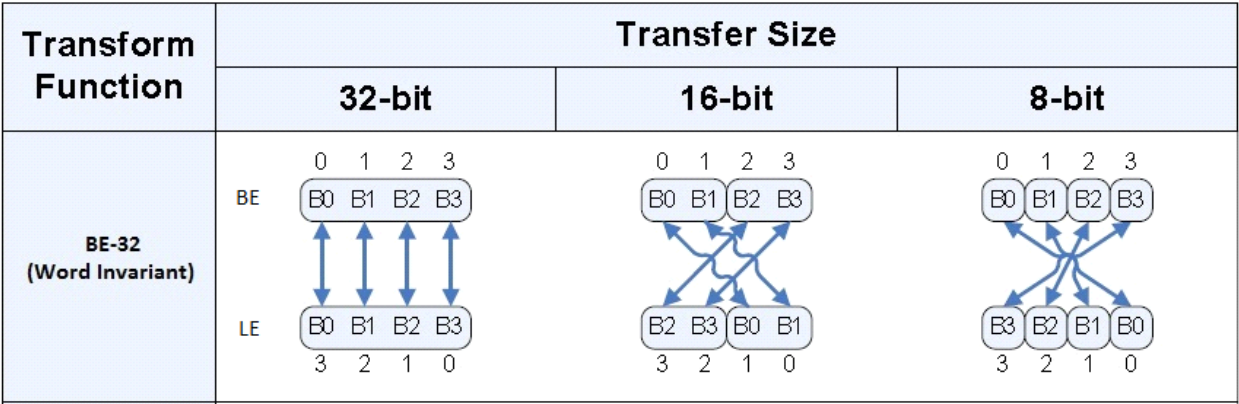


Figure 3-44 BE-LE Conversion Using BE-32 (Word Invariant) Method for 54-Bit Data Bus – Word Access

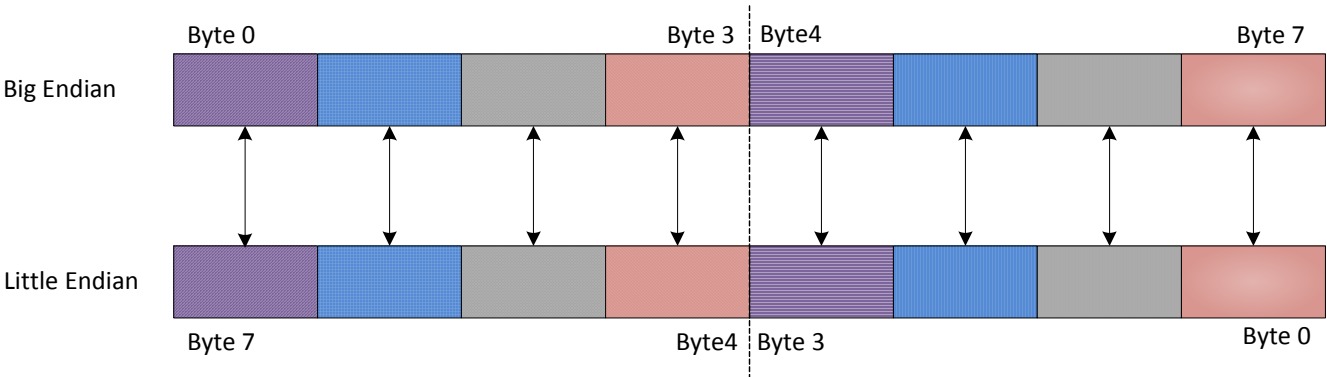
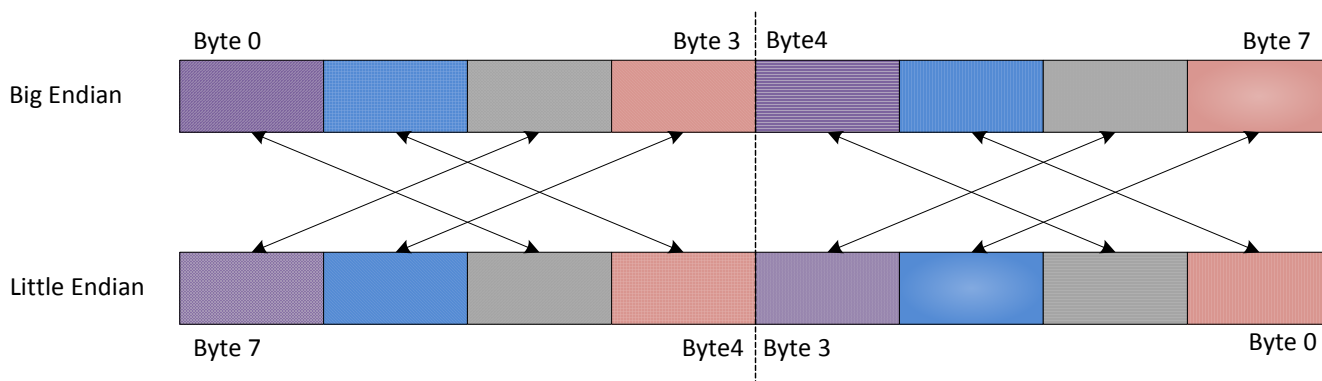
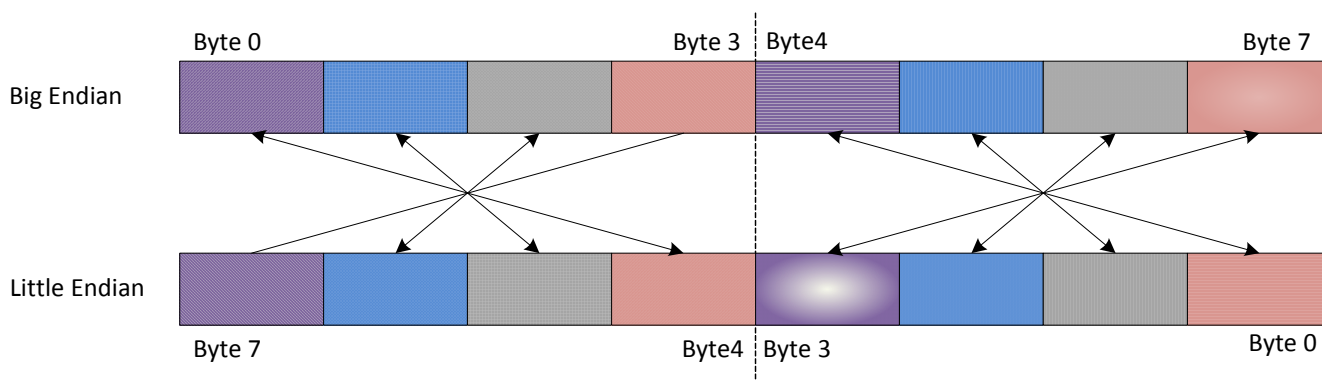


Figure 3-45 BE-LE Conversion Using BE-32 (Word Invariant) Method for 64-Bit Data Bus – Half Word Access**Figure 3-46 BE-LE Conversion Using BE-32 (Word Invariant) Method for 64-Bit Data Bus – Byte Access**

3.16.2 LLI Fetch, and Status and Control Write-Back

The DW_ahb_dmac can be programmed to use the Little Endian scheme whenever it fetches a Linked List Item (LLI) or writes Status and Control register data back to the LLI location, regardless of the endian format selected for the source and destination data transfer on the particular AHB master interface.

3.16.3 Endian Selection

The endian scheme used for the register access, for source and destination data transfer on master interfaces and LLI fetch, and for Status and Control write-back access on master interfaces is decided by the values of the coreConsultant parameters and the I/O signals. All possible combinations of endian scheme are captured in [Table 3-17](#) on page 113.

Table 3-17 Endian Scheme Selection118

DMAH_STATIC_ENDIAN_SELECT	DMAH_BIG_ENDIAN	dma_big_endian_slv/m1/m2/m3/m4	DMAH_BE32_SELECTION_PIN_EN	dma_be_select_be32_slv/m1/m2/m3/m4	Endian Scheme Used for Register Access on Slave Interface and SRC/DST Data Access on M1/M2/M3/M4 Master Interface	DMAH_LLI_ENDIAN_SELECTION_PIN_EN	dma_le_select_lli_m1/m2/m3/m4	Endian Scheme Used for LLI Fetch/ Status and Control Write Back Access on M1/M2/M3/M4 Master Interface
1	0	X	X	X	Little Endian	X	X	Little Endian
1	1	X	0	X	Big Endian AI	0	X	Big Endian AI
						1	0	Big Endian AI
							1	Little Endian
1	1	X	1	0	Big Endian AI	0	X	Big Endian AI
						1	0	Big Endian AI
							1	Little Endian
				1	Big Endian BE-32	0	X	Big Endian BE-32
						1	0	Big Endian BE-32
							1	Little Endian
0	X	0	X	X	Little Endian	X	X	Little Endian
0	X	1	0	X	Big Endian AI	0	X	Big Endian AI
						1	0	Big Endian AI
							1	Little Endian
0	X	1	1	0	Big Endian AI	0	X	Big Endian AI
						1	0	Big Endian AI
							1	Little Endian
				1	Big Endian BE-32	0	X	Big Endian BE-32
						1	0	Big Endian BE-32
							1	Little Endian

**Note**

The endian scheme used for the Status and Control write-back is the same as defined by the DMAH_LLI_ENDIAN_SELECTION_PIN_EN parameter and dma_le_select_llmN input signal, where $N = 1/2/3/4$ for the corresponding master interface even when LLP_SRC_EN = LLP_DST_EN = 0 for the corresponding channel. This is because Status and Control write-back happens to the address location derived from the base address defined by LLPx.LOC value (base address of LLI).

3.16.4 Static Endian Configuration

You can statically configure endianness under the following conditions:

- Endianness of the AHB slave interface and all configured AHB master interfaces are the same
- Endianness is known at system configuration time

Under these circumstances, you do the following:

1. Set the DMAH_STATIC_ENDIAN_SELECT parameter to True.
2. Configure the DMAC_BIG_ENDIAN parameter to either big-endian or little-endian.
3. Configure the DMAH_BE32_SELECTION_PIN_EN parameter and drive the appropriate I/O to either big-endian AI or big-endian BE-32

3.16.5 Dynamic Endian Configuration

You should dynamically configure endianness if either of the following conditions exists:

- Endianness of the AHB slave interface and all configured master interfaces are not all the same.
- Endianness of the slave interface or any master interface is not known at system configuration time.

Under either of these circumstances, do the following:

1. Set the DMAH_STATIC_ENDIAN_SELECT parameter to False.
2. Drive the appropriate I/O pin to either big-endian (AI or BE-32) or little-endian.

For example, if you were to drive dma_big_endian_mN to 1, where N is the master number, configure the DMAH_BE32_SELECTION_PIN_EN parameter to 1 and drive dma_be_select_be32_mN to 1, where N is the master number. The AMBA layer to which that master interface is attached is Big Endian BE-32. Conversely, if you were to drive dma_big_endian_mN to 0, that master interface would be Little Endian.

The dma_big_endian_slv signal works in the same way for the slave interface.

3.17 AHB Transfer Error Handling

Upon occurrence of an error in an AHB transfer, the following occurs:

1. DMA transfer in progress stops immediately
2. Relevant channel is disabled,
3. An interrupt is issued (if not masked)

If multiple channels are enabled, only the one where the AHB error was detected is disabled.

The contents of the FIFO are not cleared, but they become inaccessible and are overwritten once the channel is re-enabled to start a new sequence.

There is no support for automatically resuming the transfer from the point where the error occurred, and the full block transfer has to be re-initiated in order to be successfully completed.

The DMA does not use the hardware handshaking interface to signal the error occurrence in any way, nor does it signal the end of a transfer. In practice, this means that if a request from a peripheral is active when the error occurs – `dma_req` is high if peripheral is the flow controller; `dma_req` or `dma_single` are high if peripheral is not the flow controller – the channel is disabled without the DMA ever asserting `dma_ack` (or `dma_finish`).

The hardware handshake interface on the peripheral side has to be re-initiated by the CPU upon detection of the error interrupt. The `dma_req` signal needs to be brought low before the channel is re-enabled and then brought high when the channel has been enabled.

3.18 Last Beat of DMA Burst Indication

DW_ahb_dmac provides an additional output signal (dma_wlast) on the AHB interface to indicate the last write data during burst transfers to destination peripherals. The dma_wlast signal is available only when the parameter DMAH_WLAST_EN is set.

The dma_wlast signal is an active high signal that is asserted on completion of the last address phase of every destination MSIZE data transfer, similar to the wlast signal in the AXI protocol. The signal remains asserted until the associated last dataphase is complete. This signal is available on all AHB interfaces when the parameter DMAH_WLAST_EN is set.

The following example explains the behavior of the dma_wlast signal in different cases.

3.18.1 Example 1

Scenario: Example block transfer when the DW_ahb_dmac is the flow controller. The table lists the DMA parameters for this example.

Table 3-18 Parameters Used in Transfer Operation - Example 1

Parameter	Description
CTLx.TT_FC = 3'b011	Peripheral to peripheral transfer with DW_ahb_dmac as flow controller
CTLx.BLOCK_TS = 16	-
CTLx.SRC_TR_WIDTH = 3'b010	32 bit
CTLx.DST_TR_WIDTH = 3'b010	32 bit
CTLx.SRC_MSIZ = 3'b 010	Source burst transaction length = 8
CTLx.DEST_MSIZ = 3'b 010	Destination burst transaction length = 8
CFGx.MAX_ABRST = 1'b 0	No limit on maximum AMBA burst length
DMAH_CHx_FIFO_DEPTH = 64 bytes	-

Using equation (5), a total of 64 bytes is transferred in the block; that is, blk_size_bytes_dma = 64. This block transfer consists of two destination MSIZE data transfers of length 8 beats. The first MSIZE data transfer consists of two INCR4 AHB bursts. The second MSIZE data transfer consists of a single INCR8 AHB burst. The dma_wlast signal is asserted at the end of every destination MSIZE data transfer as shown in Figure 3-47.



Note

The dma_wlast signal indicates the last data beat of destination MSIZE data transfer only when the destination is peripheral. This signal is not applicable when the destination is memory.

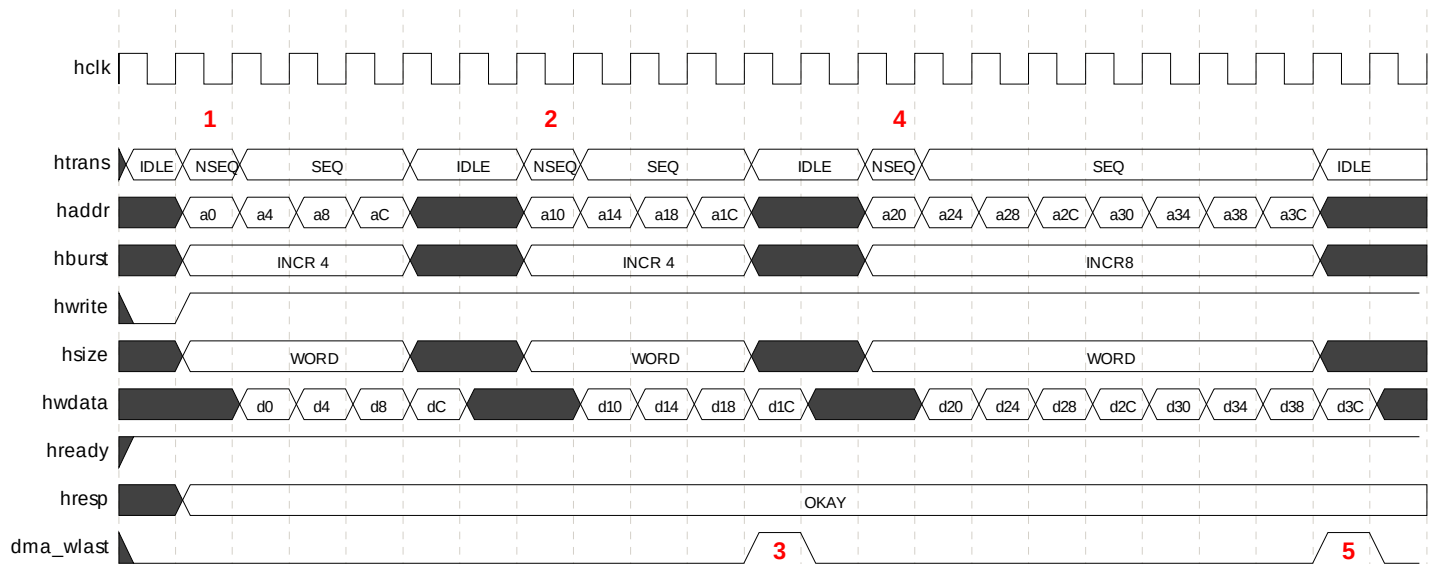
Figure 3-47 Assertion of the dma_wlast Signal

Figure 3-47 shows the following:

1. The first INCR4 AHB burst transfer of the first MSIZE data transfer.
2. The second INCR4 AHB burst transfer of the first MSIZE data transfer.
3. The signal `dma_wlast` asserted at the end of the first MSIZE data transfer.
4. A single INCR8 AHB burst transfer of the second MSIZE data transfer.
5. The `dma_wlast` signal asserted at the end of the second MSIZE data transfer.

The `dma_wlast` signal is asserted regardless of `HREADY` and is therefore pulse extended if `HREADY` is low, as shown in Figure 3-48.

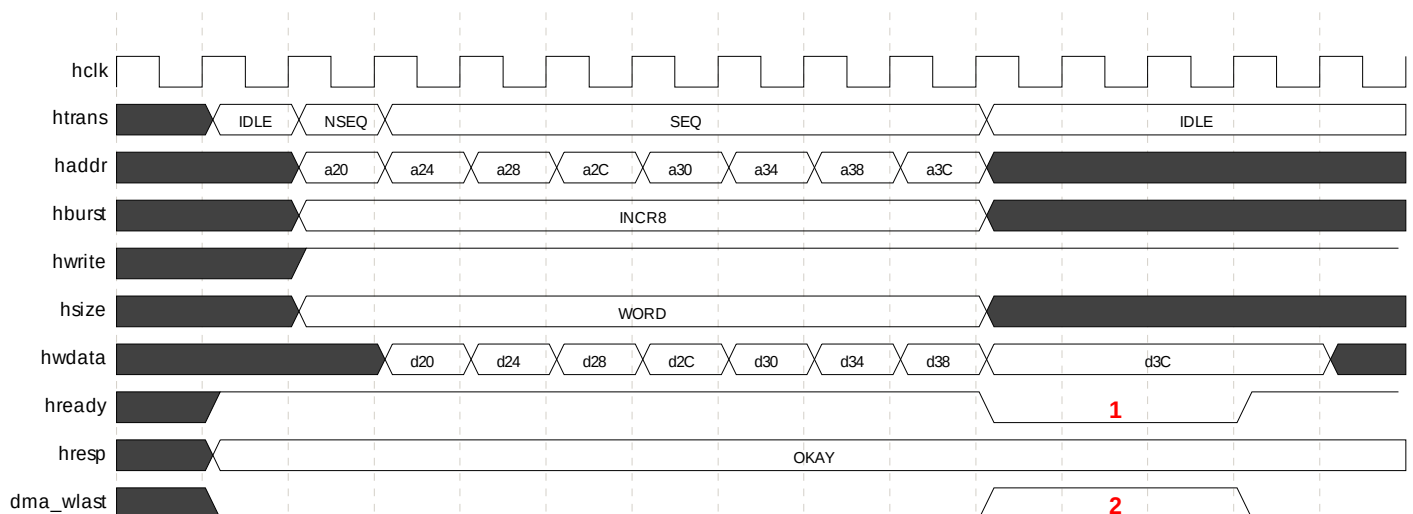
Figure 3-48 Behavior of the dma_wlast Signal During Wait State

Figure 3-48 shows the following:

1. Wait states are inserted on the last data beat of a destination MSIZE transfer.
2. The dma_wlast signal remains asserted till the last data beat completes.

Figure 3-49 shows the behavior of the dma_wlast signal when SPLIT/RETRY response is issued by a slave on the last beat of destination MSIZE data transfer.

Figure 3-49 Behavior of the dma_wlast Signal During SPLIT Response

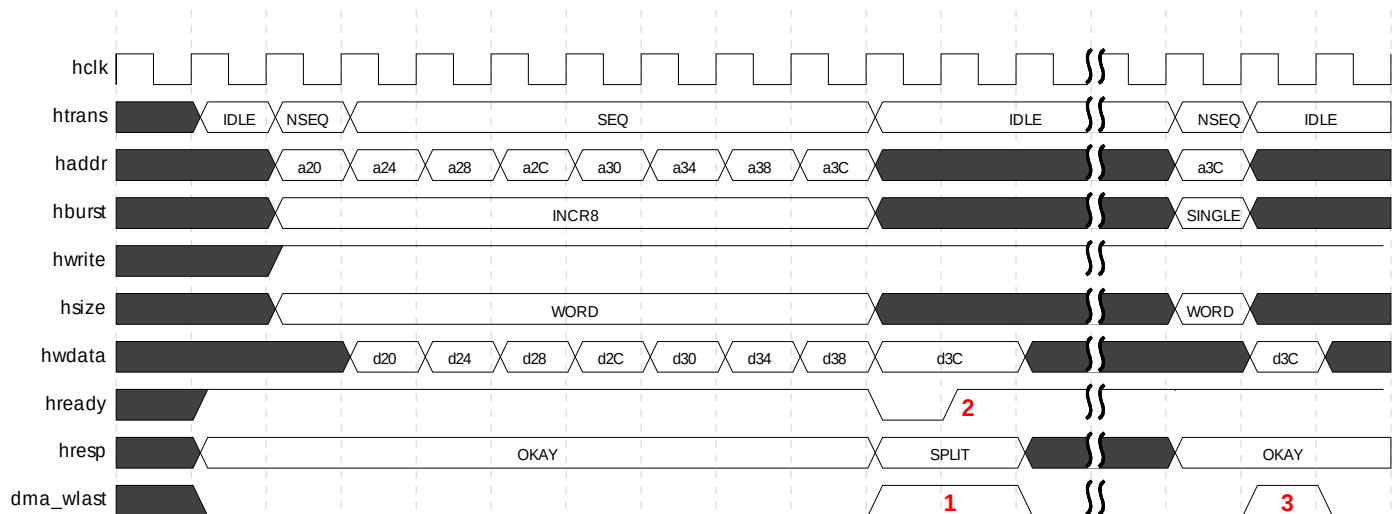


Figure 3-49 shows the following:

1. The dma_wlast asserted after the address phase of the last data beat of a destination MSIZE transfer.
2. The slave has issued a SPLIT response.
3. The dma_wlast again asserted while completing the last data beat of a destination MSIZE transfer.

4

Parameters

This chapter describes the parameters used by the DW_ahb_dmac. You use coreConsultant or coreAssembler to configure the following parameters and generate the configured code.



Attention

When using coreConsultant or coreAssembler, you can right-click on a parameter label to access a “What’s This” popup dialog that will tell you the details for that particular parameter. The information in each What’s This dialog essentially matches the information in the parameter descriptions below.

4.1 Parameter Descriptions

Table 4-1 lists the DW_ahb_dmac parameter descriptions. In this table, the values 0 and 1 occasionally appear in parentheses in the descriptions for the parameters. These are the logical values for parameter settings that appear in the coreConsultant GUI as check boxes, drop-down lists, a multiple selection, and so on.

Table 4-1 DW_ahb_dmac Parameters

Field Label	Parameter Definition
DMA Source Code Configuration	
Use DesignWare Foundation Synthesis Library	Parameter Name: USE_FOUNDATION Legal Values: True (1) or False (0) Default Value: True if DesignWare License is available; False if <i>no</i> DesignWare License is available Dependencies: Parameter is enabled if customer has both Source and DesignWare licenses Description: The component code utilizes DesignWare Foundation parts for optimal Synthesis QoR. Customers with only a DesignWare license <i>must</i> use Foundation parts. Customers with only a Source license <i>cannot</i> use Foundation parts. Customers with both Source and DesignWare licenses have the option of using DesignWare Foundation parts.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Global DMA Configuration	
Number of AHB master interfaces	Parameter Name: DMAH_NUM_MASTER_INT Legal Values: 1 to 4 Default Value: 1 Dependencies: None Description: Creates the specified number of AHB master interfaces. A channel source or destination device can be programmed to be on any of the configured AHB layers attached to the AHB master interface. This setting determines if a master interface signal set is present on the I/O or not. AHB master interface 1 signals are always present.
Number of DMA channels	Parameter Name: DMAH_NUM_CHANNELS Legal Values: 1 to 8 Default Value: 1 Dependencies: None Description: Creates the specified number of DW_ahb_dmac channels, each of which is unidirectional and transfers data from the channel source to the channel destination. The channel source and destination AHB layer, system address, and handshaking interface are under software control.
Number of handshaking interfaces	Parameter Name: DMAH_NUM_HS_INT Legal Values: 0 to 16 Default Value: 2 Dependencies: None Description: Creates the specified number of handshaking interfaces. You can program the DW_ahb_dmac to assign a handshaking interface for each channel source and destination (refer to “CFGx” on page 180). If 0 is selected, then no hardware handshaking signals are present on the I/O.
ID number	Parameter Name: DMAH_ID_NUM Legal Values: 0x0 to 0xffffffff Default Value: 0x0 Dependencies: None Description: A 32-bit value that is hardwired and read back by a read to the DW_ahb_dmac ID Register (DmaldReg).
Interrupts are active high?	Parameter Name: DMAH_INTR_POL Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: When this box is selected, the interrupt polarity for all interrupt pins on the I/O interface is active high. When this box is unselected, the interrupt polarity for all interrupt pins on the I/O interface is active low.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Interrupt pins to appear as outputs	Parameter Name: DMAH_INTR_IO Legal Values: ALL (0), TYPE (1), or COMBINED (2) Default Value: COMBINED (2) Dependencies: None Description: Selects which interrupt-related signals appear as outputs on the design. ■ All: Contents of all five Interrupt Status registers appear as output signals; 5 * DMAH_NUM_CHANNELS bits wide. ■ Type: All internal interrupts are combined by type, and appear on int_flag(_n) 5-bit bus. - bit[4] – Error interrupt - bit[3] – Destination transaction complete interrupt - bit[2] – Source transaction complete interrupt - bit[1] – Block complete interrupt - bit[0] – Transfer complete interrupt Also, the bitwise OR of all bits of the int_flag(_n) bus is driven onto the int_combined(_n) single bit output. ■ Combined: Bitwise OR of all bits of the int_flag(_n) bus is driven onto the int_combined(_n) single bit output.
Use undefined length bursts only?	Parameter Name: DMAH_INCR_BURSTS Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: When set to 1, the DW_ahb_dmac selects undefined length INCR bursts only. Setting this option to 0 allows the DW_ahb_dmac to select the largest valid defined length burst (SINGLE, INCR4, INCR8, INCR16) for the transfer.  Note It is not valid to send defined-length bursts to non-changing or decrementing addresses. If this option is set to 0, the DW_ahb_dmac selects undefined-length INCR bursts when CTLx.SINC or CTLx.DINC are set to Decrement or No Change.
Allow the AMBA burst length to be limited to a programmable maximum value?	Parameter Name: DMAH_MABRST Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: When set to 1, you can limit the maximum AMBA burst length to a value under software control by writing to the channel configuration register; this is a global parameter for all of the configured master interfaces. Setting this option to 0 allows for some logic optimization.  Note If this option is set to 0, then the maximum AMBA burst length is a function of the channel FIFO depths. The DW_ahb_dmac may try to fill or empty a channel FIFO in a single burst.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Allow the slave interface to return an error response when an illegal access is attempted?	Parameter Name: DMAH_RETURN_ERR_RESP Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: Setting this option to 1 allows the slave interface to return an error response on the hresp bus when an illegal access is attempted over the AHB slave interface. For a list of illegal accesses, refer to “Illegal Register Access” on page 246. Setting this option to 0 results in an OKAY response being returned on the hresp bus for all of the illegal accesses.
Remove Internal Pipeline Stages?	Parameter Name: DMAH_REMOVE_PIPELINING Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: Setting this parameter to 1 removes three pipeline stages internal to DW_ahb_dmac: ■ Input register on hardware handshaking interface input signals ■ Output register on master interface arbiter ■ Register on path from destination state machine to dma_ack* Removing these pipeline stages improves latency, but can result in reduced operating frequency. Setting this parameter to 0 keeps the three pipeline stages.
Add encoded parameters	Parameter Name: DMAH_ADD_ENCODED_PARAMS Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: Adding the encoded parameters gives firmware an easy and quick way of identifying the DesignWare component within an I/O memory map. Some critical design-time options determine how a driver should interact with the peripheral. There is a minimal area overhead when you include these parameters. Additionally, this option allows a self-configurable single driver to be developed for each component.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Reverse order of LLI Status WriteBack	Parameter Name: DMAH_REVERSE_WB_OVERRIDE Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: At least one of DMAH_CHx_CTL_WB_EN parameters equal to 1 Description: When the DW_ahb_dmac writes back the status information to the LLI memory location, the following order is used: 1. DSTATx 2. SSTATx 3. CTRLx When this parameter is set, the write-back order is changed to: 4. CTRLx 5. SSTATx 6. DSTATx
Include logic to enable dma_wlast[N] signal?	Parameter Name: DMAH_WLAST_EN Legal Values: True(1) or False(0) Default Value: False(0) Dependencies: This parameter is enabled only if DMAH_NUM_HS_INT parameter is greater than 0. Description: Enables the additional handshaking signal dma_wlast on all selected AHB master interfaces. The dma_wlast signal is asserted on completion of the last address phase of every destination transaction and remains asserted until the associated last data phase completes.
Configuration of AMBA Layers	
Specify AMBA Layer Parameters	
Statically configure endianness through coreConsultant?	Parameter Name: DMAH_STATIC_ENDIAN_SELECT Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: The endian scheme of the DW_ahb_dmac can be configured statically through coreConsultant or dynamically via pins on the I/O. For the static case, there is a single coreConsultant parameter that selects between Little Endian/Big Endian for all AHB master interfaces and the AHB slave interface. For the dynamic case, there is an individual pin for each of the AHB master interfaces and one for the AHB slave interface. Selection between AI and BE-32 method for Big Endian access is controlled by additional parameters defined for slave interface and each master interface.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
System is big endian?	Parameter Name: DMAH_BIG_ENDIAN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: This parameter is enabled only if DMAH_STATIC_ENDIAN_SELECT is set to true. Description: The AHB master and slave interfaces can be configured to exist in either big or little endian system. When set to 1, all master interfaces and the slave interface become big endian. Otherwise, they are set to little endian. Selection between AI and BE-32 methods for big endian access is controlled by additional parameters and inputs defined for slave interface and each master interface.
AI/BE-32 Big Endian scheme selection pin on AHB Slave and Master Interfaces?	Parameter Name: DMAH_BE32_SELECTION_PIN_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: ■ This parameter is disabled (0) if (DMAH_STATIC_ENDIAN_SELECT = 1 AND DMAH_BIG_ENDIAN = 0) ■ This parameter has effect only if DMAH_STATIC_ENDIAN_SELECT = 1 AND DMAH_BIG_ENDIAN = 1 OR DMAH_STATIC_ENDIAN_SELECT = 0 AND dma_big_endian_slv/m1/m2/m3/m4 = 1 Description: If this parameter is enabled, additional inputs are enabled to control the scheme used for big endian access (individual pin is added for AHB Slave interface and each of the AHB Master interfaces). AHB Slave interface and each AHB Master interfaces can be independently configured to support Address Invariant (AI) or Word Invariant (BE-32) Big Endian scheme if this interface is configured for Big Endian access. ■ 0: Address Invariant(AI) method is used for Big Endian access for Slave and Master 1/2/3/4 Interfaces (current scheme) ■ 1: Address Invariant(AI) or Word Invariant(BE-32) method is used for Big Endian access for Slave and Master 1/2/3/4 Interfaces depending on the value of dma_be_select_be32_slv/mN, input where $N = 1/2/3/4$

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Add Little Endian scheme selection pin for LLI access on AHB Master Interfaces?	Parameter Name: DMAH_LLI_ENDIAN_SELECTION_PIN_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: ■ This parameter is disabled (= 0) if DMAH_STATIC_ENDIAN_SELECT = 1 AND DMAH_BIG_ENDIAN = 0 ■ This parameter has effect only if DMAH_STATIC_ENDIAN_SELECT = 1 AND DMAH_BIG_ENDIAN = 1 OR DMAH_STATIC_ENDIAN_SELECT = 0 AND dma_big_endian_m1/m2/m3/m4 = 1 Description: If this parameter is enabled, additional inputs are enabled to control the endian scheme used for LLI fetch and Status and Control writeback. (Individual pin is added for each of the AHB Master interfaces.) LLI fetch and Status and Control write-back access on each AHB Master interfaces can be independently configured to support Big Endian scheme (AI or BE-32 based on the endian scheme selected for that particular Master interface) or Little Endian scheme (irrespective of the endian scheme selected for that particular Master interface). ■ 0: Endian scheme used for LLI fetch and Status & Control write-back is same as that used for data access for M1/M2/M3/M4 interfaces (current scheme) ■ 1: Endian scheme used for LLI fetch and Status & Control write-back access is same as that used for data access for M1/M2/M3/M4 interfaces or Little Endian method is used for LLI fetch and Status and Control writeback access irrespective of the endian scheme used for data access depending on the value of dma_le_select_ll_i_mN input, where $N = 1/2/3/4$.
Slave interface data bus width	Parameter Name: DMAH_S_HDATA_WIDTH Legal Values: 32, 64, 128, 256 bits Default Value: 32 Dependencies: None Description: Specifies the data bus width for the AHB slave interface.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Is Layer 1 AMBA-Lite?	Parameter Name: DMAH_M1_AHB_LITE Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: DMAH_NUM_MASTER_INT >= 1 Description: Select True (1) if master interface 1 is the only master interface on this AMBA layer. If this is the case, then this is an AMBA-Lite layer. Even though the DMAH_M1_AHB_LITE parameter is associated with master behavior, this parameter configures the DW_ahb_dmac in AHB Lite mode for slave functions as well. When the DMAH_M1_AHB_LITE parameter is configured for AHB Lite mode, the SPLIT and RETRY functions are disabled for both master and slave functions. For more information about AHB Lite, refer to the DesignWare DW_ahb Databook.
Master 1 interface data bus width	Parameter Name: DMAH_M1_HDATA_WIDTH Legal Values: 32, 64, 128, or 256 bits Default Value: 32 bits Dependencies: DMAH_NUM_MASTER_INT >= 1 Description: AHB master 1 interface data bus width.
Is Layer 2 AMBA-Lite?	Parameter Name: DMAH_M2_AHB_LITE Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: Option is available if DMAH_NUM_MASTER_INT >= 2. Description: Select True (1) if master interface 2 is the only master interface on this AMBA layer. If this is the case, then this is an AMBA-Lite layer.
Master 2 interface data bus width	Parameter Name: DMAH_M2_HDATA_WIDTH Legal Values: 32, 64, 128, or 256 bits Default Value: 32 bits Dependencies: Option is available if DMAH_NUM_MASTER_INT >= 2. Description: AHB master 2 interface data bus width.
Is Layer 3 AMBA-Lite?	Parameter Name: DMAH_M3_AHB_LITE Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: Option is available if DMAH_NUM_MASTER_INT >= 3. Description: Select Yes (1) if master interface 3 is the only master interface on this AMBA layer. If this is the case, then this is an AMBA-Lite layer.
Master 3 interface data bus width	Parameter Name: DMAH_M3_HDATA_WIDTH Legal Values: 32, 64, 128, or 256 bits Default Value: 32 bits Dependencies: Option is available if DMAH_NUM_MASTER_INT >= 3. Description: AHB master 3 interface data bus width.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Is Layer 4 AMBA-Lite?	Parameter Name: DMAH_M4_AHB_LITE Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: Option is available if DMAH_NUM_MASTER_INT = 4. Description: Select Yes (1) if master interface 4 is the only master interface on this AMBA layer. If this is the case, then this is an AMBA-Lite layer.
Master 4 interface data bus width	Parameter Name: DMAH_M4_HDATA_WIDTH Legal Values: 32, 64, 128, or 256 bits Default Value: 32 bits Dependencies: Option is available if DMAH_NUM_MASTER_INT = 4. Description: AHB master 4 interface data bus width.
Channel x Configuration, where x = 0 to 7	
Channel x FIFO depth in bytes	Parameter Name: DMAH_CHx_FIFO_DEPTH Legal Values: 8, 16, 32, 64, 128, 256 bytes Default Value: 16 bytes Dependencies: None Description: Channel x FIFO depth in bytes.
Maximum value of burst transaction size	Parameter Name: DMAH_CHx_MAX_MULT_SIZE Legal Values: 4, 8, 16, 32, 64, 128, and 256 Default Value: 8 Dependencies: None Description: Maximum value of burst transaction size that can be programmed for channel x (CTLx.SRC_MSIZ and CTLx.DEST_MSIZ). Limiting DMAH_CHx_MAX_MULT_SIZE to a maximum value will allow for some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Maximum block size in source transfer widths	Parameter Name: DMAH_CHx_MAX_BLK_SIZE Legal Values: 3, 7, 15, 31, 63, 127, 255, 511, 1023, 2047, 4095 Default Value: 31 Dependencies: None Description: The description of this parameter is dependent on what is assigned as the flow controller. DW_ahb_dmac flow controller: Maximum block size, in multiples of source transfer width, that can be programmed for channel x. A programmed value greater than this will result in erroneous behavior. Source/destination assigned as flow controller: In this case, the blocks can be greater than DMAH_CHx_MAX_BLK_SIZE in size, but the logic that keeps track of the size of a block saturates at DMAH_CHx_MAX_BLK_SIZE. This does not result in erroneous behavior, but a readback by software of the block size is incorrect when the block size exceeds the saturated value. This parameter is also used to limit the gather and scatter count registers to a maximum value of DMAH_CHx_MAX_BLK_SIZE. Limiting DMAH_CHx_MAX_BLK_SIZE to a maximum value allows some logic optimization of the implementation.
Hardcode channel x's flow control device to allow for logic optimization	Parameter Name: DMAH_CHx_FC Legal Values: DMA_FC_ONLY (0), SRC_FC_ONLY (1), DST_FC_ONLY (2), or ANY_FC (3) Default Value: DMA_FC_ONLY(0) Dependencies: None Description: Hardcodes the flow control peripheral for the channel. If ANY_FC is selected, then the flow control device is not hardcoded, and software selects the flow control device for a DMA transfer. Hardcoding the flow control device allows some logic optimization of the implementation.
Include logic to enable channel or bus locking on channel x?	Parameter Name: DMAH_CHx_LOCK_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: If set to 1, includes logic to enable channel or bus-level locking on channel x. When set to 1, then software can program the DW_ahb_dmac to assert hlock over the DMA transfer, block transfer, or transaction. When set to 1, then software can also program the DW_ahb_dmac to lock the arbitration for the master bus interface over the DMA transfer, block transfer, or transaction. Disabling this option allows some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Hardcode the master interface attached to the source of channel x	Parameter Name: DMAH_CHx_SMS Legal Values: MASTER_1 (0), MASTER_2 (1), MASTER_3 (2), MASTER_4 (3), NO_HARDCODE (4) Default Value: - MASTER_1(0) – if DMAH_NUM_MASTER_INT = 1 - NO_HARDCODE – if DMAH_NUM_MASTER_INT > 1 Description: Hardcode the AHB master interface attached to the source of channel x. If this is not hardcoded, then software can program the source of channel x to be attached to any of the configured layers. Hardcoding this value allows some logic optimization of the implementation.
Hardcode the master interface attached to the destination of channel x	Parameter Name: DMAH_CHx_DMS Legal Values: MASTER_1 (0), MASTER_2 (1), MASTER_3 (2), MASTER_4 (3), NO_HARDCODE (4) Default Value: - MASTER_1(0) – if DMAH_NUM_MASTER_INT = 1 - NO_HARDCODE – if DMAH_NUM_MASTER_INT > 1 Description: Hardcode the AHB master interface attached to the channel x destination. If this is not hardcoded, then software can program the destination of channel x to be attached to any of the configured layers. Hardcoding this value allows some logic optimization of the implementation.
Hardcode channel x's source transfer width	Parameter Name: DMAH_CHx_STW Legal Values: BYTE (8), HALFWORD (16), WORD (32), TWO_WORD (64), FOUR_WORD (128), EIGHT_WORD (256), or NO_HARDCODE (0) Default Value: WORD (32) Dependencies: None Description: Hardcode the source transfer width for transfers from the source of channel x. If this is not hardcoded, then software can program the source transfer width. Hardcoding the source transfer width allows some logic optimization of the implementation.
Hardcode channel x's destination transfer width	Parameter Name: DMAH_CHx_DTW Legal Values: BYTE (8), HALFWORD (16), WORD (32), TWO_WORD (64), FOUR_WORD (128), EIGHT_WORD (256), or NO_HARDCODE (0) Default Value: WORD (32) Dependencies: None Description: Hardcode the destination transfer width for transfers from the destination of channel x. If this is not hardcoded, then software can program the destination transfer width. Hardcoding the destination transfer width allows some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Can the source of channel <i>x</i> return a non-OK response on hresp?	Parameter Name: DMAH_CHx_SRC_NON_OK Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: Set this parameter to 1 if the source peripheral attached to channel <i>x</i> can return a non-OK response on hresp, such as a SPLIT, RETRY, or ERROR response. If set to 0, then hardware assumes that only OK responses are returned from the source peripheral attached to the channel. Setting this parameter to 0 allows some logic optimization of the implementation.
Can the destination of channel <i>x</i> return a non-OK response on hresp?	Parameter Name: DMAH_CHx_DST_NON_OK Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: None Description: Set this parameter to 1 if the destination peripheral attached to channel <i>x</i> can return a non-OKAY response on hresp, such as a SPLIT, RETRY, or ERROR response. If set to 0, then hardware assumes that only OKAY responses are returned from the destination peripheral attached to the channel. Setting this parameter to 0 allows some logic optimization of the implementation.
Include logic to enable multi-block DMA transfers on channel <i>x</i> ?	Parameter Name: DMAH_CHx_MULTI_BLK_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: Includes or excludes logic to enable multi-block DMA transfers on channel <i>x</i>. If this option is set to 0, then hardware hardwires channel <i>x</i> to perform only single block transfers. Setting this parameter to 0 allows some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Choose type of multi-blocks to be supported?	Parameter Name: DMAH_CHx_MULTI_BLK_TYPE Legal Values: ■ NO_HARDCODE (0) – Allow all types of multi-support. ■ CONT_RELOAD (1) – Allow only multi-block transfers where SARx is contiguous; DAR and CTL are reloaded from their initial values. ■ RELOAD_CONT (2) – Allow only multi-block transfers where SARx and CTLx are reloaded from their initial values; DARx is contiguous. ■ RELOAD_RELOAD (3) – Allow only multi-block transfers where SARx, DARx, and CTLx are reloaded from their initial values. ■ CONT_LL (4) – Allow only multi-block transfers where SARx is contiguous; DARx, CTLx, and LLPx are loaded from the next linked list item. ■ RELOAD_LL (5) – Allow only multi-block transfers where SARx is reloaded from its initial value; DARx, CTLx, and LLPx are loaded from the next linked list item. ■ LLP_CONT (6) – Allow only multi-block transfers where SARx, CTLx, and LLPx are loaded from the next linked list item; DARx is contiguous. ■ LLP_RELOAD (7) – Allow only multi-block transfers where SARx, CTLx, and LLPx are loaded from the next linked list item; DARx is reloaded from its initial values. ■ LLP_LL (8) – Allow only multi-block transfers where SARx, DARx, CTLx, and LLPx are loaded from the next linked list item. Default Value: NO_HARDCODE Dependencies: DMAH_CHx_MULTI_BLK_EN = 1 Description: This parameter allows you to hardcode the type of multi-block transfers that DW_ahb_dmac can perform. This results in some logic optimization of the implementation.
Hardcode Channel x LLP register to 0?	Parameter Name: DMAH_CHx_HC_LL Legal Values: True (1) or False (0) Default Value: True (1) Dependencies: (DMAH_CHx_MULTI_BLK_TYPE == 0 or DMAH_CHx_MULTI_BLK_TYPE >= 4) Description: If set to 1, hardcodes channel x Linked List Pointer register to 0 (LLPx.LOC == 0), which disables the following features: ■ Multi-block support through block chaining ■ Source and destination status fetch ■ Control and source/destination status write back Multi-block DMA transfers can still be enabled on channel x through auto-reloading of channel registers. Hardcoding this value allows some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Fetch status from destination of channel x?	Parameter Name: DMAH_CHx_STAT_DST Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: DMAH_CHx_HC_LLP==0 Description: Include or exclude logic to fetch a status register from the destination peripheral of channel x and write this status information to system memory at the end of each block transfer. Disabling this feature allows some logic optimization of the implementation.
Fetch status from source of channel x?	Parameter Name: DMAH_CHx_STAT_SRC Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: DMAH_CHx_HC_LLP == 0 Description: Include or exclude logic to fetch a status register from source peripheral of channel x and write this status information to memory at end of each block transfer. Disabling this feature allows some logic optimization of the implementation.
Hardcode the master interface attached to the LLP peripheral of channel x	Parameter Name: DMAH_CHx_LMS Legal Values: MASTER_1 (0), MASTER_2 (1), MASTER_3 (2), MASTER_4 (3), NO_HARDCODE (4) Default Value: MASTER_1 (0) Dependencies: DMAH_CHx_HC_LLP == 0 and DMAH_NUM_MASTER_INT > 1 and (DMAH_CHx_MULTI_BLK_TYPE == 0 or DMAH_CHx_MULTI_BLK_TYPE >= 4) Description: Hardcode the AHB master interface attached to the peripheral that stores the LLI information (Linked List Item) for channel x. If this is not hardcoded, then software can program the peripheral that stores the LLI information of channel x to be attached to any of the configured layers. Hardcoding this value allows some logic optimization of the implementation. LLI accesses are always 32-bit accesses (Hsize = 2) aligned to 32-bit boundaries and cannot be changed or programmed to anything other than 32-bit, even if the AHB master interface of the LLI supports more than a 32-bit data width.
Can the LLP peripheral of channel x return a non-OK response on hresp?	Parameter Name: DMAH_CHx_LLIP_NON_OK Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: DMAH_CHx_HC_LLIP == 0 and (DMAH_CHx_MULTI_BLK_TYPE == 0 or DMAH_CHx_MULTI_BLK_TYPE >= 4) Description: Set this parameter to 1 if the LLP peripheral attached to channel x can return a non-OK response on hresp, such as a SPLIT, RETRY, or ERROR response. If set to 0, then hardware assumes that only OK responses are returned from the LLP peripheral attached to the channel. Setting this parameter to 0 allows some logic optimization of the implementation.

Table 4-1 DW_ahb_dmac Parameters (Continued)

Field Label	Parameter Definition
Include logic to enable the 'gather' feature on channel x?	Parameter Name: DMAH_CHx_SRC_GAT_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: Includes or excludes logic to enable the gather feature on channel x. Setting this option to 0 allows some logic optimization of the implementation.
Include logic to enable 'scatter' feature on channel x?	Parameter Name: DMAH_CHx_DST_SCA_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: None Description: Includes or excludes logic to enable the scatter feature on channel x. Setting this option to 0 allows some logic optimization of the implementation.
Include logic to enable control register writeback after each block transfer?	Parameter Name: DMAH_CHx_CTL_WB_EN Legal Values: True (1) or False (0) Default Value: False (0) Dependencies: DMAH_CHx_HC_LLP == 0 and DMAH_CHx_STAT_SRC == 0 and DMAH_CHx_STAT_DST == 0 Description: Includes or excludes logic to enable writeback of the CTLx, SSTATx, and DSTATx registers at the end of every block transfer. Disabling this feature allows some logic optimization of the implementation. If DMAH_CHx_HC_LLP == 1, then DMAH_CHx_CTL_WB_EN is set to 0 and not changeable. If DMAH_CHx_STAT_SRC == 1 or DMAH_CHx_STAT_DST == 1, then DMAH_CHx_CTL_WB_EN is set to 1 and not changeable.

5

Signal Descriptions

This chapter details all possible I/O signals in the core. For configurable IP titles, your actual configuration might not contain all of these signals.

Inputs are on the left of the signal diagrams; outputs are on the right.

Attention: For configurable IP titles, do not use this document to determine the exact I/O footprint of the core. It is for reference purposes only.

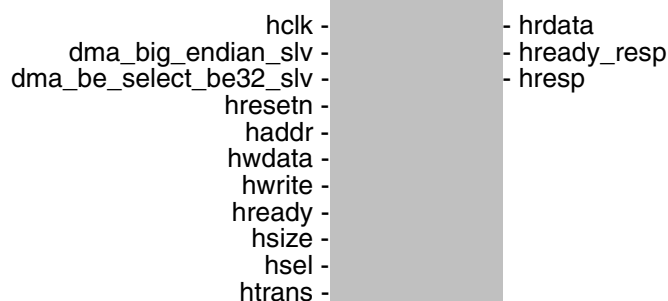
When you configure the core in coreConsultant, you must access the I/O signals for your actual configuration at workspace/report/IO.html or workspace/report/IO.xml after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the I/O signals that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the widths might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the core in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

The I/O signals are grouped as follows:

- Slave Interface on [page 136](#)
- Master N Interface on [page 139](#)
- Peripheral Handshaking Interface on [page 142](#)
- Interrupt Interface on [page 144](#)
- Debug Bus Interface on [page 146](#)

5.1 Slave Interface Signals



hclk -
 dma_big_endian_slv -
 dma_be_select_be32_slv -
 hresetn -
 haddr -
 hwddata -
 hwrite -
 hready -
 hsize -
 hsel -
 htrans -
 hrdata
 hready_resp
 hresp

Table 5-1 Slave Interface Signals

Port Name	I/O	Description
hclk	I	Bus clock. Common clock for all layers Exists: Always Active State: High. Active on rising edge. Synchronous to: N/A Registered: N/A
dma_big_endian_slv	I	Slave endianness. When active high, this signal indicates big endianness on the AMBA layer or the slave interface. When active low, this signal indicates little endianness. Exists: DMAH_STATIC_ENDIAN_SELECT==0 Active State: 1 for big endian; 0 for little endian Synchronous to: hclk Registered: No
dma_be_select_be32_slv	I	Slave interface Big Endian format selection pin. 0: Big Endian AI format is used for data transfer on slave interface. 1: Big Endian BE-32format is used for data transfer on slave interface Exists: DMAH_BE32_SELECTION_PIN_EN==1 Active State: N/A Synchronous to: hclk Registered: No

Table 5-1 Slave Interface Signals (Continued)

Port Name	I/O	Description
hresetn	I	Bus reset. Common reset for all layer. Asynchronously asserts and synchronously removed. Exists: Always Active State: Low Synchronous to: N/A Registered: N/A
haddr[(DMAH_HADDR_WIDTH-1):0]	I	Slave port address Exists: Always Active State: N/A. Synchronous to: hclk Registered: No
hwdata[(DMAH_S_HDATA_WIDTH-1):0]	I	Slave port write data Exists: Always Active State: N/A Synchronous to: hclk Registered: No
hwrite	I	Slave port transfer direction. When active high, this signal indicates a write transfer. When active low, this signal indicates a read transfer. Exists: Always Active State: 1 for Write; 0 for Read Synchronous to: hclk Registered: No
hready	I	Current transfer is complete Exists: Always Active State: High Synchronous to: hclk Registered: No
hsize[2:0]	I	Slave port transfer size Exists: Always Active State: N/A Synchronous to: hclk Registered: No
hsel	I	Slave select. Asserted when the current transfer on the AHB is intended for the DW_ahb_dmac Exists: Always Active State: High Synchronous to: hclk Registered: No

Table 5-1 Slave Interface Signals (Continued)

Port Name	I/O	Description
htrans[1:0]	I	Slave port transfer control Exists: Always Active State: N/A Synchronous to: hclk Registered: No
hrdata[(DMAH_S_HDATA_WIDTH-1):0]	O	Slave port read data Exists: Always Active State: N/A Synchronous to: hclk Registered: No
hready_resp	O	Transfer complete Exists: Always Active State: High Synchronous to: hclk Registered: Yes
hresp[1:0]	O	Slave port response type from slave. When DW_ahb_dmac is configured for AHB Lite mode and instantiated in an AHB Lite system, the HRESP0 signal is connected to the HRESP signal in the AHB Bus Fabric; the HRESP1 signal is left unconnected. Exists: Always Active State: N/A Synchronous to: hclk Registered: Yes

5.2 Master N Interface (for N = 1; N <= DMAH_NUM_MASTER_INT) Signals

dma_big_endian_mN	-	haddrN
dma_be_select_be32_mN	-	hwdataN
dma_le_select_lli_mN	-	hwriteN
hrdataN	-	hbusreqN
hgrantN	-	hlockN
hrespN	-	htransN
hreadyN	-	hburstN
	-	hsizeN
	-	hprotN

Table 5-2 Master N Interface (for N = 1; N <= DMAH_NUM_MASTER_INT) Signals

Port Name	I/O	Description
dma_big_endian_mN	I	Master endianness. When active high, this signal indicates big endianness on the AMBA layer or the master interface. When active low, this signal indicates little endianness. Exists: DMAH_STATIC_ENDIAN_SELECT==0 Active State: 1 for big endian; 0 for little endian Synchronous to: hclk Registered: No
dma_be_select_be32_mN	I	Master interface Big Endian format selection pin. 0: Big Endian AI format is used for data transfer on master interface. 1: Big Endian BE-32format is used for data transfer on master interface Exists: DMAH_BE32_SELECTION_PIN_EN==1 Active State: N/A Synchronous to: hclk Registered: No
dma_le_select_lli_mN	I	Big Endian format selection pin for LLI fetch and status and control write-back for the Master N Interface. 0 : Endian scheme used for LLI fetch and status & control write-back access on master interface is same as that used for data access for master interface. 1 : Little Endian scheme used for LLI fetch and status & control write-back access on master interface irrespective of the endian scheme used for data access on master interface. Exists: DMAH_LLI_ENDIAN_SELECTION_PIN_EN==1 Active State: N/A Synchronous to: hclk Registered: No

Table 5-2 Master N Interface (for N = 1; N <= DMAH_NUM_MASTER_INT) Signals (Continued)

Port Name	I/O	Description
hrdataN[(DMAH_M1_HDATA_WIDTH-1):0]	I	Master port read data Exists: DMAH_NUM_MASTER_INT >= N Active State: N/A Synchronous to: hclk Registered: No
hgrantN	I	Bus grant Exists: DMAH_NUM_MASTER_INT >= N Active State: High Synchronous to: hclk Registered: No
hrespN[1:0]	I	Master port response type from slave. When DW_ahb_dmac is configured for AHB Lite mode and instantiated in an AHB Lite system, the HRESP0 signal is connected to the HRESP signal in the AHB Bus Fabric; the HRESP1 signal is left unconnected. Exists: DMAH_NUM_MASTER_INT >= N Active State: N/A Synchronous to: hclk Registered: No
hreadyN	I	Indicates that the current transfer is completed. Exists: DMAH_NUM_MASTER_INT >= N Active State: High Synchronous to: hclk Registered: No
haddrN[(DMAH_HADDR_WIDTH-1):0]	O	Master port address Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes
hwdataN[(DMAH_M1_HDATA_WIDTH-1):0]	O	Master port write data Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes
hwriteN	O	Master port transfer direction. When active high, this signal indicates a write transfer. When active low, this signal indicates a read transfer. Exists: DMAH_NUM_MASTER_INT >= N Active State: 1 for Write; 0 for Read Synchronous to: hclk Registered: Yes

Table 5-2 Master N Interface (for N = 1; N <= DMAH_NUM_MASTER_INT) Signals (Continued)

Port Name	I/O	Description
hbusreqN	O	Bus request Exists: DMAH_NUM_MASTER_INT >= N Active State: High Synchronous to: hclk Registered: Yes
hlockN	O	AHB bus lock qualifier. Indicates to the arbiter that the master is performing a number of indivisible transfers. Exists: DMAH_NUM_MASTER_INT >= N Active State: High Synchronous to: hclk Registered: Yes
htransN[1:0]	O	Transfer control. Indicates type of current transfer. Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes
hburstN[2:0]	O	Burst type and length control Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes
hsizeN[2:0]	O	Master port transfer size Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes
hprotN[3:0]	O	Protection control Exists: DMAH_NUM_MASTER_INT >= N Synchronous to: hclk Registered: Yes

5.3 Peripheral Handshaking Interface Signals



```

dma_req -
dma_single -
dma_last -

```

```

- dma_ack
- dma_wlast
- dma_finish

```

Table 5-3 Peripheral Handshaking Interface Signals

Port Name	I/O	Description
dma_req[(DMAH_NUM_HS_INT_NZ-1):0]	I	Burst transaction request from peripheral Exists: (DMAH_NUM_HS_INT != 0) Active State: N/A Synchronous to: hclk Registered: Yes
dma_single[(DMAH_NUM_HS_INT_NZ-1):0]	I	Single transfer status Exists: (DMAH_NUM_HS_INT != 0) Active State: N/A Synchronous to: hclk Registered: Yes
dma_last[(DMAH_NUM_HS_INT_NZ-1):0]	I	Last transaction in block indicator. Exists: (DMAH_NUM_HS_INT != 0) Active State: N/A Synchronous to: hclk Registered: Yes
dma_ack[(DMAH_NUM_HS_INT-1):0]	O	Transaction complete acknowledge signal. Exists: (DMAH_NUM_HS_INT != 0) Active State: N/A Synchronous to: hclk Registered: No
dma_wlast[(DMAH_NUM_MASTER_INT-1):0]	O	Indicates the completion of the last address phase of every destination transaction. It remains asserted until the associated last data phase completes. Exists: (DMAH_NUM_HS_INT != 0) && (DMAH_WLAST_EN == 1) Active State: N/A Synchronous to: hclk Registered: No

Table 5-3 Peripheral Handshaking Interface Signals (Continued)

Port Name	I/O	Description
dma_finish[(DMAH_NUM_HS_INT-1):0]	O	DMA block complete signal. Exists: (DMAH_NUM_HS_INT != 0) Active State: N/A Synchronous to: hclk Registered: No

5.4 Interrupt Interface Signals



Table 5-4 Interrupt Interface Signals

Port Name	I/O	Description
intr[((5*DMAH_NUM_CHANNELS)-1):0]	O	Contents of all five Interrupt Status Registers. x is the number of configured DW_ahb_dmac channels (1 to 8). Bits[x-1:0]- contents of StatTfr Bits[2x-1:x]- contents of StatBlock Bits[3x-1:2x]- contents of StatSrcTrans Bits[4x-1:3x]- contents of StatDstTrans Bits[5x-1:4x]- contents of StatErr Exists: (DMAH_INTR_IO == 0) && (DMAH_INTR_POL == 1) Active State: High Synchronous to: hclk Registered: No
int_flag[4:0]	O	All internal interrupts for each channel are combined by type for maximum flexibility in supporting driver SoC interrupt schemes. Bit 0: logical OR of all interrupts of type IntTfr Bit 1: logical OR of all interrupts of type IntBlock Bit 2: logical OR of all interrupts of type IntSrcTran Bit 3: logical OR of all interrupts of type IntDstTran Bit 4: logical OR of all interrupts of type IntErr. Exists: (DMAH_INTR_IO == 1) && (DMAH_INTR_POL == 1) Active State: High Synchronous to: hclk Registered: No
int_combined	O	Logical OR of all individual interrupts. Exists: ((DMAH_INTR_IO == 2) (DMAH_INTR_IO == 1)) && (DMAH_INTR_POL == 1) Active State: High Synchronous to: hclk Registered: No

Table 5-4 Interrupt Interface Signals (Continued)

Port Name	I/O	Description
intr_n[((5*DMAH_NUM_CHANNELS)-1):0]	O	Contents of all five Interrupt Status Registers. x is the number of configured DW_ahb_dmac channels (1 to 8). - Bits[x-1:0]- contents of StatTfr - Bits[2x-1:x]- contents of StatBlock - Bits[3x-1:2x]- contents of StatSrcTrans - Bits[4x-1:3x]- contents of StatDstTrans - Bits[5x-1:4x]- contents of StatErr Exists: (DMAH_INTR_IO == 0) && (DMAH_INTR_POL == 0) Active State: Low Synchronous to: hclk Registered: No
int_flag_n[4:0]	O	All internal interrupts for each channel are combined by type for maximum flexibility in supporting driver SoC interrupt schemes. - Bit 0: logical OR of all interrupts of type IntTfr - Bit 1: logical OR of all interrupts of type IntBlock - Bit 2: logical OR of all interrupts of type IntSrcTran - Bit 3: logical OR of all interrupts of type IntDstTran - Bit 4: logical OR of all interrupts of type IntErr. Exists: (DMAH_INTR_IO == 1) && (DMAH_INTR_POL == 0) Active State: Low Synchronous to: hclk Registered: No
int_combined_n	O	logical OR of all individual interrupts. Exists: ((DMAH_INTR_IO == 2) (DMAH_INTR_IO == 1)) && (DMAH_INTR_POL == 0) Active State: Low Synchronous to: hclk Registered: No

5.5 Debug Bus Interface (for * = 0; * <= n) Signals



Table 5-5 Debug Bus Interface (for * = 0; * <= n) Signals

Port Name	I/O	Description
debug_*	O	Debug bus, included on the I/O for verification purposes. These signals should be left disconnected. Exists: (DMAH_DEBUG_BUS == 1) Active State: N/A Synchronous to: hclk Registered: No

6

Registers

This chapter includes information on how to program the DW_ahb_dmac.



Note

There are references to both software and hardware parameters throughout this chapter. The software parameters are the field names in each register description table and are prefixed by the register name; for example, the Block Transfer Size field in the Control register for Channel *x* is designated as “CTL*x*.BLOCK_TS.”

The hardware parameters are prefixed with DMAH_* and are configured once using Synopsys coreTools.

Shipped with the DW_ahb_dmac component is an address definition (memory map) C header file. This can be used when the DW_ahb_dmac is programmed in a C environment.

6.1 Register Memory Map

Table 6-1 shows the memory map for the DW_ahb_dmac.



Note

The address offsets of the registers are always fixed, regardless of the number of configured channels.

Table 6-1 Memory Map of DW_ahb_dmac

Name	Address Offset	Width	R/W	Description
Channel Registers				
If DMAH_NUM_CHANNELS = “dnc”, “Reg Exist?” field shows “dnc” value for register to exist.				
SAR0 (page 164)	0x000	64	R/W	Channel 0 Source Address Register Reset Value: 0x0 Reg Exist: Yes
DAR0 (page 165)	0x008	64	R/W	Channel 0 Destination Address Register Reset Value: 0x0 Reg Exist: Yes

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
LLP0 (page 167)	0x010	64	R/W	Channel 0 Linked List Pointer Register Reset Value: 0x0 Reg Exist: llp0_hc = False llp0_hc = DMAH_CH0_HC_LLP
CTL0 (page 169)	0x018	64	R/W	Channel 0 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169. Reg Exist: Yes
SSTAT0 (page 176)	0x020	64	R/W	Channel 0 Source Status Register Reset Value: 0x0 Reg Exist: stat0 = True stat0 = DMAH_CH0_STAT_SRC
DSTAT0 (page 177)	0x028	64	R/W	Channel 0 Destination Status Register Reset Value: 0x0 Reg Exist: dstat0 = True dstat0 = DMAH_CH0_STAT_DST
SSTATAR0 (page 178)	0x030	64	R/W	Channel 0 Source Status Address Register Reset Value: 0x0 Reg Exist: stat0 = True stat0 = DMAH_CH0_STAT_SRC
DSTATAR0 (page 179)	0x038	64	R/W	Channel 0 Destination Status Address Register Reset Value: 0x0 Reg Exist: dstat0 = True dstat0 = DMAH_CH0_STAT_DST
CFG0 (page 180)	0x040	64	R/W	Channel 0 Configuration Register Reset Value: 0x00000000400000e00 Reg Exist: Yes
SGR0 (page 186)	0x048	64	R/W	Channel 0 Source Gather Register Reset Value: 0x0 Reg Exist: sgr0 = True sgr0 = DMAH_CH0_SRC_GAT_EN
DSR0 (page 188)	0x050	64	R/W	Channel 0 Destination Scatter Register Reset Value: 0x0 Reg Exist: dsr0 = True dsr0 = DMAH_CH0_DST_SCA_EN

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR1 (page 164)	0x058	64	R/W	Channel 1 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 2
DAR1 (page 165)	0x060	64	R/W	Channel 1 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 2
LLP1 (page 167)	0x068	64	R/W	Channel 1 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and llp1_hc=False
CTL1 (page 169)	0x070	64	R/W	Channel 1 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169 . Reg Exist: dnc ≥ 2
SSTAT1 (page 176)	0x078	64	R/W	Channel 1 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and sstat1=True
DSTAT1 (page 177)	0x080	64	R/W	Channel 1 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and dstat1=True
SSTATAR1 (page 178)	0x088	64	R/W	Channel 1 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and sstat1=True
DSTATAR1 (page 179)	0x090	64	R/W	Channel 1 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and dstat1=True
CFG1 (page 180)	0x098	64	R/W	Channel 1 Configuration Register Reset Value: 0x00000000400000e20 Reg Exist: dnc ≥ 2
SGR1 (page 186)	0x0a0	64	R/W	Channel 1 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and sgr1 = True
DSR1 (page 188)	0x0a8	64	R/W	Channel 1 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 2 and dsr1 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR2 (page 164)	0x0b0	64	R/W	Channel 2 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 3
DAR2 (page 165)	0x0b8	64	R/W	Channel 2 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 3
LLP2 (page 167)	0x0c0	64	R/W	Channel 2 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and llp2_hc=False
CTL2 (page 169)	0x0c8	64	R/W	Channel 2 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169 . Reg Exist: dnc ≥ 3
SSTAT2 (page 176)	0x0d0	64	R/W	Channel 2 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and sstat2=True
DSTAT2 (page 177)	0x0d8	64	R/W	Channel 2 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and dstat2=True
SSTATAR2 (page 178)	0x0e0	64	R/W	Channel 2 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and sstat2=True
DSTATAR2 (page 179)	0x0e8	64	R/W	Channel 2 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and dstat2=True
CFG2 (page 180)	0x0f0	64	R/W	Channel 2 Configuration Register Reset Value: 0x0000000400000e40 Reg Exist: dnc ≥ 3
SGR2 (page 186)	0x0f8	64	R/W	Channel 2 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and sgr2 = True
DSR2 (page 188)	0x100	64	R/W	Channel 2 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 3 and dsr2 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR3 (page 164)	0x108	64	R/W	Channel 3 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 4
DAR3 (page 165)	0x110	64	R/W	Channel 3 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 4
LLP3 (page 167)	0x118	64	R/W	Channel 3 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and llp3_hc=False
CTL3 (page 169)	0x120	64	R/W	Channel 3 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169. Reg Exist: dnc ≥ 4
SSTAT3 (page 176)	0x128	64	R/W	Channel 3 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and sstat3=True
DSTAT3 (page 177)	0x130	64	R/W	Channel 3 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and dstat3=True
SSTATAR3 (page 178)	0x138	64	R/W	Channel 3 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and sstat3=True
DSTATAR3 (page 179)	0x140	64	R/W	Channel 3 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and dstat3=True
CFG3 (page 180)	0x148	64	R/W	Channel 3 Configuration Register Reset Value: 0x0000000400000e60 Reg Exist: dnc ≥ 4
SGR3 (page 186)	0x150	64	R/W	Channel 3 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and sgr3 = True
DSR3 (page 188)	0x158	64	R/W	Channel 3 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 4 and dsr3 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR4 (page 164)	0x160	64	R/W	Channel 4 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 5
DAR4 (page 165)	0x168	64	R/W	Channel 4 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 5
LLP4 (page 167)	0x170	64	R/W	Channel 4 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and llp4_hc=False
CTL4 (page 169)	0x178	64	R/W	Channel 4 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169 . Reg Exist: dnc ≥ 5
SSTAT4 (page 176)	0x180	64	R/W	Channel 4 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and sstat4=True
DSTAT4 (page 177)	0x188	64	R/W	Channel 4 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and dstat4=True
SSTATAR4 (page 178)	0x190	64	R/W	Channel 4 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and sstat4=True
DSTATAR4 (page 179)	0x198	64	R/W	Channel 4 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and dstat4=True
CFG4 (page 180)	0x1a0	64	R/W	Channel 4 Configuration Register Reset Value: 0x00000000400000e80 Reg Exist: dnc ≥ 5
SGR4 (page 186)	0x1a8	64	R/W	Channel 4 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and sgr4 = True
DSR4 (page 188)	0x1b0	64	R/W	Channel 4 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 5 and dsr4 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR5 (page 164)	0x1b8	64	R/W	Channel 5 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 6
DAR5 (page 165)	0x1c0	64	R/W	Channel 5 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 6
LLP5 (page 167)	0x1c8	64	R/W	Channel 5 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and llp5_hc=False
CTL5 (page 169)	0x1d0	64	R/W	Channel 5 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169 . Reg Exist: dnc ≥ 6
SSTAT5 (page 176)	0x1d8	64	R/W	Channel 5 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and sstat5=True
DSTAT5 (page 177)	0x1e0	64	R/W	Channel 5 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and dstat5=True
SSTATAR5 (page 178)	0x1e8	64	R/W	Channel 5 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and sstat5=True
DSTATAR5 (page 179)	0x1f0	64	R/W	Channel 5 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and dstat5=True
CFG5 (page 180)	0x1f8	64	R/W	Channel 5 Configuration Register Reset Value: 0x0000000400000ea0 Reg Exist: dnc ≥ 6
SGR5 (page 186)	0x200	64	R/W	Channel 5 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and sgr5 = True
DSR5 (page 188)	0x208	64	R/W	Channel 5 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 6 and dsr5 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR6 (6.2.2.1)	0x210	64	R/W	Channel 6 Source Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 7
DAR6 (page 165)	0x218	64	R/W	Channel 6 Destination Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 7
LLP6 (page 167)	0x220	64	R/W	Channel 6 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and llp6_hc=False
CTL6 (page 169)	0x228	64	R/W	Channel 6 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169 . Reg Exist: dnc ≥ 7
SSTAT6 (page 176)	0x230	64	R/W	Channel 6 Source Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and sstat6=True
DSTAT6 (page 177)	0x238	64	R/W	Channel 6 Destination Status Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and dstat6=True
SSTATAR6 (page 178)	0x240	64	R/W	Channel 6 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and sstat6=True
DSTATAR6 (page 179)	0x248	64	R/W	Channel 6 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and dstat6=True
CFG6 (page 180)	0x250	64	R/W	Channel 6 Configuration Register Reset Value: 0x00000000400000ec0 Reg Exist: dnc ≥ 7
SGR6 (page 186)	0x258	64	R/W	Channel 6 Source Gather Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and sgr6 = True
DSR6 (page 188)	0x260	64	R/W	Channel 6 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc ≥ 7 and dsr6 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
SAR7 (page 164)	0x268	64	R/W	Channel 7 Source Address Register Reset Value: 0x0 Reg Exist: dnc = 8
DAR7 (page 165)	0x270	64	R/W	Channel 7 Destination Address Register Reset Value: 0x0 Reg Exist: dnc = 8
CTL7 (page 169)	0x280	64	R/W	Channel 7 Control Register Reset Value: Configuration dependent. Refer to the bit table in the description for “CTLx” on page 169. Reg Exist: dnc = 8
LLP7 (page 167)	0x278	64	R/W	Channel 7 Linked List Pointer Register Reset Value: 0x0 Reg Exist: dnc = 8 and llp7_hc7=False
SSTAT7 (page 176)	0x288	64	R/W	Channel 7 Source Status Register Reset Value: 0x0 Reg Exist: dnc = 8 and sstat7=True
DSTAT7 (page 177)	0x290	64	R/W	Channel 7 Destination Status Register Reset Value: 0x0 Reg Exist: dnc = 8 and dstat7=True
SSTATAR7 (page 178)	0x298	64	R/W	Channel 7 Source Status Address Register Reset Value: 0x0 Reg Exist: dnc = 8 and sstat7=True
DSTATAR7 (page 179)	0x2a0	64	R/W	Channel 7 Destination Status Address Register Reset Value: 0x0 Reg Exist: dnc = 8 and dstat7=True
CFG7 (page 180)	0x2a8	64	R/W	Channel 7 Configuration Register Reset Value: 0x0000000400000ee0 Reg Exist: dnc = 8
SGR7 (page 186)	0x2b0	64	R/W	Channel 7 Source Gather Register Reset Value: 0x0 Reg Exist: dnc = 8 and sgr7 = True
DSR7 (page 188)	0x2b8	64	R/W	Channel 7 Destination Scatter Register Reset Value: 0x0 Reg Exist: dnc = 8 and dsr7 = True

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
Interrupt Registers				
RawBlock, RawDstTran, RawErr, RawSrcTran, RawTfr				
RawTfr (page 192)	0x2c0	64	R	Raw Status for IntTfr Interrupt Reset Value: 0x0 Reg Exist: Yes
RawBlock (page 192)	0x2c8	64	R	Raw Status for IntBlock Interrupt Reset Value: 0x0 Reg Exist: Yes
RawSrcTran (page 192)	0x2d0	64	R	Raw Status for IntSrcTran Interrupt Reset Value: 0x0 Reg Exist: Yes
RawDstTran (page 192)	0x2d8	64	R	Raw Status for IntDstTran Interrupt Reset Value: 0x0 Reg Exist: Yes
RawErr (page 192)	0x2e0	64	R	Raw Status for IntErr Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusBlock, StatusDstTran, StatusErr, StatusSrcTran, StatusTfr				
StatusTfr (page 193)	0x2e8	64	R	Status for IntTfr Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusBlock (page 193)	0x2f0	64	R	Status for IntBlock Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusSrcTran (page 193)	0x2f8	64	R	Status for IntSrcTran Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusDstTran (page 193)	0x300	64	R	Status for IntDstTran Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusErr (page 193)	0x308	64	R	Status for IntErr Interrupt Reset Value: 0x0 Reg Exist: Yes

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
MaskBlock, MaskDstTran, MaskErr, MaskSrcTran, MaskTfr				
MaskTfr (page 194)	0x310	64	R/W	Mask for IntTfr Interrupt Reset Value: 0x0 Reg Exist: Yes
MaskBlock (page 194)	0x318	64	R/W	Mask for IntBlock Interrupt Reset Value: 0x0 Reg Exist: Yes
MaskSrcTran (page 194)	0x320	64	R/W	Mask for IntSrcTran Interrupt Reset Value: 0x0 Reg Exist: Yes
MaskDstTran (page 194)	0x328	64	R/W	Mask for IntDstTran Interrupt Reset Value: 0x0 Reg Exist: Yes
MaskErr (page 194)	0x330	64	R/W	Mask for IntErr Interrupt Reset Value: 0x0 Reg Exist: Yes
ClearBlock, ClearDstTran, ClearErr, ClearSrcTran, ClearTfr				
ClearTfr (page 196)	0x338	64	W	Clear for IntTfr Interrupt Reset Value: 0x0 Reg Exist: Yes
ClearBlock (page 196)	0x340	64	W	Clear for IntBlock Interrupt Reset Value: 0x0 Reg Exist: Yes
ClearSrcTran (page 196)	0x348	64	W	Clear for IntSrcTran Interrupt Reset Value: 0x0 Reg Exist: Yes
ClearDstTran (page 196)	0x350	64	W	Clear for IntDstTran Interrupt Reset Value: 0x0 Reg Exist: Yes
ClearErr (page 196)	0x358	64	W	Clear for IntErr Interrupt Reset Value: 0x0 Reg Exist: Yes
StatusInt	0x360	64	W	Status for each interrupt type Reset Value: 0x0 Reg Exist: Yes

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
Software Handshaking Registers				
ReqSrcReg	0x368	64	R/W	Source Software Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
ReqDstReg	0x370	64	R/W	Destination Software Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
SglReqSrcReg	0x378	64	R/W	Single Source Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
SglReqDstReg	0x380	64	R/W	Single Destination Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
LstSrcReg	0x388	64	R/W	Last Source Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
LstDstReg	0x390	64	R/W	Last Destination Transaction Request Register Reset Value: 0x0 Reg Exist: Yes
Miscellaneous Registers				
DmaCfgReg	0x398	64	R/W	DMA Configuration Register Reset Value: 0x0 Reg Exist: Yes
ChEnReg	0x3a0	64	R/W	DMA Channel Enable Register Reset Value: 0x0 Reg Exist: Yes
DmaIdReg	0x3a8	64	R	DMA ID Register Reset Value: DMAH_ID_NUM Reg Exist: Yes
DmaTestReg	0x3b0	64	R/W	DMA Test Register Reset Value: 0x0 Reg Exist: Yes
Reserved	0x3b8			
Reserved	0x3c0			

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
DMA_COMP_PA RAMS_6	0x3c8	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_6” on page 207. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.
DMA_COMP_PA RAMS_5	0x3d0	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_5” on page 211. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.
DMA_COMP_PA RAMS_4	0x3d8	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_4” on page 218. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.
DMA_COMP_PA RAMS_3	0x3e0	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_3” on page 225. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.
DMA_COMP_PA RAMS_2	0x3e8	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_2” on page 232. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.
DMA_COMP_PA RAMS_1	0x3f0	64	R	Refer to the bit table in the description for “DMA_COMP_PARAMS_1” on page 237. Reset Value: Depends on user configuration; for more information, refer to the specific table for each register. Reg Exist: Dependent on setting of DMAH_ADD_ENCODED_PARAMS configuration parameter. The register exists when this parameter is set to True.

Table 6-1 Memory Map of DW_ahb_dmac (Continued)

Name	Address Offset	Width	R/W	Description
DMA Component ID Register	0x3f8	64	R	Component version register. Reset Value: Refer to the bit table in the description for “DMA Component ID Register” on page 241. Reg Exist: Yes

6.2 Register and Field Descriptions

The following sections contain the field descriptions for the individual registers.

6.2.1 Configuration and Channel Enable Registers

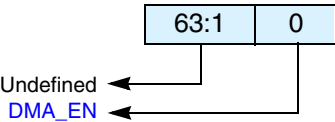
The channel registers consist of the following, where x = 0 to 7:

- [DmaCfgReg](#) – Configuration Register
- [ChEnReg](#) – Channel Enable Register

6.2.1.1 DmaCfgReg

- **Name:** DW_ahb_dmac Configuration Register
- **Size:** 64 bits
- **Address Offset:** 0x398
- **Read/Write Access:** Read/Write

This register is used to enable the DW_ahb_dmac, which must be done before any channel activity can begin.



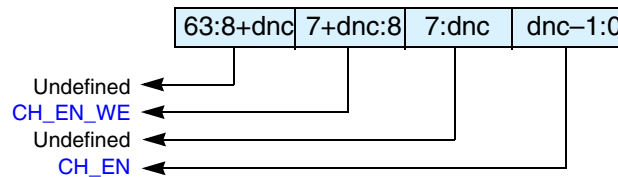
Bits	Name	R/W	Reset	Description
63:1	Undefined	N/A	0x0	Reserved
0	DMA_EN	R/W	0x0	DW_ahb_dmac Enable bit. 0 = DW_ahb_dmac Disabled 1 = DW_ahb_dmac Enabled.

If the global channel enable bit is cleared while any channel is still active, then DmaCfgReg.DMA_EN still returns 1 to indicate that there are channels still active until hardware has terminated all activity on all channels, at which point the DmaCfgReg.DMA_EN bit returns 0. For more information, refer to “[Abnormal Transfer Termination](#)” on page 283.

6.2.1.2 ChEnReg

- **Name:** DW_ahb_dmac Channel Enable Register
- **Size:** 64 bits
- **Address Offset:** 0x3a0
- **Read/Write Access:** Read/Write

This is the DW_ahb_dmac Channel Enable Register. If software needs to set up a new channel, then it can read this register in order to find out which channels are currently inactive; it can then enable an inactive channel with the required priority.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	CH_EN_WE	W	0x0	Channel enable write enable.
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	CH_EN	R/W	0x0	Enables/Disables the channel. Setting this bit enables a channel; clearing this bit disables the channel. 0 = Disable the Channel 1 = Enable the Channel The ChEnReg.CH_EN bit is automatically cleared by hardware to disable the channel after the last AMBA transfer of the DMA transfer to the destination has completed. Software can therefore poll this bit to determine when this channel is free for a new DMA transfer.

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

All bits of this register are cleared to 0 when the global DW_ahb_dmac channel enable bit, [DmaCfgReg\[0\]](#), is 0. When the global channel enable bit is 0, then a write to the [ChEnReg](#) register is ignored and a read will always read back 0.

The channel enable bit, [ChEnReg.CH_EN](#), is written only if the corresponding channel write enable bit, [ChEnReg.CH_EN_WE](#), is asserted on the same AHB write transfer. For example, writing hex 01x1 writes a 1 into [ChEnReg\[0\]](#), while [ChEnReg\[7:1\]](#) remains unchanged. Writing hex 00xx leaves [ChEnReg\[7:0\]](#) unchanged. Note that a read-modified write is not required.

For information on software disabling a channel by writing 0 to [ChEnReg.CH_EN](#), refer to “[Disabling a Channel Prior to Transfer Completion](#)” on page 282.

6.2.2 Channel Registers

The channel registers consist of the following, where $x = 0$ to 7:

- **CFG x** – Configuration register for channel x
- **CTL x** – Control register for channel x
- **DAR x** – Destination address register for channel x
- **DSR x** – Destination scatter register for channel x
- **DSTAT x** – Destination status register for channel x
- **DSTATAR x** – Destination status address register for channel x
- **LLP x** – Linked list pointer register for channel x
- **SAR x** – Source address register for channel x
- **SGR x** – Source gather register for channel x
- **SSTAT x** – Source status register for channel x
- **SSTATAR x** – Source status address register for channel x

The **SAR x** , **DAR x** , **LLP x** , **CTL x** , and **CFG x** channel registers should be programmed prior to enabling the channel. However, if an LLI update occurs before commencing data transfer, **SAR x** and **DAR x** may not need to be programmed prior to enabling the channel; refer to rows 6 to 10 in [Table 8-1](#) on page 249. It is an [Illegal Register Access](#) when a write to the **SAR x** , **DAR x** , **LLP x** , **CTL x** , **SSTAT x** , **DSTAT x** , **SSTATAR x** , **DSTATAR x** , **SGR x** , or **DSR x** registers occurs when the channel is enabled.

6.2.2.1 SARx

- **Name:** Source Address Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

SAR0 – 0x000
 SAR1 – 0x058
 SAR2 – 0x0b0
 SAR3 – 0x108
 SAR4 – 0x160
 SAR5 – 0x1b8
 SAR6 – 0x210
 SAR7 – 0x268

- **Read/Write Access:** Read/Write

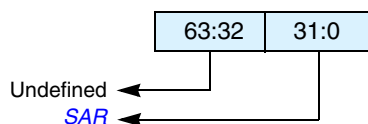
The starting source address is programmed by software before the DMA channel is enabled, or by an LLI update before the start of the DMA transfer. While the DMA transfer is in progress, this register is updated to reflect the source address of the current AHB transfer.



Note

You must program the SAR address to be aligned to `CTLx.SRC_TR_WIDTH`.

For information on how the SARx is updated at the start of each DMA block for multi-block transfers, refer to [Table 8-1](#) on page 249.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:0	<i>SAR</i>	R/W	0x0	Current Source Address of DMA transfer. Updated after each source transfer. The SINC field in the <code>CTLx</code> register determines whether the address increments, decrements, or is left unchanged on every source transfer throughout the block transfer.

6.2.2.2 **DARx**

- **Name:** Destination Address Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

DAR0 – 0x008
DAR1 – 0x060
DAR2 – 0x0b8
DAR3 – 0x110
DAR4 – 0x168
DAR5 – 0x1c0
DAR6 – 0x218
DAR7 – 0x270

- **Read/Write Access:** Read/Write

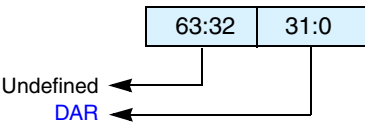
The starting destination address is programmed by software before the DMA channel is enabled, or by an LLI update before the start of the DMA transfer. While the DMA transfer is in progress, this register is updated to reflect the destination address of the current AHB transfer.



Note

You must program the DAR to be aligned to CTLx.DST_TR_WIDTH.

For information on how the DARx is updated at the start of each DMA block for multi-block transfers, refer to Table 8-1 on page 249 and “Programming Examples” on page 257.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:0	DAR	R/W	0x0	Current Destination address of DMA transfer. Updated after each destination transfer. The DINC field in the CTLx register determines whether the address increments, decrements, or is left unchanged on every destination transfer throughout the block transfer.

6.2.2.3 Hardware Realignment of SAR/DAR Registers

In a particular circumstance, during contiguous multi-block DMA transfers, the destination address can become misaligned between the end of one block and the start of the next block. When this situation occurs, DW_ahb_dmac re-aligns the destination address before the start of the next block.

Consider the following example. If the block length is 9, the source transfer width is 16 (halfword), and the destination transfer width is 32 (word) – the destination is programmed for contiguous block transfers – then the destination performs four word transfers followed by a halfword transfer to complete the block transfer to the destination. At the end of the destination block transfer, the address is aligned to a 16-bit transfer as the last AMBA transfer is halfword. This is misaligned to the programmed transfer size of 32 bits for the destination. However, for contiguous destination multi-block transfers, DW_ahb_dmac re-aligns the DAR address to the nearest 32-bit address (next 32-bit address upwards if address control is incrementing or next address downwards if address control is decrementing).

The destination address is automatically realigned by the DW_ahb_dmac in the following DMA transfer setup scenario:

- Contiguous multi-block transfers on destination side, AND
- $DST_TR_WIDTH > SRC_TR_WIDTH$, AND
- $(BLOCK_TS * SRC_TR_WIDTH) / DST_TR_WIDTH \neq \text{integer}$ (where SRC_TR_WIDTH , DST_TR_WIDTH is byte width of transfer)

6.2.2.4 LLPx

- **Name:** Linked List Pointer Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

LLP0 – 0x010
 LLP1 – 0x068
 LLP2 – 0x0c0
 LLP3 – 0x118
 LLP4 – 0x170
 LLP5 – 0x1c8
 LLP6 – 0x220
 LLP7 – 0x278

- **Read/Write Access:** Read/Write

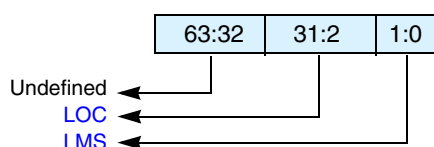
This register does not exist if the DMAH_CHx_HC_LLP configuration parameter is set to True (see [page 131](#)).



Note

You need to program this register to point to the first Linked List Item (LLI) in memory prior to enabling the channel if block chaining is enabled – rows 6 to 10 of [Table 8-1](#) on page 249. This is LLP(0) in [Figure 8-2](#) on page 248.

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:2	LOC	R/W	0x0	Starting Address In Memory of next LLI if block chaining is enabled. Note that the two LSBs of the starting address are not stored because the address is assumed to be aligned to a 32-bit boundary. LLI accesses are always 32-bit accesses (Hsize = 2) aligned to 32-bit boundaries and cannot be changed or programmed to anything other than 32-bit.

Bits	Name	R/W	Reset	Description
1:0	LMS	R/W	0x0	List Master Select. Identifies the AHB layer/interface where the memory device that stores the next linked list item resides. 00 = AHB master 1 01 = AHB master 2 10 = AHB master 3 11 = AHB master 4 This field does not exist if the configuration parameter DMAH_CHx_LMS is not set to NO_HARDCODE. In this case, the read-back value is always the hardcoded value. For more information about this parameter, see page 132. The maximum value of this field that can be read back is DMAH_NUM_MASTER_INT – 1.

The LLP register has two functions:

- The logical result of the equation $LLP.LOC \neq 0$ is used to set up the type of DMA transfer – single or multi-block. [Table 8-1](#) on page 249 shows how the method of updating the channel registers is a function of $LLP.LOC \neq 0$. If $LLP.LOC$ is set to 0x0, then transfers using linked lists are *not* enabled. This register must be programmed prior to enabling the channel in order to set up the transfer type.
- $LLP.LOC \neq 0$ contains the pointer to the next LLI for block chaining using linked lists; refer to “[Block Chaining Using Linked Lists](#)” on page 247. The $LLPx$ register can also point to the address where write-back of the control and source/destination status information occur after block completion.

6.2.2.5 CTLx

- **Name:** Control Register for Channel x
- **Size:** 64 bits
- **Address Offset:** for $x = 0$ to 7:

CTL0 – 0x018
 CTL1 – 0x070
 CTL2 – 0x0c8
 CTL3 – 0x120
 CTL4 – 0x178
 CTL5 – 0x1d0
 CTL6 – 0x228
 CTL7 – 0x280

- **Read/Write Access:** Read/Write

This register contains fields that control the DMA transfer.

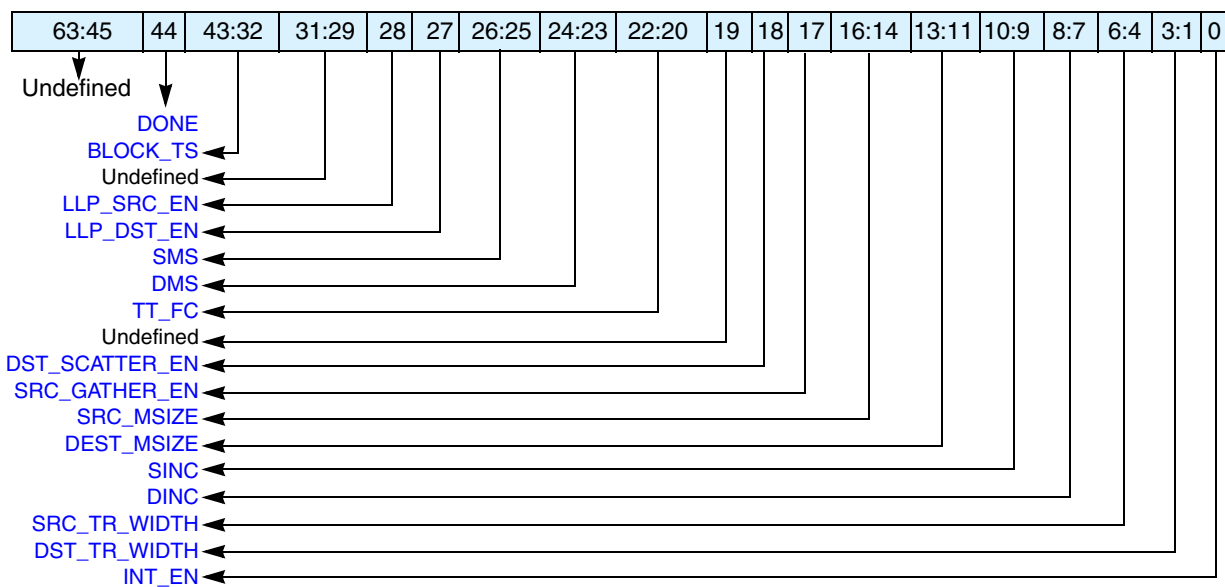
The CTLx register is part of the block descriptor (linked list item – LLI) when block chaining is enabled. It can be varied on a block-by-block basis within a DMA transfer when block chaining is enabled. For information about the behavior of this register between blocks, refer to “[Multi-Block Transfers](#)” on page 247.

If status write-back is enabled, the upper word of the control register, CTLx[63:32], is written to the control register location of the LLI in system memory at the end of the block transfer.



Note

You need to program this register prior to enabling the channel.



Bits	Name	R/W	Description
63:45	Undefined	N/A	Reserved
44	DONE	R/W	Done bit If status write-back is enabled, the upper word of the control register, CTLX[63:32], is written to the control register location of the Linked List Item (LLI) in system memory at the end of the block transfer with the done bit set. Software can poll the LLI CTLX.DONE bit to see when a block transfer is complete. The LLI CTLX.DONE bit should be cleared when the linked lists are set up in memory prior to enabling the channel. LLI accesses are always 32-bit accesses (Hsize = 2) aligned to 32-bit boundaries and cannot be changed or programmed to anything other than 32-bit. For more information, refer to “Multi-Block Transfers” on page 247. Reset Value: 0x0
b:32 (See description)	BLOCK_TS	R/W	Block Transfer Size. When the DW_ahb_dmac is the flow controller, the user writes this field before the channel is enabled in order to indicate the block size. The number programmed into BLOCK_TS indicates the total number of single transactions to perform for every block transfer; a single transaction is mapped to a single AMBA beat. Width: The width of the single transaction is determined by CTLX.SRC_TR_WIDTH. For further information on setting this field, refer to Table 6-3 on page 175 and “Transfer Operation” on page 61. Once the transfer starts, the read-back value is the total number of data items already read from the source peripheral, regardless of what is the flow controller. When the source or destination peripheral is assigned as the flow controller, then the maximum block size that can be read back saturates at DMAH_CHx_MAX_BLK_SIZE, but the actual block size can be greater. Refer to Table 8-1 on page 249. $b = \log_2(\text{DMAH_CHx_MAX_BLK_SIZE} + 1) + 31$ Bits 43:b+1 do not exist and return 0 on a read. Reset Value: 0x2
31:29	Undefined	N/A	Reserved
28	LLP_SRC_EN	R/W	Block chaining is enabled on the source side only if the LLP_SRC_EN field is high and LLPx.LOC is non-zero; for more information, see “Block Chaining Using Linked Lists” on page 247. Reset Value: 0x0 Dependencies: This field does not exist if the configuration parameter DMAH_CHx_MULTI_BLK_EN (page 130) is not selected or if DMAH_CHx_HC_LLP (page 131) is selected; in this case, the read-back value is always 0.

Bits	Name	R/W	Description
27	LLP_DST_EN	R/W	Block chaining is enabled on the destination side only if the LLP_DST_EN field is high and LLPx.LOC is non-zero. For more information, see “Block Chaining Using Linked Lists” on page 247. Reset Value: 0x0 Dependencies: This field does not exist if the configuration parameter DMAH_CHx_MULTI_BLK_EN (page 130) is not selected or if DMAH_CHx_HC_LLP (page 131) is selected; in this case, the read-back value is always 0.
26:25	SMS	R/W	Source Master Select. Identifies the Master Interface layer from which the source device (peripheral or memory) is accessed. 00 = AHB master 1 01 = AHB master 2 10 = AHB master 3 11 = AHB master 4 The maximum value of this field that can be read back is DMAH_NUM_MASTER_INT – 1. Reset Value: DMAH_CHx_SMS[1:0] Dependencies: This field does not exist if the configuration parameter DMAH_CHx_SMS (page 129) is hardcoded; in this case, the read-back value is always the hardcoded value.
24:23	DMS	R/W	Destination Master Select. Identifies the Master Interface layer where the destination device (peripheral or memory) resides. 00 = AHB master 1 01 = AHB master 2 10 = AHB master 3 11 = AHB master 4 The maximum value of this field that can be read back is DMAH_NUM_MASTER_INT – 1. Reset Value: DMAH_CHx_DMS[1:0] Dependencies: This field does not exist if the configuration parameter DMAH_CHx_DMS (page 129) is hardcoded; in this case, the read-back value is always the hardcoded value.

Bits	Name	R/W	Description
22:20	TT_FC	R/W	Transfer Type and Flow Control. The following transfer types are supported. • Memory to Memory • Memory to Peripheral • Peripheral to Memory • Peripheral to Peripheral Flow Control can be assigned to the DW_ahb_dmac, the source peripheral, or the destination peripheral. Table 6-4 on page 175 lists the decoding for this field. For more information on transfer types and flow control, refer to “Setup/Operation of DW_ahb_dmac Transfers” on page 41. Reset Value: Configuration dependent: TT_FC[0] = 1'b1 TT_FC[1] = DMAH_CHx_FC[1] (!DMAH_CHx_FC[0]) TT_FC[2] = DMAH_CHx_FC[1] ^ DMAH_CHx_FC[0] Dependencies: If the configuration parameter DMAH_CHx_FC (page 128) is set to DMA_FC_ONLY, then TT_FC[2] does not exist and TT_FC[2] always reads back 0. If DMAH_CHx_FC is set to SRC_FC_ONLY, then TT_FC[2:1] does not exist and TT_FC[2:1] always reads back 2'b10. If DMAH_CHx_FC is set to DST_FC_ONLY, then TT_FC[2:1] does not exist and TT_FC[2:1] always reads back 2'b11. For multi-block transfers using linked list operation, TT_FC must be constant for all blocks of this multi-block transfer.
19	Undefined	N/A	Reserved
18	DST_SCATTER_EN	R/W	Destination scatter enable bit: 0 = Scatter disabled 1 = Scatter enabled Scatter on the destination side is applicable only when the CTLx.DINC bit indicates an incrementing or decrementing address control. Reset Value: 0x0 Dependencies: This field does not exist if DMAH_CHx_DST_SCA_EN (page 133) is not selected; in this case, the read-back value is always 0.
17	SRC_GATHER_EN	R/W	Source gather enable bit: 0 = Gather disabled 1 = Gather enabled Gather on the source side is applicable only when the CTLx.SINC bit indicates an incrementing or decrementing address control. Reset Value: 0x0 Dependencies: This field does not exist if DMAH_CHx_SRC_GAT_EN (page 133) is not selected; in this case, the read-back value is always 0.

Bits	Name	R/W	Description
16:14	SRC_MSIZ	R/W	Source Burst Transaction Length. Number of data items, each of width CTLx.SRC_TR_WIDTH, to be read from the source every time a source burst transaction request is made from either the corresponding hardware or software handshaking interface. Table 6-2 on page 174 lists the decoding for this field; refer to “Choosing the Receive Watermark level” on page 101. NOTE: This value is not related to the AHB bus master HBURST bus. Reset Value: 0x1 Dependencies: The configuration parameter DMAH_CHx_MAX_MULT_SIZE (page 127) determines the bit width of this field. All remaining bits in this field do not exist and read back as 0.
13:11	DEST_MSIZ	R/W	Destination Burst Transaction Length. Number of data items, each of width CTLx.DST_TR_WIDTH, to be written to the destination every time a destination burst transaction request is made from either the corresponding hardware or software handshaking interface. Table 6-2 on page 174 lists the decoding for this field; refer to “Choosing the Transmit Watermark Level” on page 97. NOTE: This value is not related to the AHB bus master HBURST bus. Reset Value: 0x1 Dependencies: DMAH_CHx_MAX_MULT_SIZE (page 127) determines the bit width of this field. All surplus bits in this field do not exist and read back as 0.
10:9	SINC	R/W	Source Address Increment. Indicates whether to increment or decrement the source address on every source transfer. If the device is fetching data from a source peripheral FIFO with a fixed address, then set this field to “No change.” 00 = Increment 01 = Decrement 1x = No change NOTE: Incrementing or decrementing is done for alignment to the next CTLx.SRC_TR_WIDTH boundary. Reset Value: 0x0
8:7	DINC	R/W	Destination Address Increment. Indicates whether to increment or decrement the destination address on every destination transfer. If your device is writing data to a destination peripheral FIFO with a fixed address, then set this field to “No change.” 00 = Increment 01 = Decrement 1x = No change NOTE: Incrementing or decrementing is done for alignment to the next CTLx.DST_TR_WIDTH boundary. Reset Value: 0x0

Bits	Name	R/W	Description
6:4	SRC_TR_WIDTH	R/W	Source Transfer Width. Table 6-3 on page 175 lists the decoding for this field. Mapped to AHB bus “hsize.” For a non-memory peripheral, typically the peripheral (source) FIFO width. This value must be less than or equal to DMAH_Mx_HDATA_WIDTH, where x is the AHB layer 1 to 4 where the source resides. Reset Value: Encoded value; refer to Table 6-3 on page 175. Dependencies: This field does not exist if the parameter DMAH_CHx_STW (page 129) is hardcoded. In this case, the read-back value is always the hardcoded source transfer width, DMAH_CHx_STW.
3:1	DST_TR_WIDTH	R/W	Destination Transfer Width. Table 6-3 on page 175 lists the decoding for this field. Mapped to AHB bus “hsize.” For a non-memory peripheral, typically rgw peripheral (destination) FIFO width. This value must be less than or equal to DMAH_Mk_HDATA_WIDTH, where k is the AHB layer 1 to 4 where the destination resides. Reset Value: Encoded value; refer to Table 6-3 on page 175. Dependencies: This field does not exist if DMAH_CHx_DTW (page 129) is hardcoded. In this case, the read-back value is always the hardcoded destination transfer width, DMAH_CHx_DTW.
0	INT_EN	R/W	Interrupt Enable Bit. If set, then all interrupt-generating sources are enabled. Functions as a global mask bit for all interrupts for the channel; raw* interrupt registers still assert if CTLx.INT_EN = 0. Reset Value: 0x1

Table 6-2 CTLx.SRC_MSIZ and DEST_MSIZ Decoding

CTLx.SRC_MSIZ / CTLx.DEST_MSIZ	Number of data items to be transferred (of width CTLx.SRC_TR_WIDTH or CTLx.DST_TR_WIDTH)
000	1
001	4
010	8
011	16
100	32
101	64
110	128
111	256

Table 6-3 CTLx.SRC_TR_WIDTH and CTLx.DST_TR_WIDTH Decoding

CTLx.SRC_TR_WIDTH / CTLx.DST_TR_WIDTH	Size (bits)
000	8
001	16
010	32
011	64
100	128
101	256
11x	256

Table 6-4 CTLx.TT_FC Field Decoding

CTLx.TT_FC Field	Transfer Type	Flow Controller
000	Memory to Memory	DW_ahb_dmac
001	Memory to Peripheral	DW_ahb_dmac
010	Peripheral to Memory	DW_ahb_dmac
011	Peripheral to Peripheral	DW_ahb_dmac
100	Peripheral to Memory	Peripheral
101	Peripheral to Peripheral	Source Peripheral
110	Memory to Peripheral	Peripheral
111	Peripheral to Peripheral	Destination Peripheral

6.2.2.6 SSTATx

- **Name:** Source Status Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

SSTAT0 – 0x020
 SSTAT1 – 0x078
 SSTAT2 – 0x0d0
 SSTAT3 – 0x128
 SSTAT4 – 0x180
 SSTAT5 – 0x1d8
 SSTAT6 – 0x230
 SSTAT7 – 0x288

- **Read/Write Access:** Read/Write

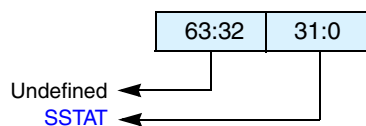
After each block transfer completes, hardware can retrieve the source status information from the address pointed to by the contents of the [SSTATARx](#) register. This status information is then stored in the SSTATx register and written out to the SSTATx register location of the LLI before the start of the next block; refer to [Figure 8-3](#) on page 250. For conditions under which the source status information is fetched, refer to “[Multi-Block Transfers](#)” on page 247. This register does not exist if DMAH_CHx_STAT_SRC ([page 132](#)) is set to False; in this case, the read-back value is always 0.



Note

This register is a temporary placeholder for the source status information on its way to the SSTATx register location of the LLI. The source status information should be retrieved by software from the SSTATx register location of the LLI, and not by a read of this register over the DW_ahb_dmac slave interface.

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:0	SSTAT	R/W	0x0	Source status information retrieved by hardware from the address pointed to by the contents of the SSTATARx register.

6.2.2.7 DSTATx

- **Name:** Destination Status Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

DSTAT0 – 0x028
 DSTAT1 – 0x080
 DSTAT2 – 0x0d8
 DSTAT3 – 0x130
 DSTAT4 – 0x188
 DSTAT5 – 0x1e0
 DSTAT6 – 0x238
 DSTAT7 – 0x290

- **Read/Write Access:** Read/Write

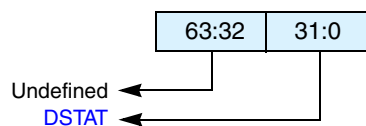
After the completion of each block transfer, hardware can retrieve the destination status information from the address pointed to by the contents of the [DSTATARx](#) register. This status information is then stored in the DSTATx register and written out to the DSTATx register location of the LLI (refer to [Figure 8-3](#) on page 250) before the start of the next block. For conditions under which the destination status information is fetched, refer to “[Multi-Block Transfers](#)” on page 247. This register does not exist if the parameter DMAH_CHx_STAT_DST ([page 132](#)) is set to False; in this case, the read-back value is always 0.



Note

This register is a temporary placeholder for the destination status information on its way to the DSTATx register location of the LLI. The destination status information should be retrieved by software from the DSTATx register location of the LLI and not by a read of this register over the DW_ahb_dmac slave interface.

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:0	DSTAT	R/W	0x0	Destination status information retrieved by hardware from the address pointed to by the contents of the DSTATARx register.

6.2.2.8 SSTATARx

- **Name:** Source Status Address Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

SSTATAR0 – 0x030
 SSTATAR1 – 0x088
 SSTATAR2 – 0x0e0
 SSTATAR3 – 0x138
 SSTATAR4 – 0x190
 SSTATAR5 – 0x1e8
 SSTATAR6 – 0x240
 SSTATAR7 – 0x298

- **Read/Write Access:** Read/Write

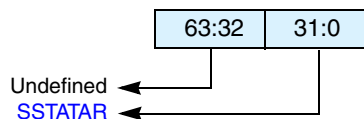
After completion of each block transfer, hardware can retrieve the source status information from the user-defined address to which the contents of the SSTATARx register point. The user can select any location in system memory that would provide a 32-bit value to indicate the status of the source transfer. For example, if the DW_apb_ssi is the source peripheral for the DMA transfer, the user can use one of the SSI registers to indicate the status of the transfer. Thus the address programmed in SSTATARx could be the address of the SSI.CTRL register or the SSI.ISR register, or it could be the address of the SSI.RXFLR register.



Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

This register does not exist if the configuration parameter DMAH_CHx_STAT_SRC ([page 132](#)) is set to False; in this case, the read-back value is always 0.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31-1:0	SSTATAR	R/W	0x0	Pointer from where hardware can fetch the source status information, which is registered in the SSTATx register and written out to the SSTATx register location of the LLI (refer to Figure 8-3 on page 250) before the start of the next block.

6.2.2.9 DSTATAR_x

- **Name:** Destination Status Address Register for Channel *x*
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for *x* = 0 to 7:

DSTATAR0 – 0x038
 DSTATAR1 – 0x090
 DSTATAR2 – 0x0e8
 DSTATAR3 – 0x140
 DSTATAR4 – 0x198
 DSTATAR5 – 0x1f0
 DSTATAR6 – 0x248
 DSTATAR7 – 0x2a0

- **Read/Write Access:** Read/Write

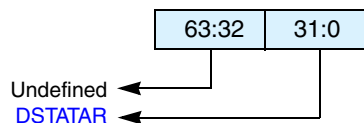
After completion of each block transfer, hardware can retrieve the destination status information from the user-defined address to which the contents of the DSTATAR_x register point. The user can select any location in system memory that would provide a 32-bit value to indicate the status of the destination transfer. For example, if the DW_apb_ssi is the destination peripheral for the DMA transfer, the user can use one of the SSI registers to indicate the status of the transfer. Thus the address programmed in DSTATAR_x could be the address of the SSI.CTRL register or the SSI.ISR register, or it could be the address of the SSI.TXFLR register.



Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

This register does not exist if the configuration parameter DMAH_CH_x_STAT_DST ([page 132](#)) is set to False; in this case, the read-back value is always 0.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31-1:0	DSTATAR	R/W	0x0	Pointer from where hardware can fetch the destination status information, which is registered in the DSTAT_x register and written out to the DSTAT _x register location of the LLI (refer to Figure 8-3 on page 250) before the start of the next block.

6.2.2.10 CFGx

- **Name:** Configuration Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

CFG0 – 0x040
 CFG1 – 0x098
 CFG2 – 0x0f0
 CFG3 – 0x148
 CFG4 – 0x1a0
 CFG5 – 0x1f8
 CFG6 – 0x250
 CFG7 – 0x2a8

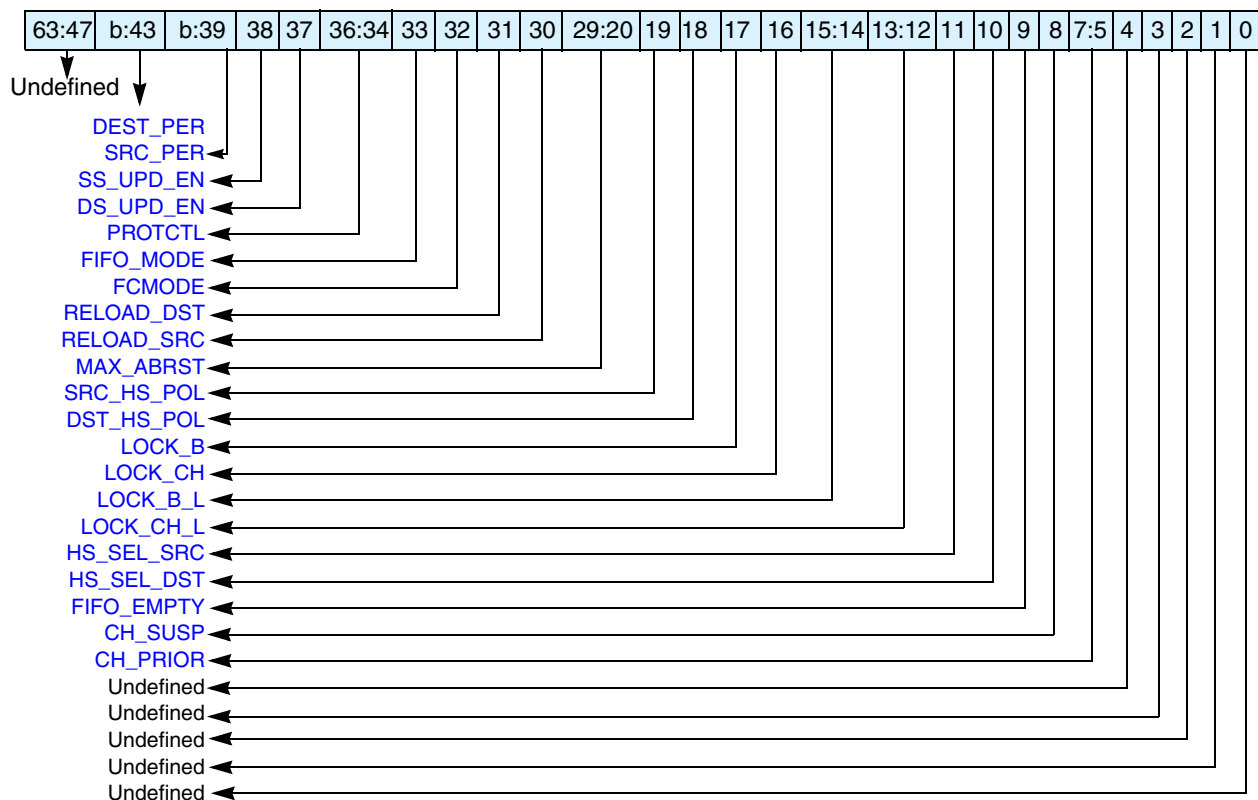
- **Read/Write Access:** Read/Write

This register contains fields that configure the DMA transfer. The channel configuration register remains fixed for all blocks of a multi-block transfer.



Note

You need to program this register prior to enabling the channel.



Bits	Name	R/W	Reset	Description
63:47	Undefined	N/A	0x0	Reserved
b:43 (see notes)	DEST_PER	R/W	0x0	Assigns a hardware handshaking interface (0 - DMAH_NUM_HS_INT-1) to the destination of channel x if the CFGx.HS_SEL_DST field is 0; otherwise, this field is ignored. The channel can then communicate with the destination peripheral connected to that interface through the assigned hardware handshaking interface. NOTE1: For correct DMA operation, only one peripheral (source or destination) should be assigned to the same handshaking interface. NOTE2: This field does not exist if the configuration parameter DMAH_NUM_HS_INT is set to 0. NOTE3: ■ b = 43 if DMAH_NUM_HS_INT is 1 ■ b = ceil(log2(DMAH_NUM_HS_INT)) + 42 if DMAH_NUM_HS_INT is greater than 1 Bits 46:(b+1) do not exist and return 0 on a read.
b:39 (see notes)	SRC_PER	R/W	0x0	Assigns a hardware handshaking interface (0 - DMAH_NUM_HS_INT-1) to the source of channel x if the CFGx.HS_SEL_SRC field is 0; otherwise, this field is ignored. The channel can then communicate with the source peripheral connected to that interface through the assigned hardware handshaking interface. NOTE1: For correct DW_ahb_dmac operation, only one peripheral (source or destination) should be assigned to the same handshaking interface. NOTE2: This field does not exist if the configuration parameter DMAH_NUM_HS_INT is set to 0. NOTE3: ■ b = 39 if DMAH_NUM_HS_INT is 1 ■ b = ceil(log2(DMAH_NUM_HS_INT)) + 38 if DMAH_NUM_HS_INT is greater than 1 Bits 42:(b+1) do not exist and return 0 on a read.
38	SS_UPD_EN	R/W	0x0	Source Status Update Enable. Source status information is fetched only from the location pointed to by the SSTATARx register, stored in the SSTATx register and written out to the SSTATx location of the LLI (refer to Figure 8-3 on page 250) if SS_UPD_EN is high. NOTE: This enable is applicable only if DMAH_CHx_STAT_SRC (page 132) is set to True. This field does not exist if the configuration parameter DMAH_CHx_STAT_SRC is set to False; in this case, the read-back value is always 0.

Bits	Name	R/W	Reset	Description
37	DS_UPD_EN	R/W	0x0	Destination Status Update Enable. Destination status information is fetched only from the location pointed to by the DSTATARx register, stored in the DSTATx register and written out to the DSTATx location of the LLI (refer to Figure 8-3 on page 250) if DS_UPD_EN is high. This field does not exist if the configuration parameter DMAH_CHx_STAT_DST is set to False; in this case, the read-back value is always 0.
36:34	PROTCTL	R/W	0x1	Protection Control bits used to drive the AHB HPROT[3:1] bus. The <i>AMBA Specification</i> recommends that the default value of HPROT indicates a non-cached, non-buffered, privileged data access. The reset value is used to indicate such an access. HPROT[0] is tied high because all transfers are data accesses, as there are no opcode fetches. There is a one-to-one mapping of these register bits to the HPROT[3:1] master interface signals. Table 6-5 on page 185 shows the mapping of bits in this field to the AHB HPROT[3:1] bus.
33	FIFO_MODE	R/W	0x0	FIFO Mode Select. Determines how much space or data needs to be available in the FIFO before a burst transaction request is serviced. 0 = Space/data available for single AHB transfer of the specified transfer width. 1 = Data available is greater than or equal to half the FIFO depth for destination transfers and space available is greater than half the fifo depth for source transfers. The exceptions are at the end of a burst transaction request or at the end of a block transfer.
32	FCMODE	R/W	0x0	Flow Control Mode. Determines when source transaction requests are serviced when the Destination Peripheral is the flow controller. 0 = Source transaction requests are serviced when they occur. Data pre-fetching is enabled. 1 = Source transaction requests are not serviced until a destination transaction request occurs. In this mode, the amount of data transferred from the source is limited so that it is guaranteed to be transferred to the destination prior to block termination by the destination. Data pre-fetching is disabled.
31	RELOAD_DST	R/W	0x0	Automatic Destination Reload. The DARx register can be automatically reloaded from its initial value at the end of every block for multi-block transfers. A new block transfer is then initiated. For conditions under which this occurs, refer to Table 8-1 on page 249. This register does not exist if the configuration parameter DMAH_CHx_MULTI_BLK_EN (page 130) is not selected; in this case, the read-back value is always 0.

Bits	Name	R/W	Reset	Description
30	RELOAD_SRC	R/W	0x0	Automatic Source Reload. The SARx register can be automatically reloaded from its initial value at the end of every block for multi-block transfers. A new block transfer is then initiated. For conditions under which this occurs, refer to Table 8-1 on page 249. This field does not exist if the configuration parameter DMAH_CHx_MULTI_BLK_EN (page 130) is not selected; in this case, the read-back value is always 0.
29:20	MAX_ABRST	R/W	0x0	Maximum AMBA Burst Length. Maximum AMBA burst length that is used for DMA transfers on this channel. A value of 0 indicates that software is not limiting the maximum AMBA burst length for DMA transfers on this channel. This field does not exist if the configuration parameter DMAH_MABRST (page 121) is not selected; in this case, the read-back value is always 0, and the maximum AMBA burst length cannot be limited by software.
19	SRC_HS_POL	R/W	0x0	Source Handshaking Interface Polarity. 0 = Active high 1 = Active low For information on this, refer to “Handshaking Interface” on page 42.
18	DST_HS_POL	R/W	0x0	Destination Handshaking Interface Polarity. 0 = Active high 1 = Active low For information on this, refer to “Handshaking Interface” on page 42.
17	LOCK_B	R/W	0x0	Bus Lock Bit. When active, the AHB bus master signal hlock is asserted for the duration specified in CFGx.LOCK_B_L. For more information, refer to “Locked DMA Transfers” on page 103. This field does not exist if the configuration parameter DMAH_CHx_LOCK_EN is set to False (page 128); in this case, the read-back value is always 0.
16	LOCK_CH	R/W	0x0	Channel Lock Bit. When the channel is granted control of the master bus interface and if the CFGx.LOCK_CH bit is asserted, then no other channels are granted control of the master bus interface for the duration specified in CFGx.LOCK_CH_L. Indicates to the master bus interface arbiter that this channel wants exclusive access to the master bus interface for the duration specified in CFGx.LOCK_CH_L. This field does not exist if the configuration parameter DMAH_CHx_LOCK_EN is set to False (page 128); in this case, the read-back value is always 0.

Bits	Name	R/W	Reset	Description
15:14	LOCK_B_L	R/W	0x0	Bus Lock Level. Indicates the duration over which CFGx.LOCK_B bit applies. 00 = Over complete DMA transfer 01 = Over complete DMA block transfer 1x = Over complete DMA transaction This field does not exist if the parameter DMAH_CHx_LOCK_EN is set to False (page 128); in this case, the read-back value is always 0.
13:12	LOCK_CH_L	R/W	0x0	Channel Lock Level. Indicates the duration over which CFGx.LOCK_CH bit applies. 00 = Over complete DMA transfer 01 = Over complete DMA block transfer 1x = Over complete DMA transaction This field does not exist if the configuration parameter DMAH_CHx_LOCK_EN is set to False (page 128); in this case, the read-back value is always 0.
11	HS_SEL_SRC	R/W	0x1	Source Software or Hardware Handshaking Select. This register selects which of the handshaking interfaces – hardware or software – is active for source requests on this channel. 0 = Hardware handshaking interface. Software-initiated transaction requests are ignored. 1 = Software handshaking interface. Hardware-initiated transaction requests are ignored. If the source peripheral is memory, then this bit is ignored.
10	HS_SEL_DST	R/W	0x1	Destination Software or Hardware Handshaking Select. This register selects which of the handshaking interfaces – hardware or software – is active for destination requests on this channel. 0 = Hardware handshaking interface. Software-initiated transaction requests are ignored. 1 = Software handshaking interface. Hardware-initiated transaction requests are ignored. If the destination peripheral is memory, then this bit is ignored.
9	FIFO_EMPTY	R	0x1	Indicates if there is data left in the channel FIFO. Can be used in conjunction with CFGx.CH_SUSP to cleanly disable a channel. For more information, refer to “Disabling a Channel Prior to Transfer Completion” on page 282. 1 = Channel FIFO empty 0 = Channel FIFO not empty

Bits	Name	R/W	Reset	Description
8	CH_SUSP	R/W	0x0	Channel Suspend. Suspends all DMA data transfers from the source until this bit is cleared. There is no guarantee that the current transaction will complete. Can also be used in conjunction with CFGx.FIFO_EMPTY to cleanly disable a channel without losing any data. 0 = Not suspended. 1 = Suspend DMA transfer from the source. For more information, refer to “Disabling a Channel Prior to Transfer Completion” on page 282.
7:5	CH_PRIOR	R/W	Channel Number For example: Chan0=0 Chan1=1	Channel priority. A priority of 7 is the highest priority, and 0 is the lowest. This field must be programmed within the following range: 0: (DMAH_NUM_CHANNELS – 1) A programmed value outside this range will cause erroneous behavior.
4:0	Undefined	N/A	0x0	Reserved

Table 6-5 PROTCTL field to HPROT Mapping

1'b1	HPROT[0]
CFGx.PROTCTL[1]	HPROT[1]
CFGx.PROTCTL[2] ->	HPROT[2]
CFGx.PROTCTL[3] ->	HPROT[3]

6.2.2.11 SGR_x

- **Name:** Source Gather Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

SGR0 – 0x048
 SGR1 – 0x0a0
 SGR2 – 0x0f8
 SGR3 – 0x150
 SGR4 – 0x1a8
 SGR5 – 0x200
 SGR6 – 0x258
 SGR7 – 0x2b0

- **Read/Write Access:** Read/Write

The Source Gather register contains two fields:

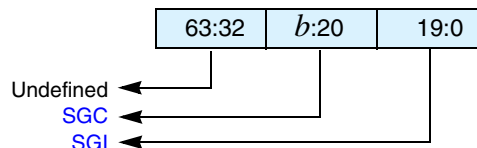
- Source gather count field (SGR_x.SGC) – Specifies the number of contiguous source transfers of CTL_x.SRC_TR_WIDTH between successive gather intervals. This is defined as a gather boundary.
- Source gather interval field (SGR_x.SGI) – Specifies the source address increment/decrement in multiples of CTL_x.SRC_TR_WIDTH on a gather boundary when gather mode is enabled for the source transfer.



Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

The CTL_x.SINC field controls whether the address increments or decrements. When the CTL_x.SINC field indicates a fixed-address control, then the address remains constant throughout the transfer and the SGR_x register is ignored. This register does not exist if the configuration parameter DMAH_CH_x_SRC_GAT_EN is set to False. For more information, see “[Scatter/Gather](#)” on page 108.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved.
<i>b</i> :20 See description	SGC	R/W	0x0	Source gather count. Source contiguous transfer count between successive gather boundaries. $b = \log_2 (\text{DMAH_CHx_MAX_BLK_SIZE} + 1) + 19$ Bits 31: <i>b</i> +1 do not exist and read back as 0.
19:0	SGL	R/W	0x0	Source gather interval.

6.2.2.12 DSRx

- **Name:** Destination Scatter Register for Channel x
- **Size:** 64 bits (upper 32 bits are reserved)
- **Address Offset:** for $x = 0$ to 7:

DSR0 – 0x050
 DSR1 – 0x0a8
 DSR2 – 0x100
 DSR3 – 0x158
 DSR4 – 0x1B0
 DSR5 – 0x208
 DSR6 – 0x260
 DSR7 – 0x2b8

- **Read/Write Access:** Read/Write

The Destination Scatter register contains two fields:

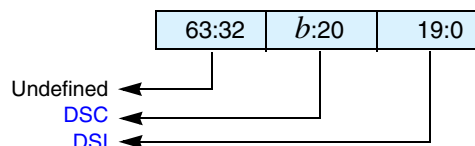
- Destination scatter count field (DSRx.DSC) – Specifies the number of contiguous destination transfers of CTLx.DST_TR_WIDTH between successive scatter boundaries.
- Destination scatter interval field (DSRx.DSI) – Specifies the destination address increment/decrement in multiples of CTLx.DST_TR_WIDTH on a scatter boundary when scatter mode is enabled for the destination transfer.



Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

The CTLx.DINC field controls whether the address increments or decrements. When the CTLx.DINC field indicates a fixed address control, then the address remains constant throughout the transfer and the DSRx register is ignored. This register does not exist if the configuration parameter DMAH_CHx_DST_SCA_EN is set to False. For more information, see “[Scatter/Gather](#)” on page 108.



Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved.
<i>b</i> :20 See description	DSC	R/W	0x0	Destination scatter count. Destination contiguous transfer count between successive scatter boundaries. $b = \log_2 (\text{DMAH_CHx_MAX_BLK_SIZE} + 1) + 19$ Bits 31: <i>b</i> +1 do not exist and read 0.
19:0	<i>DSI</i>	R/W	0x0	Destination scatter interval.

6.2.3 Interrupt Registers

The following sections describe the registers pertaining to interrupts, their status, and how to clear them. For each channel, there are five types of interrupt sources:

- **IntBlock – Block Transfer Complete Interrupt**

This interrupt is generated on DMA block transfer completion to the destination peripheral.

- **IntDstTran – Destination Transaction Complete Interrupt**

This interrupt is generated after completion of the last AHB transfer of the requested single/burst transaction from the handshaking interface (either the hardware or software handshaking interface) on the destination side.



Note

If the destination for a channel is memory, then that channel will never generate the IntDstTran interrupt. Because of this, the corresponding bit in this field will not be set.

- **IntErr – Error Interrupt**

This interrupt is generated when an ERROR response is received from an AHB slave on the HRESP bus during a DMA transfer. In addition, the DMA transfer is cancelled and the channel is disabled.

- **IntSrcTran – Source Transaction Complete Interrupt**

This interrupt is generated after completion of the last AHB transfer of the requested single/burst transaction from the handshaking interface (either the hardware or software handshaking interface) on the source side.



Note

If the source or destination is memory, then IntSrcTran/IntDstTran interrupts should be ignored, as there is no concept of a “DMA transaction level” for memory.

- **IntTfr – DMA Transfer Complete Interrupt**

This interrupt is generated on DMA transfer completion to the destination peripheral.

There are several groups of interrupt-related registers:

- [RawBlock](#), [RawDstTran](#), [RawErr](#), [RawSrcTran](#), [RawTfr](#)
- [StatusBlock](#), [StatusDstTran](#), [StatusErr](#), [StatusSrcTran](#), [StatusTfr](#)
- [MaskBlock](#), [MaskDstTran](#), [MaskErr](#), [MaskSrcTran](#), [MaskTfr](#)
- [ClearBlock](#), [ClearDstTran](#), [ClearErr](#), [ClearSrcTran](#), [ClearTfr](#)
- [StatusInt](#)

When a channel has been enabled to generate interrupts, the following is true:

- Interrupt events are stored in the Raw Status registers.
- The contents of the Raw Status registers are masked with the contents of the Mask registers.

- The masked interrupts are stored in the Status registers.
- The contents of the Status registers are used to drive the int_* port signals.
- Writing to the appropriate bit in the Clear registers clears an interrupt in the Raw Status registers and the Status registers on the same clock cycle.

The contents of each of the five Status registers is ORed to produce a single bit for each interrupt type in the Combined Status register; that is, StatusInt.

**Note**

For interrupts to propagate past the raw* interrupt register stage, CTLx.INT_EN must be set to 1'b1, and the relevant interrupt must be unmasked in the mask* interrupt register.

6.2.3.1 RawBlock, RawDstTran, RawErr, RawSrcTran, RawTfr

- **Name:** Interrupt Raw Status Registers

- **Size:** 64 bits

- **Address Offset:**

RawTfr – 0x2c0

RawBlock – 0x2c8

RawSrcTran – 0x2d0

RawDstTran – 0x2d8

RawErr – 0x2e0

- **Read/Write Access:** Read/Write

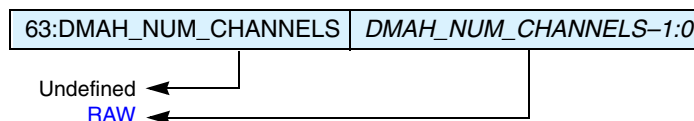
Interrupt events are stored in these Raw Interrupt Status registers before masking: RawBlock, RawDstTran, RawErr, RawSrcTran, and RawTfr. Each Raw Interrupt Status register has a bit allocated per channel; for example, RawTfr[2] is the Channel 2 raw transfer complete interrupt.

Each bit in these registers is cleared by writing a 1 to the corresponding location in the ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, ClearErr registers.



Note

Write access is available to these registers for software testing purposes only. Under normal operation, writes to these registers are not recommended.



Bits	Name	R/W	Reset	Description
63:DMAH_NUM_CHANNELS	Undefined	N/A	0x0	Reserved
DMAH_NUM_CHANNELS-1:0	RAW	R/W	0x0	Raw interrupt status.

6.2.3.2 StatusBlock, StatusDstTran, StatusErr, StatusSrcTran, StatusTfr

- **Name:** Interrupt Status Registers
- **Size:** 64 bits
- **Address Offset:**

StatusTfr – 0x2e8

StatusBlock – 0x2f0

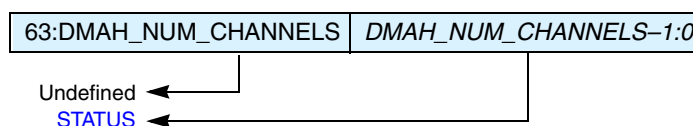
StatusSrcTran – 0x2f8

StatusDstTran – 0x300

StatusErr – 0x308

- **Read/Write Access:** Read

All interrupt events from all channels are stored in these Interrupt Status registers after masking: StatusBlock, StatusDstTran, StatusErr, StatusSrcTran, and StatusTfr. Each Interrupt Status register has a bit allocated per channel; for example, StatusTfr[2] is the Channel 2 status transfer complete interrupt. The contents of these registers are used to generate the interrupt signals (int or int_n bus, depending on interrupt polarity) leaving the DW_ahb_dmac.



Bits	Name	R/W	Reset	Description
63:DMAH_NUM_CHANNELS	Undefined	N/A	0x0	Reserved
DMAH_NUM_CHANNELS-1:0	STATUS	R	0x0	Interrupt status.

6.2.3.3 MaskBlock, MaskDstTran, MaskErr, MaskSrcTran, MaskTfr

- **Name:** Interrupt Mask Registers

- **Size:** 64 bits

- **Address Offset:**

MaskTfr – 0x310

MaskBlock – 0x318

MaskSrcTran – 0x320

MaskDstTran – 0x328

MaskErr – 0x330

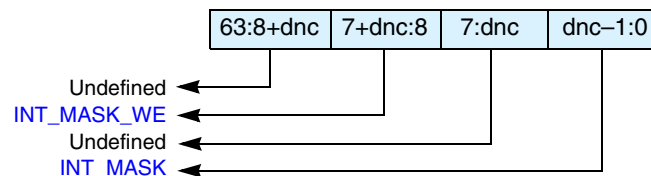
- **Read/Write Access:** Read/Write

The contents of the Raw Status registers are masked with the contents of the Mask registers: MaskBlock, MaskDstTran, MaskErr, MaskSrcTran, and MaskTfr. Each Interrupt Mask register has a bit allocated per channel; for example, MaskTfr[2] is the mask bit for the Channel 2 transfer complete interrupt.

When the source peripheral of DMA channel *i* is memory, then the source transaction complete interrupt, MaskSrcTran[*i*], must be masked to prevent an erroneous triggering of an interrupt on the int_combined signal. Similarly, when the destination peripheral of DMA channel *i* is memory, then the destination transaction complete interrupt, MaskDstTran[*i*], must be masked to prevent an erroneous triggering of an interrupt on the int_combined(_n) signal.

A channel INT_MASK bit will be written only if the corresponding mask write enable bit in the INT_MASK_WE field is asserted on the same AHB write transfer. This allows software to set a mask bit without performing a read-modified write operation. For example, writing hex 01x1 to the MaskTfr register writes a 1 into MaskTfr[0], while MaskTfr[7:1] remains unchanged. Writing hex 00xx leaves MaskTfr[7:0] unchanged.

Writing a 1 to any bit in these registers unmaskes the corresponding interrupt, thus allowing the DW_ahb_dmac to set the appropriate bit in the Status registers and int_* port signals.



Bits	Name	R/W	Description
63:8+dnc	Undefined	N/A	Reserved dnc = DMAH_NUM_CHANNELS Reset Value: 0x0
7+dnc:8	INT_MASK_WE	W	Interrupt Mask Write Enable 0 = write disabled 1 = write enabled dnc = DMAH_NUM_CHANNELS Reset Value: 0x0

Bits	Name	R/W	Description
7:dnc	Undefined	N/A	Reserved dnc = DMAH_NUM_CHANNELS If dnc = 8, then this field is not present. Reset Value: 0x0
dnc-1:0	INT_MASK	R/W	Interrupt Mask 0 = masked 1 = unmasked dnc = DMAH_NUM_CHANNELS Reset Value: 0x0

6.2.3.4 ClearBlock, ClearDstTran, ClearErr, ClearSrcTran, ClearTfr

- **Name:** Interrupt Clear Registers

- **Size:** 64 bits

- **Address Offset:**

ClearTfr – 0x338

ClearBlock – 0x340

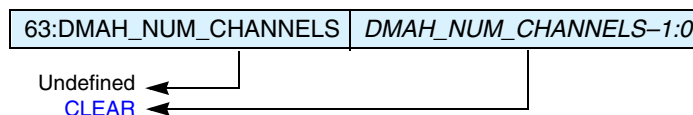
ClearSrcTran – 0x348

ClearDstTran – 0x350

ClearErr – 0x358

- **Read/Write Access:** Write

Each bit in the Raw Status and Status registers is cleared on the same cycle by writing a 1 to the corresponding location in the Clear registers: ClearBlock, ClearDstTran, ClearErr, ClearSrcTran, and ClearTfr. Each Interrupt Clear register has a bit allocated per channel; for example, ClearTfr[2] is the clear bit for the Channel 2 transfer complete interrupt. Writing a 0 has no effect. These registers are not readable.

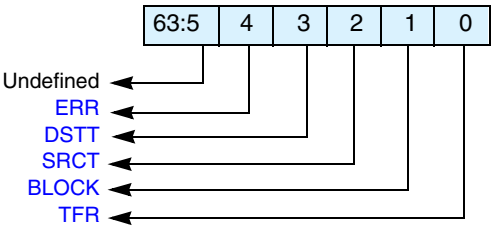


Bits	Name	R/W	Reset	Description
63:DMAH_NUM_CHANNELS	Undefined	N/A	0x0	Reserved
DMAH_NUM_CHANNELS-1:0	CLEAR	W	N/A	Interrupt clear. 0 = no effect 1 = clear interrupt

6.2.3.5 **StatusInt**

- **Name:** Combined Interrupt Status Register
- **Size:** 64 bits
- **Address Offset:** 0x360
- **Read/Write Access:** Read

The contents of each of the five Status registers – StatusTfr, StatusBlock, StatusSrcTran, StatusDstTran, StatusErr – is ORed to produce a single bit for each interrupt type in the Combined Status register (StatusInt). This register is read-only.



Bit	Name	R/W	Reset	Description
63:5	Undefined	N/A	0x0	Reserved
4	ERR	R	0x0	OR of the contents of StatusErr register.
3	DSTT	R	0x0	OR of the contents of StatusDst register.
2	SRCT	R	0x0	OR of the contents of StatusSrcTran register.
1	BLOCK	R	0x0	OR of the contents of StatusBlock register.
0	TFR	R	0x0	OR of the contents of StatusTfr register.

6.2.4 Software Handshaking Registers

The registers that comprise the software handshaking registers allow software to initiate single or burst transaction requests in the same way that handshaking interface signals do in hardware.

The software handshaking registers are:

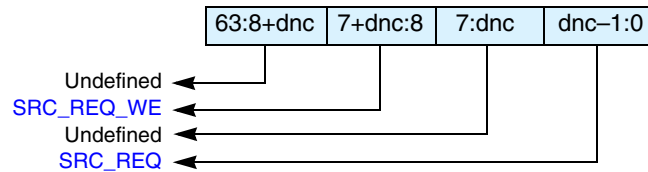
- [LstDstReg](#) – Last Destination Transaction Request Register
- [LstSrcReg](#) – Last Source Transaction Request Register
- [ReqDstReg](#) – Destination Software Transaction Request Register
- [ReqSrcReg](#) – Source Software Transaction Request Register
- [SglReqDstReg](#) – Single Destination Transaction Request Register
- [SglReqSrcReg](#) – Single Source Transaction Request Register

Setting [CFGx.HS_SEL_SRC](#) to 1 enables software handshaking on the source of channel x . Setting [CFGx.HS_SEL_DST](#) to 1 enables software handshaking on the destination of channel x .

6.2.4.1 ReqSrcReg

- **Name:** Source Software Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x368
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. ReqSrcReg[*n*] is ignored when software handshaking is not enabled for the source of channel *n*.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	SRC_REQ_WE	W	0x0	Source request write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	SRC_REQ	R/W	0x0	Source request

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

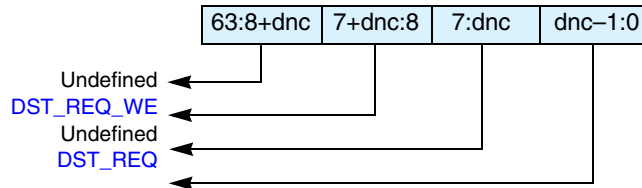
A channel SRC_REQ bit is written only if the corresponding channel write enable bit in the SRC_REQ_WE field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register. For example, writing hex 0101 writes a 1 into ReqSrcReg[0], while ReqSrcReg[7:1] remains unchanged. Writing hex 00xx leaves ReqSrcReg[7:0] unchanged. This allows software to set a bit in the ReqSrcReg register without performing a read-modified write operation.

The functionality of this register depends on whether the source is a flow control peripheral or not. For a description of when the source is not a flow controller, refer to [“Software Handshaking - Peripheral Is Not Flow Controller”](#) on page 54. For a description of when the source is a flow controller, refer to [“Software Handshaking - Peripheral Is Flow Controller”](#) on page 59.

6.2.4.2 ReqDstReg

- **Name:** Destination Software Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x370
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. ReqDstReg[*n*] is ignored when software handshaking is not enabled for the source of channel *n*.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	DST_REQ_WE	W	0x0	Destination request write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	DST_REQ	R/W	0x0	Destination request

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

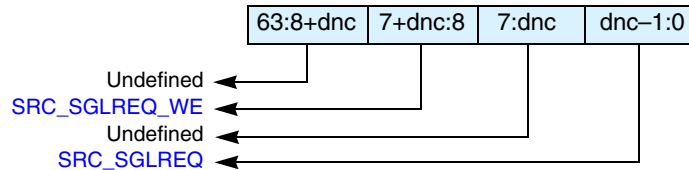
A channel DST_REQ bit is written only if the corresponding channel write enable bit in the DST_REQ_WE field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register.

The functionality of this register depends on whether the destination is a flow control peripheral or not. For a description of when the destination is not a flow controller, refer to [“Software Handshaking - Peripheral Is Not Flow Controller”](#) on page 54. For a description of when the destination is a flow controller, refer to [“Software Handshaking - Peripheral Is Flow Controller”](#) on page 59.

6.2.4.3 SglReqSrcReg

- **Name:** Single Source Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x378
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. SglReqSrcReg[*n*] is ignored when software handshaking is not enabled for the source of channel *n*.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	SRC_SGLREQ_WE	W	0x0	Single write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	SRC_SGLREQ	R/W	0x0	Source single request

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

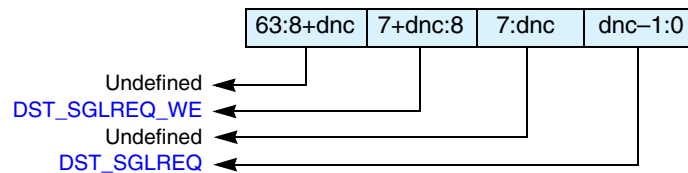
A channel SRC_SGLREQ bit is written only if the corresponding channel write enable bit in the SRC_SGLREQ_WE field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register.

The functionality of this register depends on whether the source is a flow control peripheral or not. For a description of when the source is not a flow controller, refer to [“Software Handshaking – Peripheral Is Not Flow Controller”](#) on page 54. For a description of when the source is a flow controller, refer to [“Software Handshaking – Peripheral Is Flow Controller”](#) on page 59.

6.2.4.4 SglReqDstReg

- **Name:** Single Destination Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x380
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. SglReqDstReg[*n*] is ignored when software handshaking is not enabled for the destination of channel *n*.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	DST_SGLREQ_WE	W	0x0	Destination write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	DST_SGLREQ	R/W	0x0	Destination single or burst request

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

A channel DST_SGLREQ bit is written only if the corresponding channel write enable bit in the DST_SGLREQ_WE field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register.

The functionality of this register depends on whether the destination is a flow control peripheral or not. For a description of when the destination is not a flow controller, refer to [“Software Handshaking - Peripheral Is Not Flow Controller”](#) on page 54. For a description of when the destination is a flow controller, refer to [“Software Handshaking - Peripheral Is Flow Controller”](#) on page 59.

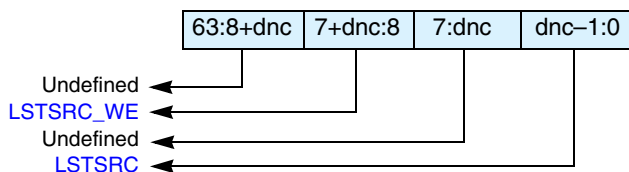
6.2.4.5 LstSrcReg

- **Name:** Last Source Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x388
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. LstSrcReg[*n*] is ignored when software handshaking is not enabled for the source of channel *n*, or when the source of channel *n* is not a flow controller.

A channel *LSTSRC* bit is written only if the corresponding channel write enable bit in the *LSTSRC_WE* field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register.

For a description of this register, refer to “[Software Handshaking – Peripheral Is Flow Controller](#)” on page 59.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	LSTSRC_WE	W	0x0	Source last transaction request write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	LSTSRC	R/W	0x0	Source last transaction request 0 = Not last transaction in current block 1 = Last transaction in current block

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

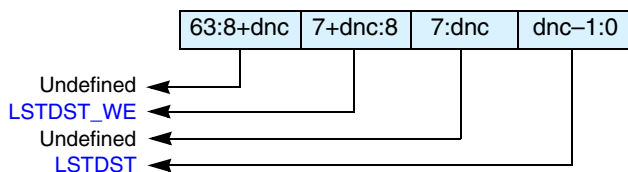
6.2.4.6 LstDstReg

- **Name:** Last Destination Transaction Request Register
- **Size:** 64 bits
- **Address Offset:** 0x390
- **Read/Write Access:** Read/Write

A bit is assigned for each channel in this register. LstDstReg[*n*] is ignored when software handshaking is not enabled for the destination of channel *n* or when the destination of channel *n* is not a flow controller.

A channel LSTDST bit is written only if the corresponding channel write enable bit in the LSTDST_WE field is asserted on the same AHB write transfer, and if the channel is enabled in the ChEnReg register.

For a description of this register, refer to “[Software Handshaking - Peripheral Is Flow Controller](#)” on page 59.



Bits	Name	R/W	Reset	Description
63:8+dnc ^a	Undefined	N/A	0x0	Reserved
7+dnc ^a :8	LSTDST_WE	W	0x0	Destination last transaction request write enable 0 = write disabled 1 = write enabled
7:dnc ^{a, b}	Undefined	N/A	0x0	Reserved
dnc ^a -1:0	LSTDST	R/W	0x0	Destination last transaction request 0 = Not last transaction in current block 1 = Last transaction in current block

a. dnc = DMAH_NUM_CHANNELS

b. If dnc = 8, then this field is not present.

6.2.5 Miscellaneous DW_ahb_dmac Registers

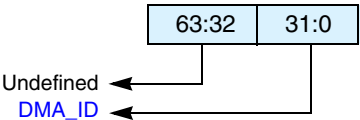
This section describes the following registers of the DW_ahb_dmac:

- [DmaIdReg](#) – DW_ahb_dmac ID
- [DmaTestReg](#) – DW_ahb_dmac Test
- [DMA Component ID Register](#) – DW_ahb_dmac Version ID

6.2.5.1 DmaIdReg

- **Name:** DW_ahb_dmac ID Register
- **Size:** 64 bits
- **Address Offset:** 0x3a8
- **Read/Write Access:** Read

This is the DW_ahb_dmac ID register, which is a read-only register that reads back the coreConsultant-configured hardcoded ID number, DMAH_ID_NUM.

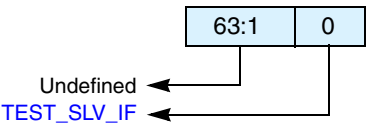


Bits	Name	R/W	Reset	Description
63:32	Undefined	N/A	0x0	Reserved
31:0	DMA_ID	R	DMAH_ID_NUM	Hardcoded DW_ahb_dmac Peripheral ID

6.2.5.2 DmaTestReg

- **Name:** DW_ahb_dmac Test Register
- **Size:** 64 bits
- **Address Offset:** 0x3b0
- **Read/Write Access:** Read/Write

This register is used to put the AHB slave interface into test mode, during which the readback value of the writable registers match the value written, assuming the DW_ahb_dmac configuration has not optimized the same registers. In normal operation, the readback value of some registers is a function of the DW_ahb_dmac state and does not match the value written.



Bits	Name	R/W	Reset	Description
63:1	Undefined	N/A	0x0	Reserved.
0	TEST_SLV_IF	R/W	0x0	Puts the AHB slave interface into test mode. In this mode, the readback value of the writable registers always matches the value written. This bit does not allow writing to read-only registers. 0 = Normal mode 1 = Test mode

6.2.5.3 DMA_COMP_PARAMS_6

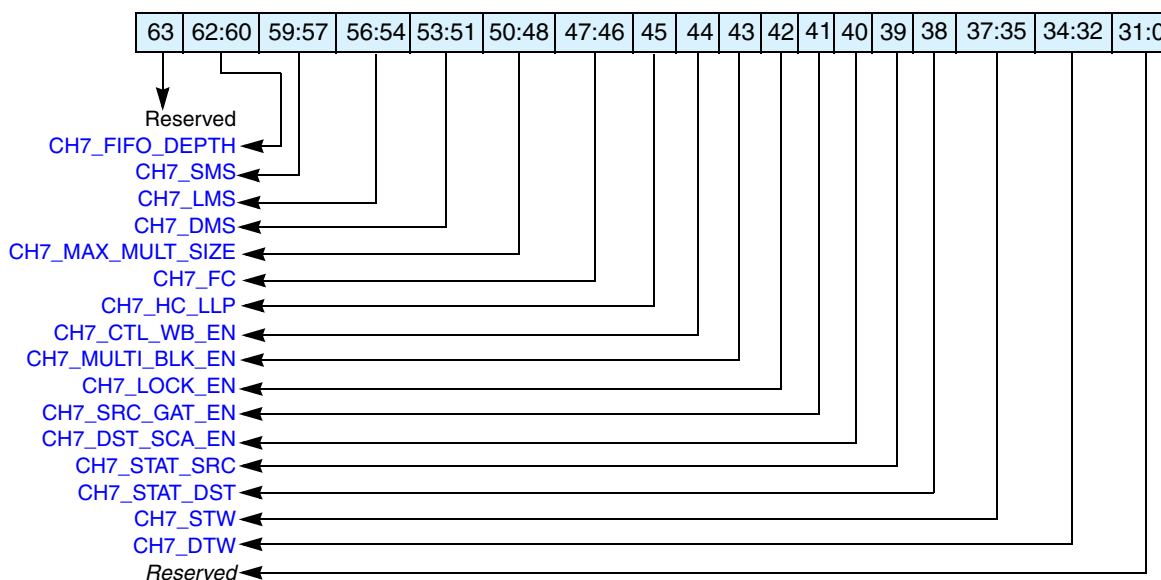
- **Name:** DW_ahb_dmac Component Parameters Register 6
- **Size:** 64 bits
- **Address Offset:** 0x3c8
- **Read/Write Access:** Read

This is a constant read-only register that contains encoded information about the component parameter settings for Channel 7. The reset value depends on coreConsultant parameter(s).



Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.



Bits	Name	R/W	Description
63	Reserved	N/A	Reserved
62:60	CH7_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH7_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128

Bits	Name	R/W	Description
59:57	CH7_SMS	R	The value of this register is derived from the DMAH_CH7_SMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
56:54	CH7_LMS	R	The value of this register is derived from the DMAH_CH7_LMS coreConsultant parameter. For a description of this parameter, refer to page 132. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
53:51	CH7_DMS	R	The value of this register is derived from the DMAH_CH7_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
50:48	CH7_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH7_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
47:46	CH7_FC	R	The value of this register is derived from the DMAH_CH7_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY

Bits	Name	R/W	Description
45	CH7_HC_LLP	R	The value of this register is derived from the DMAH_CH7_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131 . 0x0 = FALSE 0x1 = TRUE
44	CH7_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH7_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
43	CH7_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH7_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130 . 0x0 = FALSE 0x1 = TRUE
42	CH7_LOCK_EN	R	The value of this register is derived from the DMAH_CH7_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128 . 0x0 = FALSE 0x1 = TRUE
41	CH7_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH7_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
40	CH7_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH7_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
39	CH7_STAT_SRC	R	The value of this register is derived from the DMAH_CH7_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
38	CH7_STAT_DST	R	The value of this register is derived from the DMAH_CH7_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
37:35	CH7_STW	R	The value of this register is derived from the DMAH_CH7_STW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
34:32	CH7_DTW	R	The value of this register is derived from the DMAH_CH7_DTW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
31:0	Reserved	N/A	Reserved

6.2.5.4 DMA_COMP_PARAMS_5

- **Name:** DW_ahb_dmac Component Parameters Register 5
- **Size:** 64 bits
- **Address Offset:** 0x3d0
- **Read/Write Access:** Read

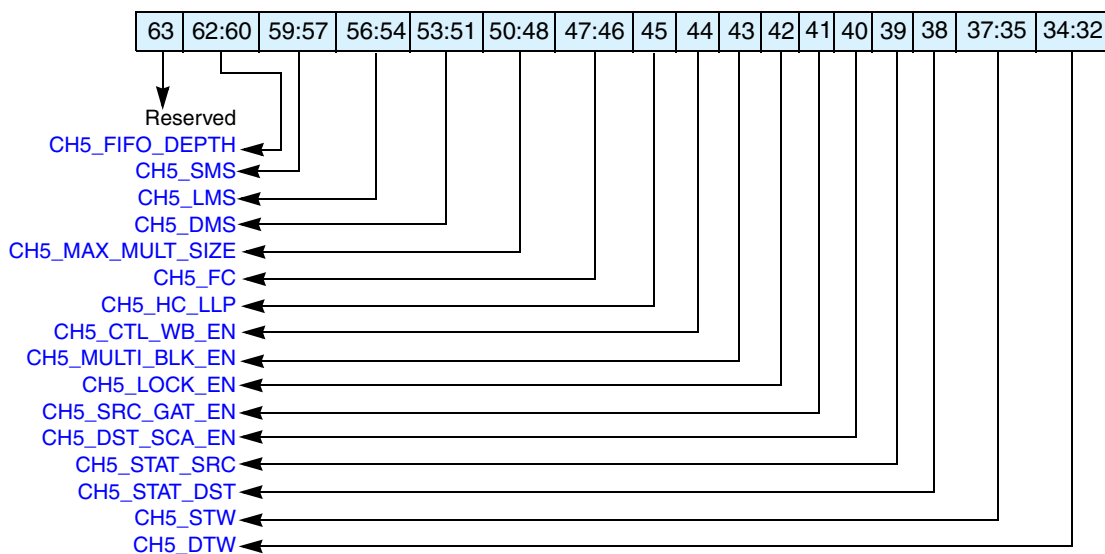
This is a constant read-only register that contains encoded information about the component parameter settings for Channel 5 and Channel 6. The reset value depends on coreConsultant parameter(s).



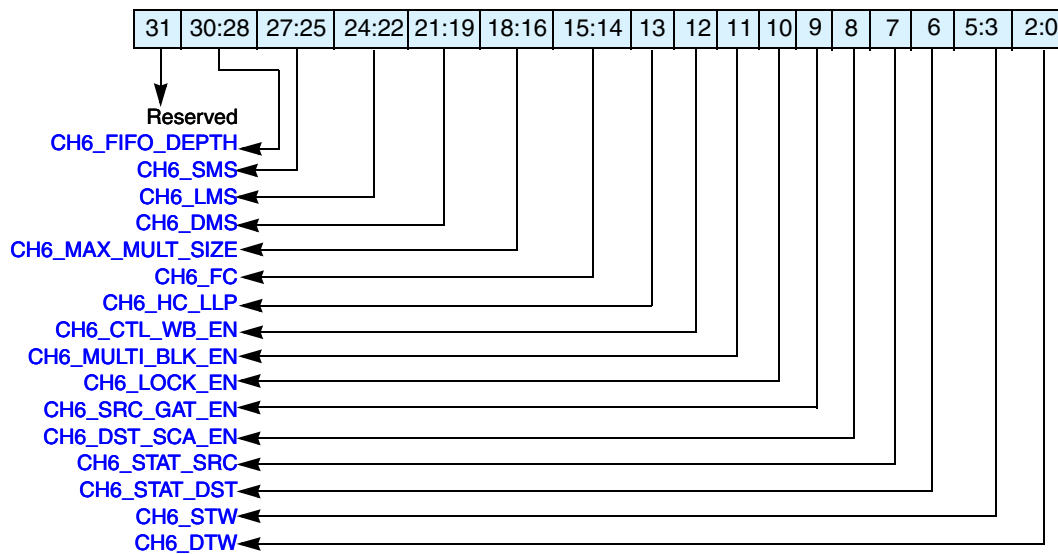
Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

Bits 63:32 include the configuration parameters for Channel 5.



Bits 31:0 include configuration parameters for Channel 6.



Bits	Name	R/W	Description
63	Reserved	N/A	Reserved
62:60	CH5_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH5_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127 . 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
59:57	CH5_SMS	R	The value of this register is derived from the DMAH_CH5_SMS coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
56:54	CH5_LMS	R	The value of this register is derived from the DMAH_CH5_LMS coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
53:51	CH5_DMS	R	The value of this register is derived from the DMAH_CH5_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
50:48	CH5_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH5_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
47:46	CH5_FC	R	The value of this register is derived from the DMAH_CH5_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
45	CH5_HC_LLP	R	The value of this register is derived from the DMAH_CH5_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
44	CH5_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH5_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
43	CH5_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH5_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
42	CH5_LOCK_EN	R	The value of this register is derived from the DMAH_CH5_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128 . 0x0 = FALSE 0x1 = TRUE
41	CH5_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH5_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
40	CH5_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH5_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
39	CH5_STAT_SRC	R	The value of this register is derived from the DMAH_CH5_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
38	CH5_STAT_DST	R	The value of this register is derived from the DMAH_CH5_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
37:35	CH5_STW	R	The value of this register is derived from the DMAH_CH5_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

Bits	Name	R/W	Description
34:32	CH5_DTW	R	The value of this register is derived from the DMAH_CH5_DTW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
31	Reserved	N/A	Reserved
30:28	CH6_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH6_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
27:25	CH6_SMS	R	The value of this register is derived from the DMAH_CH6_SMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
24:22	CH6_LMS	R	The value of this register is derived from the DMAH_CH6_LMS coreConsultant parameter. For a description of this parameter, refer to page 132. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
21:19	CH6_DMS	R	The value of this register is derived from the DMAH_CH6_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
18:16	CH6_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH6_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
15:14	CH6_FC	R	The value of this register is derived from the DMAH_CH6_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
13	CH6_HC_LLP	R	The value of this register is derived from the DMAH_CH6_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
12	CH6_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH6_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
11	CH6_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH6_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE
10	CH6_LOCK_EN	R	The value of this register is derived from the DMAH_CH6_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = FALSE 0x1 = TRUE
9	CH6_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH6_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
8	CH6_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH6_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
7	CH6_STAT_SRC	R	The value of this register is derived from the DMAH_CH6_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
6	CH6_STAT_DST	R	The value of this register is derived from the DMAH_CH6_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
5:3	CH6_STW	R	The value of this register is derived from the DMAH_CH6_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
2:0	CH6_DTW	R	The value of this register is derived from the DMAH_CH6_DTW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

6.2.5.5 DMA_COMP_PARAMS_4

- Name: DW_ahb_dmac Component Parameters Register 4
- Size: 64 bits
- Address Offset: 0x3d8
- Read/Write Access: Read

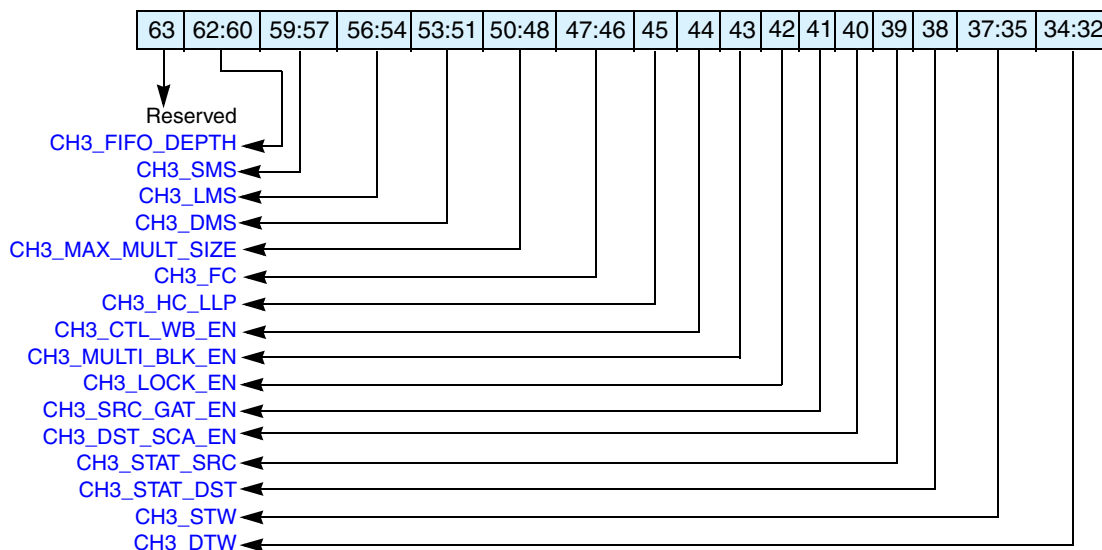
This is a constant read-only register that contains encoded information about the component parameter settings for Channel 3 and Channel 4. The reset value depends on coreConsultant parameter(s).



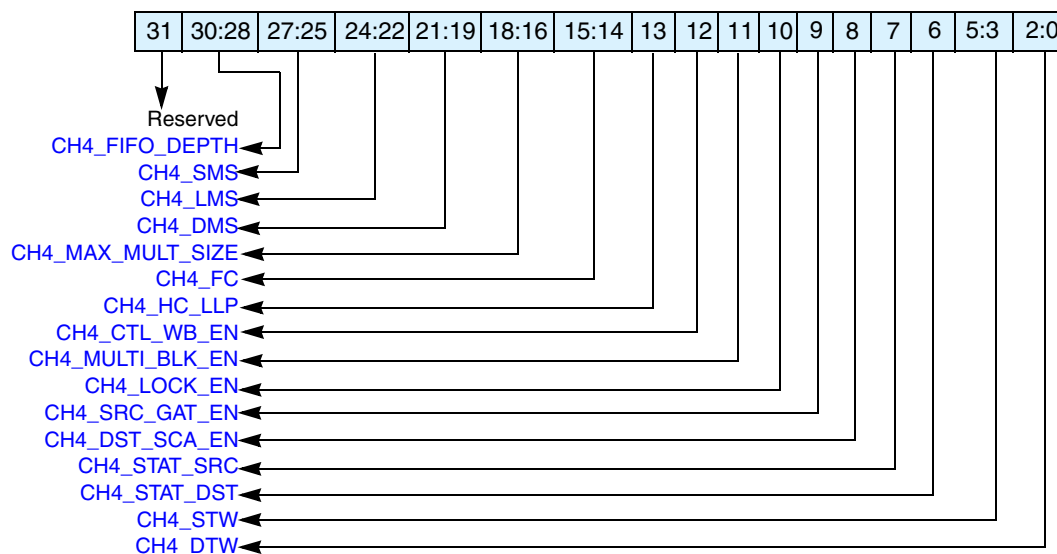
Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to [“Illegal Register Access”](#) on page 246.

Bits 63:32 include the configuration parameters for Channel 3.



Bits 31:0 include configuration parameters for Channel 4.



Bits	Name	R/W	Description
63	Reserved	N/A	Reserved
62:60	CH3_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH3_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127 . 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
59:57	CH3_SMS	R	The value of this register is derived from the DMAH_CH3_SMS coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
56:54	CH3_LMS	R	The value of this register is derived from the DMAH_CH3_LMS coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
53:51	CH3_DMS	R	The value of this register is derived from the DMAH_CH3_SMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
50:48	CH3_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH3_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
47:46	CH3_FC	R	The value of this register is derived from the DMAH_CH3_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
45	CH3_HC_LLP	R	The value of this register is derived from the DMAH_CH3_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
44	CH3_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH3_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
43	CH3_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH3_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
42	CH3_LOCK_EN	R	The value of this register is derived from the DMAH_CH3_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128 . 0x0 = FALSE 0x1 = TRUE
41	CH3_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH3_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
40	CH3_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH3_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
39	CH3_STAT_SRC	R	The value of this register is derived from the DMAH_CH3_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
38	CH3_STAT_DST	R	The value of this register is derived from the DMAH_CH3_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
37:35	CH3_STW	R	The value of this register is derived from the DMAH_CH3_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

Bits	Name	R/W	Description
34:32	CH3_DTW	R	The value of this register is derived from the DMAH_CH3_DTW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
31	Reserved	N/A	Reserved
30:28	CH4_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH4_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
27:25	CH4_SMS	R	The value of this register is derived from the DMAH_CH4_SMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
24:22	CH4_LMS	R	The value of this register is derived from the DMAH_CH4_LMS coreConsultant parameter. For a description of this parameter, refer to page 132. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
21:19	CH4_DMS	R	The value of this register is derived from the DMAH_CH4_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
18:16	CH4_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH4_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
15:14	CH4_FC	R	The value of this register is derived from the DMAH_CH4_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
13	CH4_HC_LLP	R	The value of this register is derived from the DMAH_CH4_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
12	CH4_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH4_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
11	CH4_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH4_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE
10	CH4_LOCK_EN	R	The value of this register is derived from the DMAH_CH4_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = FALSE 0x1 = TRUE
9	CH4_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH4_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
8	CH4_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH4_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
7	CH4_STAT_SRC	R	The value of this register is derived from the DMAH_CH4_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
6	CH4_STAT_DST	R	The value of this register is derived from the DMAH_CH4_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
5:3	CH4_STW	R	The value of this register is derived from the DMAH_CH4_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
2:0	CH4_DTW	R	The value of this register is derived from the DMAH_CH4_DTW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

6.2.5.6 DMA_COMP_PARAMS_3

- Name: DW_ahb_dmac Component Parameters Register 3
- Size: 64 bits
- Address Offset: 0x3e0
- Read/Write Access: Read

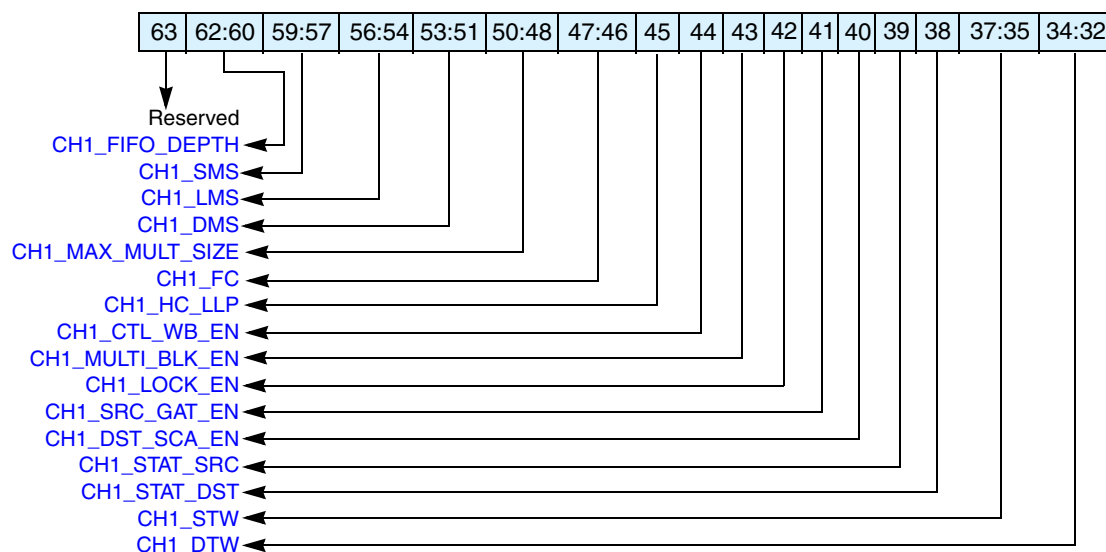
This is a constant read-only register that contains encoded information about the component parameter settings for Channel 1 and Channel 2. The reset value depends on coreConsultant parameter(s).



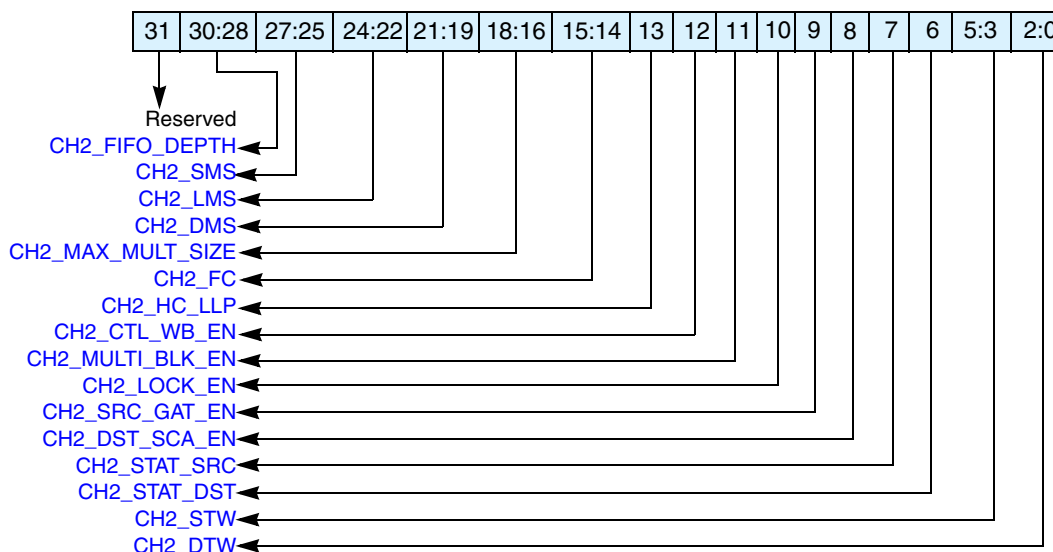
Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

Bits 63:32 include the configuration parameters for Channel 1.



Bits 31:0 include configuration parameters for Channel 2.



Bits	Name	R/W	Description
63	Reserved	N/A	Reserved
62:60	CH1_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH1_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127 . 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
59:57	CH1_SMS	R	The value of this register is derived from the DMAH_CH1_SMS coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
56:54	CH1_LMS	R	The value of this register is derived from the DMAH_CH1_LMS coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
53:51	CH1_DMS	R	The value of this register is derived from the DMAH_CH1_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
50:48	CH1_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH1_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
47:46	CH1_FC	R	The value of this register is derived from the DMAH_CH1_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
45	CH1_HC_LLP	R	The value of this register is derived from the DMAH_CH1_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
44	CH1_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH1_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
43	CH1_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH1_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
42	CH1_LOCK_EN	R	The value of this register is derived from the DMAH_CH1_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128 . 0x0 = FALSE 0x1 = TRUE
41	CH1_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH1_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
40	CH1_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH1_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
39	CH1_STAT_SRC	R	The value of this register is derived from the DMAH_CH1_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
38	CH1_STAT_DST	R	The value of this register is derived from the DMAH_CH1_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
37:35	CH1_STW	R	The value of this register is derived from the DMAH_CH1_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

Bits	Name	R/W	Description
34:32	CH1_DTW	R	The value of this register is derived from the DMAH_CH1_DTW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
31	Reserved	N/A	Reserved
30:28	CH2_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH2_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
27:25	CH2_SMS	R	The value of this register is derived from the DMAH_CH2_SMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
24:22	CH2_LMS	R	The value of this register is derived from the DMAH_CH2_LMS coreConsultant parameter. For a description of this parameter, refer to page 132. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
21:19	CH2_DMS	R	The value of this register is derived from the DMAH_CH2_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
18:16	CH2_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH2_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
15:14	CH2_FC	R	The value of this register is derived from the DMAH_CH2_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
13	CH2_HC_LLP	R	The value of this register is derived from the DMAH_CH2_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
12	CH2_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH2_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
11	CH2_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH2_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to . 0x0 = FALSE 0x1 = TRUE
10	CH2_LOCK_EN	R	The value of this register is derived from the DMAH_CH2_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = FALSE 0x1 = TRUE
9	CH2_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH2_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
8	CH2_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH2_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
7	CH2_STAT_SRC	R	The value of this register is derived from the DMAH_CH2_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
6	CH2_STAT_DST	R	The value of this register is derived from the DMAH_CH2_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
5:3	CH2_STW	R	The value of this register is derived from the DMAH_CH2_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
2:0	CH2_DTW	R	The value of this register is derived from the DMAH_CH2_DTW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

6.2.5.7 DMA_COMP_PARAMS_2

- Name: DW_ahb_dmac Component Parameters Register 2
- Size: 64 bits
- Address Offset: 0x3e8
- Read/Write Access: Read

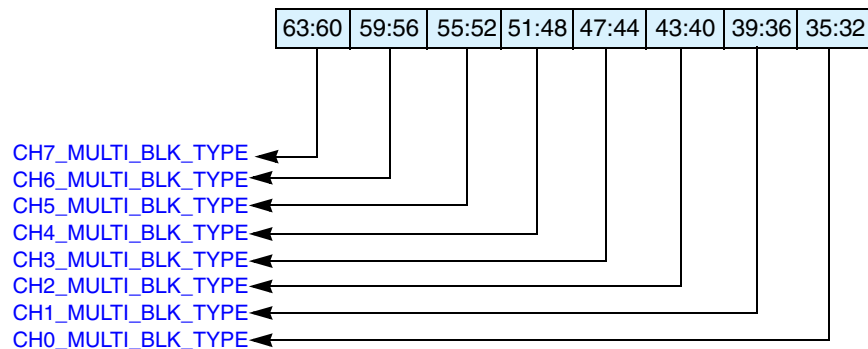
This is a constant read-only register that contains encoded information about the component parameter settings. The reset value depends on coreConsultant parameter(s).



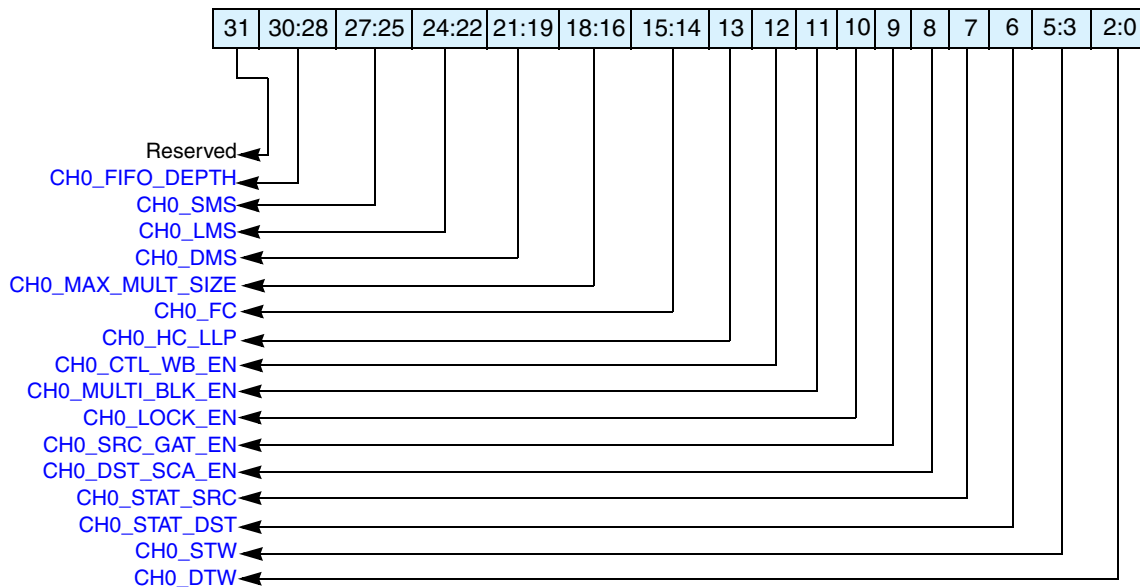
Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to “[Illegal Register Access](#)” on page 246.

Bits 63:32 include the Multi-Block type parameter for Channel 0 to Channel 7.



Bits 31:0 include the configuration parameters for Channel 0.



Bits	Name	R/W	Description
63:60	CH7_MULTI_BLK_TYPE	R	The values of these bit fields are derived from the DMAH_CHx_MULTI_BLK_TYPE coreConsultant parameter. For a description of these parameters, refer to page 131 . 0x0 = NO_HARDCODE 0x1 = CONT_RELOAD 0x2 = RELOAD_CONT 0x3 = RELOAD_RELOAD 0x4 = CONT_LL 0x5 = RELOAD_LL 0x6 = LLP_CONT 0x7 = LLP_RELOAD 0x8 = LLP_LL
59:56	CH6_MULTI_BLK_TYPE	R	
55:52	CH5_MULTI_BLK_TYPE	R	
51:48	CH4_MULTI_BLK_TYPE	R	
47:44	CH3_MULTI_BLK_TYPE	R	
43:40	CH2_MULTI_BLK_TYPE	R	
39:36	CH1_MULTI_BLK_TYPE	R	
35:32	CH0_MULTI_BLK_TYPE	R	
31	Reserved	N/A	Reserved
30:28	CH0_FIFO_DEPTH	R	The value of this register is derived from the DMAH_CH0_FIFO_DEPTH coreConsultant parameter. For a description of this parameter, refer to page 127 . 0x0 = 8 0x1 = 16 0x2 = 32 0x3 = 64 0x4 = 128
27:25	CH0_SMS	R	The value of this register is derived from the DMAH_CH0_SMS coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
24:22	CH0_LMS	R	The value of this register is derived from the DMAH_CH0_LMS coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE

Bits	Name	R/W	Description
21:19	CH0_DMS	R	The value of this register is derived from the DMAH_CH0_DMS coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = MASTER_1 0x1 = MASTER_2 0x2 = MASTER_3 0x3 = MASTER_4 0x4 = NO_HARDCODE
18:16	CH0_MAX_MULT_SIZE	R	The value of this register is derived from the DMAH_CH0_MULT_SIZE coreConsultant parameter. For a description of this parameter, refer to page 127. 0x0 = 4 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved
15:14	CH0_FC	R	The value of this register is derived from the DMAH_CH0_FC coreConsultant parameter. For a description of this parameter, refer to page 128. 0x0 = DMA 0x1 = SRC 0x2 = DST 0x3 = ANY
13	CH0_HC_LLP	R	The value of this register is derived from the DMAH_CH0_HC_LLP coreConsultant parameter. For a description of this parameter, refer to page 131. 0x0 = FALSE 0x1 = TRUE
12	CH0_CTL_WB_EN	R	The value of this register is derived from the DMAH_CH0_CTL_WB_EN coreConsultant parameter. For a description of this parameter, refer to page 133. 0x0 = FALSE 0x1 = TRUE
11	CH0_MULTI_BLK_EN	R	The value of this register is derived from the DMAH_CH0_MULTI_BLK_EN coreConsultant parameter. For a description of this parameter, refer to page 130. 0x0 = FALSE 0x1 = TRUE

Bits	Name	R/W	Description
10	CH0_LOCK_EN	R	The value of this register is derived from the DMAH_CH0_LOCK_EN coreConsultant parameter. For a description of this parameter, refer to page 128 . 0x0 = FALSE 0x1 = TRUE
9	CH0_SRC_GAT_EN	R	The value of this register is derived from the DMAH_CH0_SRC_GAT_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
8	CH0_DST_SCA_EN	R	The value of this register is derived from the DMAH_CH0_DST_SCA_EN coreConsultant parameter. For a description of this parameter, refer to page 133 . 0x0 = FALSE 0x1 = TRUE
7	CH0_STAT_SRC	R	The value of this register is derived from the DMAH_CH0_STAT_SRC coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
6	CH0_STAT_DST	R	The value of this register is derived from the DMAH_CH0_STAT_DST coreConsultant parameter. For a description of this parameter, refer to page 132 . 0x0 = FALSE 0x1 = TRUE
5:3	CH0_STW	R	The value of this register is derived from the DMAH_CH0_STW coreConsultant parameter. For a description of this parameter, refer to page 129 . 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

Bits	Name	R/W	Description
2:0	CH0_DTW	R	The value of this register is derived from the DMAH_CH0_DTW coreConsultant parameter. For a description of this parameter, refer to page 129. 0x0 = NO_HARDCODE 0x1 = 8 0x2 = 16 0x3 = 32 0x4 = 64 0x5 = 128 0x6 = 256 0x7 = reserved

6.2.5.8 DMA_COMP_PARAMS_1

- Name: DW_ahb_dmac Component Parameters Register 1
- Size: 64 bits
- Address Offset: 0x3f0
- Read/Write Access: Read

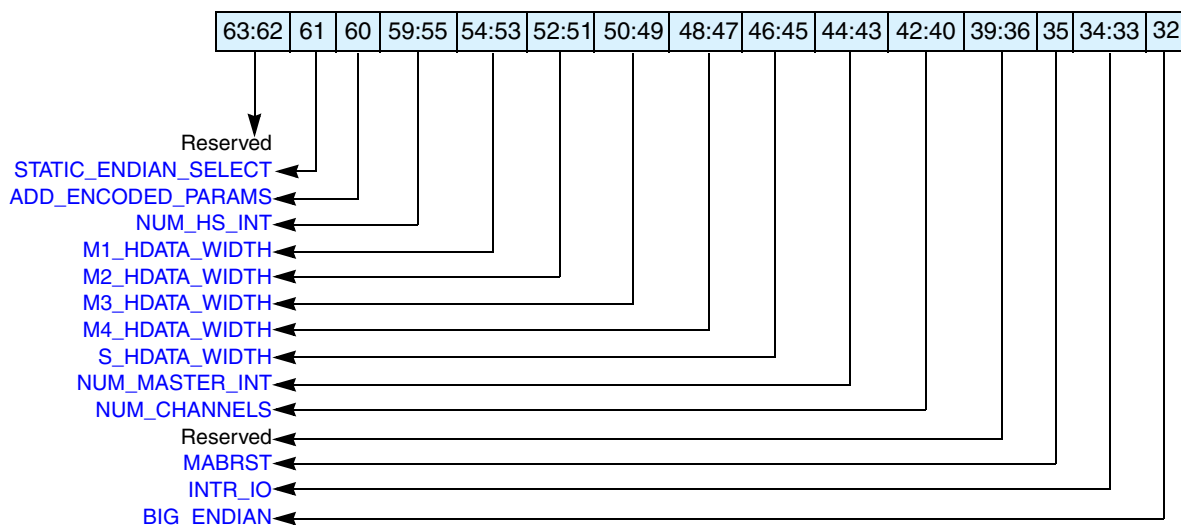
This is a constant read-only register that contains encoded information about the component parameter settings. The reset value depends on coreConsultant parameter(s).



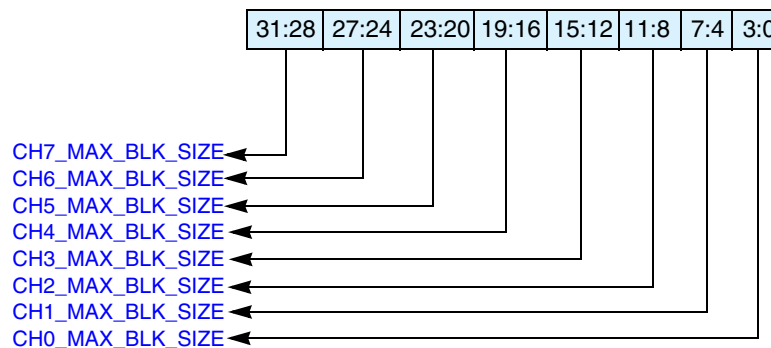
Note

If DMAH_RETURN_ERR_RESP is set to True, the DW_ahb_dmac returns an ERROR response to an illegal register access, which includes accessing registers that have been removed during DW_ahb_dmac configuration. If DMAH_RETURN_ERR_RESP is set to False, DW_ahb_dmac always returns an OK response. For more information, refer to [“Illegal Register Access”](#) on page 246.

Bits 63:32 include the top-level parameters for the DW_ahb_dmac.



Bits 31:0 include maximum block size parameters for Channel 0 to Channel 7.



Bits	Name	R/W	Description
63:62	Reserved	N/A	Reserved
61	STATIC_ENDIAN_SELECT	R	The value of this register is derived from the DMAH_STATIC_ENDIAN_SELECT coreConsultant parameter. For a description of this parameter, refer to page 123 . 0 = FALSE 1 = TRUE
60	ADD_ENCODED_PARAMS	R	The value of this register is derived from the DMAH_ADD_ENCODED_PARAMS coreConsultant parameter. For a description of this parameter, refer to page 122 . 0 = FALSE 1 = TRUE
59:55	NUM_HS_INT	R	The value of this register is derived from the DMAH_NUM_HS_INT coreConsultant parameter. For a description of this parameter, refer to page 120 . 0x00 = 0 to 0x10 = 16
54:53	M1_HDATA_WIDTH	R	The value of this register is derived from the DMAH_M1_HDATA_WIDTH coreConsultant parameter. For a description of this parameter, refer to page 126 . 0x0 = 32 bits 0x1 = 64 bits 0x2 = 128 bits 0x3 = 256 bits
52:51	M2_HDATA_WIDTH	R	The value of this register is derived from the DMAH_M2_HDATA_WIDTH coreConsultant parameter. For a description of this parameter, refer to page 126 . 0x0 = 32 bits 0x1 = 64 bits 0x2 = 128 bits 0x3 = 256 bits
50:49	M3_HDATA_WIDTH	R	The value of this register is derived from the DMAH_M3_HDATA_WIDTH coreConsultant parameter. For a description of this parameter, refer to page 126 . 0x0 = 32 bits 0x1 = 64 bits 0x2 = 128 bits 0x3 = 256 bits

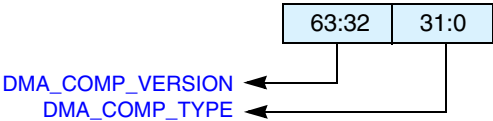
Bits	Name	R/W	Description
48:47	M4_HDATA_WIDTH	R	The value of this register is derived from the DMAH_M4_HDATA_WIDTH coreConsultant parameter. For a description of this parameter, refer to page 126 . 0x0 = 32 bits 0x1 = 64 bits 0x2 = 128 bits 0x3 = 256 bits
46:45	S_HDATA_WIDTH	R	The value of this register is derived from the DMAH_S_HDATA_WIDTH coreConsultant parameter. For a description of this parameter, refer to page 125 . 0x0 = 32 bits 0x1 = 64 bits 0x2 = 128 bits 0x3 = 256 bits
44:43	NUM_MASTER_INT	R	The value of this register is derived from the DMAH_NUM_MASTER_INT coreConsultant parameter. For a description of this parameter, refer to page 120 . 0x0 = 1 to 0x3 = 4
42:40	NUM_CHANNELS	R	The value of this register is derived from the DMAH_NUM_CHANNELS coreConsultant parameter. For a description of this parameter, refer to page 120 . 0x0 = 1 to 0x7 = 8
39:36	Reserved	N/A	Reserved
35	MABRST	R	The value of this register is derived from the DMAH_MABRST coreConsultant parameter. For a description of this parameter, refer to page 121 . 0 = FALSE 1 = TRUE
34:33	INTR_IO	R	The value of this register is derived from the DMAH_INTR_IO coreConsultant parameter. For a description of this parameter, refer to page 121 . 0x0 = ALL 0x1 = TYPE 0x2 = COMBINED 0x3 = reserved
32	BIG_ENDIAN	R	The value of this register is derived from the DMAH_BIG_ENDIAN coreConsultant parameter. For a description of this parameter, refer to page 124 . 0 = FALSE 1 = TRUE

Bits	Name	R/W	Description
31:28	CH7_MAX_BLK_SIZE	R	The values of these bit fields are derived from the DMAH_CHx_MAX_BLK_SIZE coreConsultant parameter. For a description of these parameters, refer to page 128 .
27:24	CH6_MAX_BLK_SIZE	R	
23:20	CH5_MAX_BLK_SIZE	R	
19:16	CH4_MAX_BLK_SIZE	R	
15:12	CH3_MAX_BLK_SIZE	R	
11:8	CH2_MAX_BLK_SIZE	R	
7:4	CH1_MAX_BLK_SIZE	R	
3:0	CH0_MAX_BLK_SIZE	R	
			0x0 = 3
			0x1 = 7
			0x2 = 15
			0x3 = 31
			0x4 = 63
			0x5 = 127
			0x6 = 255
			0x7 = 511
			0x8 = 1023
			0x9 = 2047
			0xa = 4095

6.2.5.9 DMA Component ID Register

- Name: DMA Component ID Register
- Size: 64 bits
- Address Offset: 0x3f8
- Read/Write Access: Read

This is the DW_ahb_dmac Component Version register, which is a read-only register that specifies the version of the packaged component in the upper 32 bits and the component type in the lower 32 bits.



Bits	Name	R/W	Reset	Description
63:32	DMA_COMP_VERSION	R	See DW_ahb_dmac releases table in “ AMBA 2 Release Notes ”	Version of the component.
31:0	DMA_COMP_TYPE	R	0x44_57_11_10	Designware Component Type number = 0x44_57_11_10. This assigned unique hex value is constant and is derived from the two ASCII letters “DW” followed by a 32-bit unsigned number.

7

Internal Parameter Descriptions

Provides a description of the internal parameters that might be indirectly referenced in expressions in the Signals, Parameters, or Registers chapters. These parameters are not visible in the coreConsultant GUI and most of them are derived automatically from visible parameters. **You must not set any of these parameters directly.**

Some expressions might refer to TCL functions or procedures (sometimes identified as **function_of**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the core in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Table 7-1 Internal Parameters

Parameter Name	Equals To
DMAH_HADDR_WIDTH	32
DMAH_NUM_HS_INT_NZ	{{function_of: DMAH_NUM_HS_INT }}
blank	intentionally left blank

Programming the DW_ahb_dmac

The DW_ahb_dmac can be programmed through software registers or the DW_ahb_dmac low-level software driver; software registers are described in more detail in [“Registers”](#) on page 147.

**Note**

There are references to both software and hardware parameters throughout this chapter. The software parameters are the field names in each register description table and are prefixed by the register name; for example, the Block Transfer Size field in the Control Register for Channel x is designated as “CTLx.BLOCK_TS.”

The hardware parameters are prefixed with an DMAH_* and are configured once using Synopsys coreConsultant.

Shipped with the DW_ahb_dmac component is an address definition (memory map) C header file. This can be used when the DW_ahb_dmac is programmed in a C environment.

8.1 Software Drivers

The family of DesignWare Synthesizable Components includes a Driver Kit for the DW_ahb_dmac component. This low-level Driver Kit allows you to program a DW_ahb_dmac component and integrate your code into a larger software system. The Driver Kit provides the following benefits to IP designers:

- Proven method of access to DW_ahb_dmac minimizing usage errors
- Rapid software development with minimum overhead
- Detailed knowledge of DW_ahb_dmac register bit fields not required
- Easy integration of DW_ahb_dmac into existing software system
- Programming at register level eliminated

You must purchase a source code license (DWC-DMA-Controller-Source) to use the DW_ahb_dmac Driver Kit. However, you can access some Driver Kit files and documentation in \$DESIGNWARE_HOME/drivers/DW_ahb_dmac/latest. For more information about the Driver Kit, refer to the [DW_ahb_dmac Driver Kit User Guide](#). For more information about purchasing the source code license and obtaining a download of the Driver Kit, contact Synopsys at designware@synopsys.com for details.

8.2 Register Access

All registers are aligned to a 64-bit boundary and are 64 bits wide. In general, the upper 32 bits of a register are reserved. A write to reserved bits within the register is ignored. A read from reserved bits in the register reads back 0. To avoid address aliasing, do one of the following:

1. The DW_ahb_dmac should not be allocated more than 1 KB of address space in the system memory map. If it is, then addresses selected above 1 KB from the base address are aliased to an address within the 1 KB space, and a transfer takes place involving this register.
2. Software should not attempt to access non-register locations when hsel is asserted.



Note

The hsel signal is asserted by the system decoder when the address on the bus is within the system address assigned for DW_ahb_dmac.

8.3 Illegal Register Access

An illegal access can be any of the following:

1. A AHB transfer of hsize greater than 64 is attempted.
2. The hsel signal is asserted, but the address does not decode to a valid address.
3. A write to the [SARx](#), [DARx](#), [LLPx](#), [CTLx](#), [SSTATx](#), [DSTATx](#), [SSTATARx](#), [DSTATARx](#), [SGRx](#), or [DSRx](#) registers occurs when the channel is enabled.
4. A read from the [ClearBlock](#), [ClearDstTran](#), [ClearErr](#), [ClearSrcTran](#), [ClearTfr](#) is attempted.
5. A write to the [StatusBlock](#), [StatusDstTran](#), [StatusErr](#), [StatusSrcTran](#), [StatusTfr](#) is attempted.
6. A write to the [StatusInt](#) register is attempted.
7. A write to either the [DmaIdReg](#) or [DMA Component ID Register](#) register is attempted.

The response to an illegal access is configured using the configuration parameter DMAH_RETURN_ERR_RESP. When DMAH_RETURN_ERR_RESP is set to True, an illegal access (read/write) returns an error response.

If DMAH_RETURN_ERR_RESP is set to False, an OKAY response is returned, a read reads back 0x0, and a write is ignored.

8.4 DW_ahb_dmac Transfer Types

A DMA transfer may consist of single or multi-block transfers. On successive blocks of a multi-block transfer, the SARx/DARx register in the DW_ahb_dmac is reprogrammed using either of the following methods:

- Block chaining using linked lists
- Auto-reloading
- Contiguous address between blocks

On successive blocks of a multi-block transfer, the CTL_x register in the DW_ahb_dmac is reprogrammed using either of the following methods:

- Block chaining using linked lists
- Auto-reloading

When block chaining, using Linked Lists is the multi-block method of choice. On successive blocks, the LLP_x register in the DW_ahb_dmac is reprogrammed using block chaining with linked lists.

A block descriptor consists of six registers: SAR_x, DAR_x, LLP_x, CTL_x, SSTAT_x, and DSTAT_x. The first four registers, along with the CFG_x register, are used by the DW_ahb_dmac to set up and describe the block transfer.

**Note**

The term Link List Item (LLI) and block descriptor are synonymous.

8.4.1 Multi-Block Transfers

Multi-block transfers are enabled by setting the DMAH_CHX_MULTI_BLK_EN configuration parameter to True.

**Note**

Multi-block transfers—in which the source and destination are swapped during the transfer—are not supported. In a multi-block transfer, the direction must not change for the duration of the transfer.

8.4.1.1 Block Chaining Using Linked Lists

To enable multi-block transfers using block chaining, you must set the configuration parameter DMAH_CHx_MULTI_BLK_EN to True and the DMAH_CHx_HC_LLQ parameter to False.

In this case, the DW_ahb_dmac reprograms the channel registers prior to the start of each block by fetching the block descriptor for that block from system memory. This is known as an LLI update.

DW_ahb_dmac block chaining uses a Linked List Pointer register (LLP_x) that stores the address in memory of the next linked list item. Each LLI contains the corresponding block descriptors:

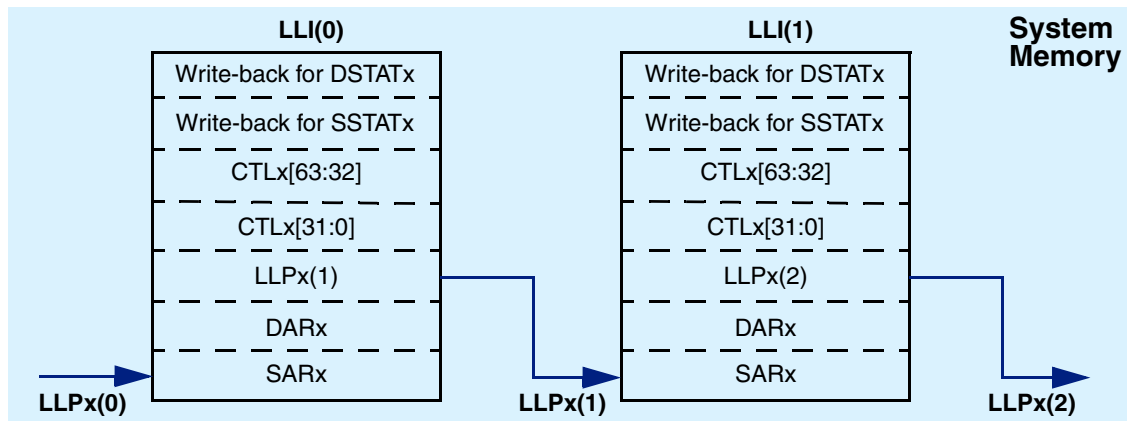
1. SAR_x
2. DAR_x
3. LLP_x
4. CTL_x
5. SSTAT_x
6. DSTAT_x

To set up block chaining, you program a sequence of Linked Lists in memory.

LLI accesses are always 32-bit accesses (Hsize = 2) aligned to 32-bit boundaries and cannot be changed or programmed to anything other than 32-bit, even if the AHB master interface of the LLI supports more than a 32-bit data width.

The SAR_x , DAR_x , LLP_x , and CTL_x registers are fetched from system memory on an LLI update. If configuration parameter $DMAH_CHx_CTL_WB_EN = \text{True}$, then the updated contents of the CTL_x , $SSTAT_x$, and $DSTAT_x$ registers are written back to memory on block completion. Figure 8-1 and Figure 8-2 show how you use chained linked lists in memory to define multi-block transfers using block chaining.

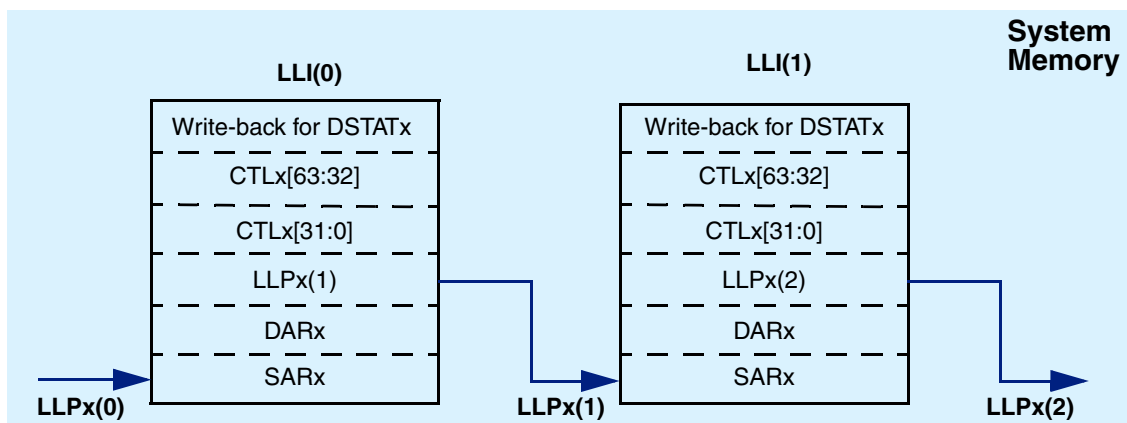
Figure 8-1 Multi-Block Transfer Using Linked Lists When $DMAH_CHx_STAT_SRC$ Set to True



It is assumed that no allocation is made in system memory for the source status when the configuration parameter $DMAH_CHx_STAT_SRC$ is set to False. If this parameter is False, then the order of a Linked List item is as follows:

1. SAR_x
2. DAR_x
3. LLP_x
4. CTL_x
5. $DSTAT_x$

Figure 8-2 Multi-Block Transfer Using Linked Lists When $DMAH_CHx_STAT_SRC$ Set to False



**Note**

In order to not confuse the SARx, DARx, LLPx, CTLx, STATx, and DSTATx register locations of the LLI with the corresponding DW_ahb_dmac memory mapped register locations, the LLI register locations are prefixed with LLI; that is, LLI.SARx, LLI.DARx, LLI.LLPx, LLI.CTLx, LLI.SSTATx, and LLI.DSTATx.

Figure 8-3 on page 250 and Figure 8-4 on page 251 show the mapping of a Linked List Item stored in memory to the channel registers block descriptor.

Rows 6 through 10 of Table 8-1 on page 249 show the required values of LLPx, CTLx, and CFGx for multi-block DMA transfers using block chaining.

**Note**

For rows 6 through 10 of Table 8-1, the LLI.CTLx, LLI.LLPx, LLI.SARx, and LLI.DARx register locations of the LLI are always affected at the start of every block transfer. The LLI.LLPx and LLI.CTLx locations are always used to reprogram the DW_ahb_dmac LLPx and CTLx registers. However, depending on the Table 8-1 row number, the LLI.SARx/LLI.DARx address may or may not be used to reprogram the DW_ahb_dmac SARx/DARx registers.

Table 8-1 Programming of Transfer Types and Channel Register Update Method

Transfer Type	LLP. LOC = 0	LLP_ SRC_EN (CTLx)	RELOAD SRC (CFGx)	LLP_ DST_EN (CTLx)	RELOAD DST (CFGx)	CTLx, LLPx Update Method	SARx Update Method	DARx Update Method	Write Back ^a
1. Single-block or last transfer of multi-block.	Yes	0	0	0	0	None, user reprograms	None (single)	None (single)	No
2. Auto-reload multi-block transfer with contiguous SAR	Yes	0	0	0	1	CTLx, LLPx are reloaded from initial values.	Con-tiguous	Auto-Reload	No
3. Auto-reload multi-block transfer with contiguous DAR.	Yes	0	1	0	0	CTLx, LLPx are reloaded from initial values	Auto-Reload	Con-tiguous	No
4. Auto-reload multi-block transfer	Yes	0	1	0	1	CTLx, LLPx are reloaded from initial values	Auto-reload	Auto-Reload	No
5. Single-block or last transfer of multi-block.	No	0	0	0	0	None, user reprograms	None (single)	None (single)	Yes
6. Linked list multi-block transfer with contiguous SAR	No	0	0	1	0	CTLx, LLPx loaded from next Linked List item.	Con-tiguous	Linked List	Yes

Table 8-1 Programming of Transfer Types and Channel Register Update Method (Continued)

Transfer Type	LLP. LOC = 0	LLP_ SRC_EN (CTLx)	RELOAD SRC (CFGx)	LLP_ DST_EN (CTLx)	RELOAD DST (CFGx)	CTLx, LLPx Update Method	SARx Update Method	DARx Update Method	Write Back ^a
7. Linked list multi-block transfer with auto-reload SAR	No	0	1	1	0	CTLx, LLPx loaded from next Linked List item.	Auto-Reload	Linked List	Yes
8. Linked list multi-block transfer with contiguous DAR	No	1	0	0	0	CTLx, LLPx loaded from next Linked List item.	Linked List	Con-tiguous	Yes
9. Linked list multi-block transfer with auto-reload DAR	No	1	0	0	1	CTLx, LLPx loaded from next Linked List item.	Linked List	Auto-Reload	Yes
10. Linked list multi-block transfer	No	1	0	1	0	CTLx, LLPx loaded from next Linked List item.	Linked List	Linked List	Yes

a. This column assumes that the configuration parameter DMAH_CHx_CTL_WB_EN = True. If DMAH_CHx_CTL_WB_EN = False, then there is never writeback of the control and status registers regardless of transfer type, and all rows of this column are “No”.

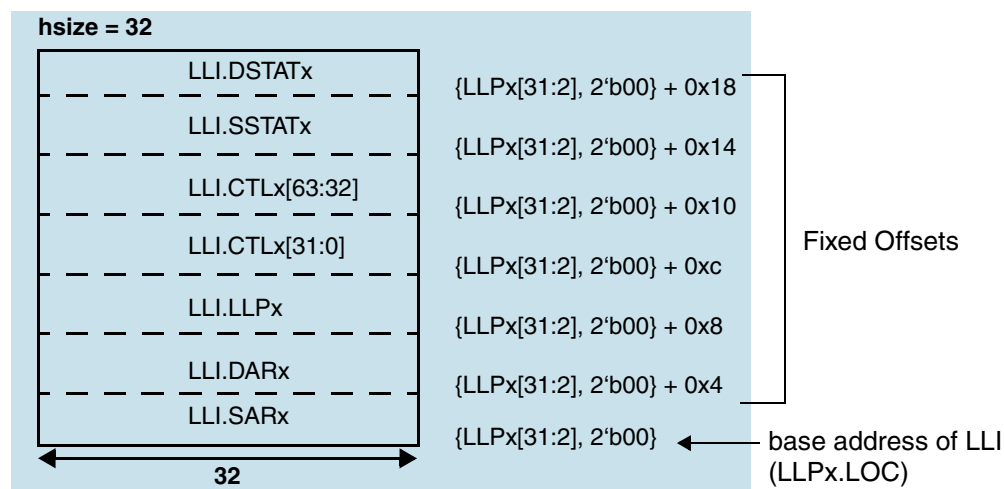
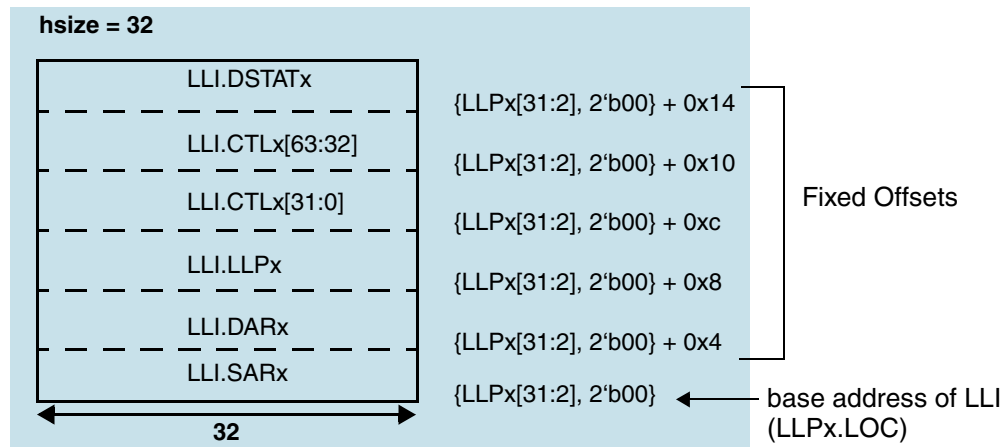
Figure 8-3 Mapping of Block Descriptor (LLI) in Memory to Channel Registers When DMAH_CHx_STAT_SRC Set to True

Figure 8-4 Mapping of Block Descriptor (LLI) in Memory to Channel Registers When DMAH_CHx_STAT_SRC Set to False**Note**

- Throughout this databook, there are descriptions about fetching the LLI.CTLx register from the location pointed to by the LLPx register. This exact location is the LLI base address (stored in LLPx register) plus the fixed offset. For example, in [Figure 8-3](#), the location of the LLI.CTLx register is LLPx.LOC + 0xc.
- Referring to [Table 8-1](#) on page 249, if the Write Back column entry is “Yes” and the configuration parameter DMAH_CHx_CTL_WB_EN = True, then the CTLx[63:32] register is always written to system memory (to LLI.CTLx[63:32]) at the end of every block transfer.

The source status is fetched and written to system memory at the end of every block transfer if the Write Back column entry is “Yes,” DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled.

The destination status is fetched and written to system memory at the end of every block transfer if the Write Back column entry is “Yes,” DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled.

8.4.2 Auto-Reloading of Channel Registers

During auto-reloading, the channel registers are reloaded with their initial values at the completion of each block and the new values used for the new block. Depending on the row number in [Table 8-1](#) on page 249, some or all of the SARx, DARx, and CTLx channel registers are reloaded from their initial value at the start of a block transfer.

8.4.3 Contiguous Address Between Blocks

In this case, the address between successive blocks is selected as a continuation from the end of the previous block.

Enabling the source or destination address to be contiguous between blocks is a function of the CTLx.LLP_SRC_EN, CFGx.RELOAD_SRC, CTLx.LLP_DST_EN, and CTLx.RELOAD_DST registers (see [Table 8-1](#) on page 249).

**Note**

You cannot select both SARx and DARx updates to be contiguous. If you want this functionality, you should increase the size of the Block Transfer (`CTLx.BLOCK_TS`), or if this is at the maximum value, use Row 10 of [Table 8-1](#) on page 249 and set up the LLI.SARx address of the block descriptor to be equal to the end SARx address of the previous block. Similarly, set up the LLI.DARx address of the block descriptor to be equal to the end DARx address of the previous block. For more information, refer to “[Multi-Block Transfer with Linked List for Source and Linked List for Destination \(Row 10\)](#)” on page 259.

8.4.4 Suspension of Transfers Between Blocks

At the end of every block transfer, an end-of-block interrupt is asserted if:

1. Interrupts are enabled, `CTLx.INT_EN = 1`, and
2. The channel block interrupt is unmasked, `MaskBlock[n] = 1`, where n is the channel number.

**Note**

The block-complete interrupt is generated at the completion of the block transfer to the destination.

For rows 6, 8, and 10 of [Table 8-1](#) on page 249, the DMA transfer does not stall between block transfers. For example, at the end-of-block N , the DW_ahb_dmac automatically proceeds to block $N + 1$.

For rows 2, 3, 4, 7, and 9 of [Table 8-1](#) on page 249 (SARx and/or DARx auto-reloaded between block transfers), the DMA transfer automatically stalls after the end-of-block interrupt is asserted, if the end-of-block interrupt is enabled and unmasked.

The DW_ahb_dmac does not proceed to the next block transfer until a write to the `ClearBlock[n]` block interrupt clear register, done by software to clear the channel block-complete interrupt, is detected by hardware.

For rows 2, 3, 4, 7, and 9 of [Table 8-1](#) on page 249 (SARx and/or DARx auto-reloaded between block transfers), the DMA transfer does not stall if either:

- Interrupts are disabled, `CTLx.INT_EN = 0`, or
- The channel block interrupt is masked, `MaskBlock[n] = 0`, where n is the channel number.

Channel suspension between blocks is used to ensure that the end-of-block ISR (interrupt service routine) of the next-to-last block is serviced before the start of the final block commences. This ensures that the ISR has cleared the `CFGx.RELOAD_SRC` and/or `CFGx.RELOAD_DST` bits before completion of the final block. The reload bits `CFGx.RELOAD_SRC` and/or `CFGx.RELOAD_DST` should be cleared in the end-of-block ISR for the next-to-last block transfer.

8.4.5 Ending Multi-Block Transfers

All multi-block transfers must end as shown in either Row 1 or Row 5 of [Table 8-1](#) on page 249. At the end of every block transfer, the DW_ahb_dmac samples the row number, and if the DW_ahb_dmac is in the Row 1 or Row 5 state, then the previous block transferred was the last block and the DMA transfer is terminated.

**Note**

Row 1 and Row 5 are used for single-block transfers or terminating multi-block transfers. Transfers initiated in rows 2, 3 or 4 can only end in row 1; similarly, transfers initiated in rows 6 through 10 can only end in row 5. Ending in the Row 5 state enables status fetch and write-back for the last block. Ending in the Row 1 state disables status fetch and write-back for the last block.

For rows 2, 3, and 4 of [Table 8-1](#) on page 249, (LLPx.LOC = 0 and [CFGx.RELOAD_SRC](#) and/or [CFGx.RELOAD_DST](#) is set), multi-block DMA transfers continue until both the [CFGx.RELOAD_SRC](#) and [CFGx.RELOAD_DST](#) registers are cleared by software. They should be programmed to 0 in the end-of-block interrupt service routine that services the next-to-last block transfer; this puts the DW_ahb_dmac into the Row 1 state.

For rows 6, 8, and 10 of [Table 8-1](#) on page 249 (both [CFGx.RELOAD_SRC](#) and [CFGx.RELOAD_DST](#) cleared), the user must set up the last block descriptor in memory so that both LLI.CTLx.LLP_SRC_EN and LLI.CTLx.LLP_DST_EN are 0.

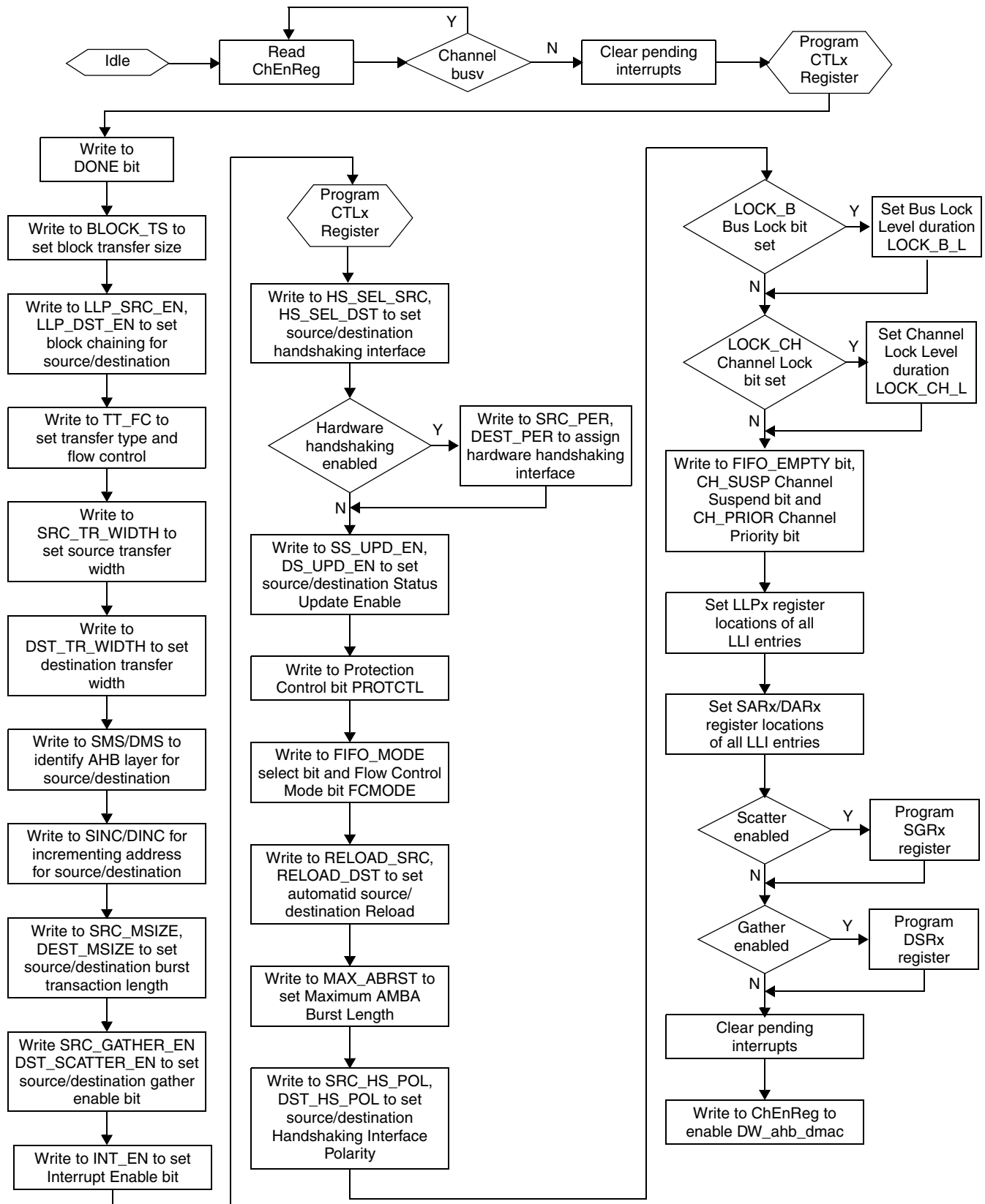
The sampling of the LLPx.LOC bit takes place exclusively at the beginning of the transfer when the channel is enabled. This determines whether writeback is enabled throughout the complete transfer, and changing the value of this bit in subsequent blocks on the same transfer does not have any effect.

**Note**

The only allowed transitions between the rows of [Table 8-1](#) on page 249 are from any row into Row 1 or Row 5. As already stated, a transition into row 1 or row 5 is used to terminate the DMA transfer; all other transitions between rows are not allowed. Software must ensure that illegal transitions between rows do not occur between blocks of a multi-block transfer. For example, if block *N* is in row 10, then the only allowed rows for block *N* + 1 are rows 10 or 5.

8.5 Programing Examples

The flow diagram in [Figure 8-5](#) on page 254 shows an overview of programming the DMA described in “[Programming Example for Linked List Multi-Block Transfer](#)” on page 255.

Figure 8-5 Flowchart for DMA Programming Example

8.5.1 Programming Example for Linked List Multi-Block Transfer

This section explains the step-by-step programming of the DW_ahb_dmac. The example demonstrates row 10 of [Table 8-1](#) on page 249 for Multi-Block Transfer with Linked List for Source and Linked List for Destination. This example uses the DW_ahb_dmac to move four blocks of contiguous data from source to destination memory using the Linked List feature.

1. Set up the chain of Linked List items – otherwise known as block descriptors – in memory. Write the control information in the LLI.CTLx register location of the block descriptor for each LLI in memory for Channel 1. In the LLI.CTLx register, the following is programmed:
 - a. Set up the transfer type for a memory-to-memory transfer:
 - `ctlx[22:20] = 3'b000;`
 - b. Set up the transfer characteristics:
 - i. Transfer width for the source in the SRC_TR_WIDTH field
 - `ctlx[6:4] = 3'b001;`
 - ii. Transfer width for the destination in the DST_TR_WIDTH field
 - `ctlx[3:1] = 3'b001;`
 - iii. Source master layer in the SMS field where the source resides
 - `ctlx[26:25] = 2'b00;`
 - iv. Destination master layer in the DMS field where the destination resides
 - `ctlx[24:23] = 2'b00;`
 - v. Incrementing address for the source in the SINC field
 - `ctlx[10:9] = 2'b00;`
 - vi. Incrementing address for the destination in the DINC field
 - `ctlx[8:7] = 2'b00;`
2. Write the channel configuration information into the CFGx register for Channel 1:
 - a. HS_SEL_SRC/HS_SEL_DST bits select which of the handshaking interfaces – hardware or software – is active for source requests on this channel.
 - `cfgx[11] = 1'b0;`
 - `cfgx[10] = 1'b0;`

These settings are ignored because both the source and destination are memory types.
 - b. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripheral by programming the SRC_PER and DEST_PER bits:
 - `cfgx[46:43] = 1'b0;`
 - `cfgx[42:39] = 1'b0;`

These settings are ignored because both the source and destination are memory types.
3. The following For loop, shown as a programming example, sets the following:
 - LLI.LLPx register locations of all LLI entries in memory (except the last) to non-zero and point to the base address of the next Linked List Item
 - LLI.SARx/LLI.DARx register locations of all LLI entries in memory point to the start source/destination block address preceding that LLI fetch

The For statement below configures the LLPx entries:

```
for(i=0 ; i < 4 ; i=i+1) begin
    if (i == 3)
        llpx = 0;    // end of LLI
    else
        llpx = llp_addr + 20; // start of next LLI

    //--: Program SAR
    `AHB_MASTER.write(0, llp_addr, sarx, AhbWord32Attrb, handle[0]);
    //--: Program DAR
    `AHB_MASTER.write(0, (llp_addr + 4), darx, AhbWord32Attrb, handle[0]);

    //--: Program LLP
    `AHB_MASTER.write(0, (llp_addr + 8), llpx, AhbWord32Attrb, handle[0]);

    //--: Program CTL
    `AHB_MASTER.write(0, (llp_addr + 12), ctlx[31:0], AhbWord32Attrb, handle[0]);
    `AHB_MASTER.write(0, (llp_addr + 16), ctlx[63:32], AhbWord32Attrb, handle[0]);

    // update pointers
    llp_addr = llp_addr + 20; // start of next LLI

    // 4 16-bit words each with scatter/gather interval in each block
    // ( will work only with scatter_gather count of 2)
    sarx = sarx + 24;
    darx = darx + 24;

end
```

4. If Gather is enabled – DMAH_CHx_SRC_GAT_EN = True and CTLx.SRC_GATHER_EN is enabled – program the SGRx register for Channel 1.
5. If Scatter is enabled – DMAH_CHx_DST_SCA_EN = True and CTLx.DST_SCATTER_EN is enabled – program the DSRx register for Channel 1.
6. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers.
7. Finally, enable the channel by writing a 1 to the ChEnReg.CH_EN bit; the transfer is performed.

8.6 Programming a Channel

Three registers – **LLPx**, **CTLx**, and **CFGx** – need to be programmed to determine whether single- or multi-block transfers occur, and which type of multi-block transfer is used. The different transfer types are shown in [Table 8-1](#) on page [249](#).

The DW_ahb_dmac can be programmed to fetch the status from the source or destination peripheral; this status is stored in the **SSTATx** and **DSTATx** registers. When the DW_ahb_dmac is programmed to fetch the status from the source or destination peripheral, it writes this status and the contents of the CTLx register back to memory at the end of a block transfer. The Write Back column of [Table 8-1](#) on page [249](#) shows when this occurs.

The “Update Method” columns indicate where the values of **SARx**, **DARx**, **CTLx**, and **LLPx** are obtained for the next block transfer when multi-block DW_ahb_dmac transfers are enabled.

**Note**

In [Table 8-1](#), all other combinations of `LLPx.LOC = 0`, `CTLx.LLP_SRC_EN`, `CFGx.RELOAD_SRC`, `CTLx.LLP_DST_EN`, and `CFGx.RELOAD_DST` are illegal, and will cause indeterminate or erroneous behavior.

8.6.1 Programming Examples

- “Single-block Transfer (Row 1)” on page 257
- “Multi-Block Transfer with Linked List for Source and Linked List for Destination (Row 10)” on page 259
- “Multi-Block Transfer with Source Address Auto-Reloaded and Destination Address Auto-Reloaded (Row 4)” on page 264
- “Multi-Block Transfer with Source Address Auto-Reloaded and Linked List Destination Address (Row 7)” on page 268
- “Multi-Block Transfer with Source Address Auto-Reloaded and Contiguous Destination Address (Row 3)” on page 273
- “Multi-Block DMA Transfer with Linked List for Source and Contiguous Destination Address (Row 8)” on page 277

8.6.1.1 Single-block Transfer (Row 1)

This section describes a single-block transfer, Row 1 in [Table 8-1](#) on page 249.

**Note**

Row 5 in [Table 8-1](#) on page 249 is also a single-block transfer with write-back of control and status information enabled at the end of the single-block transfer.

1. Read the Channel Enable register to choose a free (disabled) channel; refer to “[ChEnReg](#)” on page 162.
2. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: `ClearTfr`, `ClearBlock`, `ClearSrcTran`, `ClearDstTran`, and `ClearErr` ([page 196](#)). Reading the Interrupt Raw Status and Interrupt Status registers confirms that all interrupts have been cleared.
3. Program the following channel registers:
 - a. Write the starting source address in the `SARx` register for channel x (see [page 164](#)).
 - b. Write the starting destination address in the `DARx` register for channel x (see [page 165](#)).
 - c. Program `CTLx` and `CFGx` according to Row 1, as shown in [Table 8-1](#) on page 249. Program the `LLPx` register with 0 (see [page 167](#)).
 - d. Write the control information for the DMA transfer in the `CTLx` register for channel x (see [page 169](#)). For example, in the register, you can program the following:

- i. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control device by programming the TT_FC of the CTL_x register. [Table 6-4](#) on page 175 lists the decoding for this field.
 - ii. Set up the transfer characteristics, such as:
 - ✦ Transfer width for the source in the SRC_TR_WIDTH field. [Table 6-3](#) on page 175 lists the decoding for this field.
 - ✦ Transfer width for the destination in the DST_TR_WIDTH field. [Table 6-3](#) on page 175 lists the decoding for this field.
 - ✦ Source master layer in the SMS field where the source resides.
 - ✦ Destination master layer in the DMS field where the destination resides.
 - ✦ Incrementing/decrementing or fixed address for the source in the SINC field.
 - ✦ Incrementing/decrementing or fixed address for the destination in the DINC field.
 - e. Write the channel configuration information into the CFG_x register for channel *x* (see [page 180](#)).
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.
 This step requires programming the HS_SEL_SRC/HS_SEL_DST bits, respectively. Writing a 0 activates the hardware handshaking interface to handle source/destination requests. Writing a 1 activates the software handshaking interface to handle source and destination requests.
 - ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign a handshaking interface to the source and destination peripheral; this requires programming the SRC_PER and DEST_PER bits, respectively.
 - f. If gather is enabled (parameter DMAH_CH_x_SRC_GAT_EN = True and CTL_x.SRC_GATHER_EN is enabled), program the SGR_x register (refer to [page 186](#)) for channel *x*.
 - g. If scatter is enabled (parameter DMAH_CH_x_DST_SCA_EN = True and CTL_x.DST_SCATTER_EN), program the DSR_x register (refer to [page 188](#)) for channel *x*.
4. Ensure that bit 0 of the DmaCfgReg register is enabled before writing to ChEnReg.
 5. Source and destination request single and burst DMA transactions in order to transfer the block of data (assuming non-memory peripherals). The DW_ahb_dmac acknowledges at the completion of every transaction (burst and single) in the block and carries out the block transfer.
 6. Once the transfer completes, hardware sets the interrupts and disables the channel. At this time, you can respond to either the Block Complete or Transfer Complete interrupts, or poll for the transfer complete raw interrupt status register (RawTfr[*n*], *n* = channel number) until it is set by hardware, in order to detect when the transfer is complete. Note that if this polling is used, the software must ensure that the transfer complete interrupt is cleared by writing to the Interrupt Clear register, ClearTfr[*n*], before the channel is enabled.

8.6.1.2 Multi-Block Transfer with Linked List for Source and Linked List for Destination (Row 10)



Note

This type of multi-block transfer can only be enabled when either of the following parameters is set:

- DMAH_CHx_MULTI_BLK_TYPE = NO_HARDCODE
- or
- DMAH_CHx_MULTI_BLK_TYPE = LLP_LLP

1. Read the Channel Enable register (see “ChEnReg” on page 162) to choose a free (disabled) channel.
2. Set up the chain of Linked List Items (otherwise known as block descriptors) in memory. Write the control information in the LLI.CTLx register location of the block descriptor for each LLI in memory (see Figure 8-1 on page 248) for channel x. For example, in the register, you can program the following:
 - a. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control device by programming the TT_FC of the CTLx register. Table 6-4 on page 175 lists the decoding for this field.
 - b. Set up the transfer characteristics, such as:
 - i. Transfer width for the source in the SRC_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - ii. Transfer width for the destination in the DST_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - iii. Source master layer in the SMS field where the source resides.
 - iv. Destination master layer in the DMS field where the destination resides.
 - v. Incrementing/decrementing or fixed address for the source in the SINC field.
 - vi. Incrementing/decrementing or fixed address for the destination in the DINC field.
3. Write the channel configuration information into the CFGx register for channel x (see page 180).
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.
 This step requires programming the HS_SEL_SRC/HS_SEL_DST bits, respectively. Writing a 0 activates the hardware handshaking interface to handle source/destination requests for the specific channel. Writing a 1 activates the software handshaking interface to handle source/destination requests.
 - ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripheral. This requires programming the SRC_PER and DEST_PER bits, respectively.
4. Make sure that the LLI.CTLx register locations of all LLI entries in memory (except the last) are set as shown in Row 10 of Table 8-1 on page 249. The LLI.CTLx register of the last Linked List Item must be set as described in Row 1 or Row 5 of Table 8-1. Figure 8-1 on page 248 shows a Linked List example with two list items.
5. Make sure that the LLI.LLPx register locations of all LLI entries in memory (except the last) are non-zero and point to the base address of the next Linked List Item.

6. Make sure that the LLI.SAR_x/LLI.DAR_x register locations of all LLI entries in memory point to the start source/destination block address preceding that LLI fetch.
7. If parameter DMAH_CH_x_CTL_WB_EN = True, ensure that the LLI.CTL_x.DONE field of the LLI.CTL_x register locations of all LLI entries in memory is cleared.
8. If source status fetching is enabled (DMAH_CH_x_CTL_WB_EN = True, DMAH_CH_x_STAT_SRC = True, and CFG_x.SS_UPD_EN is enabled), program the SSTATAR_x register (see [page 178](#)) so that the source status information can be fetched from the location pointed to by the SSTATAR_x. For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).
9. If destination status fetching is enabled (DMAH_CH_x_CTL_WB_EN = True, DMAH_CH_x_STAT_DST = True, and CFG_x.DS_UPD_EN is enabled), program the DSTATAR_x register (see [page 179](#)) so that the destination status information can be fetched from the location pointed to by the DSTATAR_x register. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).
10. If gather is enabled (DMAH_CH_x_SRC_GAT_EN = True and CTL_x.SRC_GATHER_EN is enabled), program the SGR_x register (see [page 186](#)) for channel *x*.
11. If scatter is enabled (DMAH_CH_x_DST_SCA_EN = True and CTL_x.DST_SCATTER_EN is enabled) program the DSR_x register (see [page 188](#)) for channel *x*.
12. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, and ClearErr ([page 196](#)). Reading the Interrupt Raw Status and Interrupt Status registers confirms that all interrupts have been cleared.
13. Program the CTL_x and CFG_x registers according to Row 10, as shown in [Table 8-1](#) on [page 249](#).
14. Program the LLP_x register with LLP(0), the pointer to the first linked list item.
15. Finally, enable the channel by writing a 1 to the ChEnReg.CH_EN bit; the transfer is performed.
16. The DW_ahb_dmac fetches the first LLI from the location pointed to by LLP_x(0).

**Note**

The LLI.SAR_x, LLI.DAR_x, LLI.LLP_x, and LLI.CTL_x registers are fetched. The DW_ahb_dmac automatically reprograms the SAR_x, DAR_x, LLP_x, and CTL_x channel registers from the LLP_x(0).

17. Source and destination request single and burst DMA transactions to transfer the block of data (assuming non-memory peripheral). The DW_ahb_dmac acknowledges at the completion of every transaction (burst and single) in the block and carries out the block transfer.
18. Once the block of data is transferred, the source status information is fetched from the location pointed to by the SSTATAR_x register and stored in the SSTAT_x register if DMAH_CH_x_CTL_WB_EN = True, DMAH_CH_x_STAT_SRC = True, and CFG_x.SS_UPD_EN is enabled. For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).

The destination status information is fetched from the location pointed to by the **DSTATAR_x** register and stored in the **DSTAT_x** register if **DMAH_CH_x_CTL_WB_EN** = True, **DMAH_CH_x_STAT_DST** = True, and **CFG_x.DS_UPD_EN** is enabled. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of **Table 8-1** on page 249.

19. If **DMAH_CH_x_CTL_WB_EN** = True, then the **CTL_x[63:32]** register is written out to system memory. For conditions under which the **CTL_x[63:32]** register is written out to system memory, refer to the Write Back column of **Table 8-1** on page 249.

The **CTL_x[63:32]** register is written out to the same location on the same layer (**LLP_x.LMS**) where it was originally fetched; that is, the location of the **CTL_x** register of the linked list item fetched prior to the start of the block transfer. Only the second word of the **CTL_x** register is written out – **CTL_x[63:32]** – because only the **CTL_x.BLOCK_TS** and **CTL_x.DONE** fields have been updated by the DW_ahb_dmac hardware. Additionally, the **CTL_x.DONE** bit is asserted to indicate block completion. Therefore, software can poll the **LLI.CTL_x.DONE** bit of the **CTL_x** register in the **LLI** to ascertain when a block transfer has completed.

**Note**

Do not poll the **CTL_x.DONE** bit in the DW_ahb_dmac memory map; instead, poll the **LLI.CTL_x.DONE** bit in the **LLI** for that block. If the polled **LLI.CTL_x.DONE** bit is asserted, then this block transfer has completed. This **LLI.CTL_x.DONE** bit was cleared at the start of the transfer (Step 7).

20. The **SSTAT_x** register is now written out to system memory if **DMAH_CH_x_CTL_WB_EN** = True, **DMAH_CH_x_STAT_SRC** = True, and **CFG_x.SS_UPD_EN** is enabled. It is written to the **SSTAT_x** register location of the **LLI** pointed to by the previously saved **LLP_x.LOC** register.

The **DSTAT_x** register is now written out to system memory if **DMAH_CH_x_CTL_WB_EN** = True, **DMAH_CH_x_STAT_DST** = True, and **CFG_x.DS_UPD_EN** is enabled. It is written to the **DSTAT_x** register location of the **LLI** pointed to by the previously saved **LLP_x.LOC** register.

The end-of-block interrupt, **int_block**, is generated after the write-back of the control and status registers has completed.

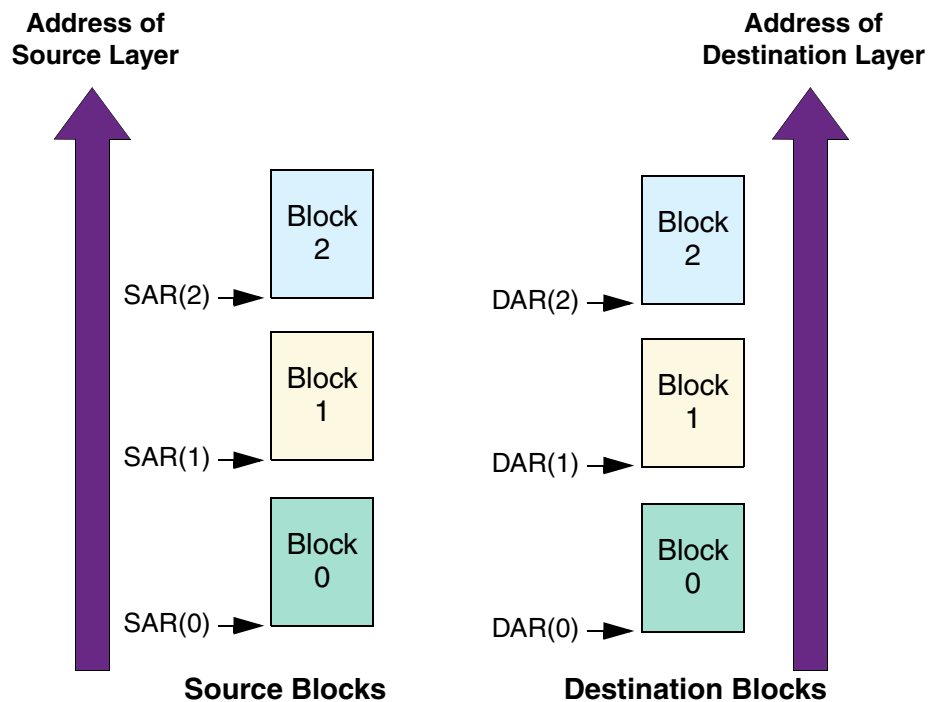
**Note**

The write-back location for the control and status registers is the **LLI** pointed to by the previous value of the **LLP_x.LOC** register, not the **LLI** pointed to by the current value of the **LLP_x.LOC** register.

21. The DW_ahb_dmac does not wait for the block interrupt to be cleared, but continues fetching the next **LLI** from the memory location pointed to by the current **LLP_x** register and automatically reprograms the **SAR_x**, **DAR_x**, **LLP_x**, and **CTL_x** channel registers. The DMA transfer continues until the DW_ahb_dmac determines that the **CTL_x** and **LLP_x** registers at the end of a block transfer match the ones described in Row 1 or Row 5 of **Table 8-1** on page 249 (as discussed earlier). The DW_ahb_dmac then knows that the previously transferred block was the last block in the DMA transfer.

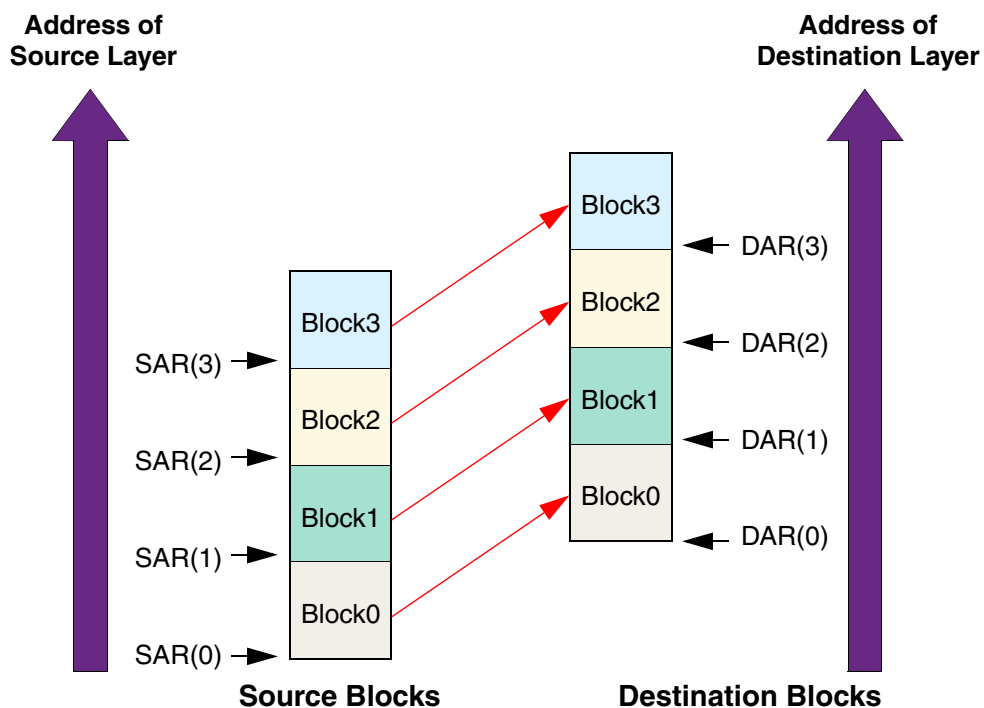
The DMA transfer might look like that shown in [Figure 8-6](#).

Figure 8-6 Multi-Block with Linked Address for Source and Destination



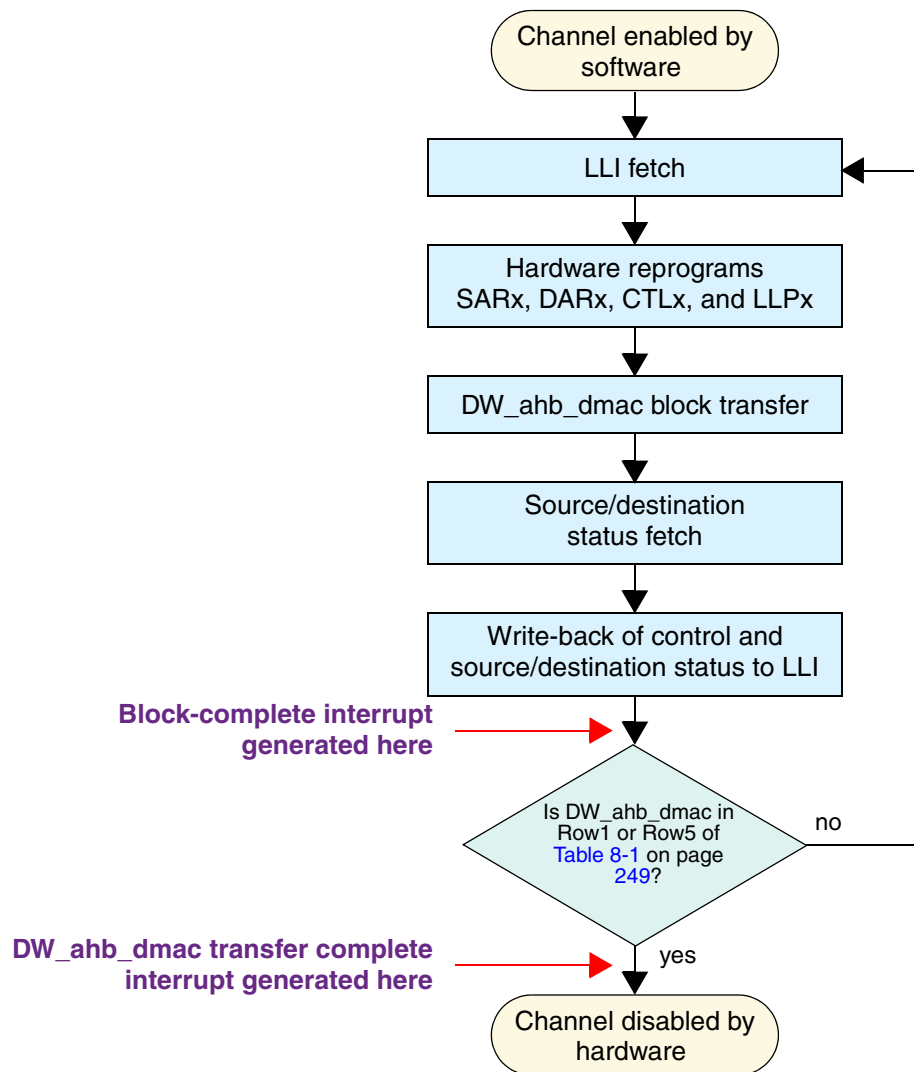
If the user needs to execute a DMA transfer where the source and destination address are contiguous, but where the amount of data to be transferred is greater than the maximum block size $CTLx.BLOCK_TS$, then this can be achieved using the type of multi-block transfer shown in [Figure 8-7](#).

Figure 8-7 Multi-Block with Linked Address for Source and Destination Where SARx and DARx Between Successive Blocks are Contiguous



The DMA transfer flow is shown in [Figure 8-8](#).

Figure 8-8 DMA Transfer Flow for Source and Destination Linked List Address



8.6.1.3 Multi-Block Transfer with Source Address Auto-Reloaded and Destination Address Auto-Reloaded (Row 4)



Note

This type of multi-block transfer can only be enabled when either of the following parameters is set:

- DMAH_CHx_MULTI_BLK_TYPE = NO_HARDCODE
- or
- DMAH_CHx_MULTI_BLK_TYPE = RELOAD_RELOAD

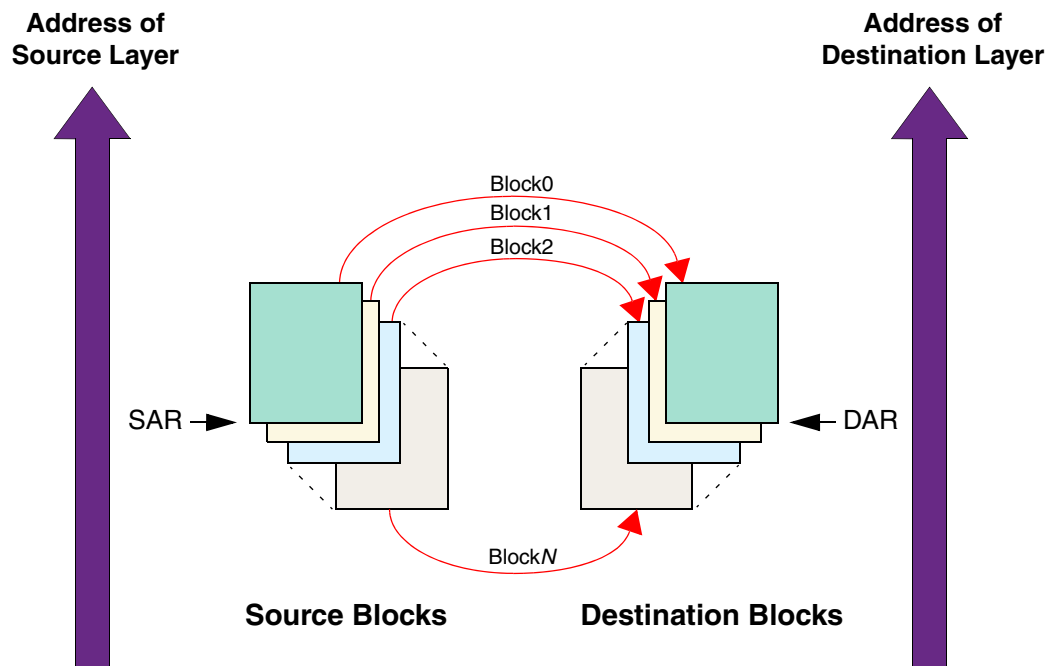
1. Read the Channel Enable register (see “ChEnReg” on page 162) to choose an available (disabled) channel.

2. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, and ClearErr ([page 196](#)). Reading the Interrupt Raw Status and Interrupt Status registers ([page 193](#)) confirms that all interrupts have been cleared.
3. Program the following channel registers:
 - a. Write the starting source address in the [SARx](#) register for channel *x* (see [page 164](#)).
 - b. Write the starting destination address in the [DARx](#) register for channel *x* (see [page 165](#)).
 - c. Program CTLx and CFGx according to Row 4, as shown in [Table 8-1](#) on [page 249](#). Program the LLPx register with 0 (see [page 167](#)).
 - d. Write the control information for the DMA transfer in the CTLx register for channel *x* (see [page 169](#)). For example, in the register, you can program the following:
 - i. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control device by programming the TT_FC of the CTLx register. [Table 6-4](#) on [page 175](#) lists the decoding for this field.
 - ii. Set up the transfer characteristics, such as:
 - ✦ Transfer width for the source in the SRC_TR_WIDTH field; [Table 6-3](#) on [page 175](#) lists the decoding for this field.
 - ✦ Transfer width for the destination in the DST_TR_WIDTH field; [Table 6-3](#) on [page 175](#) lists the decoding for this field.
 - ✦ Source master layer in the SMS field where the source resides.
 - ✦ Destination master layer in the DMS field where the destination resides.
 - ✦ Incrementing/decrementing or fixed address for the source in the SINC field.
 - ✦ Incrementing/decrementing or fixed address for the destination in the DINC field.
 - e. If gather is enabled (DMAH_CHx_SRC_GAT_EN = True and CTLx.SRC_GATHER_EN is enabled), program the SGRx register (see [page 186](#)) for channel *x*.
 - f. If scatter is enabled (DMAH_CHx_DST_SCA_EN = True and CTLx.DST_SCATTER_EN), program the DSRx register (see [page 188](#)) for channel *x*.
 - g. Write the channel configuration information into the CFGx register for channel *x* (see [page 180](#)). Ensure that the reload bits, CFGx.RELOAD_SRC and CFGx.RELOAD_DST, are enabled.
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.
This step requires programming the [HS_SEL_SRC](#)/[HS_SEL_DST](#) bits, respectively. Writing a 0 activates the hardware handshaking interface to handle source/destination requests for the specific channel. Writing a 1 activates the software handshaking interface to handle source/destination requests.
 - ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripheral. This requires programming the [SRC_PER](#) and [DEST_PER](#) bits, respectively.
4. Ensure that bit 0 of the [DmaCfgReg](#) register is enabled before writing to [ChEnReg](#).

5. Source and destination request single and burst DW_ahb_dmac transactions to transfer the block of data (assuming non-memory peripherals). The DW_ahb_dmac acknowledges on completion of each burst/single transaction and carries out the block transfer.
6. When the block transfer has completed, the DW_ahb_dmac reloads the SAR_x, DAR_x, and CTL_x registers. Hardware sets the block-complete interrupt. The DW_ahb_dmac then samples the row number, as shown in [Table 8-1](#) on page 249. If the DW_ahb_dmac is in Row 1, then the DMA transfer has completed. Hardware sets the transfer complete interrupt and disables the channel. You can either respond to the Block Complete or Transfer Complete interrupts, or poll for the transfer complete raw interrupt status register (RawTfr[*n*], where *n* is the channel number) until it is set by hardware, in order to detect when the transfer is complete. Note that if this polling is used, software must ensure that the transfer complete interrupt is cleared by writing to the Interrupt Clear register, ClearTfr[*n*], before the channel is enabled. If the DW_ahb_dmac is not in Row 1, the next step is performed.
7. The DMA transfer proceeds as follows:
 - a. If interrupts are enabled (CTL_x.INT_EN = 1) and the block-complete interrupt is unmasked (MaskBlock[*x*] = 1'b1, where *x* is the channel number), hardware sets the block-complete interrupt when the block transfer has completed. It then stalls until the block-complete interrupt is cleared by software. If the next block is to be the last block in the DMA transfer, then the block-complete ISR (interrupt service routine) should clear the reload bits in the CFG_x.RELOAD_SRC and CFG_x.RELOAD_DST registers. This puts the DW_ahb_dmac into Row 1, as shown in [Table 8-1](#) on page 249. If the next block is not the last block in the DMA transfer, then the reload bits should remain enabled to keep the DW_ahb_dmac in Row 4.
 - b. If interrupts are disabled (CTL_x.INT_EN = 0) or the block-complete interrupt is masked (MaskBlock[*x*] = 1'b0, where *x* is the channel number), then hardware does not stall until it detects a write to the block-complete interrupt clear register; instead, it immediately starts the next block transfer. In this case, software must clear the reload bits in the CFG_x.RELOAD_SRC and CFG_x.RELOAD_DST registers to put the DW_ahb_dmac into Row 1 of [Table 8-1](#) on page 249 before the last block of the DMA transfer has completed.

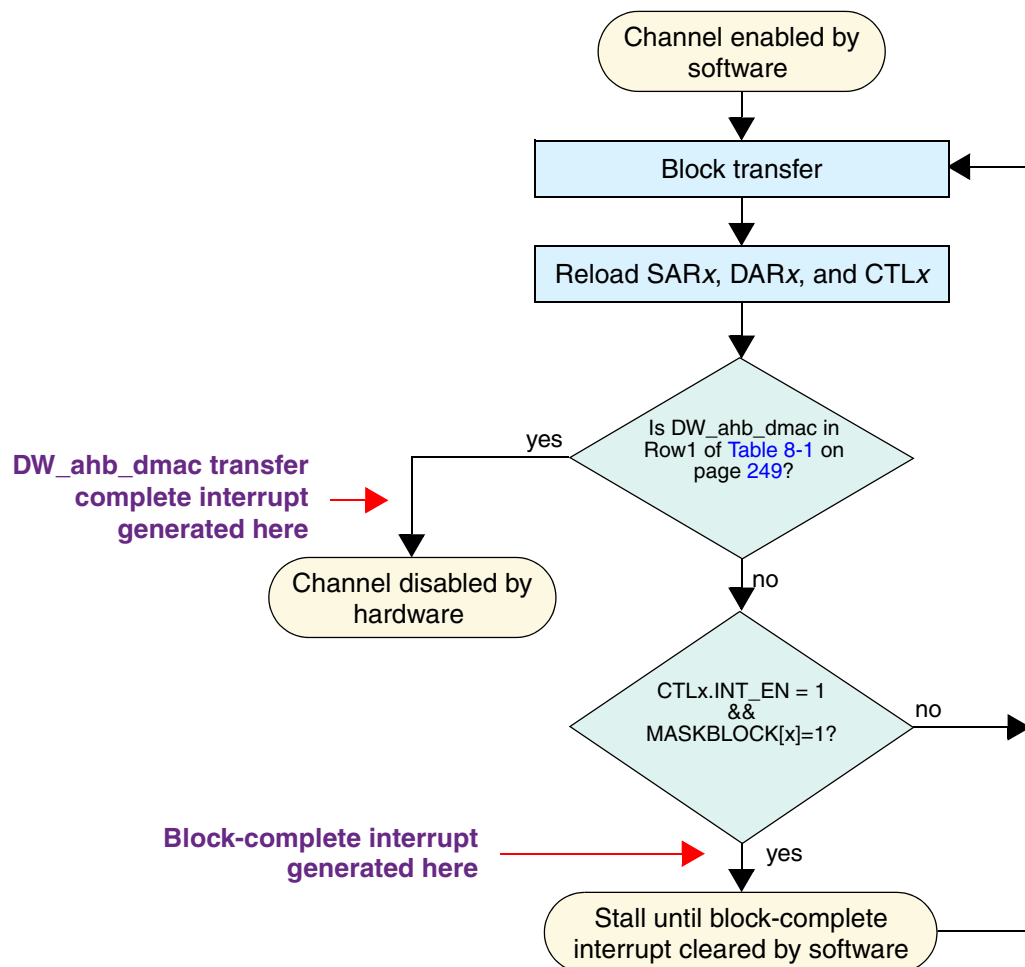
The transfer is similar to that shown in [Figure 8-9](#).

Figure 8-9 Multi-Block DMA Transfer with Source and Destination Address Auto-Reloaded



The DMA transfer flow is shown in [Figure 8-10](#).

Figure 8-10 DMA Transfer Flow for Source and Destination Address Auto-Reloaded



8.6.1.4 Multi-Block Transfer with Source Address Auto-Reloaded and Linked List Destination Address (Row 7)



Note

This type of multi-block transfer can only be enabled when either of the following parameters is set:

- DMAH_CHx_MULTI_BLK_TYPE = 0
- or
- DMAH_CHx_MULTI_BLK_TYPE = RELOAD_LL

1. Read the Channel Enable register (see “ChEnReg” on page 162) in order to choose a free (disabled) channel.
2. Set up the chain of linked list items (otherwise known as block descriptors) in memory. Write the control information in the LLI.CTLx register location of the block descriptor for each LLI in memory

(see [Figure 8-1](#) on page 248) for channel x . For example, in the register you can program the following:

- a. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control peripheral by programming the TT_FC of the CTL x register. [Table 6-4](#) on page 175 lists the decoding for this field.
- b. Set up the transfer characteristics, such as:
 - i. Transfer width for the source in the SRC_TR_WIDTH field. [Table 6-3](#) on page 175 lists the decoding for this field.
 - ii. Transfer width for the destination in the DST_TR_WIDTH field. [Table 6-3](#) on page 175 lists the decoding for this field.
 - iii. Source master layer in the SMS field where the source resides.
 - iv. Destination master layer in the DMS field where the destination resides.
 - v. Incrementing/decrementing or fixed address for the source in the SINC field.
 - vi. Incrementing/decrementing or fixed address for the destination in the DINC field.
3. Write the starting source address in the SAR x register for channel x (see [page 164](#)).

**Note**

The values in the LLI.SAR x register locations of each of the Linked List Items (LLIs) set up in memory, although fetched during an LLI fetch, are not used.

4. Write the channel configuration information into the CFG x register for channel x (see [page 180](#)).
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.
This step requires programming the HS_SEL_SRC/HS_SEL_DST bits. Writing a 0 activates the hardware handshaking interface to handle source/destination requests for the specific channel. Writing a 1 activates the software handshaking interface source/destination requests.
 - ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripheral; this requires programming the SRC_PER and DEST_PER bits, respectively.
5. Make sure that the LLI.CTL x register locations of all LLIs in memory (except the last) are set as shown in Row 7 of [Table 8-1](#) on page 249, while the LLI.CTL x register of the last Linked List item must be set as described in Row 1 or Row 5 of [Table 8-1](#). [Figure 8-1](#) shows a Linked List example with two list items.
6. Ensure that the LLI.LLP x register locations of all LLIs in memory (except the last) are non-zero and point to the next Linked List Item.
7. Ensure that the LLI.DAR x register location of all LLIs in memory point to the start destination block address preceding that LLI fetch.
8. If DMAH_CH x _CTL_WB_EN = True, ensure that the LLI.CTL x .DONE fields of the LLI.CTL x register locations of all LLIs in memory are cleared.

9. If source status fetching is enabled (DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled), program the [SSTATARx](#) register (see [page 178](#)) so that the source status information can be fetched from the location pointed to by the [SSTATARx](#). For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).
10. If destination status fetching is enabled (DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled), program the [DSTATARx](#) register (see [page 179](#)) so that the destination status information can be fetched from the location pointed to by the [DSTATARx](#) register. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).
11. If gather is enabled (DMAH_CHx_SRC_GAT_EN = True and [CTLx.SRC_GATHER_EN](#) is enabled), program the [SGRx](#) register (see [page 186](#)) for channel *x*.
12. If scatter is enabled (DMAH_CHx_DST_SCA_EN = True and [CTLx.DST_SCATTER_EN](#), program the [DSRx](#) register (see [page 188](#)) for channel *x*.
13. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, and ClearErr ([page 196](#)). Reading the Interrupt Raw Status and Interrupt Status registers ([page 193](#)) confirms that all interrupts have been cleared.
14. Program the [CTLx](#) and [CFGx](#) registers according to Row 7, as shown in [Table 8-1](#) on [page 249](#).
15. Program the [LLPx](#) register with LLPx(0), the pointer to the first Linked List item.
16. Ensure that bit 0 of the [DmaCfgReg](#) register is enabled before writing to [ChEnReg](#).
17. The DW_ahb_dmac fetches the first LLI from the location pointed to by LLPx(0).



The LLI.SARx, LLI.DARx, LLI.LLPx, and LLI.CTLx registers are fetched. The LLI.SARx register – although fetched – is not used.

18. Source and destination request single and burst DW_ahb_dmac transactions in order to transfer the block of data (assuming non-memory peripherals). The DW_ahb_dmac acknowledges at the completion of every transaction (burst and single) in the block and carries out the block transfer.
19. Once the block of data is transferred, the source status information is fetched from the location pointed to by the [SSTATARx](#) register and stored in the [SSTATx](#) register if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled. For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).

The destination status information is fetched from the location pointed to by the [DSTATARx](#) register and stored in the [DSTATx](#) register if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on [page 249](#).

20. If DMAH_CHx_CTL_WB_EN = True, then the CTLx[63:32] register is written out to system memory. For conditions under which the CTLx[63:32] register is written out to system memory, refer to the Write Back column of Table 8-1 on page 249.

The CTLx[63:32] register is written out to the same location on the same layer (LLPx.LMS) where it was originally fetched; that is, the location of the CTLx register of the linked list item fetched prior to the start of the block transfer. Only the second word of the CTLx register is written out – CTLx[63:32] – because only the CTLx.BLOCK_TS and CTLx.DONE fields have been updated by hardware within the DW_ahb_dmac. The LLI.CTLx.DONE bit is asserted to indicate block completion. Therefore, software can poll the LLI.CTLx.DONE bit field of the CTLx register in the LLI to ascertain when a block transfer has completed.

**Note**

Do not poll the CTLx.DONE bit in the DW_ahb_dmac memory map. Instead, poll the LLI.CTLx.DONE bit in the LLI for that block. If the polled LLI.CTLx.DONE bit is asserted, then this block transfer has completed. This LLI.CTLx.DONE bit was cleared at the start of the transfer (Step 8).

21. The SSTATx register is now written out to system memory if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled. It is written to the SSTATx register location of the LLI pointed to by the previously saved LLPx.LOC register.

The DSTATx register is now written out to system memory if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled. It is written to the DSTATx register location of the LLI pointed to by the previously saved LLPx.LOC register.

The end-of-block interrupt, int_block, is generated after the write-back of the control and status registers has completed.

**Note**

The write-back location for the control and status registers is the LLI pointed to by the previous value of the LLPx.LOC register, not the LLI pointed to by the current value of the LLPx.LOC register.

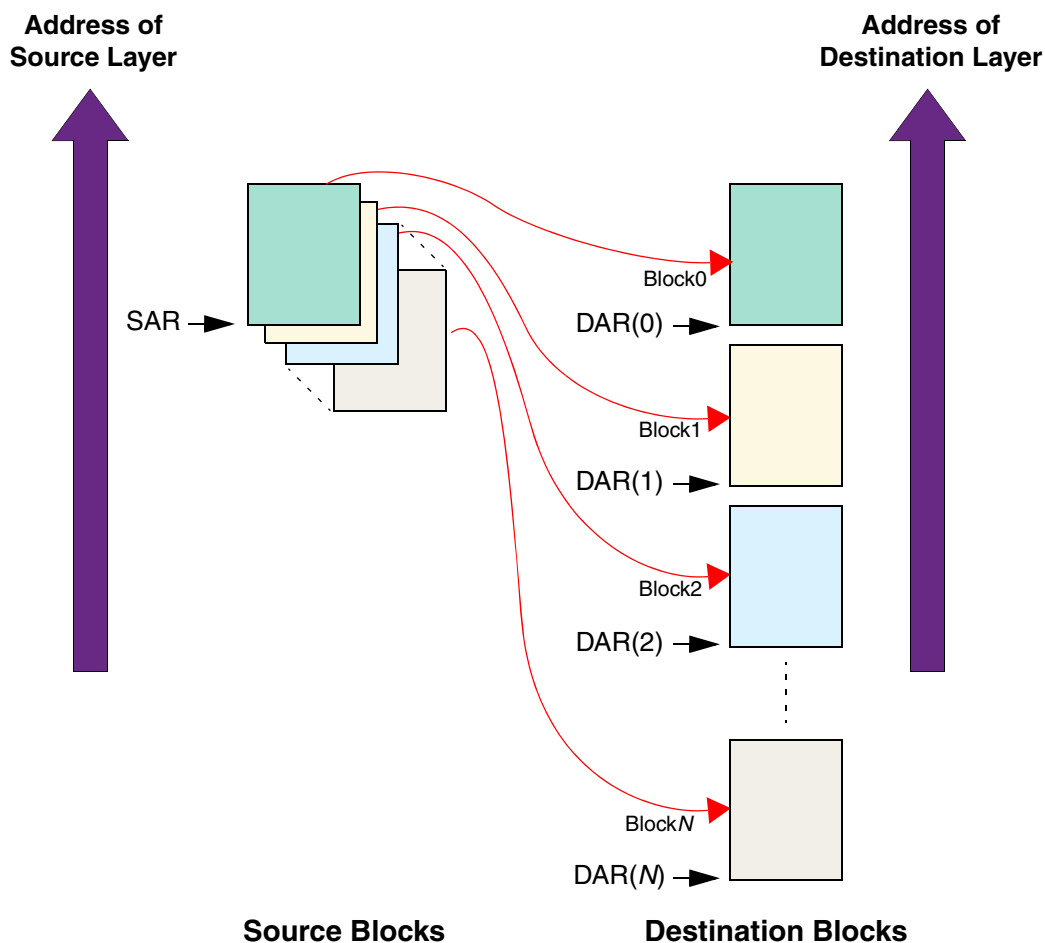
22. The DW_ahb_dmac reloads the SARx register from the initial value. Hardware sets the block-complete interrupt. The DW_ahb_dmac samples the row number, as shown in Table 8-1 on page 249. If the DW_ahb_dmac is in Row 1 or Row 5, then the DMA transfer has completed. Hardware sets the transfer complete interrupt and disables the channel. You can either respond to the Block Complete or Transfer Complete interrupts, or poll for the transfer complete raw interrupt status register (RawTfr[n], n = channel number) until it is set by hardware, in order to detect when the transfer is complete. Note that if this polling is used, software must ensure that the transfer complete interrupt is cleared by writing to the Interrupt Clear register, ClearTfr[n], before the channel is enabled. If the DW_ahb_dmac is not in Row 1 or Row 5 as shown in Table 8-1 on page 249, the following steps are performed.
23. The DMA transfer proceeds as follows:
- If interrupts are enabled (CTLx.INT_EN = 1) and the block-complete interrupt is unmasked (MaskBlock[x] = 1'b1, where x is the channel number), hardware sets the block-complete interrupt when the block transfer has completed. It then stalls until the block-complete interrupt is cleared by software. If the next block is to be the last block in the DMA transfer, then the

block-complete ISR (interrupt service routine) should clear the $CFGx.RELOAD_SRC$ source reload bit. This puts the DW_ahb_dmac into Row 1, as shown in Table 8-1 on page 249. If the next block is not the last block in the DMA transfer, then the source reload bit should remain enabled to keep the DW_ahb_dmac in Row 7, as shown in Table 8-1 on page 249.

- b. If interrupts are disabled ($CTLx.INT_EN = 0$) or the block-complete interrupt is masked ($MaskBlock[x] = 1'b0$, where x is the channel number), then hardware does not stall until it detects a write to the block-complete interrupt clear register; instead, it immediately starts the next block transfer. In this case, software must clear the source reload bit, $CFGx.RELOAD_SRC$ in order to put the device into Row 1 of Table 8-1 on page 249 before the last block of the DMA transfer has completed.
24. The DW_ahb_dmac fetches the next LLI from memory location pointed to by the current $LLPx$ register and automatically reprograms the $DARx$, $CTLx$, and $LLPx$ channel registers. Note that the $SARx$ is not reprogrammed, since the reloaded value is used for the next DMA block transfer. If the next block is the last block of the DMA transfer, then the $CTLx$ and $LLPx$ registers just fetched from the LLI should match Row 1 or Row 5 of Table 8-1 on page 249.

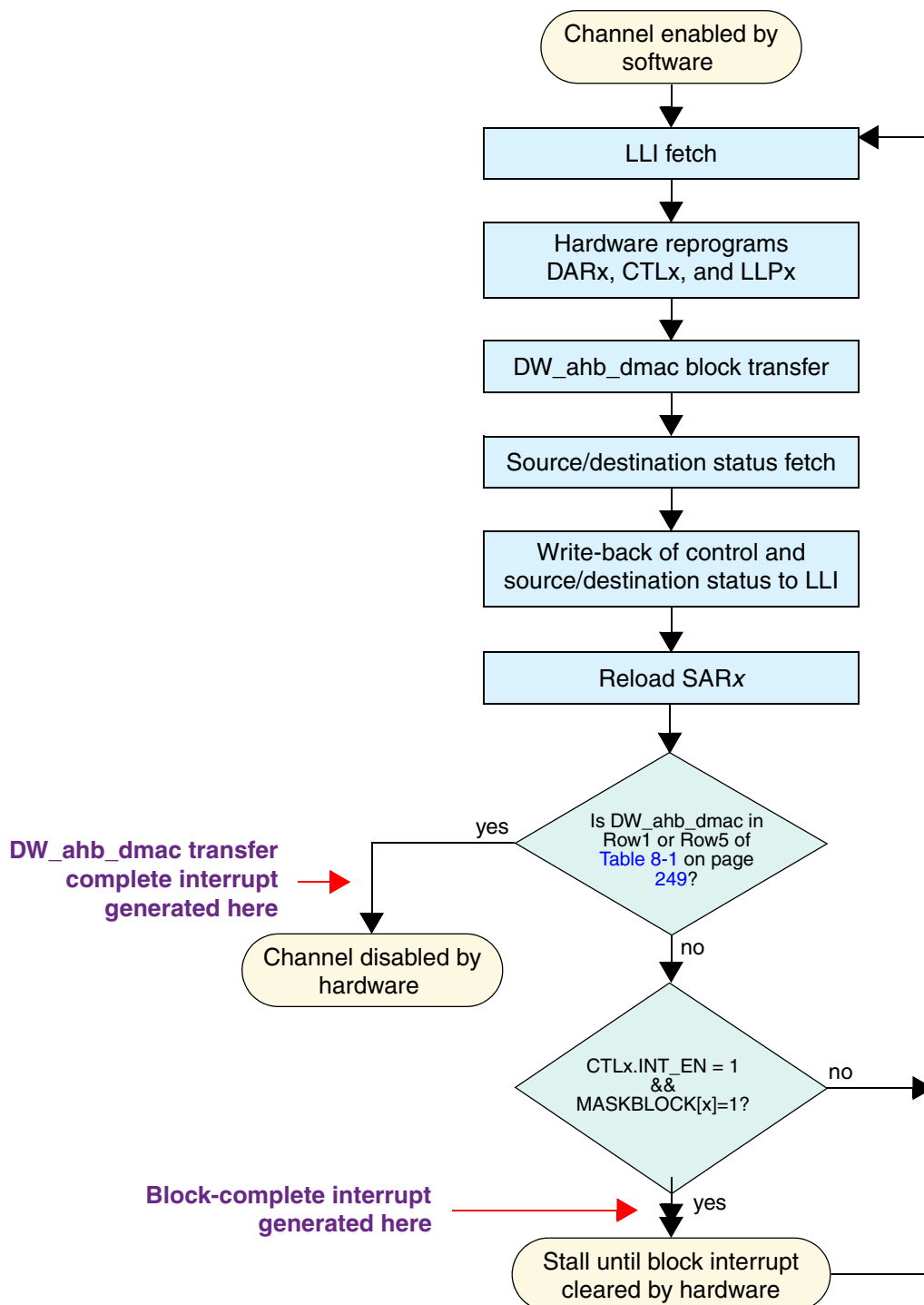
The DMA transfer might look like that shown in Figure 8-11.

Figure 8-11 Multi-Block DMA Transfer with Source Address Auto-Reloaded and Linked List Destination Address



The DMA transfer flow is shown in [Figure 8-12](#).

Figure 8-12 DMA Transfer Flow for Source Address Auto-Reloaded and Linked List Destination Address



8.6.1.5 Multi-Block Transfer with Source Address Auto-Reloaded and Contiguous Destination

Address (Row 3)**Note**

This type of multi-block transfer can only be enabled when either of the following parameters is set:

- DMAH_CHx_MULTI_BLK_TYPE = 0
- or
- DMAH_CHx_MULTI_BLK_TYPE = RELOAD_CONT

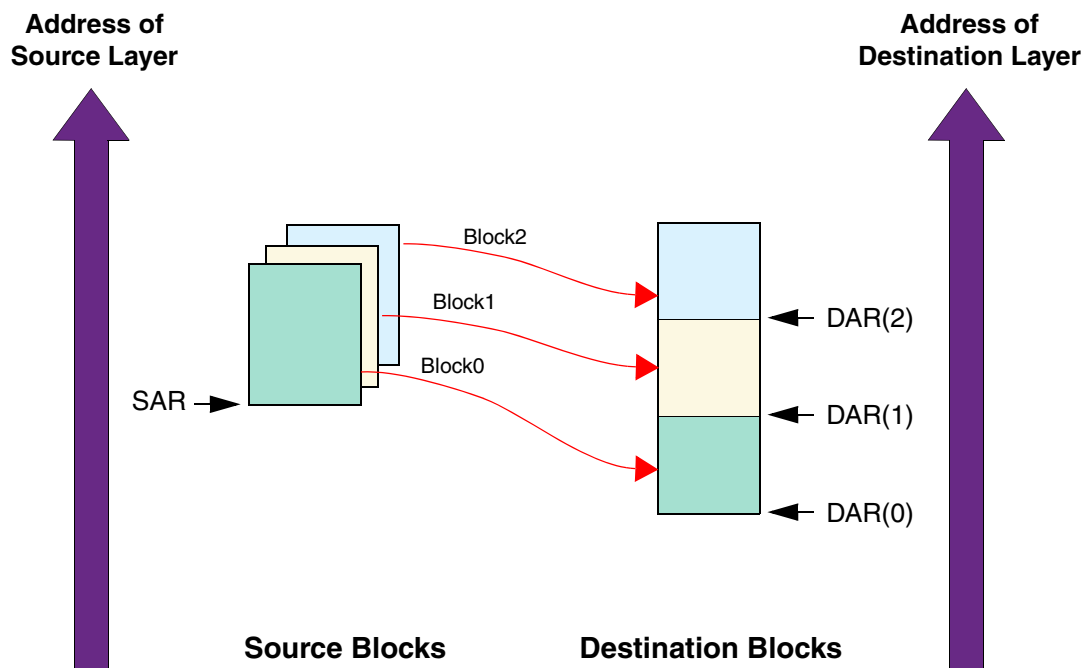
1. Read the Channel Enable register (see “ChEnReg” on page 162) to choose a free (disabled) channel.
2. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, and ClearErr (page 196). Reading the Interrupt Raw Status and Interrupt Status registers (page 193) confirms that all interrupts have been cleared.
3. Program the following channel registers:
 - a. Write the starting source address in the SARx register for channel x (see page 164).
 - b. Write the starting destination address in the DARx register for channel x (see page 165).
 - c. Program CTLx and CFGx according to Row 3, shown in Table 8-1 on page 249. Program the LLPx register with 0 (see page 167).
 - d. Write the control information for the DMA transfer in the CTLx register for channel x (see page 169). For example, in the register, you can program the following:
 - i. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control device by programming the TT_FC of the CTLx register. Table 6-4 on page 175 lists the decoding for this field.
 - ii. Set up the transfer characteristics, such as:
 - ✧ Transfer width for the source in the SRC_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - ✧ Transfer width for the destination in the DST_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - ✧ Source master layer in the SMS field where the source resides.
 - ✧ Destination master layer in the DMS field where the destination resides.
 - ✧ Incrementing/decrementing or fixed address for the source in the SINC field.
 - ✧ Incrementing/decrementing or fixed address for the destination in the DINC field.
 - e. If gather is enabled (DMAH_CHx_SRC_GAT_EN = True and CTLx.SRC_GATHER_EN is enabled), program the SGRx register (see page 186) for channel x.
 - f. If scatter is enabled (DMAH_CHx_DST_SCA_EN = True and CTLx.DST_SCATTER_EN is enabled), program the DSRx register (see page 188) for channel x.
 - g. Write the channel configuration information into the CFGx register for channel x (see page 180).
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.

This step requires programming the [HS_SEL_SRC/HS_SEL_DST](#) bits, respectively. Writing a 0 activates the hardware handshaking interface to handle source/destination requests for the specific channel. Writing a 1 activates the software handshaking interface to handle source/destination requests.

- ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripheral. This requires programming the [SRC_PER](#) and [DEST_PER](#) bits, respectively.
4. Ensure that bit 0 of the [DmaCfgReg](#) register is enabled before writing to [ChEnReg](#).
5. Source and destination request single and burst DW_ahb_dmac transactions to transfer the block of data (assuming non-memory peripherals). The DW_ahb_dmac acknowledges at the completion of every transaction (burst and single) in the block and carries out the block transfer.
6. When the block transfer has completed, the DW_ahb_dmac reloads the SARx register; the DARx register remains unchanged. Hardware sets the block-complete interrupt. The DW_ahb_dmac then samples the row number, as shown in [Table 8-1](#) on page 249. If the DW_ahb_dmac is in Row 1, then the DMA transfer has completed. Hardware sets the transfer-complete interrupt and disables the channel. You can either respond to the Block Complete or Transfer Complete interrupts, or poll for the transfer complete raw interrupt status register (RawTfr[n], n = channel number) until it is set by hardware, in order to detect when the transfer is complete. Note that if this polling is used, software must ensure that the transfer complete interrupt is cleared by writing to the Interrupt Clear register, ClearTfr[n], before the channel is enabled. If the DW_ahb_dmac is not in Row 1, the next step is performed.
7. The DMA transfer proceeds as follows:
 - a. If interrupts are enabled (CTLx.INT_EN = 1) and the block-complete interrupt is unmasked (MaskBlock[x] = 1'b1, where x is the channel number), hardware sets the block-complete interrupt when the block transfer has completed. It then stalls until the block-complete interrupt is cleared by software. If the next block is to be the last block in the DMA transfer, then the block-complete ISR (interrupt service routine) should clear the source reload bit, CFGx.RELOAD_SRC. This puts the DW_ahb_dmac into Row 1, as shown in [Table 8-1](#) on page 249. If the next block is not the last block in the DMA transfer, then the source reload bit should remain enabled to keep the DW_ahb_dmac in Row 3, as shown in [Table 8-1](#) on page 249.
 - b. If interrupts are disabled (CTLx.INT_EN = 0) or the block-complete interrupt is masked (MaskBlock[x] = 1'b0, where x is the channel number), then hardware does not stall until it detects a write to the block-complete interrupt clear register; instead, it starts the next block transfer immediately. In this case, software must clear the source reload bit, CFGx.RELOAD_SRC, to put the device into Row 1 of [Table 8-1](#) on page 249 before the last block of the DMA transfer has completed.

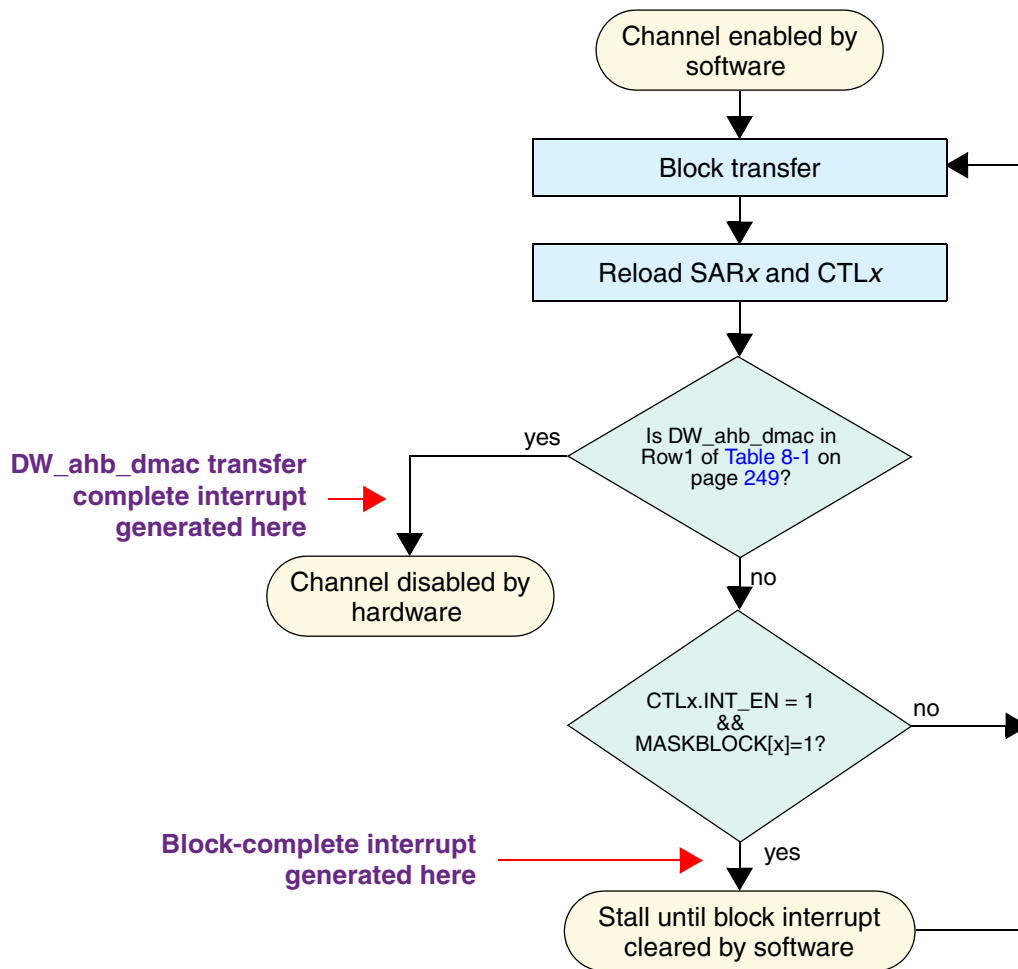
The transfer is similar to that shown in [Figure 8-13](#).

Figure 8-13 Multi-Block DMA Transfer with Source Address Auto-Reloaded and Contiguous Destination Address



The DMA transfer flow is shown in [Figure 8-14](#).

Figure 8-14 DMA Transfer Flow for Source Address Auto-Reloaded and Contiguous Destination Address



8.6.1.6 Multi-Block DMA Transfer with Linked List for Source and Contiguous Destination Address (Row 8)



Note

This type of multi-block transfer can only be enabled when either of the following parameters is set:

- DMAH_CHx_MULTI_BLK_TYPE = 0
- or
- DMAH_CHx_MULTI_BLK_TYPE = LLP_CONT

1. Read the Channel Enable register (see "ChEnReg" on page 162) to choose a free (disabled) channel.
2. Set up the linked list in memory. Write the control information in the LLI. CTLx register location of the block descriptor for each LLI in memory (see [Figure 8-1](#) on page 248) for channel x. For example, in the register, you can program the following:

- a. Set up the transfer type (memory or non-memory peripheral for source and destination) and flow control device by programming the TT_FC of the CTLx register. Table 6-4 on page 175 lists the decoding for this field.
- b. Set up the transfer characteristics, such as:
 - i. Transfer width for the source in the SRC_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - ii. Transfer width for the destination in the DST_TR_WIDTH field. Table 6-3 on page 175 lists the decoding for this field.
 - iii. Source master layer in the SMS field where the source resides.
 - iv. Destination master layer in the DMS field where the destination resides.
 - v. Incrementing/decrementing or fixed address for the source in the SINC field.
 - vi. Incrementing/decrementing or fixed address for the destination in the DINC field.
3. Write the starting destination address in the DARx register for channel x (see page 165).

**Note**

The values in the LLI.DARx register location of each Linked List Item (LLI) in memory, although fetched during an LLI fetch, are not used.

4. Write the channel configuration information into the CFGx register for channel x (see page 180).
 - i. Designate the handshaking interface type (hardware or software) for the source and destination peripherals; this is not required for memory.
This step requires programming the HS_SEL_SRC/HS_SEL_DST bits. Writing a 0 activates the hardware handshaking interface to handle source/destination requests for the specific channel. Writing a 1 activates the software handshaking interface to handle source/destination requests.
 - ii. If the hardware handshaking interface is activated for the source or destination peripheral, assign the handshaking interface to the source and destination peripherals. This requires programming the SRC_PER and DEST_PER bits, respectively.
5. Ensure that all LLI.CTLx register locations of the LLI (except the last) are set as shown in Row 8 of Table 8-1 on page 249, while the LLI.CTLx register of the last Linked List item must be set as described in Row 1 or Row 5 of Table 8-1. Figure 8-1 on page 248 shows a Linked List example with two list items.
6. Ensure that the LLI.LLPx register locations of all LLIs in memory (except the last) are non-zero and point to the next Linked List Item.
7. Ensure that the LLI.SARx register location of all LLIs in memory point to the start source block address preceding that LLI fetch.
8. If DMAH_CHx_CTL_WB_EN = True, ensure that the LLI.CTLx.DONE fields of the LLI.CTLx register locations of all LLIs in memory are cleared.
9. If source status fetching is enabled (DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled), program the SSTATARx register (see page 178) so that the source status information can be fetched from the location pointed

to by [SSTATARx](#). For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on page 249.

10. If destination status fetching is enabled (DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled), program the [DSTATARx](#) register (see [page 179](#)) so that the destination status information can be fetched from the location pointed to by the [DSTATARx](#) register. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on page 249.
11. If gather is enabled (DMAH_CHx_SRC_GAT_EN = True and [CTLx](#).SRC_GATHER_EN is enabled), program the [SGRx](#) register (see [page 186](#)) for channel x.
12. If scatter is enabled (DMAH_CHx_DST_SCA_EN = True and [CTLx](#).DST_SCATTER_EN) program the [DSRx](#) register (see [page 188](#)) for channel x.
13. Clear any pending interrupts on the channel from the previous DMA transfer by writing to the Interrupt Clear registers: ClearTfr, ClearBlock, ClearSrcTran, ClearDstTran, and ClearErr ([page 196](#)). Reading the Interrupt Raw Status and Interrupt Status registers ([page 193](#)) confirms that all interrupts have been cleared.
14. Program the [CTLx](#) and [CFGx](#) registers according to Row 8, as shown in [Table 8-1](#) on page 249.
15. Program the [LLPx](#) register with LLPx(0), the pointer to the first Linked List item.
16. Ensure that bit 0 of the [DmaCfgReg](#) register is enabled before writing to [ChEnReg](#).
17. The DW_ahb_dmac fetches the first LLI from the location pointed to by LLPx(0).



Note

The LLI.SARx, LLI.DARx, LLI.LLPx, and LLI.CTLx registers are fetched. The LLI.DARx register location of the LLI – although fetched – is not used. The [DARx](#) register in the DW_ahb_dmac remains unchanged.

18. Source and destination request single and burst DW_ahb_dmac transactions to transfer the block of data (assuming non-memory peripherals). The DW_ahb_dmac acknowledges at the completion of every transaction (burst and single) in the block and carries out the block transfer.
19. Once the block of data is transferred, the source status information is fetched from the location pointed to by the [SSTATARx](#) register and stored in the [SSTATx](#) register if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_SRC = True, and CFGx.SS_UPD_EN is enabled. For conditions under which the source status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on page 249.

The destination status information is fetched from the location pointed to by the [DSTATARx](#) register and stored in the [DSTATx](#) register if DMAH_CHx_CTL_WB_EN = True, DMAH_CHx_STAT_DST = True, and CFGx.DS_UPD_EN is enabled. For conditions under which the destination status information is fetched from system memory, refer to the Write Back column of [Table 8-1](#) on page 249.
20. If DMAH_CHx_CTL_WB_EN = True, then the [CTLx](#)[63:32] register is written out to system memory. For conditions under which the [CTLx](#)[63:32] register is written out to system memory, refer to the Write Back column of [Table 8-1](#) on page 249.

The **CTLx**[63:32] register is written out to the same location on the same layer (**LLPx.LMS**) where it was originally fetched; that is, the location of the **CTLx** register of the linked list item fetched prior to the start of the block transfer. Only the second word of the **CTLx** register is written out, **CTLx**[63:32], because only the **CTLx.BLOCK_TS** and **CTLx.DONE** fields have been updated by hardware within the **DW_ahb_dmac**. Additionally, the **CTLx.DONE** bit is asserted to indicate block completion. Therefore, software can poll the **LLI.CTLx.DONE** bit field of the **CTLx** register in the **LLI** to ascertain when a block transfer has completed.

**Note**

Do not poll the **CTLx.DONE** bit in the **DW_ahb_dmac** memory map. Instead, poll the **LLI.CTLx.DONE** bit in the **LLI** for that block. If the polled **LLI.CTLx.DONE** bit is asserted, then this block transfer has completed. This **LLI.CTLx.DONE** bit was cleared at the start of the transfer (Step 8).

21. The **SSTATx** register is now written out to system memory if **DMAH_CHx_CTL_WB_EN** = True, **DMAH_CHx_STAT_SRC** = True, and **CFGx.SS_UPD_EN** is enabled. It is written to the **SSTATx** register location of the **LLI** pointed to by the previously saved **LLPx.LOC** register.

The **DSTATx** register is now written out to system memory if **DMAH_CHx_CTL_WB_EN** = True, **DMAH_CHx_STAT_DST** = True, and **CFGx.DS_UPD_EN** is enabled. It is written to the **DSTATx** register location of the **LLI** pointed to by the previously saved **LLPx.LOC** register.

The end-of-block interrupt, **int_block**, is generated after the write-back of the control and status registers has completed.

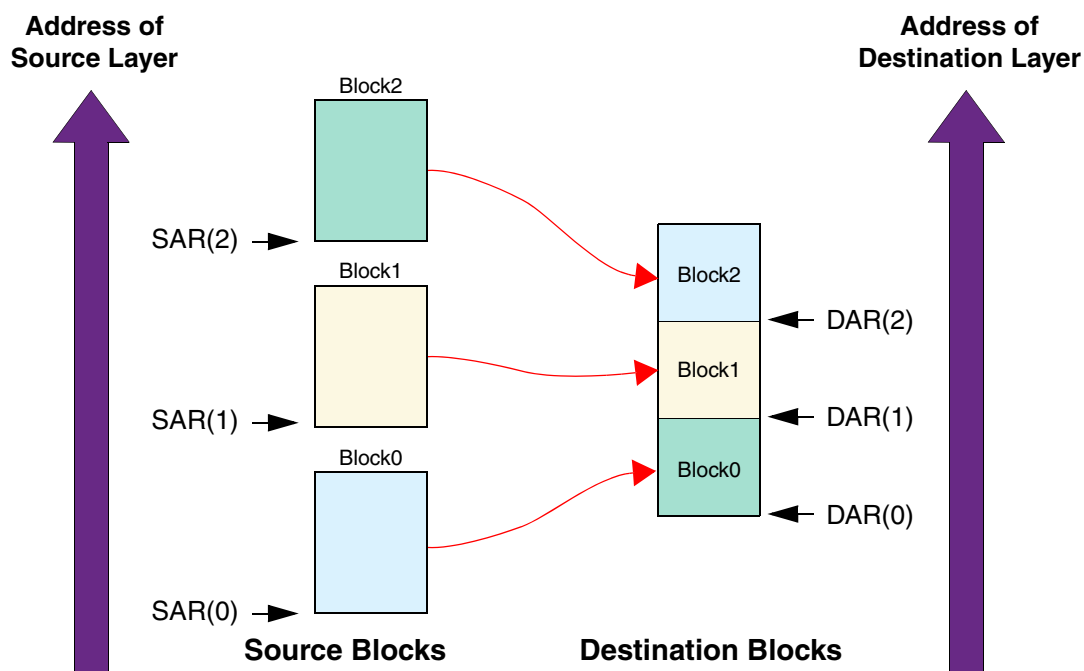
**Note**

The write-back location for the control and status registers is the **LLI** pointed to by the previous value of the **LLPx.LOC** register, not the **LLI** pointed to by the current value of the **LLPx.LOC** register.

22. The **DW_ahb_dmac** does not wait for the block interrupt to be cleared, but continues and fetches the next **LLI** from the memory location pointed to by the current **LLPx** register and automatically reprograms the **SARx**, **CTLx**, and **LLPx** channel registers. The **DARx** register is left unchanged. The DMA transfer continues until the **DW_ahb_dmac** samples that the **CTLx** and **LLPx** registers at the end of a block transfer match those described in Row 1 or Row 5 of [Table 6-2](#) on page 174 (as discussed earlier). The **DW_ahb_dmac** then knows that the previously transferred block was the last block in the DMA transfer.

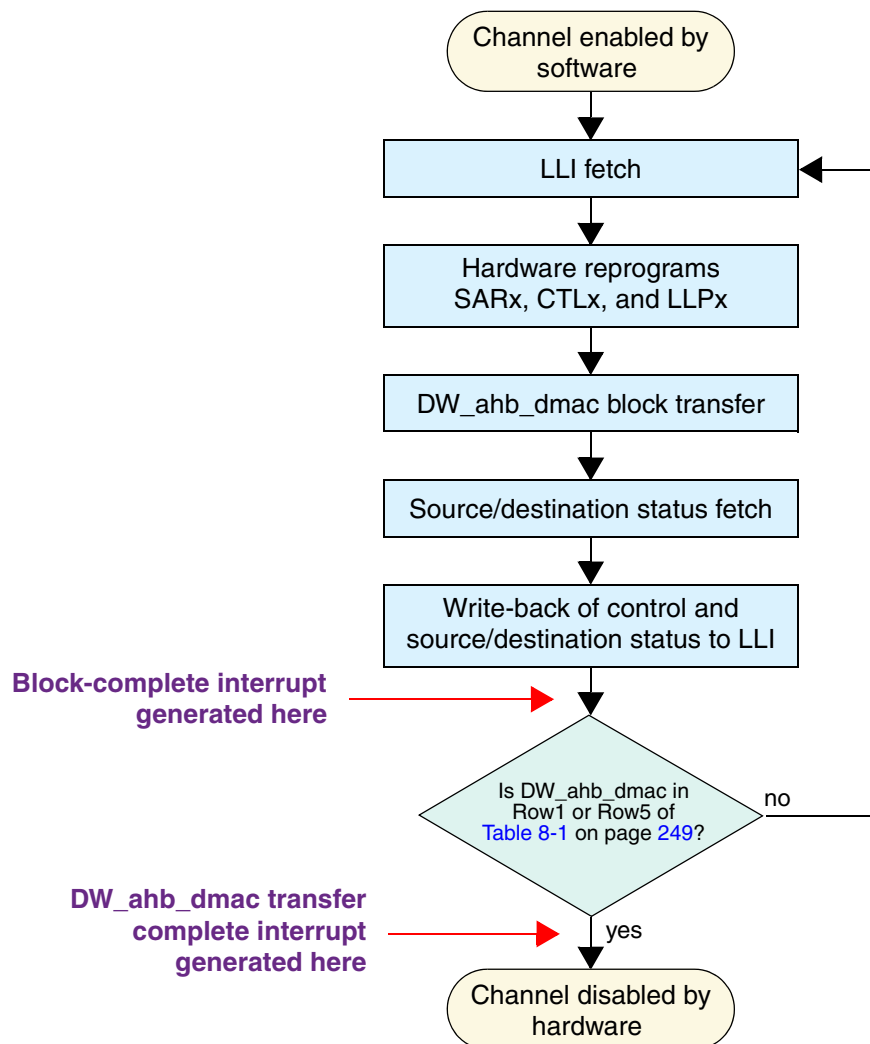
The DW_ahb_dmac transfer might look like that shown in [Figure 8-15](#). Note that the destination address is decrementing.

Figure 8-15 Multi-Block DMA Transfer with Linked List Source Address and Contiguous Destination Address



The DMA transfer flow is shown in [Figure 8-16](#).

Figure 8-16 DMA Transfer Flow for Source Address Auto-Reloaded and Contiguous Destination Address



8.7 Disabling a Channel Prior to Transfer Completion

Under normal operation, software enables a channel by writing a 1 to the channel enable register, [ChEnReg.CH_EN](#), and hardware disables a channel on transfer completion by clearing the [ChEnReg.CH_EN](#) register bit.

The recommended way for software to disable a channel without losing data is to use the [CH_SUSP](#) bit in conjunction with the [FIFO_EMPTY](#) bit in the Channel Configuration Register ([CFGx](#)).

1. If software wishes to disable a channel prior to the DMA transfer completion, then it can set the [CFGx.CH_SUSP](#) bit to tell the DW_ahb_dmac to halt all transfers from the source peripheral. Therefore, the channel FIFO receives no new data.
2. Software can now poll the [CFGx.FIFO_EMPTY](#) bit until it indicates that the channel FIFO is empty.
3. The [ChEnReg.CH_EN](#) bit can then be cleared by software once the channel FIFO is empty.

When `CTLx.SRC_TR_WIDTH < CTLx.DST_TR_WIDTH` and the `CFGx.CH_SUSP` bit is high, the `CFGx.FIFO_EMPTY` is asserted once the contents of the FIFO do not permit a single word of `CTLx.DST_TR_WIDTH` to be formed. However, there may still be data in the channel FIFO, but not enough to form a single transfer of `CTLx.DST_TR_WIDTH`. In this scenario, once the channel is disabled, the remaining data in the channel FIFO is not transferred to the destination peripheral.

It is permissible to remove the channel from the suspension state by writing a 0 to the `CFGx.CH_SUSP` register. The DMA transfer completes in the normal manner.

**Note**

If a channel is disabled by software, an active single or burst transaction is not guaranteed to receive an acknowledgement.

8.7.1 Abnormal Transfer Termination

A DW_ahb_dmac DMA transfer may be terminated abruptly by software by clearing the channel enable bit, `ChEnReg.CH_EN`. You must not assume that the channel is disabled immediately after the `ChEnReg`. The `CH_EN` bit is cleared over the AHB slave interface. Consider this as a request to disable the channel. You must poll `ChEnReg.CH_EN` and confirm that the channel is disabled by reading back 0. A case where the channel is not disabled after a channel disable request is where either the source or destination has received a split or retry response. The DW_ahb_dmac must keep re-attempting the transfer to the system HADDR that originally received the split or retry response until an OKAY response is returned; to do otherwise is an AMBA protocol violation.

Software may terminate all channels abruptly by clearing the global enable bit in the DW_ahb_dmac Configuration Register (`DmaCfgReg[0]`). Again, you must not assume that all channels are disabled immediately after the `DmaCfgReg[0]` is cleared over the AHB slave interface. Consider this as a request to disable all channels. You must poll `ChEnReg` and confirm that all channels are disabled by reading back 0.

**Note**

If the channel enable bit is cleared while there is data in the channel FIFO, this data is not sent to the destination peripheral and is not present when the channel is re-enabled. For read-sensitive source peripherals, such as a source FIFO, this data is therefore lost. When the source is not a read-sensitive device (such as memory), disabling a channel without waiting for the channel FIFO to empty may be acceptable, since the data is available from the source peripheral upon request and is not lost.

If a channel is disabled by software, an active single or burst transaction is not guaranteed to receive an acknowledgement.

If the DW_ahb_dmac is configured to use defined length bursts (`DMAH_INCR_BURSTS = 0`), disabling the channel via software prior to completing a transfer is not allowed. Clearing the `CH_EN` bit prior to channel suspend may violate the AHB protocol.

8.8 Defined-Length Burst Support on DW_ahb_dmac

By default, the DW_ahb_dmac support incremental (INCR) bursts only. To achieve better performance, defined length bursts, such as INCR4, INCR8 and INCR16 are required. The DW_ahb_dmac can be configured to use defined-length bursts by setting the configuration parameter `DMAH_INCR_BURSTS` to 0. In this mode, the DW_ahb_dmac will select the largest valid defined-length burst to complete the transfer.

9

Verification

This chapter provides an overview of the testbench available for DW_ahb_dmac verification. Once the DW_ahb_dmac has been configured and the verification environment set up, simulations can be run automatically. For information on running simulations for DW_ahb_dmac in coreAssembler or coreConsultant, see [“Building and Verifying a Component or Subsystem”](#) on page 27.

**Note**

The DW_ahb_dmac verification testbench is built with DesignWare Verification IP (VIP). Please make sure you have the supported version of the VIP components for this release, otherwise, you may experience some tool compatibility problems. For more information about supported tools in this release, refer to the [DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide](#).

9.1 Overview of Vera Tests

The DW_ahb_dmac verification testbench performs the following set of tests that have been written to exhaustively verify the functionality and have also achieved maximum RTL code coverage:

- A single block transfer
- A single block transfer with register write back
- An LLP block chaining transfer
- A reloading transfer
- A transfer that is suspended and the channel terminated
- A transfer that is suspended and then resumed (without terminating the channel)
- A transfer where the various peripherals (SRC, DST and LLP) issue error responses
- A transfer where the channel is terminated part way through (without suspend)
- A transfer where the channel is terminated part way through (without suspend) using the global DW_ahb_dmac enable bit; terminates transfers running on all other channels as well

Within each of these transfers, all parameters are randomized.

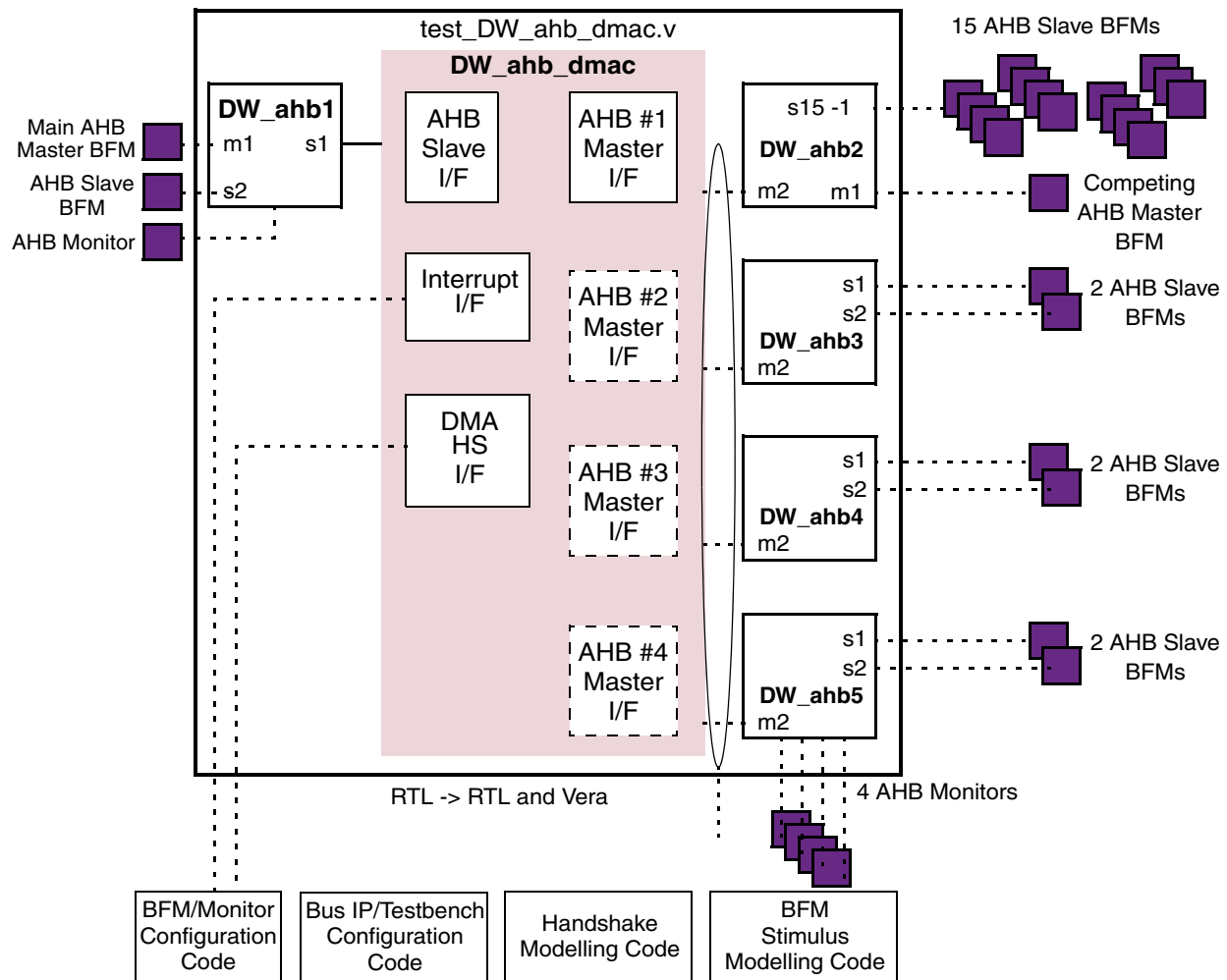
The testbench constantly verifies whether conditions including the following are met:

- Transfers are correct
- Registers are updated correctly
- Registers are written out correctly
- Interrupts are set correctly
- FIFO mode and flow control mode is not violated
- Bus and channel locking is correct
- Channel arbitration is correct

9.2 Overview of DW_ahb_dmac Testbench

As illustrated in [Figure 9-1](#), the DW_ahb_dmac testbench is a Verilog testbench that includes:

- Verilog DUT (DW_ahb_dmac) and supporting bus IP (DW_ahb)
- Vera BFM (AHB master and slave) and monitors (AHB only)
- Vera test harness and stimulus code

Figure 9-1 DW_ahb_dmac Testbench

The file, `test_DW_ahb_dmac.v`, shows the instantiation of the top-level design in a testbench and resides in the `workspace/sim/testbench` directory. The testbench tests your configuration specified in the Specify Configuration task of coreConsultant and is self-checking. When a coreKit has been configured, the verification environment is stored in `workspace/sim`. Files in `workspace/sim/testbench` form the actual testbench for DW_ahb_dmac. The `workspace/sim/test_name` directory contains the `test.vrh` header file and the encrypted `test.vrp` file.

The Competing AHB master BFM is used to compete with the DW_ahb_dmac when requesting the AHB bus and also to generate AHB traffic to modulate the otherwise uninhibited flow of data between the DW_ahb_dmac and AHB slaves. When the AMBA-lite version of the DW_ahb_dmac is being verified, the competing AHB master BFM is not used, and the DW_ahb components are replaced with AMBA-lite versions of the bus IP.

**Note**

The top-level RTL code contains checkers to check for illegal programming of the DW_ahb_dmac. These monitors are removed upon synthesis. Because all simulators apart from VCS use a GTECH netlist, then the monitors are triggered only when you illegally program the DW_ahb_dmac while running an RTL simulation using the VCS simulator. An example of an illegal programming condition is where you program the source master select field of the control register, CTL0.SMS = 3, in a DW_ahb_dmac configuration where DMAH_NUM_MASTER_INT = 2. This triggers the following monitor illegal programming warning:

```
DW_ahb_dmac_CHECKER_ILLEGAL_PROG : SMS field in ctl register  
channel 0 refers to master interface 4, but there are only 2 master  
interfaces in the system ***
```


10

Integration Considerations

After you have configured, tested, and synthesized your component with the coreTools flow, you can integrate the component into your own design environment.

10.1 Performance

This section discusses performance and the parameters—software and hardware configuration parameters—that affect the performance of the DW_ahb_dmac.

10.1.1 Area

This section provides information to help you configure area for your configuration.

The following table includes synthesis results that have been generated using the TSMC 65nm technology library.

Table 10-1 Synthesis Results Using TSMC 65nm Technology Library

Configuration	Operating Frequency	Gate Count
Default Configuration	166.67 MHz	6959 gates
Default Configuration with Clock Gating	285.71 MHz	6519 gates
Minimum Configuration: DMAH_NUM_MASTER_INT=1 DMAH_NUM_CHANNELS=1 DMAH_NUM_HS_INT=0 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=2 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32	166.67 MHz	5206 gates

Table 10-1 Synthesis Results Using TSMC 65nm Technology Library (Continued)

Configuration	Operating Frequency	Gate Count
<i>Continued</i> Minimum Configuration DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CH0_FIFO_DEPTH=8 DMAH_CHx_STAT_DST=0 DMAH_CHx_STAT_SRC=0 DMAH_CH0_MAX_MULT_SIZE=4 DMAH_CH0_MAX_BLK_SIZE=3 DMAH_CHx_FC=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_SMS=0 DMAH_CHx_DMS=0 DMAH_CHx_LMS=0 DMAH_CHx_STW=32 DMAH_CHx_DTW=32 DMAH_CH0_SRC_NON_OK=0 DMAH_CH0_DST_NON_OK=0 DMAH_CHx_LL_P_NON_OK=0 DMAH_CHx_MULTI_BLK_EN=0 DMAH_CHx_HC_LL_P=1 DMAH_CHx_SRC_GAT_EN=0 DMAH_CHx_DST_SCA_EN=0	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 5206 gates
Maximum Configuration: DMAH_NUM_MASTER_INT=4 DMAH_NUM_CHANNELS=8 DMAH_NUM_HS_INT=16 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=1 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0	166.67 MHz	146062 gates

Table 10-1 Synthesis Results Using TSMC 65nm Technology Library (Continued)

Configuration	Operating Frequency	Gate Count
<i>Continued</i> Maximum Configuration DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CHx_FIFO_DEPTH=16 DMAH_CHx_STAT_DST =1 DMAH_CHx_STAT_SRC =1 DMAH_CHx_MAX_MULT_SIZE=256 DMAH_CHx_MAX_BLK_SIZE=4095 DMAH_CHx_FC=3 DMAH_CHx_LOCK_EN=1 DMAH_CHx_SMS=4 DMAH_CHx_DMS=4 DMAH_CHx_LMS=4 DMAH_CHx_STW=0 DMAH_CHx_DTW=0 DMAH_CHx_SRC_NON_OK=1 DMAH_CHx_DST_NON_OK=1 DMAH_CHx_LLQ_NON_OK=1 DMAH_CHx_MULTI_BLK_EN=1 DMAH_CHx_HC_LLQ=0 DMAH_CHx_SRC_GAT_EN=1 DMAH_CHx_DST_SCA_EN=1 DMAH_CHx_MULTI_BLK_TYPE=8 DMAH_CHx_CTL_WB_EN=1	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 146062 gates

Note: x = 0 to 7

The following table includes synthesis results that have been generated using the TSMC 28nm technology library.

Table 10-2 Synthesis Results Using TSCM 28nm Technology Library

Configuration	Operating Frequency	Gate Count
Default Configuration	166.67 MHz	6832 gates
Default Configuration with Clock Gating	285.71 MHz	6470 gates
Minimum Configuration: DMAH_NUM_MASTER_INT=1 DMAH_NUM_CHANNELS=1 DMAH_NUM_HS_INT=0 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=2 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CH0_FIFO_DEPTH=8 DMAH_CHx_STAT_DST=0 DMAH_CHx_STAT_SRC=0 DMAH_CH0_MAX_MULT_SIZE=4 DMAH_CH0_MAX_BLK_SIZE=3 DMAH_CHx_FC=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_SMS=0 DMAH_CHx_DMS=0 DMAH_CHx_LMS=0 DMAH_CHx_STW=32 DMAH_CHx_DTW=32 DMAH_CH0_SRC_NON_OK=0 DMAH_CH0_DST_NON_OK=0 DMAH_CHx_LLP_NON_OK=0 DMAH_CHx_MULTI_BLK_EN=0	166.67 MHz	5111 gates

Table 10-2 Synthesis Results Using TSCM 28nm Technology Library (Continued)

Configuration	Operating Frequency	Gate Count
<i>Continued</i> Minimum Configuration: DMAH_CHx_HC_LLP=1 DMAH_CHx_SRC_GAT_EN=0 DMAH_CHx_DST_SCA_EN=0	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 5111 gates
Maximum Configuration: DMAH_NUM_MASTER_INT=4 DMAH_NUM_CHANNELS=8 DMAH_NUM_HS_INT=16 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=1 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CHx_FIFO_DEPTH=16 DMAH_CHx_STAT_DST=1 DMAH_CHx_STAT_SRC=1 DMAH_CHx_MAX_MULT_SIZE=256 DMAH_CHx_MAX_BLK_SIZE=4095 DMAH_CHx_FC=3 DMAH_CHx_LOCK_EN=1 DMAH_CHx_SMS=4 DMAH_CHx_DMS=4 DMAH_CHx_LMS=4 DMAH_CHx_STW=0 DMAH_CHx_DTW=0 DMAH_CHx_SRC_NON_OK=1 DMAH_CHx_DST_NON_OK=1 DMAH_CHx_LLQ_NON_OK=1 DMAH_CHx_MULTI_BLK_EN=1 DMAH_CHx_HC_LLQ=0 DMAH_CHx_SRC_GAT_EN=1	166.67 MHz	141125 gates

Table 10-2 Synthesis Results Using TSCM 28nm Technology Library (Continued)

Configuration	Operating Frequency	Gate Count
<i>Continued</i> Maximum Configuration: DMAH_CHx_DST_SCA_EN=1 DMAH_CHx_MULTI_BLK_TYPE=8 DMAH_CHx_CTL_WB_EN=1	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 141125 gates

Note: x = 0 to 7

10.1.2 Power Consumption

The following table provides information about the power consumption of the DW_ahb_dmac using the TSMC 65nm technology library and how it affects performance.

Table 10-3 Power Consumption of DW_ahb_dmac Using TSMC 65nm Technology Library

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
Default Configuration	166.67 MHz	1.9131 μ W	785.4383 μ W
Default Configuration with Clock Gating	285.71 MHz	56.3772 μ W	20.7952 μ W
Minimum Configuration: DMAH_NUM_MASTER_INT=1 DMAH_NUM_CHANNELS=1 DMAH_NUM_HS_INT=0 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=2 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CH0_FIFO_DEPTH=8 DMAH_CHx_STAT_DST=0 DMAH_CHx_STAT_SRC=0 DMAH_CH0_MAX_MULT_SIZE=4 DMAH_CH0_MAX_BLK_SIZE=3 DMAH_CHx_FC=0	166.67 MHz	1.4542 μ W	582.9334 μ W

Table 10-3 Power Consumption of DW_ahb_dmac Using TSMC 65nm Technology Library (Continued)

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
<i>Continued</i> Minimum Configuration: DMAH_CHx_LOCK_EN=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_SMS=0 DMAH_CHx_DMS=0 DMAH_CHx_LMS=0 DMAH_CHx_STW=32 DMAH_CHx_DTW=32 DMAH_CH0_SRC_NON_OK=0 DMAH_CH0_DST_NON_OK=0 DMAH_CHx_LLIP_NON_OK=0	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 1.4542 μ W	<i>Cont'd</i> 582.9334 μ W
Maximum Configuration: DMAH_NUM_MASTER_INT=4 DMAH_NUM_CHANNELS=8 DMAH_NUM_HS_INT=16 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=1 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0 DMAH_ID_NUM=0x02080901 DMAH_CHx_FIFO_DEPTH=16 DMAH_CHx_STAT_DST =1 DMAH_CHx_STAT_SRC =1 DMAH_CHx_MAX_MULT_SIZE=256 DMAH_CHx_MAX_BLK_SIZE=4095 DMAH_CHx_FC=3 DMAH_CHx_LOCK_EN=1 DMAH_CHx_SMS=4 DMAH_CHx_DMS=4 DMAH_CHx_LMS=4	166.67 MHz	39.5220 μ W	12.4878 mW

Table 10-3 Power Consumption of DW_ahb_dmac Using TSMC 65nm Technology Library (Continued)

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
<i>Continued</i> Maximum Configuration: DMAH_CHx_STW=0 DMAH_CHx_DTW=0 DMAH_CHx_SRC_NON_OK=1 DMAH_CHx_DST_NON_OK=1 DMAH_CHx_LLQ_NON_OK=1 DMAH_CHx_MULTI_BLK_EN=1 DMAH_CHx_HC_LLQ=0 DMAH_CHx_SRC_GAT_EN=1 DMAH_CHx_DST_SCA_EN=1 DMAH_CHx_MULTI_BLK_TYPE=8 DMAH_CHx_CTL_WB_EN=1	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 39.5220 μ W	<i>Cont'd</i> 12.4878 mW

Note: x = 0 to 7

The following table provides information about the power consumption of the DW_ahb_dmac using the TSMC 28nm technology library and how it affects performance.

Table 10-4 Power Consumption of DW_ahb_dmac Using TSMC 28nm Technology Library

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
Default Configuration	166.67 MHz	700.6328 μ W	541.6402 μ W
Default Configuration with Clock Gating	285.71 MHz	25.8536 μ W	9.5665 μ W
Minimum Configuration: DMAH_NUM_MASTER_INT=1 DMAH_NUM_CHANNELS=1 DMAH_NUM_HS_INT=0 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=2 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0	166.67 MHz	524.6405 μ W	400.4807 μ W

Table 10-4 Power Consumption of DW_ahb_dmac Using TSMC 28nm Technology Library (Continued)

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
<i>Continued</i> Minimum Configuration: DMAH_ID_NUM=0x02080901 DMAH_CH0_FIFO_DEPTH=8 DMAH_CHx_STAT_DST=0 DMAH_CHx_STAT_SRC=0 DMAH_CH0_MAX_MULT_SIZE=4 DMAH_CH0_MAX_BLK_SIZE=3 DMAH_CHx_FC=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_LOCK_EN=0 DMAH_CHx_SMS=0 DMAH_CHx_DMS=0 DMAH_CHx_LMS=0 DMAH_CHx_STW=32 DMAH_CHx_DTW=32 DMAH_CH0_SRC_NON_OK=0 DMAH_CH0_DST_NON_OK=0 DMAH_CHx_LLP_NON_OK=0 DMAH_CHx_MULTI_BLK_EN=0 DMAH_CHx_HC_LLP=1 DMAH_CHx_SRC_GAT_EN=0 DMAH_CHx_DST_SCA_EN=0	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 524.6405 μ W	<i>Cont'd</i> 400.4807 μ W
Maximum Configuration: DMAH_NUM_MASTER_INT=4 DMAH_NUM_CHANNELS=8 DMAH_NUM_HS_INT=16 DMAH_MABRST=0 DMAH_RETURN_ERR_RESP=0 DMAH_INTR_POL=1 DMAH_INTR_IO=1 DMAH_BIG_ENDIAN=0 DMAH_S_HDATA_WIDTH=32 DMAH_M1_HDATA_WIDTH=32 DMAH_M2_HDATA_WIDTH=32 DMAH_M3_HDATA_WIDTH=32 DMAH_M4_HDATA_WIDTH=32 DMAH_M1_AHB_LITE=0 DMAH_M2_AHB_LITE=0 DMAH_M3_AHB_LITE=0 DMAH_M4_AHB_LITE=0	166.67 MHz	14.0693 mW	8.7554 mW

Table 10-4 Power Consumption of DW_ahb_dmac Using TSMC 28nm Technology Library (Continued)

Configuration	Operating Frequency	Static Power Consumption	Dynamic Power Consumption
<i>Continued</i> Maximum Configuration: DMAH_ID_NUM=0x02080901 DMAH_CHx_FIFO_DEPTH=16 DMAH_CHx_STAT_DST =1 DMAH_CHx_STAT_SRC =1 DMAH_CHx_MAX_MULT_SIZE=256 DMAH_CHx_MAX_BLK_SIZE=4095 DMAH_CHx_FC=3 DMAH_CHx_LOCK_EN=1 DMAH_CHx_SMS=4 DMAH_CHx_DMS=4 DMAH_CHx_LMS=4 DMAH_CHx_STW=0 DMAH_CHx_DTW=0 DMAH_CHx_SRC_NON_OK=1 DMAH_CHx_DST_NON_OK=1 DMAH_CHx_LLQ_NON_OK=1 DMAH_CHx_MULTI_BLK_EN=1 DMAH_CHx_HC_LLQ=0 DMAH_CHx_SRC_GAT_EN=1 DMAH_CHx_DST_SCA_EN=1 DMAH_CHx_MULTI_BLK_TYPE=8 DMAH_CHx_CTL_WB_EN=1	<i>Cont'd</i> 166.67 MHz	<i>Cont'd</i> 14.0693 mW	<i>Cont'd</i> 8.7554 mW

Note: x = 0 to 7

10.2 1KB Boundary Crossing

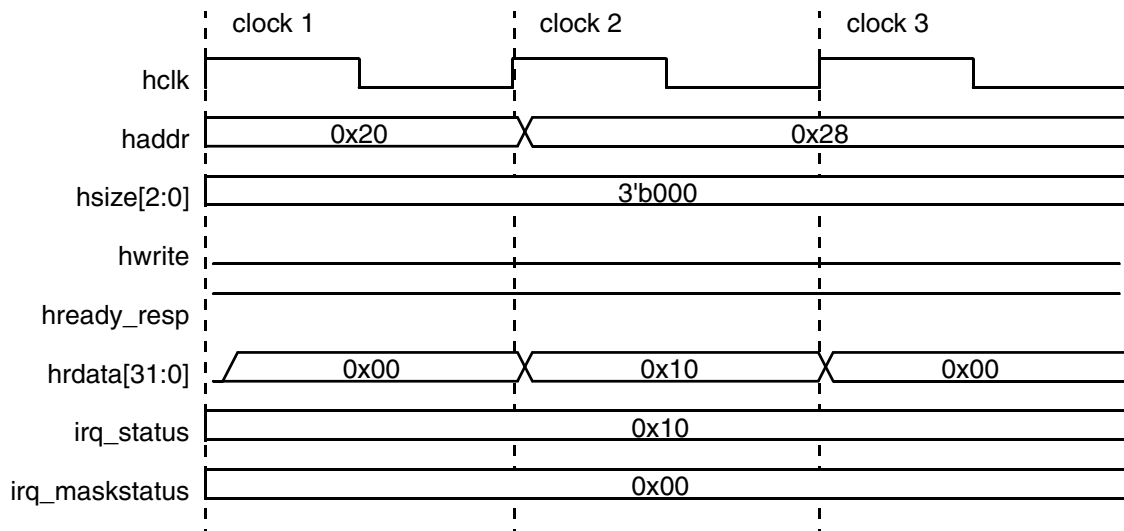
The AHB protocol requires that no AHB burst cross a 1KB address boundary. DW_ahb_dmac handles this situation automatically. If a DMA transfer is set up by software such that during the transfer one of the AHB transfers could cross a 1KB boundary, DW_ahb_dmac will automatically set up the AHB transfers such that the end of the 1KB boundary completes one AHB transfer and the start of the 1KB boundary starts another AHB transfer.

10.3 Read Accesses

For reads, registers less than the full access width return zeros in the unused upper bits. An AHB read takes two hclk cycles. The two cycles can be thought of as a control and data cycle, respectively. As shown in the following figure, the address and control is driven from clock 1 (control cycle); the read data for this access is driven by the slave interface onto the bus from clock 2 (data cycle) and is sampled by the master on clock

3. The operation of the AHB bus is pipelined, so while the read data from the first access is present on the bus for the master to sample, the control for the next access is present on the bus for the slave to sample.

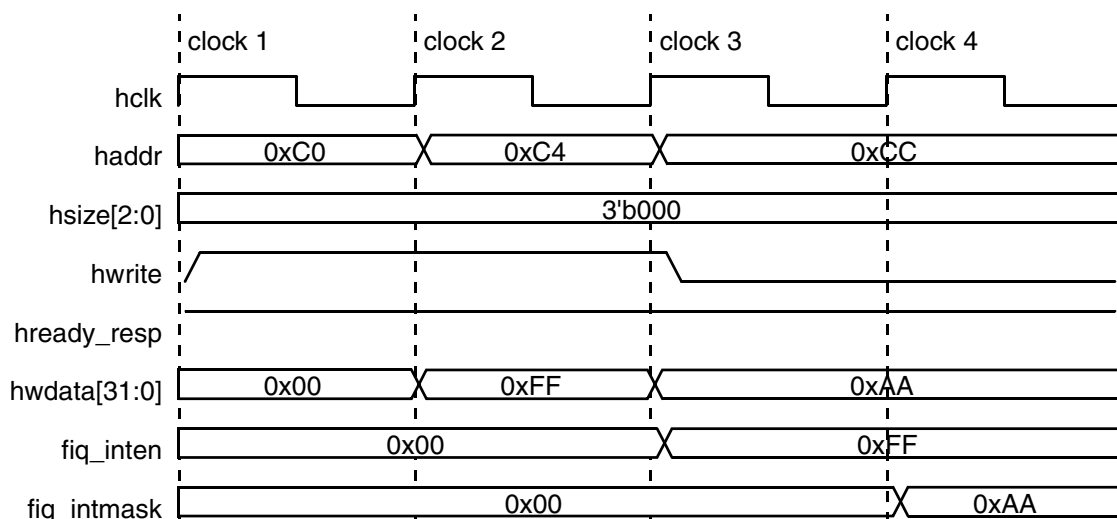
Figure 10-1 AHB Read



10.4 Write Accesses

When writing to a register, bit locations larger than the register width or allocation are ignored. Only pertinent bits are written to the register. Similar to the read case, a write access may be thought of as comprising a control and data cycle. As illustrated in the following figure, the address and control is driven from clock1 (control cycle), and the write data is driven by the bus from clock 2 (data cycle) and sampled by the destination register on clock 3.

Figure 10-2 AHB Write

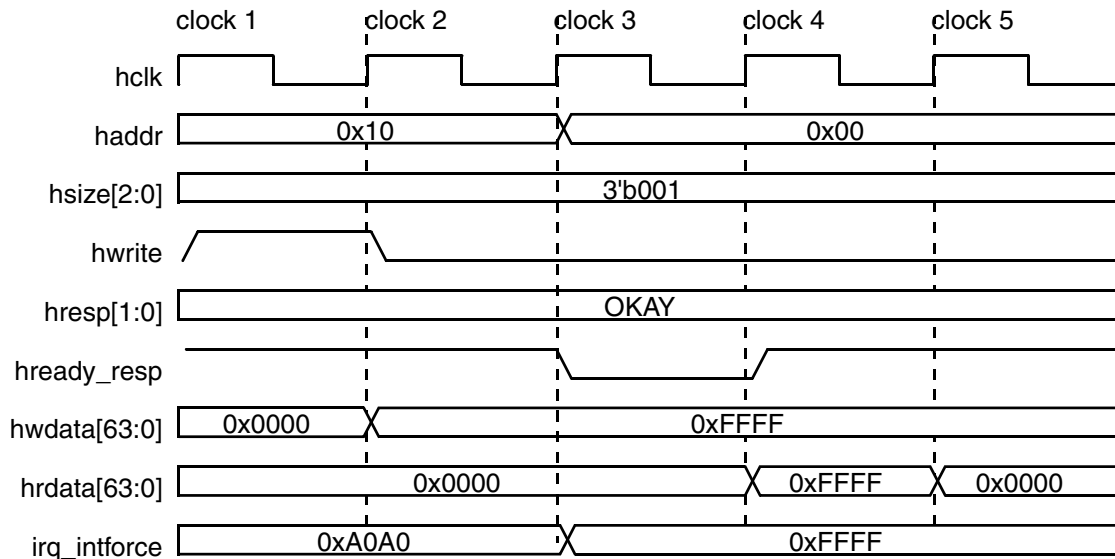


The operation of the AHB bus is pipelined, so while the write data for the first write is present on the bus for the slave to sample, the control for the next write is present on the bus for the slave to sample.

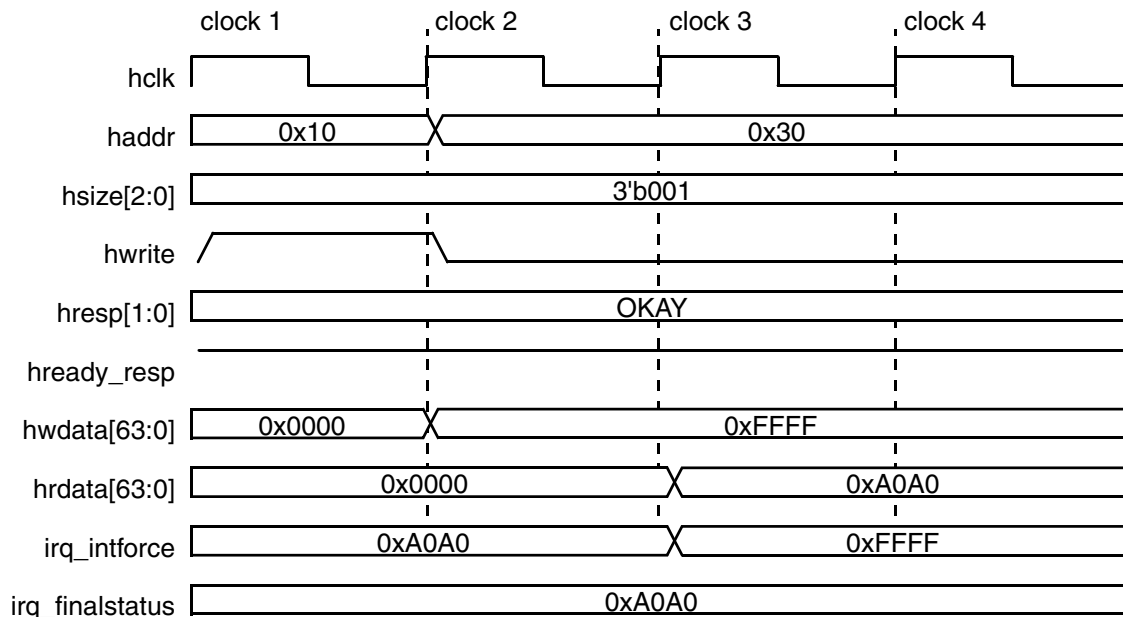
10.5 Consecutive Write-Read

This is a specific case for the AHB slave interface. The AMBA specification says that for a read after a write to the same address, the newly written data must be read back, not the old data. To comply with this, the slave interface in the DW_ahb_ictl inserts a “wait state” when it detects a read immediately after a write to the same address. As shown in the following figure, the control for a write is driven on clock 1, followed by the write data and the control for a read from the same address on clock 2.

Figure 10-3 AHB Wait State Read/Write



Sensing the read after a write to the same address, the slave interface drives `hready_resp` low from clock 3; `hready_resp` is driven high on clock 4 when the new write data can be read; and the bus samples `hready_resp` high on clock 5 and reads the newly written data. The following figure shows a normal consecutive write-read access.

Figure 10-4 AHB Consecutive Read/Write

10.6 Accessing Top-level Constraints

To get SDC constraints out of coreConsultant, you need to first complete the synthesis activity and then use the “write_sdc” command to write out the results:

1. This cC command sets synthesis to write out scripts only, without running DC:

```
set_activity_parameter Synthesize ScriptsOnly 1
```

2. This cC command autocompletes the activity:

```
autocomplete_activity Synthesize
```

3. Finally, this cC command writes out SDC constraints:

```
write_sdc <filename>
```

10.7 Coherency

Coherency is where bits within a register are logically connected. For instance, part of a register is read at time 1 and another part is read at time 2. Being coherent means that the part read at time 2 is at the same value it was when the register was read at time 1. The unread part is stored into a shadow register and this is read at time 2. When there is no coherency, no shadow registers are involved.

A bus master may need to be able to read the contents of a register, regardless of the data bus width, and be guaranteed of the coherency of the value read. A bus master may need to be able to write a register coherently regardless of the data bus width and use that register only when it has been fully programmed. This may need to be the case regardless of the relationship between the clocks.

Coherency enables a value to be read that is an accurate reflection of the state of the counter, independent of the data bus width, the counter width, and even the relationship between the clocks. Additionally, a value written in one domain is transferred to another domain in a seamless and coherent fashion.

Throughout this appendix the following terms are used:

- **Writing.** A bus master programs a configuration register. An example is programming the load value of a counter into a register.
- **Transferring.** The programmed register is in a different clock domain to where it is used, therefore, it needs to be transferred to the other clock domain.
- **Loading.** Once the programmed register is transferred into the correct clock domain, it needs to be loaded or used to perform its function. For example, once the load value is transferred into the counter domain, it gets loaded into the counter.

10.7.1 Writing Coherently

Writing coherently means that all the bits of a register can be written at the same time. A peripheral may have programmable registers that are wider than the width of the connected APB data bus, which prevents all the bits being programmed at the same time unless additional coherency circuitry is provided.

The programmable register could be the load value for a counter that may exist in a different clock domain. Not only does the value to be programmed need to be coherent, it also needs to be transferred to a different clock domain and then loaded into the counter. Depending on the function of the programmable register, a qualifier may need to be generated with the data so that it knows when the new value is currently transferred and when it should be loaded into the counter.

Depending on the system and on the register being programmed, there may be no need for any special coherency circuitry. One example that requires coherency circuitry is a 32-bit timer within an 8-bit APB system. The value is entirely programmed only after four 8-bit wide write transfers. It is safe to transfer or use the register when the last byte is currently written. An example where no coherency is required is a 16-bit wide timer within a 16-bit APB system. The value is entirely programmed after a single 16-bit wide write transfer.

Coherency circuitry enables the value to be loaded into the counter only when fully programmed and crossed over clock domains if the peripheral clock is not synchronous to the processor clock. While the load register is being programmed, the counter has access to the previous load value in case it needs to reload the counter.

Coherency circuitry is only added in cores where it is needed. The coherency circuitry incorporates an upper byte method that requires users to program the load register in LSB to MSB order when the peripheral width is smaller than the register width. When the upper byte is programmed, the value can be transferred and loaded into the load register. When the lower bytes are being programmed, they need to be stored in shadow registers so that the previous load register is available to the counter if it needs to reload. When the upper byte is programmed, the contents of the shadow registers and the upper byte are loaded into the load register.

The upper byte is the top byte of a register. A register can be transferred and loaded into the counter only when it has been fully programmed. A new value is available to the counter once this upper byte is written into the register. The following table shows the relationship between the register width and the peripheral bus width for the generation of the correct upper byte. The numbers in the table represent bytes, Byte 0 is the LSB and Byte 3 is the MSB. NCR means that no coherency circuitry is required, as the entire register is written with one access.

Table 10-5 Upper Byte Generation

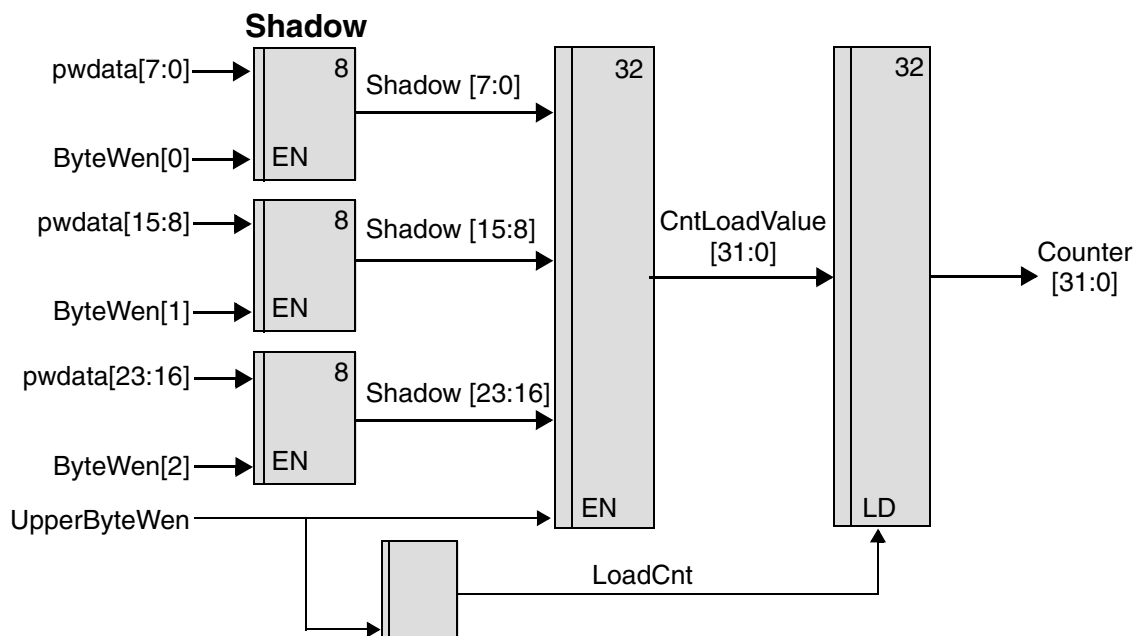
	Upper Byte Bus Width		
Load Register Width	8	16	32
1 - 8	NCR	NCR	NCR
9 - 16	1	NCR	NCR
17 - 24	2	2	NCR
25 - 32	3	2 (or 3)	NCR

There are three relationship cases to be considered for the processor and peripheral clocks:

- Identical
- Synchronous (phase coherent but of an integer fraction)
- Asynchronous

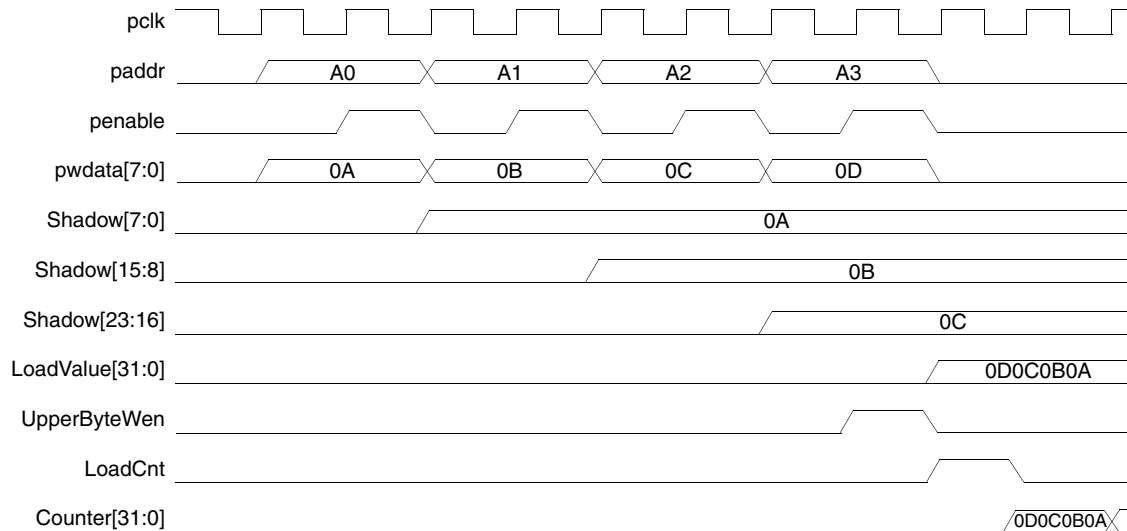
10.7.1.1 Identical Clocks

The following figure illustrates an RTL diagram for the circuitry required to implement the coherent write transaction when the APB bus clock and peripheral clocks are identical.

Figure 10-5 Coherent Loading – Identical Synchronous Clocks

The following figure shows a 32-bit register that is written over an 8-bit data bus, as well as the shadow registers being loaded and then loaded into the counter when fully programmed. The LoadCnt signal lasts for one cycle and is used to load the counter with CntLoadValue.

Figure 10-6 Coherent Loading – Identical Synchronous Clocks



Each of the bytes that make up the load register are stored into shadow registers until the final byte is written. The shadow register is up to three bytes wide. The contents of the shadow registers and the final byte are transferred into the CntLoadValue register when the final byte is written. The counter uses this register to load/initialize itself. If the counter is operating in a periodic mode, it reloads from this register each time the count expires.

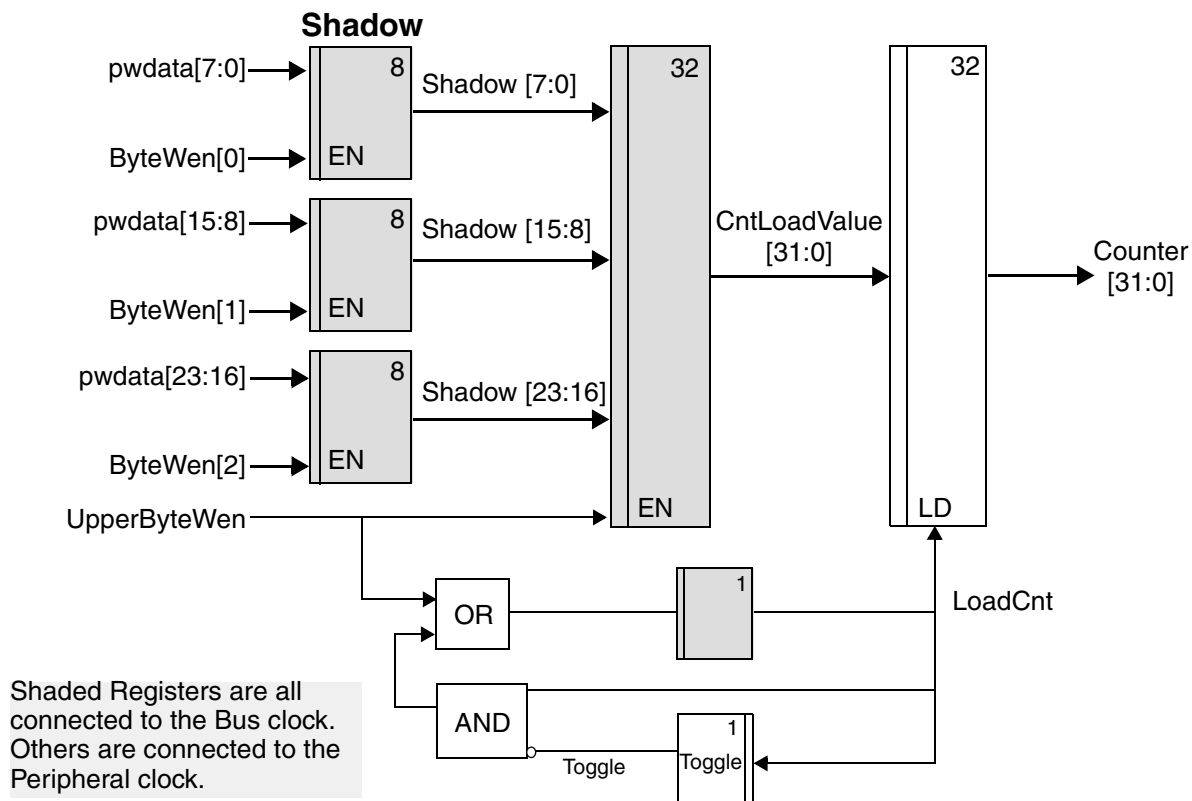
By using the shadow registers, the CntLoadValue is kept stable until it can be changed in one cycle. This allows the counter to be loaded in one access and the state of the counter is not affected by the latency in programming it. When there is a new value to be loaded into the counter initially, this is signaled by LoadCnt = 1. After the upper byte is written, the LoadCnt goes to zero.

10.7.1.2 Synchronous Clocks

When the clocks are synchronous but do not have identical periods, the circuitry needs to be extended so that the LoadCnt signal is kept high until a rising edge of the counter clock occurs. This extension is necessary so that the value can be loaded, using LoadCnt, into the counter on the first counter clock edge. At the rising edge of the counter clock if LoadCnt is high, then a register clocked with the counter clock toggles, otherwise it keeps its current value. A circuit detecting the toggling is used to clear the original LoadCnt by looking for edge changes. The value is loaded into the counter when a toggle has been detected. Once it is loaded, the counter should be free to increment or decrement by normal rules.

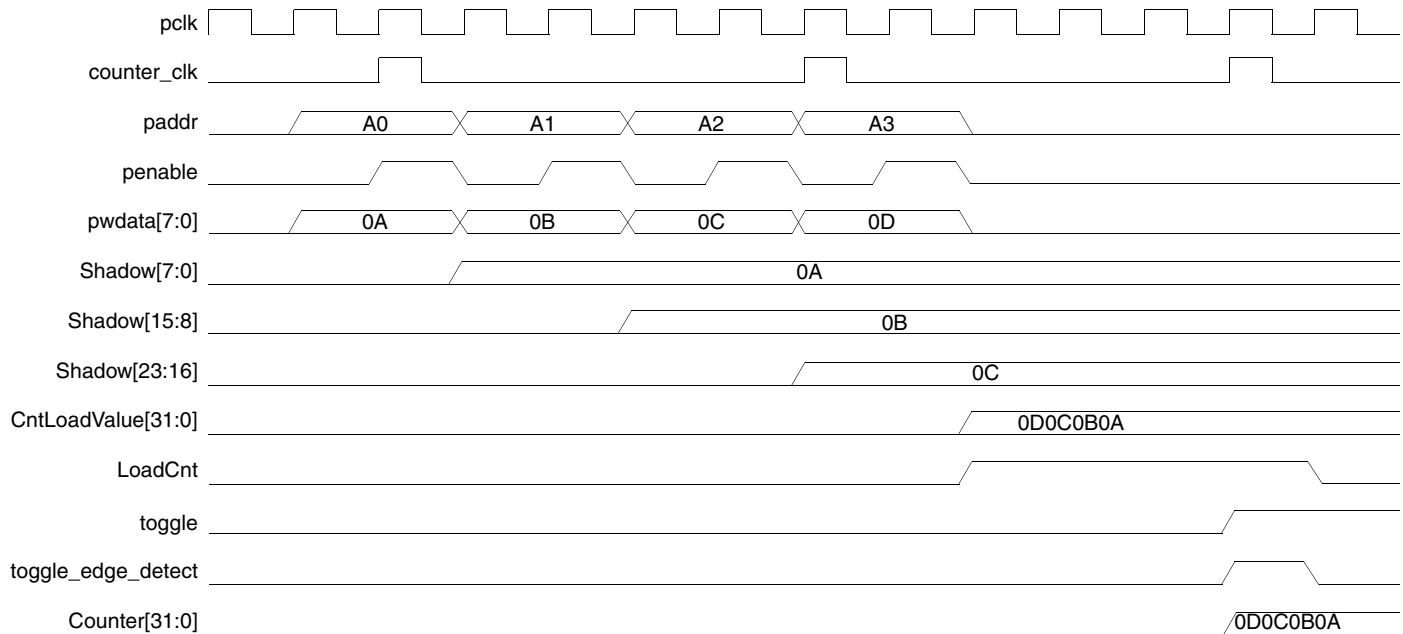
The following figure shows an RTL diagram for the circuitry required to implement the coherent write when the bus and peripheral clocks are synchronous.

Figure 10-7 Coherent Loading – Synchronous Clocks



The following figure shows a 32-bit register being written over an 8-bit data bus, as well as the shadow registers being loaded and then loaded into the counter when fully programmed. The LoadCnt signal is extended until a change in the toggle is detected and is used to load the counter.

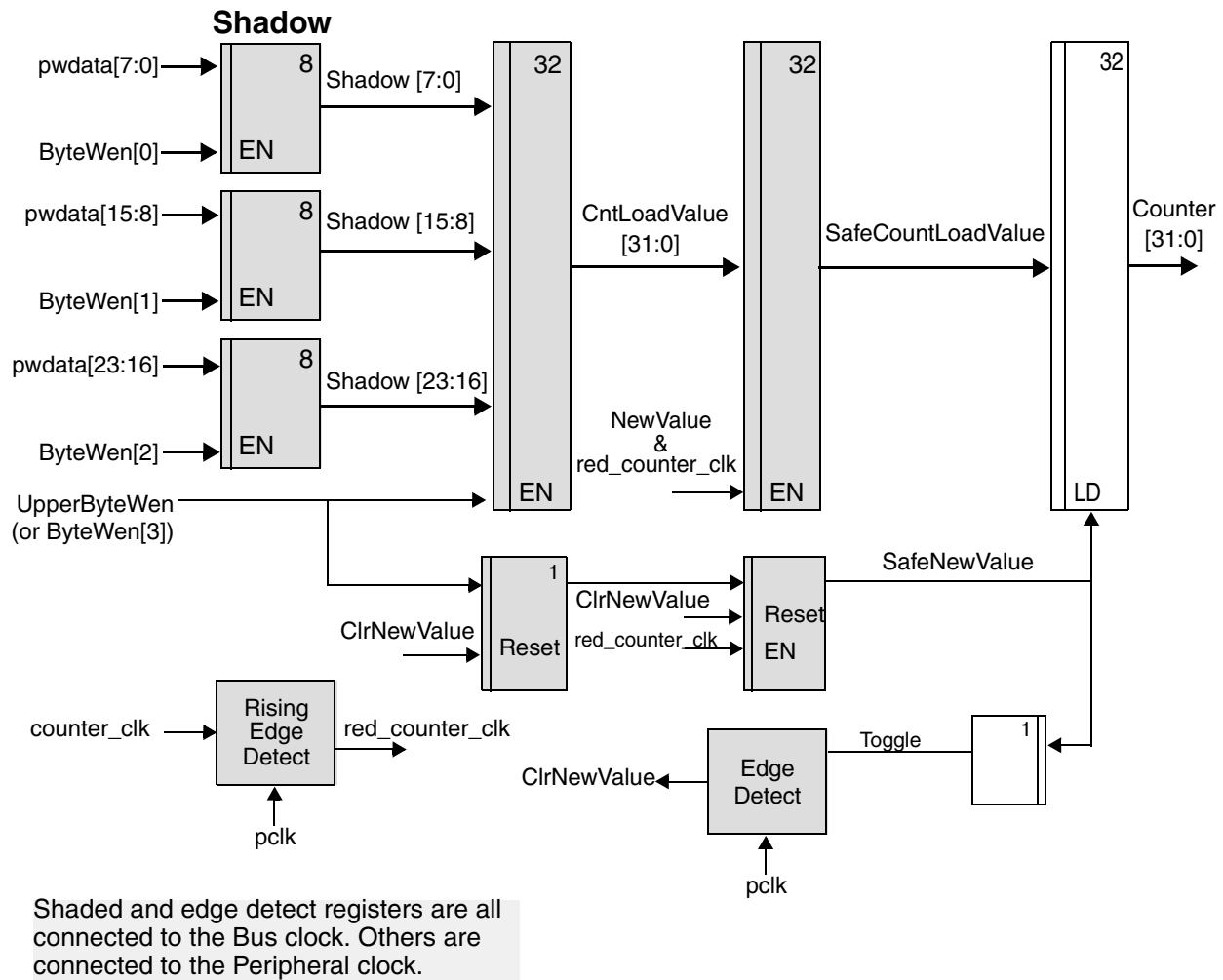
Figure 10-8 Coherent Loading – Synchronous Clocks



10.7.1.3 Asynchronous Clocks

When the clocks are asynchronous, the processor clock needs to be three-times the speed of the peripheral clock for the re-timing to operate correctly. The high pulse time of the peripheral clock needs to be greater than the period of the processor clock. The following figure shows an RTL diagram for the circuitry required to implement the coherent write when the bus and peripheral clocks are asynchronous.

Figure 10-9 Coherent Loading – Asynchronous Clocks



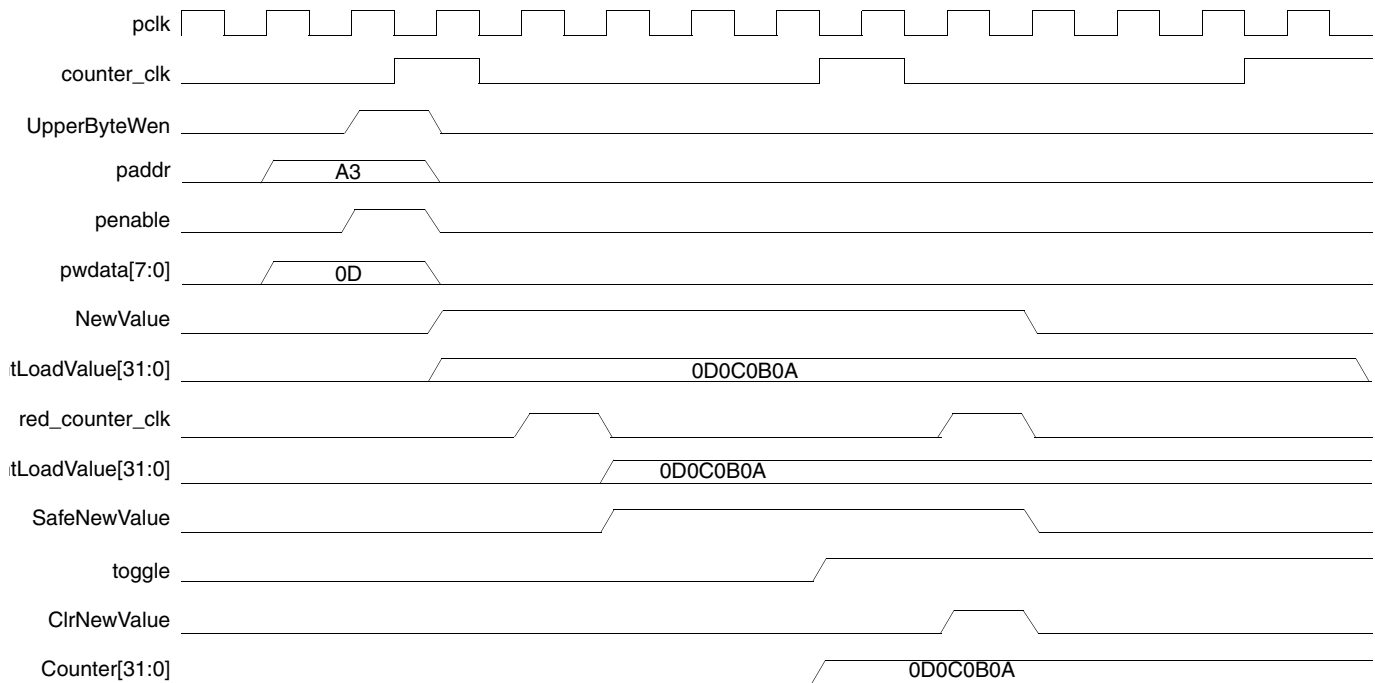
When the clocks are asynchronous, you need to transfer the contents of the register from one clock domain to another. It is not desirable to transfer the entire register through meta-stability registers, as coherency is not guaranteed with this method. The circuitry needed requires the processor clock to be used to re-time the peripheral clock. Upon a rising edge of the re-timed clock, the new value signal, NewValue, is transferred into a safe new value signal, SafeNewValue, which happens after the edge of the peripheral clock has occurred.

Every time there is a rising edge of the peripheral clock detected, the CntLoadValue is transferred into a SafeCntLoadValue. This value is used to transfer the load value across the clock domains. The SafeCntLoadValue only changes a number of bus clock cycles after the peripheral clock edge changes. A

counter running on the peripheral clock is able to use this value safely. It could be up to two peripheral clock periods before the value is loaded into the counter. Along with this loaded value, there also is a single bit transferred that is used to qualify the loading of the value into the counter.

The timing diagram depicted in the following figure does not show the shadow registers being loaded. This is identical to the loading for the other clock modes.

Figure 10-10 Coherent Loading – Asynchronous Clocks



The NewValue signal is extended until a change in the toggle is detected and is used to update the safe value. The SafeNewValue is used to load the counter at the rising edge of the peripheral clock. Each time a new value is written the toggle bit is flipped and the edge detection of the toggle is used to remove both the NewValue and the SafeNewValue.

10.7.2 Reading Coherently

For writing to registers, an upper-byte concept is proposed for solving coherency issues. For read transactions, a lower-byte concept is required. The following table provides the relationship between the register width and the bus width for the generation of the correct lower byte.

Table 10-6 Lower Byte Generation

	Lower Byte Bus Width		
Counter Register Width	8	16	32
1 - 8	NCR	NCR	NCR
9 - 16	0	NCR	NCR

Table 10-6 Lower Byte Generation

	Lower Byte Bus Width		
17 - 24	0	0	NCR
25 - 32	0	0	NCR

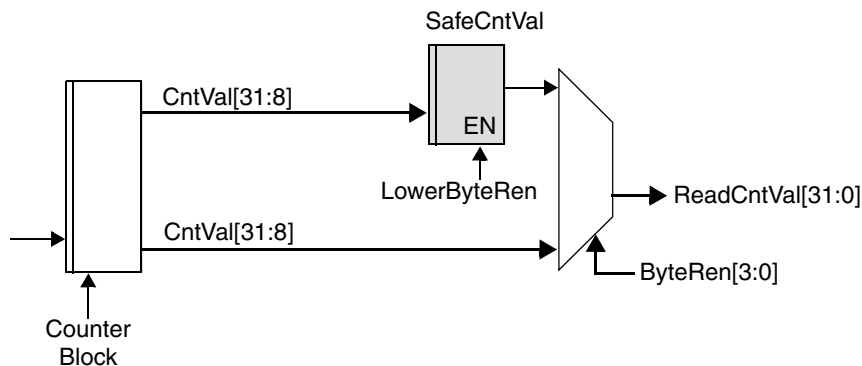
Depending on the bus width and the register width, there may be no need to save the upper bits because the entire register is read in one access, in which case there is no problem with coherency. When the lower byte is read, the remaining upper bytes within the counter register are transferred into a holding register. The holding register is the source for the remaining upper bytes. Users must read LSB to MSB for this solution to operate correctly. NCR means that no coherency circuitry is required, as the entire register is read with one access.

There are two cases regarding the relationship between the processor and peripheral clocks to be considered as follows:

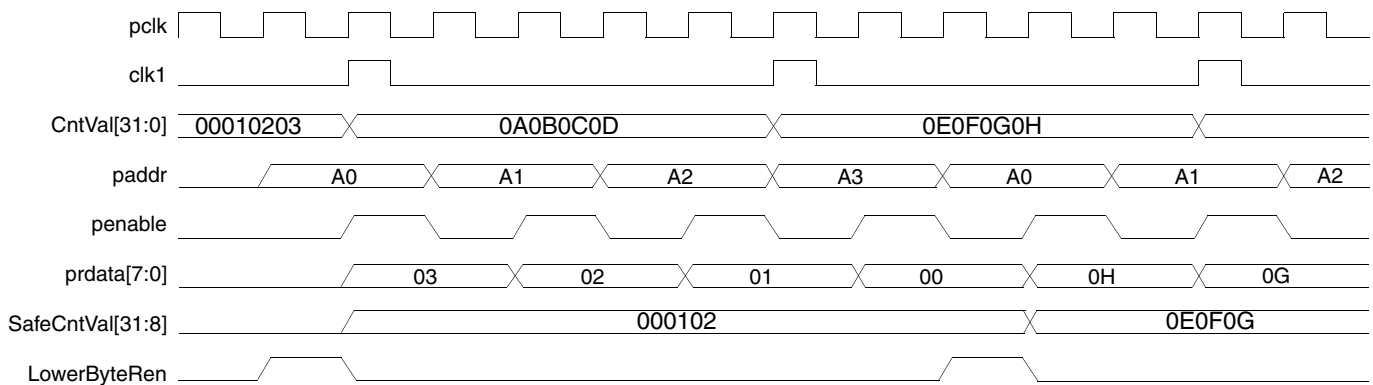
- Identical and/or synchronous
- Asynchronous

10.7.2.1 Synchronous Clocks

When the clocks are identical and/or synchronous, the remaining unread bits (if any) need to be saved into a holding register once a read is started. The first read byte must be the lower byte provided in the previous table, which causes the other bits to be moved into the holding register, *SafeCntVal*, provided that the register cannot be read in one access. The upper bytes of the register are read from the holding register rather than the actual register so that the value read is coherent. This is illustrated in the following figure and in the timing diagram after it.

Figure 10-11 Coherent Registering – Synchronous Clocks

Shaded registers are clocked with the processor clock.

Figure 10-12 Coherent Registering – Synchronous Clocks

10.7.2.2 Asynchronous Clocks

When the clocks are asynchronous, the processor clock needs to be three times the speed of the peripheral clock for the re-timing to operate correctly. The high pulse time of the peripheral clock needs to be greater than the period of the processor clock.

To safely transfer a counter value from the counter clock domain to the bus clock domain, the counter clock signal should be transferred to the bus clock domain. When the rising edge detect of this re-timed counter clock signal is detected, it is safe to use the counter value to update a shadow register that holds the current value of the counter.

While reading the counter contents it may take multiple APB transfers to read the value.



Note

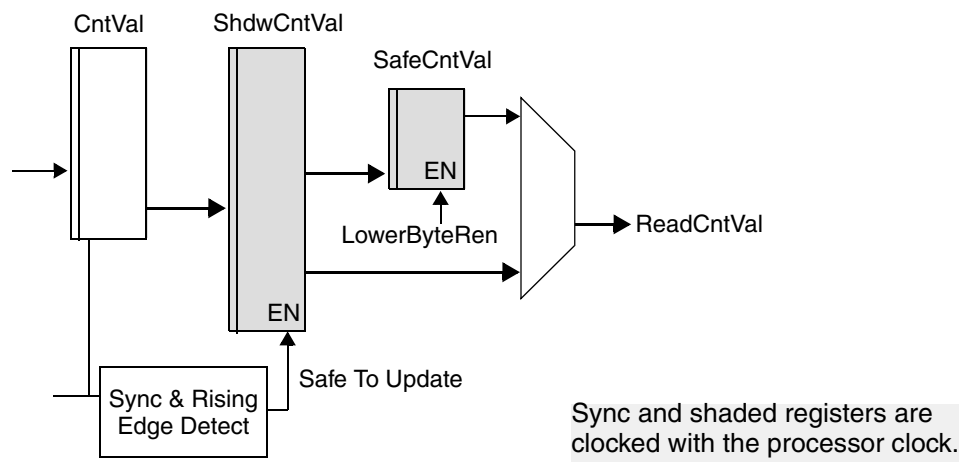
You must read LSB to MSB when the bus width is narrower than the counter width.

Once a read transaction has started, the value of the upper register bits need to be stored into a shadow register so that they can be read with subsequent read accesses. Storing these upper bits preserves the coherency of the value that is being read. When the processor reads the current value it actually reads the contents of the shadow register instead of the actual counter value. The holding register is read when the bus width is narrower than the counter width. When the LSB is read, the value comes from the shadow register; when the remaining bytes are read they come from the holding register. If the data bus width is wide enough to read the counter in one access, then the holding registers do not exist.

The counter clock is registered and successively pipelined to sense a rising edge on the counter clock. Having detected the rising edge, the value from the counter is known to be stable and can be transferred into the shadow register. The coherency of the counter value is maintained before it is transferred, because the value is stable.

The following figure illustrates the synchronization of the counter clock and the update of the shadow register.

Figure 10-13 Coherency and Shadow Registering – Asynchronous Clocks



A

Error and Warning Messages

This appendix lists and describes errors and warnings that you may encounter when configuring, verifying, or synthesizing the DesignWare component using the coreConsultant GUI.

A.1 Warnings During Simulation

When the Vera testbench is compiled, multiple copies of warnings similar to the following are generated:

```
Warning: call to potentially undefined super method "run" at  
        <line number> in <file name>"
```

These messages can be ignored. The warning occurs because the run() task in the super class is defined as a virtual task, and the super class is an abstract class. This means that run() does not have to be defined. However, the super class always has run() defined, and the compiler should know this.

A.2 Warnings During Synthesis

If you use your own synthesis scripts and perform the ungroup command in Design Compiler (DC) on the design, DC generates warning messages that indicate that some of the DesignWare cells cannot be ungrouped. These messages are generated because few of the DesignWare components used in the design (like W_ram_r_w_s_dff) have no hierarchy and cannot be ungrouped further.

You can ignore these messages.

B

Channel Locking and Deadlock

This appendix explains how locking can cause deadlock. Deadlock situations can occur where multiple channels are enabled concurrently and no channel can proceed with its DMA transfer. This will only occur for configurations where `DMAH_NUM_MASTER_INT > 1` and `DMAH_NUM_CHANNELS > 1`.

The methods used to avoid deadlock are:

- Hardware will detect when a potential deadlock occurs between two channels and deal with it. Refer to [“Deadlock Prevention by Hardware”](#) on page 318.
- Programming restrictions to avoid potential deadlock situations. Refer to [“Programming Restrictions to Avoid Deadlock”](#) on page 319.



Refer to [“Channel Locking”](#) on page 103 for details of the actual value of channel locking enable and channel locking level used in the DMA transfer. These may differ from the programmed values, `CFGx_LOCK_CH` and `CFGx.LOCK_CH_L`. All references to channel locking enable and channel locking level in this section refer to the actual values used and not the programmed values.

B.1 Hardware Detection of Deadlock

Three cases are outlined that can potentially cause a deadlock situation.

B.1.1 Case 1

Consider a channel programmed as follows:

B.1.1.1 Channel 0

CTL0.SMS = 0	Source on AHB Layer 0
CTL0.DMS = 1	Destination on AHB Layer 1
CFG0.LOCK_CH_L = 0	Locking over complete DMA transfer
CFG0.LOCK_CH = 1	Channel Locking enabled

B.1.1.2 Channel 1

CTL1.SMS = 1	Source on AHB Layer 1
CTL1.DMS = 0	Destination on AHB Layer 0
CFG1.LOCK_CH_L = 0	Locking over complete DMA transfer
CFG1.LOCK_CH = 1	Channel Locking enabled

Consider the following scenario. Channel 0 and Channel 1 are both enabled. The source of Channel 0 is granted ownership of Layer 0 and then locks out all other channel access to Layer 0 until it completes the DMA transfer. Before the Channel 0 destination requests Layer 1, the source of Channel 1 requests and is granted Layer 1. Channel 1 now locks all other channel accesses to Layer 1.

This is a deadlock situation whereby neither Channel 1 nor Channel 0 can complete because neither channel destination can gain access to its respective AHB layer.

B.1.1.3 Case 1 potential deadlock

Hardware flags a potential deadlock situation between two channels (channels x and y) if:

- Both channels are enabled
- $CTL_x.SMS = CTL_y.DMS$
- $CTL_y.SMS = CTL_x.DMS$
- Both channels have channel locking enabled

B.1.2 Case 2

This case occurs only when channel locking is enabled at the DMA transaction level.

B.1.2.1 Case 2 potential deadlock

Hardware flags a potential deadlock situation between two channels if:

1. Both channels are enabled
2. $CTLx.SMS = CTLy.SMS$
3. $CTLx.DMS = CTLy.DMS$
4. $CTLx.SMS \neq CTLx.DMS$ and $CTLy.SMS \neq CTLy.DMS$
5. Both channels have channel locking enabled
6. Either channel x or channel y has channel locking enabled at the transaction level

B.1.3 Case 3

Another potential deadlock occurs when $DMAH_NUM_MASTER_INT > 2$ && $DMAH_NUM_CHANNELS > 2$. The following shows two examples when channel locking is enabled at any level and where a potential deadlock situation can occur:

B.1.3.1 Example 1

$CTL0.SMS = 0$	Channel 0 Source on AHB Layer 0
$CTL0.DMS = 1$	Channel 0 Destination on AHB Layer 1
$CTL1.SMS = 1$	Channel 1 Source on AHB Layer 1
$CTL1.DMS = 2$	Channel 1 Destination on AHB Layer 2
$CTL2.SMS = 2$	Channel 2 Source on AHB Layer 2
$CTL2.DMS = 0$	Channel 2 Destination on AHB Layer 0

For example, suppose all three channels employ channel locking at the block transfer level. If the source of all three channels start before any of the destinations start, then a deadlock situation will arise.

1. Layer 0 locked by the source of channel 0
2. Layer 1 locked by the source of channel 1
3. Layer 2 locked by the source of channel 2

Then:

1. The destination of channel 0 cannot proceed as Layer 1 is locked by channel 1
2. The destination of channel 1 cannot proceed as Layer 2 is locked by channel 2
3. The destination of channel 2 cannot proceed as Layer 0 is locked by channel 0

This is a deadlocked situation.

B.1.3.2 Example 2

CTL0.SMS = 0	Channel 0 Source on AHB Layer 0
CTL0.DMS = 1	Channel 0 Destination on AHB Layer 1
CTL1.SMS = 1	Channel 1 Source on AHB Layer 1
CTL1.DMS = 2	Channel 1 Destination on AHB Layer 2
CTL2.SMS = 2	Channel 2 Source on AHB Layer 2
CTL2.DMS = 3	Channel 2 Destination on AHB Layer 3
CTL3.SMS = 3	Channel 3 Source on AHB Layer 3
CTL3.DMS = 3	Channel 3 Destination on AHB Layer 3

This example is similar to “[Example 1](#)” on page 317. Suppose all four channels employ channel locking at the block transfer level. If the source of all four channels start before any of the destinations start, then a deadlock situation will arise.

B.1.3.3 Case 3 potential deadlock

Hardware flags a potential deadlock situation between two channels if:

1. DMAH_NUM_MASTER_INT > 2 && DMAH_NUM_CHANNELS > 2
2. Both channels are enabled
3. Both channels have channel locking enabled at any level
4. The channel source and destination are on different layers
5. The source of one channel is on the same layer as the destination of the other channel

B.1.4 Deadlock Prevention by Hardware

Hardware detects a potential deadlock situation between two channels, as given in [Case 1 potential deadlock](#), [Case 2 potential deadlock](#), and [Case 3 potential deadlock](#). It then stalls one channel until the enable bit of the other channel in the Channel Enable Register, [ChEnReg.Ch_En](#), is cleared by hardware. The channel that is first enabled by software is allowed to complete its DMA transfer before the other channel can begin its DMA transfer. In this case, a lower priority channel can be given precedence over a higher one if it is enabled first over the AHB slave interface; for example, the programmed channel priorities are overridden.



Note

This preventative method of avoiding deadlock assumes that no two channels are enabled by software at the same time. To avoid enabling two or more channels at the same time, writes to ChEnReg.ChEn should enable only one channel per write. Also, when enabling the DMAC with a write to DmaCfgReg.DMA_EN, there should be no more than one ChEnReg.ChEn bit already enabled.

B.2 Programming Restrictions to Avoid Deadlock

The previous section details scenarios where hardware detects potential deadlock situations and takes preventative action. This section details scenarios where hardware does not detect potential deadlock situations, but it is left up to software to avoid programming the DW_ahb_dmac in a way that may cause deadlock. Failure to comply with these programming restrictions may lead to deadlock.

1. If
 - a. Channel locking is enabled at the DMA transfer level
 - b. The `LLPx.LOC` field is non-zero and
 - c. Block chaining is enabled on the source or destination side.

In equation form, this is:

```
((CFGx.LOCK_B == 1 && CFG.LOCK_B_L == 2'b00) ||
 (CFGx.LOCK_CH == 1 && CFG.LOCK_CH_L == 2'b00)) &&
 (LLPx.LOC != 0) &&
 ((CTLx.LLP_SRC_EN == 1) || ((CTLx.LLP_DST_EN == 1))
```

then the following condition must be complied with:

```
(LLPx.LMS == CTLx.SMS) || (LLPx.LMS == CTLx.DMS)
```

2. If:

`CTLx.SMS != CTLx.DMS && DMAH_NUM_MASTER_INT > 2 && DMAH_NUM_CHANNELS > 2`
 then channel transaction level locking is prohibited. Software must ensure that if channel locking is enabled, then the locking level used for channel locking is not at the transaction level.
3. If the DMA block size in bytes is less than or equal to the FIFO depth in bytes (the source block can complete before the destination block starts) and if the source and destination are on different layers, only transaction-level locking is allowed for channel locking.

C

DW_ahb_dmac Application Notes

This appendix provides application notes for the DW_ahb_dmac.

C.1 Interoperability Between DW_ahb_dmac and PrimeCell Hardware Handshaking Interface

You may have already designed a peripheral with a handshaking interface that conforms to the ARM PrimeCell handshaking interface for use with the ARM PrimeCell DMA. This handshaking interface can be mapped to conform with the DW_ahb_dmac handshaking interface with glue logic, as shown in [Figure C-1](#) through [Figure C-3](#). [Table C-1](#) lists PrimeCell handshaking signals.

Table C-1 PrimeCell Handshaking Signals

ARM PrimeCell DMA Handshaking Signal	Direction Relative to Peripheral	Description
dmacclr	Input	DMA request acknowledge clear.
dmactc	Input	DMA terminal count. Indicates the transaction is complete and the block of data is transferred.
dmacbreq	Output	DMA burst transfer request.
dmacsreq	Output	DMA single transfer request.
dmaclbreq	Output	DMA last burst transfer request.
dmacslreq	Output	DMA last single transfer request.

Figure C-1 Mapping Between ARM PrimeCell and DW_ahb_dmac Peripheral Handshaking – Peripheral (source or destination) is Flow Controller

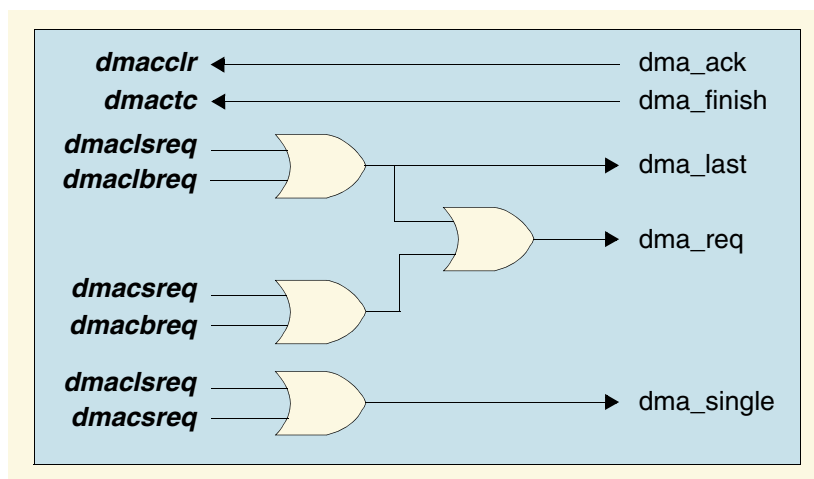


Figure C-2 Mapping Between ARM PrimeCell and DW_ahb_dmac Source Peripheral Handshaking – Source Peripheral Not Flow Controller

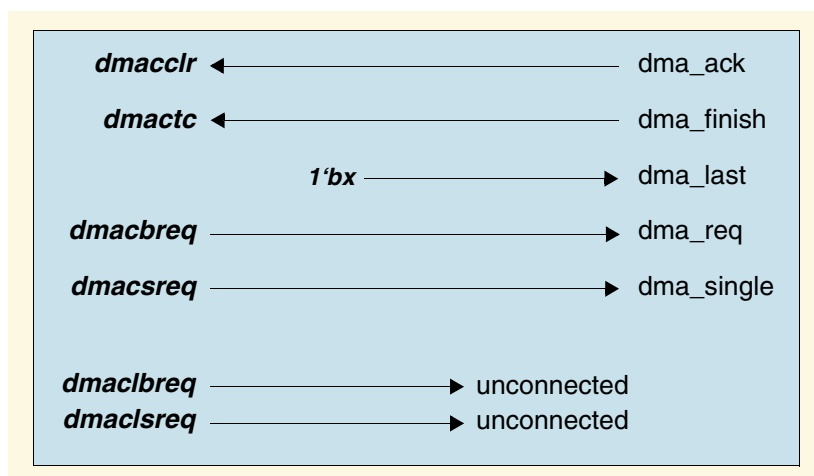
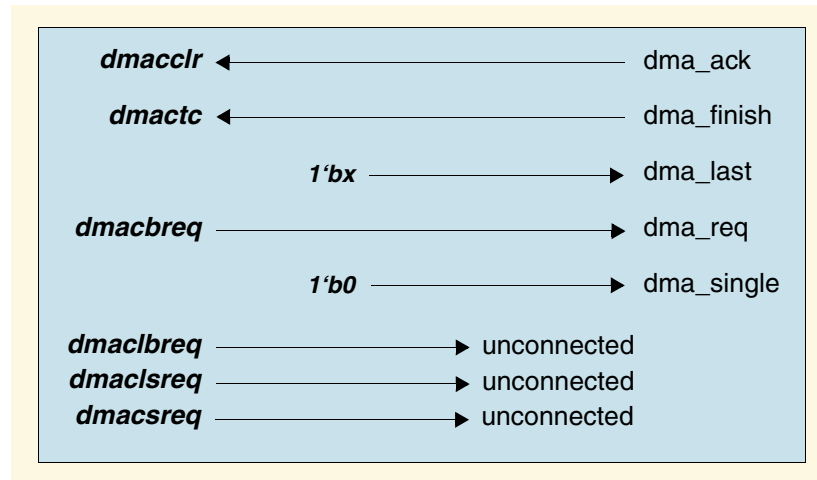
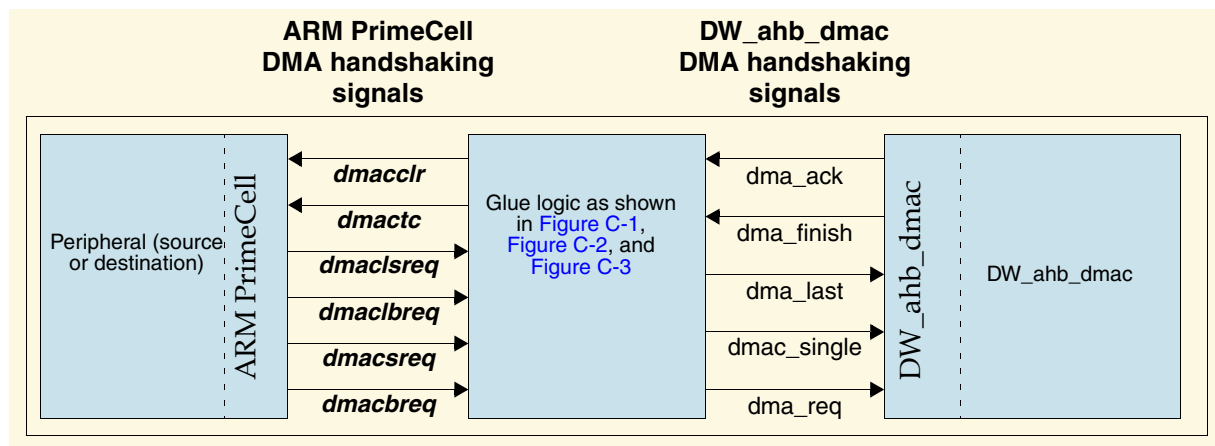


Figure C-3 Mapping Between ARM PrimeCell and DW_ahb_dmac Destination Peripheral Handshaking – Destination Peripheral Not Flow Controller



This glue logic may be used to allow a peripheral designed with the ARM PrimeCell handshaking interface to operate with DW_ahb_dmac. This logic is shown in the [Figure C-4](#).

Figure C-4 DMA Operation Using ARM PrimeCell Handshaking Interface with DW_ahb_dmac



C.2 Mapping of PrimeCell Software Handshaking Registers to DW_ahb_dmac

This application note describes the mapping of the PrimeCell software handshaking registers to the equivalent DW_ahb_dmac software handshaking interface registers. Software that is used to control the PrimeCell DMA Controller software interface registers can be changed to conform to the DW_ahb_dmac software interface by the mapping described in the following sections.

C.2.1 PrimeCell Software Handshaking Registers

The ARM PrimeCell DMA Controller (PL080 and PL081) has four software registers that are used to control the DMA handshaking. These registers allow DMA requests to be generated by software.

- DMACSoftBReq (16 bits) – Generates DMA burst requests
- DMACSoftSReq (16 bits) – Generates DMA single requests

- DMACSoftLBReq (16 bits) – Generates DMA last burst requests
- DMACSoftLSReq (16 bits) – Generates DMA last single requests

A DMA request can be generated for a peripheral (Source or Destination) by writing a 1 to the corresponding register bit of the previous registers.

C.2.2 DW_ahb_dmac Software Handshaking Registers

DW_ahb_dmac has six registers allocated for software handshaking:

- ReqSrcReg (64 bits) – Generates DMA requests for source peripheral
- ReqDstReg (64 bits) – Generates DMA requests for destination peripheral
- SglReqSrcReg (64 bits) – Generates DMA single requests for source peripheral
- SglReqDstReg (64 bits) – Generates DMA single requests for destination peripheral
- LstSrcReg (64 bits) – Generates DMA last requests for source peripheral
- LstDstReg (64 bits) – Generates DMA last requests for destination peripheral

A DMA request can be generated for a source or destination peripheral by writing 1 to the “request” and “request write enable” bits of the respective channel bits of the previous registers. For the exact register bit encoding of these software handshaking interface registers, refer to [“Software Handshaking Registers”](#) on page 198 in this databook and to the PrimeCell DMAC databook for the PrimeCell registers.

C.2.3 Register Interface Mapping

This section provides the mapping of the ARM PrimeCell DMAC handshaking registers to DW_ahb_dmac software handshaking registers when the peripheral (source or destination is flow controller) is not the flow controller and when the peripheral is the flow controller.

C.2.3.1 Peripheral is Not Flow Controller

[Table C-2](#) describes the mapping of the DW_ahb_dmac software handshaking registers with respect to the PrimeCell DMAC handshaking registers when the peripheral is not the flow controller. The mapping is true

for the source peripheral when it is not the flow controller, and the destination peripheral when it is not the flow controller.

Table C-2 Mapping of PrimeCell Handshaking Registers to DW_ahb_dmac DMA is Flow Controller

Operation	ARM PrimeCell	DW_ahb_dmac	Notes
Burst transaction outside Single Transaction Region	Write to DMACSoftBReq	SglReqSrcReg / SglReqDstReg followed by ReqSrcReg / ReqDstReg OR ReqSrcReg / ReqDstReg followed by SglReqSrcReg / SglReqDstReg	When converting from PrimeCell programming for writes to DMACSoftBReq, a corresponding write is required to ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. Writing a 1 to the ReqSrcReg[x]/ReqDstReg[x] register is always interpreted as a burst transaction request, where x is the channel number. However, in order for a burst transaction request to start, software must write a 1 to the SglReqSrcReg[x]/SglReqDstReg[x] register.
Single transaction Inside the Single Transaction Region	Write to DMACSoftSReq	Write to SglReqSrcReg / SglReqDstReg	When converting from PrimeCell programming for writes to DMACSoftSReq, a corresponding write is required on SglReqSrcReg or SglReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. This is only true when a block transfer to the peripheral is in the Single Transaction Region .
Single transaction Outside the Single Transaction Region	DMACSoftSReq	No equivalent	When the peripheral is not the flow controller, then a single transaction cannot be initiated outside the Single Transaction Region in the DW_ahb_dmac.

Table C-2 Mapping of PrimeCell Handshaking Registers to DW_ahb_dmac DMA is Flow Controller (Continued)

Operation	ARM PrimeCell	DW_ahb_dmac	Notes
Burst transaction Inside the Single Transaction Region	Write to DMACSoftBReq	ReqSrcReg/ ReqDstReg followed by SglReqSrcReg/ SglReqDstReg	When converting from PrimeCell programming for writes to DMACSoftBReq, a corresponding write is required on ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. Writing a 1 to the ReqSrcReg[x]/ReqDstReg[x] register is always interpreted as a burst transaction request, where x is the channel number. However, in order for a burst transaction request to start, software must write a 1 to the SglReqSrcReg[x]/SglReqDstReg[x] register. The order of the writes to the ReqSrcReg/ReqDstReg and SglReqSrcReg/SglReqDstReg is important.

C.2.3.2 Peripheral is Flow Controller

[Table C-3](#) provides the mapping of the DW_ahb_dmac software handshaking registers with respect to the PrimeCell DMAC handshaking registers when (source or destination) peripheral is the flow controller.

Table C-3 Mapping of PrimeCell Handshaking Registers to DW_ahb_dmac Peripheral is Flow Controller

Operation	ARM PrimeCell	DW_ahb_dmac	Notes
Burst transaction	DMACSoftBReq	SglReqSrcReg/ SglReqDstReg followed by ReqSrcReg/ ReqDstReg	When converting from PrimeCell programming for writes to DMACSoftBReq, a corresponding write is required on ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. This initiates a transaction. The type of transaction, single or burst, depends on the value in the corresponding SglReqSrcReg/ SglReqDstReg register. If it is 0, it is a burst transaction.

Table C-3 Mapping of PrimeCell Handshaking Registers to DW_ahb_dmac Peripheral is Flow Controller

Operation	ARM PrimeCell	DW_ahb_dmac	Notes
Single transaction	DMACSoftSReq	SglReqSrcReg / SglReqDstReg followed by ReqSrcReg / ReqDstReg	When converting from PrimeCell programming for writes to DMACSoftSReq, a corresponding write is required on ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. This initiates a transaction. The type of transaction, single or burst, depends on the value in the corresponding SglReqSrcReg/SglReqDstReg register. If it is 1, it is a single transaction.
Last burst transaction	DMACSoftLBReq	LstSrcReg / LstDstReg followed by SglReqSrcReg / SglReqDstReg followed by ReqSrcReg / ReqDstReg ^a	When converting from PrimeCell programming for writes to DMACSoftLBReq, a corresponding write is required on ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. This initiates a transaction. The type of transaction, single or burst, depends on the value in the corresponding SglReqSrcReg/SglReqDstReg register. If it is 0, it is a burst transaction. Furthermore, the transaction is the last transaction in the block if the corresponding bit in the LstSrcReg/LstDstReg register is 1.

Table C-3 Mapping of PrimeCell Handshaking Registers to DW_ahb_dmac Peripheral is Flow Controller

Operation	ARM PrimeCell	DW_ahb_dmac	Notes
Last single transaction	DMACSoftLSReq	LstSrcReg/LstDstReg followed by SglReqSrcReg/ SglReqDstReg followed by ReqSrcReg/ ReqDstReg ^a	When converting from PrimeCell programming for writes to DMACSoftLSReq, a corresponding write is required on ReqSrcReg or ReqDstReg in DW_ahb_dmac, depending on the bit location (source or destination peripheral) in the DMACSoftBReq register. This initiates a transaction. The type of transaction, single or burst, depends on the value in the corresponding SglReqSrcReg/SglReqDstReg register. If it is 1, it is a single transaction. Furthermore, the transaction is the last transaction in the block if the corresponding bit in the LstSrcReg/LstDstReg register is 1.

a. The order before the write to the ReqSrcReg/ReqDstReg register is not important.

D

Configuring DW_ahb_dmac to Match ARM PrimeCell PL080/PL081

This appendix provides the configuration parameter settings for DW_ahb_dmac so that it matches or is equivalent to the ARM PrimeCell PL080 and PL081 devices.

D.1 ARM PL080 Equivalent

The ARM PL080 equivalent is a two-master, eight-channel device. The channel parameter settings are the same for all channels, so only one set of parameters is listed for channel x , where $x = 0$ to 7. The configuration parameters for the PL080 equivalent are described in [Table D-1](#).

D.2 ARM PL081 Equivalent

The PL081 ARM equivalent is a one-master, two-channel device. The parameter settings are the same as the PL080, except for the following two parameters:

- DMAH_NUM_MASTER_INT = 1
- DMAH_NUM_CHANNELS = 2

Again, the channel parameters are the same for all channels, so only one set of parameters is listed for channel x , where $x = 0$ to 2. For the configuration settings for the PL081 equivalent (except for number of masters and number of channels), refer to [Table D-1](#).

For more information about the configuration parameters for DW_ahb_dmac, refer to “[Parameters](#)” on page [119](#).

Table D-1 ARM PL080 Equivalent Configuration Settings

DW_ahb_dmac Configuration Parameter	Setting
DMAH_NUM_MASTER_INT	2
DMAH_NUM_CHANNELS	8
DMAH_NUM_HS_INT	16
DMAH_MABRST	0

Table D-1 ARM PL080 Equivalent Configuration Settings

DW_ahb_dmac Configuration Parameter	Setting
DMAH_RETURN_ERR_RESP	0
DMAH_INTR_POL	1
DMAH_INTR_IO	1
DMAH_BIG_ENDIAN	0
DMAH_S_HDATA_WIDTH	32
DMAH_M1_HDATA_WIDTH	32
DMAH_M2_HDATA_WIDTH	32
DMAH_M1_AHB_LITE	0
DMAH_M2_AHB_LITE	0
DMAH_ID_NUM	0x02080901
Channel Register (x = 0 to 7)	
DMAH_CHx_FIFO_DEPTH	16
DMAH_CHx_STAT_DST	0
DMAH_CHx_STAT_SRC	0
DMAH_CHx_MAX_MULT_SIZE	256
DMAH_CHx_MAX_BLK_SIZE	4095
DMAH_CHx_FC	3
DMAH_CHx_LOCK_EN	0
DMAH_CHx_SMS	4
DMAH_CHx_DMS	4
DMAH_CHx_LMS	4
DMAH_CHx_STW	0
DMAH_CHx_DTW	0
DMAH_CHx_SRC_NON_OK	1
DMAH_CHx_DST_NON_OK	1
DMAH_CHx_LLQ_NON_OK	1
DMAH_CHx_MULTI_BLK_EN	1
DMAH_CHx_HC_LLQ	0

Table D-1 ARM PL080 Equivalent Configuration Settings

DW_ahb_dmac Configuration Parameter	Setting
DMAH_CHx_SRC_GAT_EN	0
DMAH_CHx_DST_SCA_EN	0
DMAH_CHx_MULTI_BLK_TYPE	8
DMAH_CHx_CTL_WB_EN	0

E

Glossary

active command queue	Command queue from which a model is currently taking commands; see also command queue.
activity	A set of functions in coreConsultant that step you through configuration, verification, and synthesis of a selected core.
AHB	Advanced High-performance Bus — high-performance system backbone bus. AHB supports the efficient connection of processors, on-chip memories and off-chip external memory interfaces (ARM Limited specification).
AMBA	Advanced Microcontroller Bus Architecture — a trademarked name by ARM Limited that defines an on-chip communication standard for high speed microcontrollers.
APB	Advanced Peripheral Bus — optimized for minimal power consumption and reduced interface complexity to support peripheral functions (ARM Limited specification).
APB bridge	DW_apb submodule that converts protocol between the AHB bus and APB bus.
application design	Overall chip-level design into which a subsystem or subsystems are integrated.
arbiter	AMBA bus submodule that arbitrates bus activity between masters and slaves.
BFM	Bus-Functional Model — A simulation model used for early hardware debug. A BFM simulates the bus cycles of a device and models device pins, as well as certain on-chip functions. See also Full-Functional Model.
big-endian	Data format in which most significant byte comes first; normal order of bytes in a word.
blocked command stream	A command stream that is blocked due to a blocking command issued to that stream; see also command stream, blocking command, and non-blocking command.
blocking command	A command that prevents a testbench from advancing to next testbench statement until this command executes in model. Blocking commands typically return data to the testbench from the model.

bus bridge	Logic that handles the interface and transactions between two bus standards, such as AHB and APB. See APB bridge.
command channel	Manages command streams. Models with multiple command channels execute command streams independently of each other to provide full-duplex mode function.
command stream	The communication channel between the testbench and the model.
component	A generic term that can refer to any synthesizable IP or verification IP in the DesignWare Library. In the context of synthesizable IP, this is a configurable block that can be instantiated as a single entity (VHDL) or module (Verilog) in a design.
configuration	The act of specifying parameters for a core prior to synthesis; can also be used in the context of VIP.
configuration intent	Range of values allowed for each parameter associated with a reusable core.
core	Any configurable block of synthesizable IP that can be instantiated as a single entity (VHDL) or module (Verilog) in a design. Core is the preferred term for a big piece of IIP. Anything that requires coreConsultant for configuration, as well as anything in the DesignWare Cores library, is a core.
core developer	Person or company who creates or packages a reusable core. All the cores in the DesignWare Library are developed by Synopsys.
core integrator	Person who uses coreConsultant or coreAssembler to incorporate reusable cores into a system-level design.
coreAssembler	Synopsys product that enables automatic connection of a group of cores into a subsystem. Generates RTL and gate-level views of the entire subsystem.
coreConsultant	A Synopsys product that lets you configure a core and generate the design views and synthesis views you need to integrate the core into your design. Can also synthesize the core and run the unit-level testbench supplied with the core.
coreKit	An unconfigured core and associated files, including the core itself, a specified synthesis methodology, interfaces definitions, and optional items such as verification environment files and core-specific documentation.
cycle command	A command that executes and causes HDL simulation time to advance.
decoder	Software or hardware subsystem that translates from and “encoded” format back to standard format.
design context	Aspects of a component or subsystem target environment that affect the synthesis of the component or subsystem.
design creation	The process of capturing a design as parameterized RTL.
Design View	A simulation model for a core generated by coreConsultant.
DesignWare Synthesizable Components	The Synopsys name for the collection of AMBA-compliant coreKits and verification models delivered with DesignWare and used with coreConsultant or coreAssembler to quickly build DesignWare Synthesizable Component designs.

DesignWare cores	A specific collection of synthesizable cores that are licensed individually. For more information, refer to www.synopsys.com/designware .
DesignWare Library	A collection of synthesizable IP and verification IP components that is authorized by a single DesignWare license. Products include SmartModels, VMT model suites, DesignWare Memory Models, Building Block IP, and the DesignWare Synthesizable Components.
dual role device	Device having the capabilities of function and host (limited).
endian	Ordering of bytes in a multi-byte word; see also little-endian and big-endian.
Full-Functional Mode	A simulation model that describes the complete range of device behavior, including code execution. See also BFM.
GPIO	General Purpose Input Output.
GTECH	A generic technology view used for RTL simulation of encrypted source code by non-Synopsys simulators.
hard IP	Non-synthesizable implementation IP.
HDL	Hardware Description Language – examples include Verilog and VHDL.
IIP	Implementation Intellectual Property — A generic term for synthesizable HDL and non-synthesizable “hard” IP in all of its forms (coreKit, component, core, MacroCell, and so on).
implementation view	The RTL for a core. You can simulate, synthesize, and implement this view of a core in a real chip.
instantiate	The act of placing a core or model into a design.
interface	Set of ports and parameters that defines a connection point to a component.
IP	Intellectual property — A term that encompasses simulation models and synthesizable blocks of HDL code.
little-endian	Data format in which the least-significant byte comes first.
MacroCell	Bigger IP blocks (6811, 8051, memory controller) available in the DesignWare Library and delivered with coreConsultant.
master	Device or model that initiates and controls another device or peripheral.
model	A Verification IP component or a Design View of a core.
monitor	A device or model that gathers performance statistics of a system.
non-blocking command	A testbench command that advances to the next testbench statement without waiting for the command to complete.
peripheral	Generally refers to a small core that has a bus connection, specifically an APB interface.

RTL	Register Transfer Level. A higher level of abstraction that implies a certain gate-level structure. Synthesis of RTL code yields a gate-level design.
SDRAM	Synchronous Dynamic Random Access Memory; high-speed DRAM adds a separate clock signal to control signals.
SDRAM controller	A memory controller with specific connections for SDRAMs.
slave	Device or model that is controlled by and responds to a master.
SoC	System on a chip.
soft IP	Any implementation IP that is configurable. Generally referred to as synthesizable IP.
static controller	Memory controller with specific connections for Static memories such as asynchronous SRAMs, Flash memory, and ROMs.
subsystem	In relation to coreAssembler, highest level of RTL that is automatically generated.
synthesis intent	Attributes that a core developer applies to a top-level design, ports, and core.
synthesizable IP	A type of Implementation IP that can be mapped to a target technology through synthesis. Sometimes referred to as Soft IP.
technology-independent	Design that allows the technology (that is, the library that implements the gate and via widths for gates) to be specified later during synthesis.
Testsuite Regression Environment (TRE)	A collection of files for stand-alone verification of the configured component. The files, tests, and functionality vary from component to component.
VIP	Verification Intellectual Property — A generic term for a simulation model in any form, including a Design View.
workspace	A network location that contains a personal copy of a component or subsystem. After you configure the component or subsystem (using coreConsultant or coreAssembler), the workspace contains the configured component/subsystem and generated views needed for integration of the component/subsystem at the top level.
wrap, wrapper	Code, usually VHDL or Verilog, that surrounds a design or model, allowing easier interfacing. Usually requires an extra, sometimes automated, step to create the wrapper.
zero-cycle command	A command that executes without HDL simulation time advancing.

Index

A

Accessing registers [246](#)
 active command queue
 definition [333](#)
 activity
 definition [333](#)
 Address register, re-alignment of [166](#)
 AHB
 definition [333](#)
 AHB master interface
 arbitration for [105](#)
 AMBA
 definition [333](#)
 APB
 definition [333](#)
 APB bridge
 definition [333](#)
 application design
 definition [333](#)
 arbiter
 definition [333](#)
 Arbitration, for AHB master interface [105](#)
 Auto-reloading, of channel registers [251](#)

B

BFM
 definition [333](#)
 big-endian
 definition [333](#)
 Block
 chaining, about [247](#)
 size [43](#)
 Block diagram, of DW_ahb_dmac [17](#)
 blocked command stream
 definition [333](#)
 blocking command
 definition [333](#)

bus bridge
 definition [334](#)
 Bus locking, about [103](#)

C

Channel
 locking
 about [103](#)
 and deadlock [315](#)
 levels of [104](#)
 registers [160](#), [163](#)
 auto-reloading of [251](#)
 configuration [180](#)
 control [169](#)
 destination address [165](#)
 destination status [177](#)
 destination status address [179](#)
 linked list pointer [167](#)
 source address [164](#)
 source status [176](#)
 source status address [178](#)
 Channel configuration register (CFG) [180](#)
 Coherency
 about [301](#)
 read [308](#)
 write [302](#)
 Combined interrupt status register [197](#)
 command channel
 definition [334](#)
 command stream
 definition [334](#)
 component
 definition [334](#)
 configuration
 definition [334](#)
 configuration intent
 definition [334](#)
 Configuration parameters

- ARM PrimeCell equivalents [329](#)
- DMAH_ADD_ENCODED_PARAMS [122](#)
- DMAH_BIG_ENDIAN [124](#)
- DMAH_CHx_CTL_WB_EN [133](#)
- DMAH_CHx_DMS [129](#)
- DMAH_CHx_DST_NON_OK [130](#)
- DMAH_CHx_DST_SCA_EN [133](#)
- DMAH_CHx_DTW [129](#)
- DMAH_CHx_FIFO_DEPTH [127](#)
- DMAH_CHx_HC_LLP [131](#)
- DMAH_CHx_LMS [132](#)
- DMAH_CHx_LOCK_EN [128](#)
- DMAH_CHx_MAX_BLK_SIZE_BITS [128](#)
- DMAH_CHx_MULTI_BLK_EN [130](#)
- DMAH_CHx_MULTI_BLK_TYPE [131](#)
- DMAH_CHx_SMS [129](#)
- DMAH_CHx_SRC_GAT_EN [133](#)
- DMAH_CHx_SRC_NON_OK [130](#)
- DMAH_CHx_STAT_DST [127](#), [132](#)
- DMAH_CHx_STAT_SRC [132](#)
- DMAH_CHx_STW [129](#)
- DMAH_FC [128](#)
- DMAH_ID_NUM [120](#)
- DMAH_INTR_IO [121](#)
- DMAH_INTR_POL [120](#)
- DMAH_M1_AHB_LITE [126](#)
- DMAH_M1_HDATA_WIDTH [126](#)
- DMAH_M2_AHB_LITE [126](#)
- DMAH_M2_HDATA_WIDTH [126](#)
- DMAH_M3_AHB_LITE [126](#)
- DMAH_M3_HDATA_WIDTH [126](#)
- DMAH_M4_AHB_LITE [127](#)
- DMAH_M4_HDATA_WIDTH [127](#)
- DMAH_MABRST [121](#)
- DMAH_NUM_CHANNELS [120](#)
- DMAH_NUM_HS_INT [120](#)
- DMAH_NUM_MASTER_INT [120](#)
- DMAH_RETURN_ERR_RESP [122](#)
- DMAH_REVERSE_WB_OVERRIDE [123](#)
- DMAH_S_HDATA_WIDTH [125](#)
- Control register (CTL) [169](#)
- core
 - definition [334](#)
- core developer
 - definition [334](#)
- core integrator
 - definition [334](#)
- coreAssembler
 - definition [334](#)
- overview of usage flow [34](#)
- coreConsultant
 - definition [334](#)
 - overview of usage flow [28](#)
- coreKit
 - definition [334](#)
- Customer Support [12](#)
- cycle command
 - definition [334](#)
- D**
- Deadlock, and channel locking [315](#)
- decoder
 - definition [334](#)
- design context
 - definition [334](#)
- design creation
 - definition [334](#)
- Design View
 - definition [334](#)
- DesignWare cores
 - definition [335](#)
- DesignWare Library
 - definition [335](#)
- DesignWare Synthesizable Components
 - definition [334](#)
- Destination
 - burst transaction size [43](#)
 - peripheral
 - and transaction requests [103](#)
 - single transaction size [43](#)
- Destination address register (DAR) [165](#)
- Destination software transaction request register [200](#)
- Destination status address register (DSTATAR) [179](#)
- Destination status register (DSTAT) [177](#)
- Disabling DMA channel
 - prior to transfer completion [282](#)
- DMA transfers
 - hierarchy of [21](#)
 - locked [103](#)
- DMAH_ADD_ENCODED_PARAMS [122](#)
- DMAH_BIG_ENDIAN [124](#)
- DMAH_CHx_CTL_WB_EN [133](#)
- DMAH_CHx_DMS [129](#)
- DMAH_CHx_DST_NON_OK [130](#)
- DMAH_CHx_DST_SCA_EN [133](#)

- DMAH_CHx_DTW [129](#)
- DMAH_CHx_FIFO_DEPTH [127](#)
- DMAH_CHx_HC_LLQ [131](#)
- DMAH_CHx_LMS [132](#)
- DMAH_CHx_LOCK_EN [128](#)
- DMAH_CHx_MAX_BLK_SIZE_BITS [128](#)
- DMAH_CHx_MAX_MULT_SIZE [127](#)
- DMAH_CHx_MULTI_BLK_EN [130](#)
- DMAH_CHx_MULTI_BLK_TYPE [131](#)
- DMAH_CHx_SMS [129](#)
- DMAH_CHx_SRC_GAT_EN [133](#)
- DMAH_CHx_SRC_NON_OK [130](#)
- DMAH_CHx_STAT_DST [132](#)
- DMAH_CHx_STAT_SRC [132](#)
- DMAH_CHx_STW [129](#)
- DMAH_FC [128](#)
- DMAH_ID_NUM [120](#)
- DMAH_INTR_IO [121](#)
- DMAH_INTR_POL [120](#)
- DMAH_M1_AHB_LITE [126](#)
- DMAH_M1_HDATA_WIDTH [126](#)
- DMAH_M2_AHB_LITE [126](#)
- DMAH_M2_HDATA_WIDTH [126](#)
- DMAH_M3_AHB_LITE [126](#)
- DMAH_M3_HDATA_WIDTH [126](#)
- DMAH_M4_AHB_LITE [127](#)
- DMAH_M4_HDATA_WIDTH [127](#)
- DMAH_MABRST [121](#)
- DMAH_NUM_CHANNELS [120](#)
- DMAH_NUM_HS_INT [120](#)
- DMAH_NUM_MASTER_INT [120](#)
- DMAH_RETURN_ERR_RESP [122](#)
- DMAH_REVERSE_WB_OVERRIDE [123](#)
- DMAH_S_HDATA_WIDTH [125](#)
- dual role device
 - definition [335](#)
- DW_ahb_dmac
 - block diagram of [17](#)
 - channel enable register [162](#)
 - configuration register [161](#)
 - description [119](#)
 - features of [18](#), [19](#)
 - functional description of [17](#), [41](#)
 - ID register [205](#)
 - memory map of [147](#)
 - programming interface [147](#)
 - programming of [245](#)
 - test register [206](#)
 - testbench
 - overview of [286](#)
 - overview of tests [285](#)
 - version ID register [241](#)
- E**
- Early-terminated burst transaction [47](#)
- endian
 - definition [335](#)
- Environment, licenses [26](#)
- F**
- FIFO, readiness of [101](#)
- Flow control
 - about [41](#)
 - configurations of [95](#)
- Full-Functional Mode
 - definition [335](#)
- Functional description [17](#), [119](#)
- G**
- Generating
 - hardware handshaking signals [53](#)
 - requests [101](#)
- GPIO
 - definition [335](#)
- GTECH
 - definition [335](#)
- H**
- Handshaking interface
 - overview [42](#)
- hard IP
 - definition [335](#)
- Hardware handshaking interface
 - generating signals for [53](#)
 - mapping DW_ahb_dmac to PrimeCell [321](#)
 - peripheral is flow controller [57](#)
 - peripheral not flow controller [48](#)
- HDL
 - definition [335](#)
- I**
- IIP
 - definition [335](#)
- implementation view

- definition 335
- instantiate
 - definition 335
- interface
 - definition 335
- Interrupt clear registers 196
- Interrupt mask registers 194
- Interrupt raw status registers 192
- Interrupt registers 190
 - clear 196
 - combined interrupt status 197
 - mask 194
 - raw status 192
 - status 193
- Interrupt Status registers 193
- IP
 - definition 335

L

- Last destination transaction request register 204
- Last source transaction request register 203
- Licenses 26
- Linked list pointer register (LLP) 167
- little-endian
 - definition 335

M

- MacroCell
 - definition 335
- master
 - definition 335
- Memory map, of DW_ahb_dmac 147
- Memory peripherals 44
- model
 - definition 335
- monitor
 - definition 335

N

- non-blocking command
 - definition 335

O

- Output files
 - GTECH 39
 - RTL-level 39
 - Simulation model 39
 - synthesis 40

- verification 40

P

- peripheral
 - definition 335
- Peripheral burst transaction request, about 97
- Peripheral interrupt request interface
 - about 94
- Peripheral interrupt request interface, about 94
- PrimeCell
 - equivalent configuration parameters 329
 - mapping hardware handshaking interface to DW_ahb_dmac 321
 - mapping software handshaking registers to DW_ahb_dmac 323
- Programming interface 147

R

- Read coherency
 - about 308
 - and asynchronous clocks 310
 - and synchronous clocks 309
- Register access 246
- Registers
 - channel enable 162
 - DW_ahb_dmac configuration 161
 - DW_ahb_dmac ID 205
 - DW_ahb_dmac test 206
 - DW_ahb_dmac version ID 241
- Requesting, DMA transfers 42
- RTL
 - definition 336

S

- SDRAM
 - definition 336
- SDRAM controller
 - definition 336
- Setting up, transfers 60
- Single destination transaction request register 202
- Single source transaction request register 201
- Single transactions
 - peripheral is flow controller 60
 - peripheral not flow controller 55
- slave
 - definition 336
- SoC
 - definition 336

- SoC Platform
 - AHB contained in [15](#)
 - APB, contained in [15](#)
 - defined [15](#)
- soft IP
 - definition [336](#)
- Software handshaking interface
 - about [44](#)
 - mapping PrimeCell to DW_ahb_dmac [323](#)
 - peripheral as flow controller [59](#)
 - peripheral is flow controller [59](#)
 - peripheral not flow controller [54](#)
 - registers [45](#)
- Software handshaking registers [198](#)
 - destination software transaction request [200](#)
 - last destination transaction request [204](#)
 - last source transaction request [203](#)
 - single destination transaction request [202](#)
 - single source transaction request [201](#)
 - source software transaction request [199](#)
- Source
 - burst transaction size [43](#)
 - single transaction size [43](#)
- Source address register (SAR) [164](#)
- Source software transaction request register [199](#)
- Source status address register (SSTATAR) [178](#)
- Source status register (SSTAT) [176](#)
- static controller
 - definition [336](#)
- subsystem
 - definition [336](#)
- synthesis intent
 - definition [336](#)
- synthesizable IP
 - definition [336](#)
- T**
- technology-independent
 - definition [336](#)
- Testsuite Regression Environment (TRE)
 - definition [336](#)
- Transaction requests
 - from destination peripheral [103](#)
 - generating [101](#)
- Transactions
 - early-terminated burst [47](#)
 - single
 - peripheral is flow controller [60](#)
 - peripheral not flow controller [55](#)
- Transfers
 - operation of [61](#)
 - setting up [60](#)
- TRE
 - definition [336](#)
- U**
- USE_FOUNDATION [119](#)
- V**
- Vera, overview of tests [285](#)
- Verification
 - and Vera tests [285](#)
- VIP
 - definition [336](#)
- W**
- workspace
 - definition [336](#)
- wrap
 - definition [336](#)
- wrapper
 - definition [336](#)
- Write coherency
 - about [302](#)
 - and asynchronous clocks [307](#)
 - and identical clocks [303](#)
 - and synchronous clocks [304](#)
- Z**
- zero-cycle command
 - definition [336](#)

